UVM Simulation Code

Q1.0 Print HELLO_WORLD

```
`include "uvm_macros.svh"
import uvm_pkg::*;

module hello_world;
initial
    `uvm_info("info", "HELLO_WORLD",UVM_LOW);
endmodule
```

Command line for Run code

[b23n_sarif_727@prac3 uvm_lab]$ vcs -sverilog hello.sv -ntb_opts uvm -kdb -lca -debug_access+all -full64

Simulation results:

```
UVM_INFO hello.sv(7) @ 0: reporter [info] HELLO_WORLD
        V C S   S i m u l a t i o n   R e p o r t
Time: 0
```

Method-2

```
`include "uvm_macros.svh"
import uvm_pkg::*;

class my_test extends uvm_test;
    `uvm_component_utils(my_test)

    function new(string name ,uvm_component parent);
        super.new(name,parent);
    endfunction
```

```
    task run_phase(uvm_phase phase);
        `uvm_info("NAME", "SARIF_AHMAD", UVM_LOW)
    endtask
endclass

module top;
initial begin
    run_test("my_test");
end
endmodule
```

Simulation Results:

```
UVM_INFO @ 0: reporter [RNTST] Running test my_test...
UVM_INFO hello.sv(13) @ 0: uvm_test_top [NAME] SARIF_AHMAD

--- UVM Report Summary ---

** Report counts by severity
UVM_INFO :    2
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
** Report counts by id
[NAME]     1
[RNTST]     1
$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_root.svh", line 437.
```

$finish at simulation time 0

V C S S i m u l a t i o n R e p o r t

Time: 0

ADDER Code of UVM

Command line for Run code

[b23n_sarif_727@prac3 adder]$ ls

adder_design.sv csrc scoreboard.sv simv ucli.key

adder_pkg.sv driver.sv sequence_item.sv simv.daidir vc_hdrs.h

add_if.sv environment.sv sequencer.sv test.sv

agent.sv monitor.sv sequence.sv top.sv

```
//
module adder(input clk,reset,input [7:0] in1,in2, output reg [8:0] out);

always@(posedge clk or posedge reset) begin
        if(reset)
                out <= 0;
        else
                out <= in1 + in2;
end
endmodule

//
`include "uvm_macros.svh"
import uvm_pkg::*;
```

```
class seq_item extends uvm_sequence_item;
	rand bit [7:0] ip1,ip2;
	bit [8:0] out;

	function new(string name="seq_item");
		super.new(name);
	endfunction

	`uvm_object_utils_begin(seq_item)
	`uvm_field_int(ip1,UVM_ALL_ON);
	`uvm_field_int(ip2,UVM_ALL_ON);
	`uvm_object_utils_end

	constraint c1 {ip1 <100; ip2<100;}
endclass

//
`include "uvm_macros.svh"
import uvm_pkg::*;

class base_seq extends uvm_sequence #(seq_item);
	seq_item req;
	`uvm_object_utils(base_seq)

	function new(string name="base_seq");
		super.new(name);
	endfunction
```

```
	task body();
		`uvm_info(get_type_name(), "Base seq: Inside Body", UVM_LOW);
		`uvm_do(req);
	endtask
endclass

//
`include "uvm_macros.svh"
import uvm_pkg::*;

class seqcr extends uvm_sequencer #(seq_item);
	`uvm_component_utils(seqcr)

	function new(string name = "seqcr", uvm_component parent = null);
		super.new(name, parent);
	endfunction

	function void build_phase(uvm_phase phase);
		super.build_phase(phase);
	endfunction

endclass

//
`include "uvm_macros.svh"
import uvm_pkg::*;
```

```
class driver extends uvm_driver#(seq_item);
	virtual add_if vif;
	`uvm_component_utils(driver)

	function new(string name = "driver", uvm_component parent = null);
		super.new(name, parent);
	endfunction

	function void build_phase(uvm_phase phase);
		super.build_phase(phase);
		if(!uvm_config_db#(virtual add_if) :: get(this, "", "vif", vif))
			`uvm_fatal(get_type_name(), "Not set at top level");
	endfunction

	task run_phase (uvm_phase phase);
		forever begin
			seq_item_port.get_next_item(req);
			`uvm_info(get_type_name, $sformatf("ip1 = %0d, ip2 = %0d", 
req.ip1, req.ip2), UVM_LOW);
			vif.ip1 <= req.ip1;
			vif.ip2 <= req.ip2;
			seq_item_port.item_done();
		end
	endtask
endclass

//
```

```
`include "uvm_macros.svh"
import uvm_pkg::*;

class monitor extends uvm_monitor;
	virtual add_if vif;
	uvm_analysis_port #(seq_item) item_collect_port;
	seq_item mon_item;
	`uvm_component_utils(monitor)

	function new(string name = "monitor", uvm_component parent = null);
		super.new(name, parent);
		item_collect_port = new("item_collect_port", this);
		mon_item = new();
	endfunction

	function void build_phase(uvm_phase phase);
		super.build_phase(phase);
		if(!uvm_config_db#(virtual add_if) :: get(this, "", "vif", vif))
			`uvm_fatal(get_type_name(), "Not set at top level");
	endfunction

	task run_phase (uvm_phase phase);
		forever begin
			wait(!vif.reset);
			@(posedge vif.clk);
			mon_item.ip1 = vif.ip1;
			mon_item.ip2 = vif.ip2;
```

```
                `uvm_info(get_type_name, $sformatf("ip1 = %0d, ip2 = %0d", mon_item.ip1, mon_item.ip2), UVM_HIGH);
                @(posedge vif.clk);
                mon_item.out = vif.out;
                item_collect_port.write(mon_item);
            end
        endtask
endclass

//
`include "uvm_macros.svh"
import uvm_pkg::*;

class scoreboard extends uvm_scoreboard;
        uvm_analysis_imp #(seq_item, scoreboard) item_collect_export;
        seq_item item_q[$];
        `uvm_component_utils(scoreboard)

        function new(string name = "scoreboard", uvm_component parent = null);
                super.new(name, parent);
                item_collect_export = new("item_collect_export", this);
        endfunction

        function void build_phase(uvm_phase phase);
                super.build_phase(phase);
       endfunction

        function void write(seq_item req);
```

```
        item_q.push_back(req);
    endfunction

    task run_phase (uvm_phase phase);
        seq_item sb_item;
        forever begin
            wait(item_q.size > 0);
            if(item_q.size > 0) begin
                sb_item = item_q.pop_front();
                $display("----------------------------------------");
                if(sb_item.ip1 + sb_item.ip2 == sb_item.out) begin
                    `uvm_info(get_type_name, $sformatf("Matched: ip1 = %0d, ip2 = %0d, out = %0d", sb_item.ip1, sb_item.ip2, sb_item.out),UVM_LOW);
                end
                else begin
                    `uvm_error(get_name, $sformatf("NOT matched: ip1 = %0d, ip2 = %0d, out = %0d", sb_item.ip1, sb_item.ip2, sb_item.out));
                end
                $display("----------------------------------");
            end
        end
    endtask
endclass

//
`include "uvm_macros.svh"
import uvm_pkg::*;
```

```
class agent extends uvm_agent;
	`uvm_component_utils(agent)
	driver drv;
	seqcr seqr;
	monitor mon;

	function new(string name = "agent", uvm_component parent = null);
		super.new(name, parent);
	endfunction

	function void build_phase(uvm_phase phase);
		super.build_phase(phase);
		if(get_is_active == UVM_ACTIVE) begin
			drv = driver::type_id::create("drv", this);
			seqr = seqcr::type_id::create("seqr", this);
		end
		mon = monitor::type_id::create("mon", this);
	endfunction

	function void connect_phase(uvm_phase phase);
		if(get_is_active == UVM_ACTIVE) begin
			drv.seq_item_port.connect(seqr.seq_item_export);
		end
	endfunction
endclass

//
```

```
`include "uvm_macros.svh"
import uvm_pkg::*;

class env extends uvm_env;
	`uvm_component_utils(env)
	agent agt;
	scoreboard sb;

	function new(string name = "env", uvm_component parent = null);
		super.new(name, parent);
	endfunction

	function void build_phase(uvm_phase phase);
		super.build_phase(phase);
		agt = agent::type_id::create("agt", this);
		sb = scoreboard::type_id::create("sb", this);
	endfunction

	function void connect_phase(uvm_phase phase);
		agt.mon.item_collect_port.connect(sb.item_collect_export);
	endfunction
endclass

//
`include "uvm_macros.svh"
import uvm_pkg::*;
```

```
class base_test extends uvm_test;
	env env_o;
	base_seq bseq;
	`uvm_component_utils(base_test)

	function new(string name = "base_test", uvm_component parent = null);
		super.new(name, parent);
	endfunction

	function void build_phase(uvm_phase phase);
		super.build_phase(phase);
		env_o = env::type_id::create("env_o", this);
	endfunction

	task run_phase(uvm_phase phase);
		phase.raise_objection(this);
		bseq = base_seq::type_id::create("bseq");
		repeat(10) begin
			#5; bseq.start(env_o.agt.seqr);
		end

		phase.drop_objection(this);
		`uvm_info(get_type_name, "End of testcase", UVM_LOW);
	endtask
endclass

//
```

```
package adder_pkg;

`include "uvm_macros.svh"
import uvm_pkg::*;
`include "sequence_item.sv";
`include "sequence.sv";
`include "sequencer.sv";
`include "driver.sv";
`include "monitor.sv";
`include "agent.sv";
`include "scoreboard.sv";
`include "environment.sv";
`include "test.sv";
endpackage

//
`include "uvm_macros.svh"
import uvm_pkg::*;

interface add_if(input logic clk, reset);
        logic [7:0] ip1;
        logic [7:0] ip2;
        logic [8:0] out;

endinterface

//
```

```
`include "uvm_macros.svh"
import uvm_pkg::*;
import adder_pkg::*;

interface add_if(input logic clk, reset);
	logic [7:0] ip1;
	logic [7:0] ip2;
	logic [8:0] out;

endinterface

module tb_top;
bit clk;
bit reset;

always #2 clk = ~clk;

initial begin
	reset = 1;
	#5;
	reset = 0;
end

add_if vif(clk, reset);

adder DUT(.clk(vif.clk),.reset(vif.reset),.in1(vif.ip1),.in2(vif.ip2),.out(vif.out));
```

```
initial begin
        uvm_config_db#(virtual add_if)::set(uvm_root::get(), "*", "vif", vif);
end

initial begin
        run_test("base_test");
end
endmodule
```

```
[b23n_sarif_727@prac3 adder]$ vcs -sverilog adder_design.sv adder_pkg.sv top.sv -ntb_opts uvm -kdb -lca -debug_access+all -full64
[b23n_sarif_727@prac3 adder]$ ./simv
```

Simulation Results:

```
UVM_INFO @ 0: reporter [RNTST] Running test base_test...
UVM_INFO sequence.sv(14) @ 5: uvm_test_top.env_o.agt.seqr@@bseq [base_seq] Base seq: Inside Body
UVM_INFO driver.sv(22) @ 5: uvm_test_top.env_o.agt.drv [driver] ip1 = 5, ip2 = 45
----------------------------------------
UVM_INFO scoreboard.sv(31) @ 10: uvm_test_top.env_o.sb [scoreboard] Matched: ip1 = 5, ip2 = 45, out = 50
----------------------------------
UVM_INFO sequence.sv(14) @ 10: uvm_test_top.env_o.agt.seqr@@bseq [base_seq] Base seq: Inside Body
UVM_INFO driver.sv(22) @ 10: uvm_test_top.env_o.agt.drv [driver] ip1 = 99, ip2 = 71
UVM_INFO sequence.sv(14) @ 15: uvm_test_top.env_o.agt.seqr@@bseq [base_seq] Base seq: Inside Body
UVM_INFO driver.sv(22) @ 15: uvm_test_top.env_o.agt.drv [driver] ip1 = 20, ip2 = 5
----------------------------------------
```

UVM_INFO scoreboard.sv(31) @ 18: uvm_test_top.env_o.sb [scoreboard] Matched: ip1 = 99, ip2 = 71, out = 170

UVM_INFO sequence.sv(14) @ 20: uvm_test_top.env_o.agt.seqr@@bseq [base_seq] Base seq: Inside Body

UVM_INFO driver.sv(22) @ 20: uvm_test_top.env_o.agt.drv [driver] ip1 = 58, ip2 = 99

UVM_INFO sequence.sv(14) @ 25: uvm_test_top.env_o.agt.seqr@@bseq [base_seq] Base seq: Inside Body

UVM_INFO driver.sv(22) @ 25: uvm_test_top.env_o.agt.drv [driver] ip1 = 37, ip2 = 74

UVM_INFO scoreboard.sv(31) @ 26: uvm_test_top.env_o.sb [scoreboard] Matched: ip1 = 58, ip2 = 99, out = 157

UVM_INFO sequence.sv(14) @ 30: uvm_test_top.env_o.agt.seqr@@bseq [base_seq] Base seq: Inside Body

UVM_INFO driver.sv(22) @ 30: uvm_test_top.env_o.agt.drv [driver] ip1 = 69, ip2 = 78

UVM_INFO scoreboard.sv(31) @ 34: uvm_test_top.env_o.sb [scoreboard] Matched: ip1 = 37, ip2 = 74, out = 111

UVM_INFO sequence.sv(14) @ 35: uvm_test_top.env_o.agt.seqr@@bseq [base_seq] Base seq: Inside Body

UVM_INFO driver.sv(22) @ 35: uvm_test_top.env_o.agt.drv [driver] ip1 = 72, ip2 = 11

UVM_INFO sequence.sv(14) @ 40: uvm_test_top.env_o.agt.seqr@@bseq [base_seq] Base seq: Inside Body

UVM_INFO driver.sv(22) @ 40: uvm_test_top.env_o.agt.drv [driver] ip1 = 79, ip2 = 27

UVM_INFO scoreboard.sv(31) @ 42: uvm_test_top.env_o.sb [scoreboard] Matched: ip1 = 72, ip2 = 11, out = 83

UVM_INFO sequence.sv(14) @ 45: uvm_test_top.env_o.agt.seqr@@bseq [base_seq] Base seq: Inside Body

UVM_INFO driver.sv(22) @ 45: uvm_test_top.env_o.agt.drv [driver] ip1 = 47, ip2 = 82

UVM_INFO scoreboard.sv(31) @ 50: uvm_test_top.env_o.sb [scoreboard] Matched: ip1 = 47, ip2 = 82, out = 129

UVM_INFO sequence.sv(14) @ 50: uvm_test_top.env_o.agt.seqr@@bseq [base_seq] Base seq: Inside Body

UVM_INFO driver.sv(22) @ 50: uvm_test_top.env_o.agt.drv [driver] ip1 = 65, ip2 = 95

UVM_INFO test.sv(27) @ 50: uvm_test_top [base_test] End of testcase

UVM_INFO /usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_objection.svh(1274) @ 50: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase

--- UVM Report Summary ---

** Report counts by severity

UVM_INFO :   29

UVM_WARNING :   0

UVM_ERROR :   0

UVM_FATAL :   0

** Report counts by id

[RNTST]   1

[TEST_DONE]   1

[base_seq]   10

[base_test]   1

[driver]   10

[scoreboard]   6

$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_root.svh", line 437.

$finish at simulation time 50

V C S S i m u l a t i o n R e p o r t

Time: 50

CPU Time: 0.130 seconds; Data structure size: 0.5Mb

Tue Jun 2 15:28:44 2026

- (31) → line number in that file
- @ 10 → simulation time = 10

Short answer: 31 is the line number in scoreboard.sv where the uvm_info message was generated.

D-flipflop of UVM Code:

Command line for Run code

[b23n_sarif_727@prac3 dflipflop]$ ls

agent.sv dff_design.sv dff_pkg.sv environment.sv scoreboard.sv sequencer.sv simv tb_top.sv ucli.key

csrc dff_if.sv driver.sv monitor.sv sequence_item.sv sequence.sv simv.daidir test.sv vc_hdrs.h

Simulation run command:

[b23n_sarif_727@prac3 dflipflop]$ vcs -sverilog dff_design.sv dff_if.sv dff_pkg.sv tb_top.sv -ntb_opts uvm -kdb -lca -debug_access+all -full64

[b23n_sarif_727@prac3 dflipflop]$ ./simv

//

module dff (

```
        input  clk,
      input  reset,
      input  d,
      output reg q
);

always @(posedge clk or posedge reset) begin
        if(reset)
                q <= 0;
        else
                q <= d;
end
endmodule

//
interface dff_if(input logic clk, reset);
        logic d;
      logic q;
endinterface

//
package dff_pkg;
```

```
import uvm_pkg::*;
`include "uvm_macros.svh"

`include "sequence_item.sv"
`include "sequence.sv"
`include "sequencer.sv"
`include "driver.sv"
`include "monitor.sv"
`include "scoreboard.sv"
`include "agent.sv"
`include "environment.sv"
`include "test.sv"

endpackage

//
class dff_seq_item extends uvm_sequence_item;
        rand bit d;
        bit q;

        `uvm_object_utils_begin(dff_seq_item)
      `uvm_field_int(d,UVM_ALL_ON)
      `uvm_field_int(q,UVM_ALL_ON)
```

```
	`uvm_object_utils_end

	function new(string name="dff_seq_item");
		super.new(name);
	endfunction

endclass

//
class dff_sequence extends uvm_sequence #(dff_seq_item);
	`uvm_object_utils(dff_sequence)

	function new(string name="dff_sequence");
		super.new(name);
	endfunction

	task body();
		repeat(10) begin
			`uvm_do(req)
		end
	endtask

endclass
```

```
//
class dff_sequencer extends uvm_sequencer #(dff_seq_item);
        `uvm_component_utils(dff_sequencer)

        function new(string name,uvm_component parent);
                super.new(name,parent);
       endfunction

endclass

//
class dff_driver extends uvm_driver #(dff_seq_item);
        virtual dff_if vif;
        `uvm_component_utils(dff_driver)

        function new(string name,uvm_component parent);
                super.new(name,parent);
        endfunction

        function void build_phase(uvm_phase phase);
                if(!uvm_config_db#(virtual dff_if)::get(this,"","vif",vif))
                        `uvm_fatal("DRV","Interface not found")
        endfunction
```

```
	task run_phase(uvm_phase phase);
		forever begin
			seq_item_port.get_next_item(req);
			@(posedge vif.clk);
			vif.d <= req.d;
			`uvm_info("DRIVER",$sformatf("Driving
d=%0d",req.d),UVM_LOW)
			seq_item_port.item_done();
		end
	endtask
endclass

//
class dff_monitor extends uvm_monitor;
	virtual dff_if vif;
	dff_seq_item item;
	uvm_analysis_port #(dff_seq_item) mon_ap;
	`uvm_component_utils(dff_monitor)

	function new(string name,uvm_component parent);
		super.new(name,parent);
		mon_ap = new("mon_ap",this);
	endfunction
```

```
    function void build_phase(uvm_phase phase);
        if(!uvm_config_db#(virtual dff_if)::get(this,"","vif",vif))
            `uvm_fatal("MON","Interface not found")
    endfunction

    task run_phase(uvm_phase phase);
        forever begin
            item = dff_seq_item::type_id::create("item");
            @(posedge vif.clk);
            item.d = vif.d;

            @(posedge vif.clk);
            item.q = vif.q;
            mon_ap.write(item);
        end
    endtask
endclass

//
class dff_scoreboard extends uvm_scoreboard;
    `uvm_component_utils(dff_scoreboard)
    uvm_analysis_imp #(dff_seq_item,dff_scoreboard) sb_imp;
```

```
	function new(string name,uvm_component parent);
		super.new(name,parent);
		sb_imp = new("sb_imp",this);
	endfunction

	function void write(dff_seq_item item);
		if(item.d == item.q)
			`uvm_info("SCOREBOARD",$sformatf("PASS d=%0d q=%0d",item.d,item.q),UVM_LOW)
		else
			`uvm_error("SCOREBOARD",$sformatf("FAIL d=%0d q=%0d",item.d,item.q))
	endfunction

endclass

//
class dff_agent extends uvm_agent;
	dff_driver drv;
	dff_monitor mon;
	dff_sequencer seqr;

	`uvm_component_utils(dff_agent)
```

```
    function new(string name,uvm_component parent);
            super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
            drv  = dff_driver::type_id::create("drv",this);
          mon  = dff_monitor::type_id::create("mon",this);
          seqr = dff_sequencer::type_id::create("seqr",this);
    endfunction

    function void connect_phase(uvm_phase phase);
            drv.seq_item_port.connect(seqr.seq_item_export);
    endfunction

endclass

//
class dff_env extends uvm_env;
        dff_agent agt;
        dff_scoreboard sb;
        `uvm_component_utils(dff_env)
```

```
    function new(string name,uvm_component parent);
        super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
        agt = dff_agent::type_id::create("agt",this);
        sb = dff_scoreboard::type_id::create("sb",this);
    endfunction

    function void connect_phase(uvm_phase phase);
        agt.mon.mon_ap.connect(sb.sb_imp);
    endfunction

endclass

//
class dff_test extends uvm_test;
    dff_env env;
    dff_sequence seq;
    `uvm_component_utils(dff_test)

    function new(string name,uvm_component parent);
        super.new(name,parent);
```

```
    endfunction

    function void build_phase(uvm_phase phase);
        env = dff_env::type_id::create("env",this);
    endfunction

    task run_phase(uvm_phase phase);
        phase.raise_objection(this);

        seq = dff_sequence::type_id::create("seq");
        seq.start(env.agt.seqr);
        phase.drop_objection(this);
    endtask
endclass

//
module tb_top;

import uvm_pkg::*;
import dff_pkg::*;

bit clk;
bit reset;
```

```
always #5 clk = ~clk;

initial begin
        clk = 0;
        reset = 1;
        #10;
        reset = 0;
end

dff_if vif(clk,reset);

dff dut(.clk(vif.clk),.reset(vif.reset),.d(vif.d),.q(vif.q));

initial begin
        uvm_config_db#(virtual dff_if)::set(uvm_root::get(),"*","vif",vif);
        run_test("dff_test");
end

endmodule
```

Simulation Results:

```
UVM_INFO @ 0: reporter [RNTST] Running test dff_test...
```

UVM_INFO driver.sv(20) @ 5: uvm_test_top.env.agt.drv [DRIVER] Driving d=0

UVM_INFO scoreboard.sv(13) @ 15: uvm_test_top.env.sb [SCOREBOARD] PASS d=0 q=0

UVM_INFO driver.sv(20) @ 15: uvm_test_top.env.agt.drv [DRIVER] Driving d=0

UVM_INFO driver.sv(20) @ 25: uvm_test_top.env.agt.drv [DRIVER] Driving d=0

UVM_INFO scoreboard.sv(13) @ 35: uvm_test_top.env.sb [SCOREBOARD] PASS d=0 q=0

UVM_INFO driver.sv(20) @ 35: uvm_test_top.env.agt.drv [DRIVER] Driving d=1

UVM_INFO driver.sv(20) @ 45: uvm_test_top.env.agt.drv [DRIVER] Driving d=1

UVM_INFO scoreboard.sv(13) @ 55: uvm_test_top.env.sb [SCOREBOARD] PASS d=1 q=1

UVM_INFO driver.sv(20) @ 55: uvm_test_top.env.agt.drv [DRIVER] Driving d=1

UVM_INFO driver.sv(20) @ 65: uvm_test_top.env.agt.drv [DRIVER] Driving d=1

UVM_INFO scoreboard.sv(13) @ 75: uvm_test_top.env.sb [SCOREBOARD] PASS d=1 q=1

UVM_INFO driver.sv(20) @ 75: uvm_test_top.env.agt.drv [DRIVER] Driving d=0

UVM_INFO driver.sv(20) @ 85: uvm_test_top.env.agt.drv [DRIVER] Driving d=1

UVM_INFO scoreboard.sv(13) @ 95: uvm_test_top.env.sb [SCOREBOARD] PASS d=0 q=0

UVM_INFO driver.sv(20) @ 95: uvm_test_top.env.agt.drv [DRIVER] Driving d=1

UVM_INFO /usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_objection.svh(1274) @ 95: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase

--- UVM Report Summary ---

** Report counts by severity

UVM_INFO : 17

UVM_WARNING : 0

UVM_ERROR : 0

UVM_FATAL : 0

** Report counts by id

[DRIVER] 10

[RNTST] 1

[SCOREBOARD] 5

[TEST_DONE] 1

$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_root.svh", line 437.

$finish at simulation time 95

V C S S i m u l a t i o n R e p o r t

Time: 95

CPU Time: 0.130 seconds; Data structure size: 0.5Mb

Wed Jun 17 15:50:26 2026

Better D-flip flop of UVM Code

```
//
class dff_seq_item extends uvm_sequence_item;
        rand bit d;
        bit q;

        `uvm_object_utils_begin(dff_seq_item)
      `uvm_field_int(d,UVM_ALL_ON)
```

```
	`uvm_field_int(q,UVM_ALL_ON)
	  `uvm_object_utils_end

	  function new(string name="dff_seq_item");
		super.new(name);
	endfunction

endclass

//
class dff_sequence extends uvm_sequence #(dff_seq_item);
	  `uvm_object_utils(dff_sequence)
	  dff_seq_item req;

	  function new(string name="dff_sequence");
		super.new(name);
	endfunction

	  task body();
		`uvm_info(get_type_name(),"d-ff sequence :inside body", UVM_LOW);
		repeat(10) begin
			`uvm_do(req)
		end
	  endtask

endclass
```

```
//
class dff_sequencer extends uvm_sequencer #(dff_seq_item);
        `uvm_component_utils(dff_sequencer)

        function new(string name="dff_sequencer", uvm_component parent=null);
                super.new(name,parent);
       endfunction

        function void build_phase(uvm_phase phase);
                super.build_phase(phase);
        endfunction

endclass

//
class dff_driver extends uvm_driver #(dff_seq_item);
        virtual dff_if vif;
        `uvm_component_utils(dff_driver)

        function new(string name ="dff_driver", uvm_component parent=null);
                super.new(name,parent);
        endfunction

        function void build_phase(uvm_phase phase);
                super.build_phase(phase);
                if(!uvm_config_db#(virtual dff_if)::get(this,"","vif",vif))
                        `uvm_fatal(get_type_name(),"Interface not found")
```

```
	endfunction

	task run_phase(uvm_phase phase);
		forever begin
			seq_item_port.get_next_item(req);
			@(posedge vif.clk);
			vif.d <= req.d;
			`uvm_info(get_type_name(),$sformatf("Driving
d=%0d",req.d),UVM_LOW)
			seq_item_port.item_done();
		end
	endtask
endclass

//
class dff_monitor extends uvm_monitor;
	virtual dff_if vif;
	dff_seq_item item;
	uvm_analysis_port #(dff_seq_item) mon_ap;
	`uvm_component_utils(dff_monitor)

	function new(string name ="dff_monitor",uvm_component parent=null);
		super.new(name,parent);
		mon_ap = new("mon_ap",this);
	endfunction

	function void build_phase(uvm_phase phase);
		if(!uvm_config_db#(virtual dff_if)::get(this,"","vif",vif))
```

```
                `uvm_fatal("get_type_name()","Interface not found")
        endfunction

        task run_phase(uvm_phase phase);
                forever begin
                        item = dff_seq_item::type_id::create("item");
                        @(posedge vif.clk);
                        item.d = vif.d;
                        `uvm_info(get_type_name(), $sformatf("MON:observe the data
d:%d", item.d),UVM_LOW);


                        @(posedge vif.clk);
                        item.q = vif.q;
                        `uvm_info(get_type_name(), $sformatf("MON:observe the output
data q:%d", item.q),UVM_LOW);
                        mon_ap.write(item);
                end
        endtask
endclass

//
class dff_scoreboard extends uvm_scoreboard;
        `uvm_component_utils(dff_scoreboard)
        uvm_analysis_imp #(dff_seq_item,dff_scoreboard) sb_imp;

        function new(string name="dff_scoreboard", uvm_component parent=null);
                super.new(name,parent);
                sb_imp = new("sb_imp",this);
```

```
        endfunction

        function void write(dff_seq_item item);
                if(item.d == item.q)
                        `uvm_info(get_type_name(),$sformatf("PASS d=%0d
q=%0d",item.d,item.q),UVM_LOW)
                else
                        `uvm_error(get_type_name(),$sformatf("FAIL d=%0d
q=%0d",item.d,item.q))
        endfunction

endclass

//
class dff_agent extends uvm_agent;
        dff_driver drv;
        dff_monitor mon;
        dff_sequencer seqr;

        `uvm_component_utils(dff_agent)

    function new(string name="dff_agent", uvm_component parent=null);
            super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
            super.build_phase(phase);
            drv  = dff_driver::type_id::create("drv",this);
```

```
        mon  = dff_monitor::type_id::create("mon",this);
        seqr = dff_sequencer::type_id::create("seqr",this);
    endfunction

    function void connect_phase(uvm_phase phase);
          drv.seq_item_port.connect(seqr.seq_item_export);
    endfunction

endclass

//
class dff_env extends uvm_env;
        dff_agent agt;
        dff_scoreboard sb;
        `uvm_component_utils(dff_env)

        function new(string name="dff_env",uvm_component parent=null);
                super.new(name,parent);
      endfunction

      function void build_phase(uvm_phase phase);
                super.build_phase(phase);
                agt = dff_agent::type_id::create("agt",this);
                sb = dff_scoreboard::type_id::create("sb",this);
        endfunction

        function void connect_phase(uvm_phase phase);
```

```
        super.connect_phase(phase);
        agt.mon.mon_ap.connect(sb.sb_imp);
    endfunction

endclass

//
class dff_test extends uvm_test;
    dff_env env;
   dff_sequence seq;
    `uvm_component_utils(dff_test)

    function new(string name="dff_test",uvm_component parent=null);
        super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
        env = dff_env::type_id::create("env",this);
    endfunction

    task run_phase(uvm_phase phase);
        phase.raise_objection(this);

        seq = dff_sequence::type_id::create("seq");
        seq.start(env.agt.seqr);
        phase.drop_objection(this);
```

```
                `uvm_info(get_type_name(),"End Of Testcases",UVM_LOW);
        endtask
endclass

//
module tb_top;

import uvm_pkg::*;
import dff_pkg::*;

bit clk;
bit reset;

always #5 clk = ~clk;

initial begin
        clk = 0;
        reset = 1;
        #10;
        reset = 0;
end

dff_if vif(clk,reset);

dff dut(.clk(vif.clk),.reset(vif.reset),.d(vif.d),.q(vif.q));

initial begin
```

```
        uvm_config_db#(virtual dff_if)::set(uvm_root::get(),"*","vif",vif);
        run_test("dff_test");
end


endmodule
```

Simulation Run command

[b23n_sarif_727@prac3 dflipflop]$ vcs -sverilog dff_design.sv dff_if.sv dff_pkg.sv tb_top.sv -ntb_opts uvm -kdb -lca -debug_access+all -full64

Simulation results:

UVM_INFO @ 0: reporter [RNTST] Running test dff_test...

UVM_INFO sequence.sv(11) @ 0: uvm_test_top.env.agt.seqr@@seq [dff_sequence] d-ff sequence :inside body

UVM_INFO monitor.sv(23) @ 5: uvm_test_top.env.agt.mon [dff_monitor] MON:observe the data d:0

UVM_INFO driver.sv(21) @ 5: uvm_test_top.env.agt.drv [dff_driver] Driving d=0

UVM_INFO monitor.sv(27) @ 15: uvm_test_top.env.agt.mon [dff_monitor] MON:observe the output data q:0

UVM_INFO scoreboard.sv(13) @ 15: uvm_test_top.env.sb [dff_scoreboard] PASS d=0 q=0

UVM_INFO driver.sv(21) @ 15: uvm_test_top.env.agt.drv [dff_driver] Driving d=0

UVM_INFO monitor.sv(23) @ 25: uvm_test_top.env.agt.mon [dff_monitor] MON:observe the data d:0

UVM_INFO driver.sv(21) @ 25: uvm_test_top.env.agt.drv [dff_driver] Driving d=0

UVM_INFO monitor.sv(27) @ 35: uvm_test_top.env.agt.mon [dff_monitor] MON:observe the output data q:0

UVM_INFO scoreboard.sv(13) @ 35: uvm_test_top.env.sb [dff_scoreboard] PASS d=0 q=0

UVM_INFO driver.sv(21) @ 35: uvm_test_top.env.agt.drv [dff_driver] Driving d=1

UVM_INFO monitor.sv(23) @ 45: uvm_test_top.env.agt.mon [dff_monitor] MON:observe the data d:1

UVM_INFO driver.sv(21) @ 45: uvm_test_top.env.agt.drv [dff_driver] Driving d=1

UVM_INFO monitor.sv(27) @ 55: uvm_test_top.env.agt.mon [dff_monitor] MON:observe the output data q:1

UVM_INFO scoreboard.sv(13) @ 55: uvm_test_top.env.sb [dff_scoreboard] PASS d=1 q=1

UVM_INFO driver.sv(21) @ 55: uvm_test_top.env.agt.drv [dff_driver] Driving d=1

UVM_INFO monitor.sv(23) @ 65: uvm_test_top.env.agt.mon [dff_monitor] MON:observe the data d:1

UVM_INFO driver.sv(21) @ 65: uvm_test_top.env.agt.drv [dff_driver] Driving d=1

UVM_INFO monitor.sv(27) @ 75: uvm_test_top.env.agt.mon [dff_monitor] MON:observe the output data q:1

UVM_INFO scoreboard.sv(13) @ 75: uvm_test_top.env.sb [dff_scoreboard] PASS d=1 q=1

UVM_INFO driver.sv(21) @ 75: uvm_test_top.env.agt.drv [dff_driver] Driving d=0

UVM_INFO monitor.sv(23) @ 85: uvm_test_top.env.agt.mon [dff_monitor] MON:observe the data d:0

UVM_INFO driver.sv(21) @ 85: uvm_test_top.env.agt.drv [dff_driver] Driving d=1

UVM_INFO monitor.sv(27) @ 95: uvm_test_top.env.agt.mon [dff_monitor] MON:observe the output data q:0

UVM_INFO scoreboard.sv(13) @ 95: uvm_test_top.env.sb [dff_scoreboard] PASS d=0 q=0

UVM_INFO driver.sv(21) @ 95: uvm_test_top.env.agt.drv [dff_driver] Driving d=1

UVM_INFO test.sv(22) @ 95: uvm_test_top [dff_test] End Of Testcases

UVM_INFO /usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_objection.svh(1274) @ 95: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase

--- UVM Report Summary ---

** Report counts by severity

UVM_INFO : 29

UVM_WARNING : 0

UVM_ERROR : 0

UVM_FATAL : 0

** Report counts by id

[RNTST] 1

[TEST_DONE] 1

[dff_driver] 10

[dff_monitor] 10

[dff_scoreboard] 5

[dff_sequence] 1

[dff_test] 1

$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_root.svh", line 437.

$finish at simulation time 95

V C S S i m u l a t i o n R e p o r t

Time: 95

CPU Time: 0.130 seconds; Data structure size: 0.5Mb

Wed Jun 17 18:18:01 2026

UVM code of AND Gate

[b23n_sarif_727@prac3 and_gate]$ ls

agent.sv and_if.sv csrc env.sv scoreboard.sv sequencer.sv simv tb_top.sv ucli.key

and_gate.sv and_pkg.sv driver.sv monitor.sv sequence_item.sv sequence.sv simv.daidir test.sv vc_hdrs.h

[b23n_sarif_727@prac3 and_gate]$ vcs -sverilog and_gate.sv and_if.sv and_pkg.sv tb_top.sv -ntb_opts uvm -kdb -lca -debug_access+all -full64

```
[b23n_sarif_727@prac3 and_gate]$ ./simv

//
module and_gate(
        input logic a,
        input logic b,
        output logic y
);

assign y = a & b;

endmodule

//
interface and_if;
        logic a;
        logic b;
        logic y;
endinterface

//
package and_pkg;
```

```
import uvm_pkg::*;
`include "uvm_macros.svh"

`include "sequence_item.sv"
`include "sequence.sv"
`include "sequencer.sv"
`include "driver.sv"
`include "monitor.sv"
`include "scoreboard.sv"
`include "agent.sv"
`include "env.sv"
`include "test.sv"

endpackage

//
class and_seq_item extends uvm_sequence_item;
        rand bit a;
        rand bit b;
        bit y;

        `uvm_object_utils_begin(and_seq_item)
        `uvm_field_int(a,UVM_ALL_ON)
```

```
	`uvm_field_int(b,UVM_ALL_ON)
	`uvm_field_int(y,UVM_ALL_ON)
	`uvm_object_utils_end

	function new(string name = "and_seq_item");
		super.new(name);
	endfunction

endclass

//
class and_sequence extends uvm_sequence #(and_seq_item);
	`uvm_object_utils(and_sequence)
	and_seq_item req;

	function new(string name="and_sequence");
		super.new(name);
	endfunction

	task body();
		`uvm_info(get_type_name(),"Base sequence:inside body",UVM_LOW);
		repeat(10) begin
			`uvm_do(req);
```

```
        end
    endtask
endclass

//
class and_sequencer extends uvm_sequencer #(and_seq_item);
    `uvm_component_utils(and_sequencer)

    function new(string name="and_sequencer", uvm_component parent=null);
        super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
    endfunction

endclass

//
class and_driver extends uvm_driver #(and_seq_item);
    virtual and_if vif;
    `uvm_component_utils(and_driver)
```

```
	function new(string name="and_driver", uvm_component parent=null);
		super.new(name,parent);
	endfunction

	function void build_phase(uvm_phase phase);
		super.build_phase(phase);
		if(!uvm_config_db #(virtual and_if)::get(this,"","vif",vif))
			`uvm_fatal(get_type_name(),"Interface Not Found");
	endfunction

	task run_phase(uvm_phase phase);
		forever begin
			seq_item_port.get_next_item(req);
			vif.a <= req.a;
			vif.b <= req.b;
			`uvm_info(get_type_name(), $sformatf("a=%0d b=%0d",req.a,req.b),UVM_LOW);

			seq_item_port.item_done();
			#5;
		end
	endtask
endclass
```

```
//
class and_monitor extends uvm_monitor;
	virtual and_if vif;
	and_seq_item item;
	uvm_analysis_port #(and_seq_item) mon_ap;
	`uvm_component_utils(and_monitor)

	function new(string name="and_monitor", uvm_component parent=null);
		super.new(name,parent);
		mon_ap = new("mon_ap",this);
	endfunction

	function void build_phase(uvm_phase phase);
		super.build_phase(phase);
		if(!uvm_config_db #(virtual and_if)::get(this,"","vif",vif))
			`uvm_fatal(get_type_name(),"Interface Not Found");
	endfunction

	task run_phase(uvm_phase phase);
		forever begin
			#5;
			item = and_seq_item::type_id::create("item");
			item.a = vif.a;
```

```
                item.b = vif.b;
                item.y = vif.y;

                mon_ap.write(item);
            end
        endtask
endclass

//
class and_scoreboard extends uvm_scoreboard;
        `uvm_component_utils(and_scoreboard)
        uvm_analysis_imp #(and_seq_item,and_scoreboard) sb_imp;

        function new(string name="and_scoreboard",uvm_component parent=null);
            super.new(name,parent);
            sb_imp =new("sb_imp",this);
        endfunction

        function void build_phase(uvm_phase phase);
            super.build_phase(phase);
        endfunction

        function void write(and_seq_item item);
```

```
        if((item.a & item.b) == item.y)
            `uvm_info("SCOREBOARD", $sformatf("PASS a=%0d b=%0d y=%0d",item.a,item.b,item.y),UVM_LOW)
        else
            `uvm_error("SCOREBOARD",$sformatf("FAIL a=%0d b=%0d y=%0d",item.a,item.b,item.y))
    endfunction


endclass


//
class and_agent extends uvm_agent;
    and_driver drv;
   and_monitor mon;
   and_sequencer seqr;
    `uvm_component_utils(and_agent)


    function new(string name="and_agent", uvm_component parent=null);
        super.new(name,parent);
    endfunction


    function void build_phase(uvm_phase phase);
        drv  = and_driver::type_id::create("drv",this);
        mon  = and_monitor::type_id::create("mon",this);
```

```
        seqr = and_sequencer::type_id::create("seqr",this);

    endfunction

      function void connect_phase(uvm_phase phase);
            drv.seq_item_port.connect(seqr.seq_item_export);
      endfunction

endclass

//
class and_env extends uvm_env;
      and_agent agt;
      and_scoreboard sb;
      `uvm_component_utils(and_env)

      function new(string name="and_env", uvm_component parent=null);
            super.new(name,parent);
    endfunction

      function void build_phase(uvm_phase phase);
            agt = and_agent::type_id::create("agt",this);
            sb = and_scoreboard::type_id::create("sb",this);
```

```
    endfunction

    function void connect_phase(uvm_phase phase);
        agt.mon.mon_ap.connect(sb.sb_imp);
    endfunction

endclass

//
class and_test extends uvm_test;
    and_env env;
   and_sequence seq;
    `uvm_component_utils(and_test)

    function new(string name="and_test", uvm_component parent=null);
        super.new(name,parent);
   endfunction

    function void build_phase(uvm_phase phase);
        env = and_env::type_id::create("env",this);
    endfunction

    task run_phase(uvm_phase phase);
```

```
        phase.raise_objection(this);
        seq = and_sequence::type_id::create("seq");
        seq.start(env.agt.seqr);
        phase.drop_objection(this);
    endtask

endclass

//
module tb_top;

import uvm_pkg::*;
import and_pkg::*;

and_if vif();

and_gate dut(.a(vif.a),.b(vif.b),.y(vif.y));

initial begin
    uvm_config_db#(virtual and_if)::set(uvm_root::get(),"*","vif",vif);
    run_test("and_test");
end
```

endmodule

Simulation Results:

UVM_INFO @ 0: reporter [RNTST] Running test and_test...

UVM_INFO sequence.sv(11) @ 0: uvm_test_top.env.agt.seqr@@seq [and_sequence] Base sequence:inside body

UVM_INFO driver.sv(21) @ 0: uvm_test_top.env.agt.drv [and_driver] a=1 b=1

UVM_INFO scoreboard.sv(17) @ 5: uvm_test_top.env.sb [SCOREBOARD] PASS a=1 b=1 y=1

UVM_INFO driver.sv(21) @ 5: uvm_test_top.env.agt.drv [and_driver] a=0 b=1

UVM_INFO scoreboard.sv(17) @ 10: uvm_test_top.env.sb [SCOREBOARD] PASS a=0 b=1 y=0

UVM_INFO driver.sv(21) @ 10: uvm_test_top.env.agt.drv [and_driver] a=0 b=0

UVM_INFO scoreboard.sv(17) @ 15: uvm_test_top.env.sb [SCOREBOARD] PASS a=0 b=0 y=0

UVM_INFO driver.sv(21) @ 15: uvm_test_top.env.agt.drv [and_driver] a=0 b=0

UVM_INFO scoreboard.sv(17) @ 20: uvm_test_top.env.sb [SCOREBOARD] PASS a=0 b=0 y=0

UVM_INFO driver.sv(21) @ 20: uvm_test_top.env.agt.drv [and_driver] a=0 b=0

UVM_INFO scoreboard.sv(17) @ 25: uvm_test_top.env.sb [SCOREBOARD] PASS a=0 b=0 y=0

UVM_INFO driver.sv(21) @ 25: uvm_test_top.env.agt.drv [and_driver] a=0 b=1

UVM_INFO scoreboard.sv(17) @ 30: uvm_test_top.env.sb [SCOREBOARD] PASS a=0 b=1 y=0

UVM_INFO driver.sv(21) @ 30: uvm_test_top.env.agt.drv [and_driver] a=0 b=0

UVM_INFO scoreboard.sv(17) @ 35: uvm_test_top.env.sb [SCOREBOARD] PASS a=0 b=0 y=0

UVM_INFO driver.sv(21) @ 35: uvm_test_top.env.agt.drv [and_driver] a=0 b=0

UVM_INFO scoreboard.sv(17) @ 40: uvm_test_top.env.sb [SCOREBOARD] PASS a=0 b=0 y=0

UVM_INFO driver.sv(21) @ 40: uvm_test_top.env.agt.drv [and_driver] a=0 b=1

UVM_INFO scoreboard.sv(17) @ 45: uvm_test_top.env.sb [SCOREBOARD] PASS a=0 b=1 y=0

UVM_INFO driver.sv(21) @ 45: uvm_test_top.env.agt.drv [and_driver] a=1 b=0

UVM_INFO /usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_objection.svh(1274) @ 45: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase

--- UVM Report Summary ---

** Report counts by severity

UVM_INFO : 22

UVM_WARNING : 0

UVM_ERROR : 0

UVM_FATAL : 0

** Report counts by id

[RNTST] 1

[SCOREBOARD] 9

[TEST_DONE] 1

[and_driver] 10

[and_sequence] 1

$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_root.svh", line 437.

$finish at simulation time 45

V C S S i m u l a t i o n R e p o r t

Time: 45

UVM Code for 4-bit Counter Circuit

[b23n_sarif_727@prac3 counter]$ ls

agent.sv counter_if.sv csrc env.sv scoreboard.sv sequencer.sv simv tb_top.sv ucli.key

counter_design.sv counter_pkg.sv driver.sv monitor.sv sequence_item.sv sequence.sv simv.daidir test.sv vc_hdrs.h

[b23n_sarif_727@prac3 counter]$

```
//
module counter(
	input clk,
	input rst,
	output reg [3:0] count
);

always @(posedge clk or posedge rst) begin
	if (rst)
		count <= 4'd0;
	else
		count <= count + 1'b1;
end

endmodule

//
interface counter_if (input logic clk,input logic rst);
```

```
        logic [3:0] count;
endinterface

//
package counter_pkg;

import uvm_pkg::*;
`include "uvm_macros.svh"

`include "sequence_item.sv"
`include "sequence.sv"
`include "sequencer.sv"
`include "driver.sv"
`include "monitor.sv"
`include "scoreboard.sv"
`include "agent.sv"
`include "env.sv"
`include "test.sv"

endpackage

//
class counter_seq_item extends uvm_sequence_item;
```

```
        bit [3:0] count;

        `uvm_object_utils_begin(counter_seq_item)
        `uvm_field_int(count,UVM_ALL_ON)
        `uvm_object_utils_end

        function new(string name="counter_seq_item");
                super.new(name);
        endfunction

endclass

//
class counter_sequence extends uvm_sequence #(counter_seq_item);
        counter_seq_item req;
        `uvm_object_utils(counter_sequence)

        function new(string name="counter_sequence");
                super.new(name);
        endfunction

        task body();
                repeat(20) begin
```

```
                `uvm_do(req)
            end
        endtask
endclass

//
class counter_sequencer extends uvm_sequencer #(counter_seq_item);
        `uvm_component_utils(counter_sequencer)

        function new(string name="counter_sequencer", uvm_component parent=null);
            super.new(name,parent);
        endfunction

        function void build_phase(uvm_phase phase);
            super.build_phase(phase);
        endfunction

endclass

//
class counter_driver extends uvm_driver #(counter_seq_item);
        `uvm_component_utils(counter_driver)
        virtual counter_if vif;
```

```
    function new(string name="counter_driver", uvm_component parent=null);
        super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);

        if (!uvm_config_db #(virtual counter_if)::get(this,"","vif",vif))
            `uvm_fatal(get_type_name(),"Interface Not Found");
    endfunction

    task run_phase(uvm_phase phase);
        forever begin
            seq_item_port.get_next_item(req);
            `uvm_info("DRIVER","Counter Running", UVM_LOW)
            seq_item_port.item_done();
            #10;
        end
    endtask
endclass

//
```

```
class counter_monitor extends uvm_monitor;
	virtual counter_if vif;
	counter_seq_item item;
	uvm_analysis_port #(counter_seq_item) mon_ap;
	`uvm_component_utils(counter_monitor)

	function new(string name="counter_monitor", uvm_component parent=null);
		super.new(name,parent);
		mon_ap = new("mon_ap",this);
	endfunction

	function void build_phase(uvm_phase phase);
		if(!uvm_config_db#(virtual counter_if)::get(this,"","vif",vif))
		begin
			`uvm_fatal("MON","Interface Not Found")
		end
	endfunction

	task run_phase(uvm_phase phase);
		forever begin
			wait(!vif.rst);
			@(posedge vif.clk);
			item = counter_seq_item::type_id::create("item");
```

```
                item.count = vif.count;
                mon_ap.write(item);
            end
        endtask
endclass

//
class counter_scoreboard extends uvm_scoreboard;
        uvm_analysis_imp #(counter_seq_item,counter_scoreboard) sb_imp;

        bit [3:0] expected_count;
        `uvm_component_utils(counter_scoreboard)

        function new(string name="counter_scoreboard", uvm_component parent=null);
            super.new(name,parent);
            sb_imp = new("sb_imp",this);
            expected_count = 0;
        endfunction

        function void write(counter_seq_item item);
            if(item.count == expected_count) begin
                `uvm_info("SCOREBOARD", $sformatf("PASS count=%0d",
item.count),UVM_LOW)
            end
```

```
            else begin
                `uvm_error("SCOREBOARD", $sformatf("FAIL Expected=%0d Actual=%0d",expected_count, item.count))
            end
            expected_count++;
        endfunction

endclass

//
class counter_agent extends uvm_agent;
        counter_driver drv;
        counter_monitor mon;
        counter_sequencer seqr;
        `uvm_component_utils(counter_agent)

        function new(string name="counter_agent", uvm_component parent=null);
            super.new(name,parent);
       endfunction

        function void build_phase(uvm_phase phase);
            drv  = counter_driver::type_id::create("drv",this);
            mon  = counter_monitor::type_id::create("mon",this);
            seqr = counter_sequencer::type_id::create("seqr",this);
```

```
    endfunction

      function void connect_phase(uvm_phase phase);
            drv.seq_item_port.connect(seqr.seq_item_export);
      endfunction

endclass

//
class counter_env extends uvm_env;
      counter_agent agt;
    counter_scoreboard sb;
      `uvm_component_utils(counter_env)

      function new(string name="counter_env", uvm_component parent=null);
            super.new(name,parent);
      endfunction

      function void build_phase(uvm_phase phase);
            agt = counter_agent::type_id::create("agt",this);
            sb = counter_scoreboard::type_id::create("sb",this);
      endfunction
```

```
	function void connect_phase(uvm_phase phase);
		agt.mon.mon_ap.connect(sb.sb_imp);
	endfunction

endclass

//
class counter_test extends uvm_test;
	counter_env env;
	counter_sequence seq;
	`uvm_component_utils(counter_test)

	function new(string name="counter_test", uvm_component parent=null);
		super.new(name,parent);
	endfunction

	function void build_phase(uvm_phase phase);
		env = counter_env::type_id::create("env",this);
	endfunction

	task run_phase(uvm_phase phase);
		phase.raise_objection(this);
```

```
        seq = counter_sequence::type_id::create("seq");
        seq.start(env.agt.seqr);
        #200;
        phase.drop_objection(this);
    endtask

endclass

//
module tb_top;

import uvm_pkg::*;
import counter_pkg::*;

bit clk;
bit rst;

always #5 clk = ~clk;

initial begin
    clk = 0;
    rst = 1;
```

```
        #10;
        rst = 0;
end

counter_if vif(clk,rst);

counter dut(.clk(vif.clk),.rst(vif.rst),.count(vif.count));

initial begin
        uvm_config_db#(virtual counter_if)::set(uvm_root::get(),"*","vif",vif);
        run_test("counter_test");
end

endmodule
```

```
[b23n_sarif_727@prac3 counter]$ vcs -sverilog counter_design.sv counter_if.sv counter_pkg.sv tb_top.sv -ntb_opts uvm -kdb -lca -debug_access+all -full64

[b23n_sarif_727@prac3 counter]$ ./simv
```

Simulation Results:

```
UVM_INFO @ 0: reporter [RNTST] Running test counter_test...
UVM_INFO driver.sv(20) @ 0: uvm_test_top.env.agt.drv [DRIVER] Counter Running
UVM_INFO driver.sv(20) @ 10: uvm_test_top.env.agt.drv [DRIVER] Counter Running
UVM_INFO scoreboard.sv(16) @ 15: uvm_test_top.env.sb [SCOREBOARD] PASS count=0
```

UVM_INFO driver.sv(20) @ 20: uvm_test_top.env.agt.drv [DRIVER] Counter Running

UVM_INFO scoreboard.sv(16) @ 25: uvm_test_top.env.sb [SCOREBOARD] PASS count=1

UVM_INFO driver.sv(20) @ 30: uvm_test_top.env.agt.drv [DRIVER] Counter Running

UVM_INFO scoreboard.sv(16) @ 35: uvm_test_top.env.sb [SCOREBOARD] PASS count=2

UVM_INFO driver.sv(20) @ 40: uvm_test_top.env.agt.drv [DRIVER] Counter Running

UVM_INFO scoreboard.sv(16) @ 45: uvm_test_top.env.sb [SCOREBOARD] PASS count=3

UVM_INFO driver.sv(20) @ 50: uvm_test_top.env.agt.drv [DRIVER] Counter Running

UVM_INFO scoreboard.sv(16) @ 55: uvm_test_top.env.sb [SCOREBOARD] PASS count=4

UVM_INFO driver.sv(20) @ 60: uvm_test_top.env.agt.drv [DRIVER] Counter Running

UVM_INFO scoreboard.sv(16) @ 65: uvm_test_top.env.sb [SCOREBOARD] PASS count=5

UVM_INFO driver.sv(20) @ 70: uvm_test_top.env.agt.drv [DRIVER] Counter Running

UVM_INFO scoreboard.sv(16) @ 75: uvm_test_top.env.sb [SCOREBOARD] PASS count=6

UVM_INFO driver.sv(20) @ 80: uvm_test_top.env.agt.drv [DRIVER] Counter Running

UVM_INFO scoreboard.sv(16) @ 85: uvm_test_top.env.sb [SCOREBOARD] PASS count=7

UVM_INFO driver.sv(20) @ 90: uvm_test_top.env.agt.drv [DRIVER] Counter Running

UVM_INFO scoreboard.sv(16) @ 95: uvm_test_top.env.sb [SCOREBOARD] PASS count=8

UVM_INFO driver.sv(20) @ 100: uvm_test_top.env.agt.drv [DRIVER] Counter Running

UVM_INFO scoreboard.sv(16) @ 105: uvm_test_top.env.sb [SCOREBOARD] PASS count=9

UVM_INFO driver.sv(20) @ 110: uvm_test_top.env.agt.drv [DRIVER] Counter Running

UVM_INFO scoreboard.sv(16) @ 115: uvm_test_top.env.sb [SCOREBOARD] PASS count=10

UVM_INFO driver.sv(20) @ 120: uvm_test_top.env.agt.drv [DRIVER] Counter Running

UVM_INFO scoreboard.sv(16) @ 125: uvm_test_top.env.sb [SCOREBOARD] PASS count=11

UVM_INFO driver.sv(20) @ 130: uvm_test_top.env.agt.drv [DRIVER] Counter Running

UVM_INFO scoreboard.sv(16) @ 135: uvm_test_top.env.sb [SCOREBOARD] PASS count=12

UVM_INFO driver.sv(20) @ 140: uvm_test_top.env.agt.drv [DRIVER] Counter Running

UVM_INFO scoreboard.sv(16) @ 145: uvm_test_top.env.sb [SCOREBOARD] PASS count=13

UVM_INFO driver.sv(20) @ 150: uvm_test_top.env.agt.drv [DRIVER] Counter Running

UVM_INFO scoreboard.sv(16) @ 155: uvm_test_top.env.sb [SCOREBOARD] PASS count=14

UVM_INFO driver.sv(20) @ 160: uvm_test_top.env.agt.drv [DRIVER] Counter Running

UVM_INFO scoreboard.sv(16) @ 165: uvm_test_top.env.sb [SCOREBOARD] PASS count=15

UVM_INFO driver.sv(20) @ 170: uvm_test_top.env.agt.drv [DRIVER] Counter Running

UVM_INFO scoreboard.sv(16) @ 175: uvm_test_top.env.sb [SCOREBOARD] PASS count=0

UVM_INFO driver.sv(20) @ 180: uvm_test_top.env.agt.drv [DRIVER] Counter Running

UVM_INFO scoreboard.sv(16) @ 185: uvm_test_top.env.sb [SCOREBOARD] PASS count=1

UVM_INFO driver.sv(20) @ 190: uvm_test_top.env.agt.drv [DRIVER] Counter Running

UVM_INFO scoreboard.sv(16) @ 195: uvm_test_top.env.sb [SCOREBOARD] PASS count=2

UVM_INFO scoreboard.sv(16) @ 205: uvm_test_top.env.sb [SCOREBOARD] PASS count=3

UVM_INFO scoreboard.sv(16) @ 215: uvm_test_top.env.sb [SCOREBOARD] PASS count=4

UVM_INFO scoreboard.sv(16) @ 225: uvm_test_top.env.sb [SCOREBOARD] PASS count=5

UVM_INFO scoreboard.sv(16) @ 235: uvm_test_top.env.sb [SCOREBOARD] PASS count=6

UVM_INFO scoreboard.sv(16) @ 245: uvm_test_top.env.sb [SCOREBOARD] PASS count=7

UVM_INFO scoreboard.sv(16) @ 255: uvm_test_top.env.sb [SCOREBOARD] PASS count=8

UVM_INFO scoreboard.sv(16) @ 265: uvm_test_top.env.sb [SCOREBOARD] PASS count=9

UVM_INFO scoreboard.sv(16) @ 275: uvm_test_top.env.sb [SCOREBOARD] PASS count=10

UVM_INFO scoreboard.sv(16) @ 285: uvm_test_top.env.sb [SCOREBOARD] PASS count=11

UVM_INFO scoreboard.sv(16) @ 295: uvm_test_top.env.sb [SCOREBOARD] PASS count=12

UVM_INFO scoreboard.sv(16) @ 305: uvm_test_top.env.sb [SCOREBOARD] PASS count=13

UVM_INFO scoreboard.sv(16) @ 315: uvm_test_top.env.sb [SCOREBOARD] PASS count=14

UVM_INFO scoreboard.sv(16) @ 325: uvm_test_top.env.sb [SCOREBOARD] PASS count=15

UVM_INFO scoreboard.sv(16) @ 335: uvm_test_top.env.sb [SCOREBOARD] PASS count=0

UVM_INFO scoreboard.sv(16) @ 345: uvm_test_top.env.sb [SCOREBOARD] PASS count=1

UVM_INFO scoreboard.sv(16) @ 355: uvm_test_top.env.sb [SCOREBOARD] PASS count=2

UVM_INFO scoreboard.sv(16) @ 365: uvm_test_top.env.sb [SCOREBOARD] PASS count=3

UVM_INFO scoreboard.sv(16) @ 375: uvm_test_top.env.sb [SCOREBOARD] PASS count=4

UVM_INFO scoreboard.sv(16) @ 385: uvm_test_top.env.sb [SCOREBOARD] PASS count=5

UVM_INFO /usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_objection.svh(1274) @ 390: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase

--- UVM Report Summary ---

** Report counts by severity

UVM_INFO : 60

UVM_WARNING : 0

UVM_ERROR : 0

UVM_FATAL : 0

** Report counts by id

[DRIVER] 20

[RNTST] 1

[SCOREBOARD] 38

[TEST_DONE] 1

$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_root.svh", line 437.

$finish at simulation time 390

V C S S i m u l a t i o n R e p o r t

Time: 390

Print method in UVM: -

```
//
`include "uvm_macros.svh"
import uvm_pkg::*;

class seq_item extends uvm_sequence_item;
        rand bit [7:0] data;
        rand bit [7:0] addr;

        `uvm_object_utils_begin(seq_item)
        `uvm_field_int(data, UVM_ALL_ON)
        `uvm_field_int(addr, UVM_ALL_ON)
        `uvm_object_utils_end

        function new(string name = "seq_item");
                super.new(name);
        endfunction

endclass

module tb;

seq_item s1;

initial begin
        s1 = seq_item::type_id::create("s1");
        s1.randomize();
```

```
        s1.print();
end
endmodule
```

Run command:

```
[b23n_sarif_727@prac3 uvm_lab]$ vcs -sverilog print_method.sv -ntb_opts uvm -kdb -lca -debug_access+all -full64
```

Simulation Outputs:

```
----------------------------
Name    Type      Size  Value
----------------------------
s1      seq_item  -     @456
  data  integral  8     'hb9
  addr  integral  8     'h6c
----------------------------
        V C S   S i m u l a t i o n   R e p o r t
Time: 0
CPU Time:      0.150 seconds;       Data structure size:   0.5Mb
Thu Jun 18 19:40:04 2026
```

But I want to print 5 data in the above code- modified code

```
//
`include "uvm_macros.svh"
import uvm_pkg::*;

class seq_item extends uvm_sequence_item;
        rand bit [7:0] data;
        rand bit [7:0] addr;
```

```
	`uvm_object_utils_begin(seq_item)
	`uvm_field_int(data, UVM_ALL_ON)
	`uvm_field_int(addr, UVM_ALL_ON)
	`uvm_object_utils_end

	function new(string name = "seq_item");
		super.new(name);
	endfunction
endclass

module tb;
seq_item s1;
initial begin
	s1 = seq_item::type_id::create("s1");
	repeat(5) begin
		s1.randomize();
		s1.print();
	end
end
endmodule
```

Simulation Outputs:

```
----------------------------
Name    Type      Size  Value
----------------------------
s1      seq_item  -     @456
  data  integral  8     'hb9
```

```
  addr  integral  8     'h6c
----------------------------
----------------------------
Name    Type      Size  Value
----------------------------
s1      seq_item  -     @456
  data  integral  8     'h5b
  addr  integral  8     'h80
----------------------------
----------------------------
Name    Type      Size  Value
----------------------------
s1      seq_item  -     @456
  data  integral  8     'h3a
  addr  integral  8     'h56
----------------------------
----------------------------
Name    Type      Size  Value
----------------------------
s1      seq_item  -     @456
  data  integral  8     'h7
  addr  integral  8     'h52
----------------------------
----------------------------
Name    Type      Size  Value
----------------------------
s1      seq_item  -     @456
```

```
  data  integral  8     'he0
  addr  integral  8     'h4c
----------------------------
        V C S   S i m u l a t i o n   R e p o r t
Time: 0
```

do_print() Method in UVM

```
//
`include "uvm_macros.svh"
import uvm_pkg::*;

class seq_item extends uvm_sequence_item;

        bit [7:0] data;
        `uvm_object_utils(seq_item)

        function new(string name = "seq_item");
                super.new(name);
        endfunction
// Custom print method
      function void do_print(uvm_printer printer);
                super.do_print(printer);
                printer.print_field("data", data, 8);
        endfunction
endclass

module tb;
```

```
seq_item s1;
initial begin
        s1 = seq_item::type_id::create("s1");
        s1.data = 25;
        s1.print();
end
endmodule
```

Simulation Outputs:

```
----------------------------
Name    Type      Size  Value
----------------------------
s1      seq_item  -     @456
  data  integral  8     'h19
----------------------------
          V C S   S i m u l a t i o n   R e p o r t
Time: 0
```

copy () Method in UVM

```
//
`include "uvm_macros.svh"
import uvm_pkg::*;

class seq_item extends uvm_sequence_item;
        rand bit [7:0] data;

        `uvm_object_utils_begin(seq_item)
```

```
    `uvm_field_int(data, UVM_ALL_ON)
    `uvm_object_utils_end

    function new(string name = "seq_item");
        super.new(name);
    endfunction
endclass

module tb;
seq_item s1, s2;
initial begin
    // Create objects
    s1 = seq_item::type_id::create("s1");
    s2 = seq_item::type_id::create("s2");

    // Assign value
    s1.data = 55;

    // Copy s1 data into s2
    s2.copy(s1);

    // Print copied object
    s2.print();
end
endmodule
```

Simulation Outputs:

```
Name    Type      Size  Value
----------------------------
s2      seq_item  -     @460
  data  integral  8     'h37
----------------------------
          V C S   S i m u l a t i o n   R e p o r t
Time: 0
```

clone() Method in UVM

```
//
`include "uvm_macros.svh"
import uvm_pkg::*;

class seq_item extends uvm_sequence_item;
        rand bit [7:0] data;

        `uvm_object_utils_begin(seq_item)
        `uvm_field_int(data, UVM_ALL_ON)
        `uvm_object_utils_end

        function new(string name = "seq_item");
                super.new(name);
        endfunction
endclass

module tb;
```

```
seq_item s1, s2;
initial begin
        s1 = seq_item::type_id::create("s1");
        s1.data = 99;
        $cast(s2 ,s1.clone());
        s2.print();
end
endmodule
```

Run command:

```
[b23n_sarif_727@prac3 uvm_lab]$ vcs -sverilog clone_method.sv -ntb_opts uvm -kdb -lca -debug_access+all -full64 vcs -sverilog clone_met
```

Simulation Outputs:

```
----------------------------
Name    Type      Size  Value
----------------------------
s1      seq_item  -     @460
  data  integral  8     'h63
----------------------------
        V C S   S i m u l a t i o n   R e p o r t
Time: 0
```

do_copy() Method in UVM

```
//
`include "uvm_macros.svh"
import uvm_pkg::*;

class seq_item extends uvm_sequence_item;
```

```
        bit [7:0] data;

        `uvm_object_utils_begin(seq_item)
        `uvm_field_int(data, UVM_ALL_ON)
        `uvm_object_utils_end

        function new(string name = "seq_item");
                super.new(name);
        endfunction

        function void do_copy(uvm_object rhs);
                seq_item temp;
                super.do_copy(rhs);
                $cast(temp, rhs);
                data = temp.data;
        endfunction
endclass

module tb;
seq_item s1, s2;

initial begin
        s1 = seq_item::type_id::create("s1");
       s2 = seq_item::type_id::create("s2");
        s1.data = 45;
        s2.copy(s1);
        s2.print();
```

```
end
endmodule
```

Simulation Outputs:

```
----------------------------
Name    Type      Size  Value
----------------------------
s2      seq_item  -     @460
  data  integral  8     'h2d
----------------------------
          V C S   S i m u l a t i o n   R e p o r t
Time: 0
```

compare() Method in UVM:-

```
//
`include "uvm_macros.svh"
import uvm_pkg::*;

class seq_item extends uvm_sequence_item;
        bit [7:0] data;

        `uvm_object_utils_begin(seq_item)
        `uvm_field_int(data, UVM_ALL_ON)
        `uvm_object_utils_end

        function new(string name="seq_item");
                super.new(name);
        endfunction
```

```
endclass

module tb;
seq_item s1, s2;

initial begin
        s1 = seq_item::type_id::create("s1");
        s2 = seq_item::type_id::create("s2");

        s1.data = 25;
        s2.data = 25;

        if(s1.compare(s2))
                `uvm_info("COMPARE","DATA MATCHED", UVM_LOW)
        else
                `uvm_error("COMPARE", "DATA MISMATCH")
end

endmodule
```

Simulation Outputs:

```
UVM_INFO compare_uvm.sv(28) @ 0: reporter [COMPARE] DATA MATCHED
        V C S  S i m u l a t i o n  R e p o r t
Time: 0
CPU Time:      0.130 seconds;       Data structure size:   0.5Mb
Fri Jun 19 10:48:19 2026
```

do_compare() Method in UVM

```
//
`include "uvm_macros.svh"
import uvm_pkg::*;

class seq_item extends uvm_sequence_item;
        bit [7:0] data;

        `uvm_object_utils(seq_item)

        function new(string name="seq_item");
                super.new(name);
       endfunction

        function bit do_compare(uvm_object rhs, uvm_comparer comparer);
                seq_item temp;
                $cast(temp, rhs);
                return (data == temp.data);
        endfunction
endclass

module tb;
seq_item s1, s2;
```

```
initial begin
        s1 = seq_item::type_id::create("s1");
      s2 = seq_item::type_id::create("s2");

        s1.data = 50;
        s2.data = 50;

        if(s1.compare(s2))
                `uvm_info("COMPARE", "MATCHED", UVM_LOW)
        else
                `uvm_error("COMPARE", "MISMATCH")
end
endmodule
```

Simulation Outputs:

```
UVM_INFO do_compare.sv(32) @ 0: reporter [COMPARE] MATCHED
        V C S  S i m u l a t i o n  R e p o r t
Time: 0
CPU Time:      0.140 seconds;       Data structure size:   0.5Mb
Fri Jun 19 10:58:47 2026
```

Pack Methods in UVM: -

```
//
`include "uvm_macros.svh"
```

```
import uvm_pkg::*;

class seq_item extends uvm_sequence_item;
	bit [7:0] data;

	`uvm_object_utils_begin(seq_item)
	`uvm_field_int(data, UVM_ALL_ON)
	`uvm_object_utils_end

	function new(string name="seq_item");
		super.new(name);
	endfunction
endclass

module tb;
seq_item s1;

bit bits[];

initial begin
	s1 = seq_item::type_id::create("s1");
	s1.data = 15;
	s1.pack(bits);

	foreach(bits[i])
		$display("bits[%0d] = %0b",i, bits[i]);
end
```

endmodule

Simulation Outputs:

bits[0] = 0

bits[1] = 0

bits[2] = 0

bits[3] = 0

bits[4] = 1

bits[5] = 1

bits[6] = 1

bits[7] = 1

V C S S i m u l a t i o n R e p o r t

Time: 0

CPU Time: 0.140 seconds; Data structure size: 0.5Mb

Fri Jun 19 11:10:18 2026

Unpack Methods in UVM

//

`include "uvm_macros.svh"

import uvm_pkg::*;

class seq_item extends uvm_sequence_item;

    bit [7:0] data;

    `uvm_object_utils_begin(seq_item)

    `uvm_field_int(data, UVM_ALL_ON)

    `uvm_object_utils_end

```
        function new(string name="seq_item");
                super.new(name);
        endfunction
endclass

module tb;
seq_item s1, s2;

bit bits[];

initial begin
        s1 = seq_item::type_id::create("s1");
       s2 = seq_item::type_id::create("s2");
        s1.data = 25;


        s1.pack(bits);
        s2.unpack(bits);
        s2.print();
end
endmodule
```

Simulation Outputs:

```
----------------------------
Name    Type      Size  Value
----------------------------
s2      seq_item  -     @460
  data  integral  8     'h19
----------------------------
```

V C S S i m u l a t i o n R e p o r t

Time: 0

CPU Time: 0.140 seconds; Data structure size: 0.5Mb

Fri Jun 19 11:21:50 2026

1.0 Type Override

```
//
`include "uvm_macros.svh"
import uvm_pkg::*;
class base_driver extends uvm_component;
    `uvm_component_utils(base_driver)

    function new(string name="base_driver", uvm_component parent=null);
        super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
        `uvm_info("BASE_DRV","Base Driver Created",UVM_LOW)
  endfunction
endclass

class new_driver extends base_driver;
    `uvm_component_utils(new_driver)

    function new(string name="new_driver", uvm_component parent=null);
        super.new(name,parent);
    endfunction
```

```
    function void build_phase(uvm_phase phase);
        `uvm_info("NEW_DRV","New Driver Created",UVM_LOW)
    endfunction
endclass

class my_test extends uvm_test;
    `uvm_component_utils(my_test)
    base_driver drv;

    function new(string name="my_test",uvm_component parent=null);
        super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);

        base_driver::type_id::set_type_override(new_driver::get_type());
        drv = base_driver::type_id::create("drv", this);
    endfunction
endclass

module top;
initial begin
    run_test("my_test");
end
endmodule
```

Run command:

[b23n_sarif_727@prac3 uvm_lab]$ vcs -sverilog type_override.sv -ntb_opts uvm -kdb -lca -debug_access+all -full64

Simulation Outputs:

UVM_INFO @ 0: reporter [RNTST] Running test my_test...

UVM_INFO type_override.sv(24) @ 0: uvm_test_top.drv [NEW_DRV] New Driver Created

--- UVM Report Summary ---

** Report counts by severity

UVM_INFO : 2

UVM_WARNING : 0

UVM_ERROR : 0

UVM_FATAL : 0

** Report counts by id

[NEW_DRV] 1

[RNTST] 1

$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_root.svh", line 437.

$finish at simulation time 0

V C S S i m u l a t i o n R e p o r t

Time: 0

CPU Time: 0.140 seconds; Data structure size: 0.5Mb

Fri Jun 19 18:38:20 2026

[b23n_sarif_727@prac3 uvm_lab]$

2.0 Instance Override

```
//
`include "uvm_macros.svh"
import uvm_pkg::*;
class base_driver extends uvm_component;
	`uvm_component_utils(base_driver)

	function new(string name="base_driver",uvm_component parent=null);
		super.new(name,parent);
	endfunction

	function void build_phase(uvm_phase phase);
		`uvm_info("BASE_DRV","Base Driver Created", UVM_LOW)
	endfunction
endclass

class new_driver extends base_driver;
	`uvm_component_utils(new_driver)

	function new(string name="new_driver",uvm_component parent=null);
		super.new(name,parent);
	endfunction

	function void build_phase(uvm_phase phase);
		`uvm_info("NEW_DRV","New Driver Created",UVM_LOW)
	endfunction
endclass
```

```
class my_test extends uvm_test;
        `uvm_component_utils(my_test)

        base_driver drv1;
      base_driver drv2;

        function new(string name="my_test",uvm_component parent=null);
                super.new(name,parent);
      endfunction

        function void build_phase(uvm_phase phase);
                super.build_phase(phase);


        base_driver::type_id::set_inst_override(new_driver::get_type(),"uvm_test_top.drv1
");
                drv1 = base_driver::type_id::create("drv1", this);
                drv2 = base_driver::type_id::create("drv2", this);
        endfunction
endclass

module top;
initial begin
        run_test("my_test");
end
endmodule
```

Simulation Outputs:

```
UVM_INFO @ 0: reporter [RNTST] Running test my_test...
```

UVM_INFO instance_override.sv(24) @ 0: uvm_test_top.drv1 [NEW_DRV] New Driver Created

UVM_INFO instance_override.sv(12) @ 0: uvm_test_top.drv2 [BASE_DRV] Base Driver Created

--- UVM Report Summary ---

** Report counts by severity

UVM_INFO : 3

UVM_WARNING : 0

UVM_ERROR : 0

UVM_FATAL : 0

** Report counts by id

[BASE_DRV] 1

[NEW_DRV] 1

[RNTST] 1

$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_root.svh", line 437.

$finish at simulation time 0

V C S S i m u l a t i o n R e p o r t

Time: 0

CPU Time: 0.140 seconds; Data structure size: 0.5Mb

Fri Jun 19 19:48:57 2026

lock() Method in UVM Sequencer

//

`include "uvm_macros.svh"

import uvm_pkg::*;

```
class txn extends uvm_sequence_item;
	`uvm_object_utils(txn)

	function new(string name="txn");
		super.new(name);
	endfunction
endclass

class my_sequencer extends uvm_sequencer #(txn);
	`uvm_component_utils(my_sequencer)

	function new(string name, uvm_component parent);
		super.new(name,parent);
	endfunction
endclass

class seq1 extends uvm_sequence #(txn);
	`uvm_object_utils(seq1)

	function new(string name="seq1");
		super.new(name);
	endfunction

	task body();
		m_sequencer.lock(this);
		`uvm_info("SEQ1","Sequencer Locked by SEQ1",UVM_LOW)
```

```
            #20;
            m_sequencer.unlock(this);
            `uvm_info("SEQ1","Sequencer Unlocked by SEQ1",UVM_LOW)
        endtask
endclass

class seq2 extends uvm_sequence #(txn);
        `uvm_object_utils(seq2)
        function new(string name="seq2");
            super.new(name);
        endfunction

        task body();
            `uvm_info("SEQ2","SEQ2 Started",UVM_LOW)
        endtask
endclass

class my_test extends uvm_test;
        `uvm_component_utils(my_test)

        my_sequencer seqr;

        function new(string name,uvm_component parent);
            super.new(name,parent);
       endfunction

        function void build_phase(uvm_phase phase);
```

```
        super.build_phase(phase);
        seqr = my_sequencer::type_id::create("seqr", this);
    endfunction

    task run_phase(uvm_phase phase);
        seq1 s1;
        seq2 s2;

        phase.raise_objection(this);
        s1 = seq1::type_id::create("s1");
        s2 = seq2::type_id::create("s2");

        fork
            s1.start(seqr);
            s2.start(seqr);
        join
        phase.drop_objection(this);
    endtask
endclass

module top;
initial begin
    run_test("my_test");
end
endmodule
```

Simulation Outputs:

UVM_INFO @ 0: reporter [RNTST] Running test my_test...

UVM_INFO lock_method.sv(30) @ 0: uvm_test_top.seqr@@s1 [SEQ1] Sequencer Locked by SEQ1

UVM_INFO lock_method.sv(44) @ 0: uvm_test_top.seqr@@s2 [SEQ2] SEQ2 Started

UVM_INFO lock_method.sv(33) @ 20: uvm_test_top.seqr@@s1 [SEQ1] Sequencer Unlocked by SEQ1

UVM_INFO /usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_objection.svh(1274) @ 20: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase

--- UVM Report Summary ---

** Report counts by severity

UVM_INFO : 5

UVM_WARNING : 0

UVM_ERROR : 0

UVM_FATAL : 0

** Report counts by id

[RNTST] 1

[SEQ1] 2

[SEQ2] 1

[TEST_DONE] 1

$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_root.svh", line 437.

$finish at simulation time 20

V C S S i m u l a t i o n R e p o r t

Time: 20

CPU Time: 0.140 seconds;

grab() Method in UVM Sequencer

```
//
`include "uvm_macros.svh"
import uvm_pkg::*;

class txn extends uvm_sequence_item;
        rand bit [7:0] data;
        `uvm_object_utils(txn)

        function new(string name="txn");
                super.new(name);
        endfunction
endclass

class seq1 extends uvm_sequence #(txn);
        `uvm_object_utils(seq1)

        task body();
                txn req;
                m_sequencer.grab(this);
                `uvm_info("SEQ1", "SEQ1 Grabbed Sequencer",UVM_LOW)

                repeat(3) begin
                        `uvm_do(req)
                        `uvm_info("SEQ1",$sformatf("Data = %0d", req.data),UVM_LOW)
                end
```

```
            m_sequencer.ungrab(this);
            `uvm_info("SEQ1","SEQ1 Released Sequencer",UVM_LOW)
        endtask
endclass

class seq2 extends uvm_sequence #(txn);
        `uvm_object_utils(seq2)

        task body();
            txn req;

            repeat(5) begin
                `uvm_do(req)
                `uvm_info("SEQ2",$sformatf("Data = %0d", req.data),UVM_LOW)
            end
        endtask
endclass

class my_driver extends uvm_driver #(txn);
        `uvm_component_utils(my_driver)

        function new(string name="my_driver", uvm_component parent=null);
            super.new(name,parent);
        endfunction

        task run_phase(uvm_phase phase);
            txn req;
```

```
        forever begin
            seq_item_port.get_next_item(req);
            `uvm_info("DRIVER", $sformatf("Received Data = %0d",
req.data),UVM_LOW)
            seq_item_port.item_done();
        end
    endtask
endclass

class my_test extends uvm_test;
    `uvm_component_utils(my_test)
    uvm_sequencer #(txn) seqr;
   my_driver drv;

    function new(string name="my_test",uvm_component parent=null);
        super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
        seqr = uvm_sequencer #(txn)::type_id::create("seqr", this);
        drv  = my_driver::type_id::create("drv", this);
    endfunction

    function void connect_phase(uvm_phase phase);
        drv.seq_item_port.connect(seqr.seq_item_export);
    endfunction

    task run_phase(uvm_phase phase);
```

```
        seq1 s1;
        seq2 s2;

        phase.raise_objection(this);
        s1 = seq1::type_id::create("s1");
        s2 = seq2::type_id::create("s2");

        fork
            s1.start(seqr);
            s2.start(seqr);
        join
        phase.drop_objection(this);
    endtask
endclass

module top;
initial begin
    run_test("my_test");
end
endmodule
```

Simulation Outputs:

UVM_INFO @ 0: reporter [RNTST] Running test my_test...

UVM_INFO grab_method.sv(20) @ 0: uvm_test_top.seqr@@s1 [SEQ1] SEQ1 Grabbed Sequencer

UVM_INFO grab_method.sv(56) @ 0: uvm_test_top.drv [DRIVER] Received Data = 173

UVM_INFO grab_method.sv(24) @ 0: uvm_test_top.seqr@@s1 [SEQ1] Data = 173

UVM_INFO grab_method.sv(56) @ 0: uvm_test_top.drv [DRIVER] Received Data = 242

UVM_INFO grab_method.sv(24) @ 0: uvm_test_top.seqr@@s1 [SEQ1] Data = 242

UVM_INFO grab_method.sv(56) @ 0: uvm_test_top.drv [DRIVER] Received Data = 33

UVM_INFO grab_method.sv(24) @ 0: uvm_test_top.seqr@@s1 [SEQ1] Data = 33

UVM_INFO grab_method.sv(28) @ 0: uvm_test_top.seqr@@s1 [SEQ1] SEQ1 Released Sequencer

UVM_INFO grab_method.sv(56) @ 0: uvm_test_top.drv [DRIVER] Received Data = 28

UVM_INFO grab_method.sv(40) @ 0: uvm_test_top.seqr@@s2 [SEQ2] Data = 28

UVM_INFO grab_method.sv(56) @ 0: uvm_test_top.drv [DRIVER] Received Data = 87

UVM_INFO grab_method.sv(40) @ 0: uvm_test_top.seqr@@s2 [SEQ2] Data = 87

UVM_INFO grab_method.sv(56) @ 0: uvm_test_top.drv [DRIVER] Received Data = 128

UVM_INFO grab_method.sv(40) @ 0: uvm_test_top.seqr@@s2 [SEQ2] Data = 128

UVM_INFO grab_method.sv(56) @ 0: uvm_test_top.drv [DRIVER] Received Data = 89

UVM_INFO grab_method.sv(40) @ 0: uvm_test_top.seqr@@s2 [SEQ2] Data = 89

UVM_INFO grab_method.sv(56) @ 0: uvm_test_top.drv [DRIVER] Received Data = 158

UVM_INFO grab_method.sv(40) @ 0: uvm_test_top.seqr@@s2 [SEQ2] Data = 158

UVM_INFO /usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_objection.svh(1274) @ 0: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase

--- UVM Report Summary ---

** Report counts by severity

UVM_INFO :   20

UVM_WARNING :   0

UVM_ERROR :   0

UVM_FATAL :   0

** Report counts by id

[DRIVER] 8

[RNTST] 1

[SEQ1] 5

[SEQ2] 5

[TEST_DONE] 1

$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_root.svh", line 437.

$finish at simulation time 0

V C S S i m u l a t i o n R e p o r t

Time: 0

CPU Time: 0.130 seconds; Data structure size: 0.5Mb

Sat Jun 20 10:11:19 2026

m_sequencer in UVM

```
//
`include "uvm_macros.svh"
import uvm_pkg::*;


class txn extends uvm_sequence_item;
    `uvm_object_utils(txn)


    function new(string name="txn");
        super.new(name);
    endfunction
endclass
```

```
class my_sequencer extends uvm_sequencer #(txn);
	`uvm_component_utils(my_sequencer)

	function new(string name,uvm_component parent);
		super.new(name,parent);
	endfunction
endclass

class my_seq extends uvm_sequence #(txn);
	`uvm_object_utils(my_seq)

	function new(string name="my_seq");
		super.new(name);
	endfunction

	task body();
		if(m_sequencer != null)
			`uvm_info("SEQ","m_sequencer is available", UVM_LOW)
		else
			`uvm_error("SEQ","m_sequencer is NULL")
	endtask
endclass

class my_test extends uvm_test;
	`uvm_component_utils(my_test)
	my_sequencer seqr;
```

```
    function new(string name,uvm_component parent);
        super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
        seqr = my_sequencer::type_id::create("seqr", this);
    endfunction

    task run_phase(uvm_phase phase);
        my_seq seq;

        phase.raise_objection(this);
        seq = my_seq::type_id::create("seq");
        seq.start(seqr);
        phase.drop_objection(this);
    endtask
endclass

`include "uvm_macros.svh"
import uvm_pkg::*;

module top;
initial
    run_test("my_test");
endmodule
```

Simulation Outputs:

```
UVM_INFO @ 0: reporter [RNTST] Running test my_test...
```

UVM_INFO m_sequencer.sv(30) @ 0: uvm_test_top.seqr@@seq [SEQ] m_sequencer is available

UVM_INFO /usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_objection.svh(1274) @ 0: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase

--- UVM Report Summary ---

** Report counts by severity

UVM_INFO : 3

UVM_WARNING : 0

UVM_ERROR : 0

UVM_FATAL : 0

** Report counts by id

[RNTST] 1

[SEQ] 1

[TEST_DONE] 1

$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_root.svh", line 437.

$finish at simulation time 0

V C S S i m u l a t i o n R e p o r t

Time: 0

CPU Time: 0.130 seconds; Data structure size: 0.5Mb

Sat Jun 20 10:42:15 2026

p_sequencer in UVM

//

`include "uvm_macros.svh"

import uvm_pkg::*;

```
class txn extends uvm_sequence_item;
	`uvm_object_utils(txn)

	function new(string name="txn");
		super.new(name);
	endfunction
endclass

class my_sequencer extends uvm_sequencer #(txn);
	`uvm_component_utils(my_sequencer)

	int mode = 100;

	function new(string name,uvm_component parent);
		super.new(name,parent);
	endfunction
endclass

class my_seq extends uvm_sequence #(txn);
	`uvm_object_utils(my_seq)
	`uvm_declare_p_sequencer(my_sequencer)

	function new(string name="my_seq");
		super.new(name);
	endfunction
```

```
	task body();
		`uvm_info("SEQ",$sformatf("mode =
%0d",p_sequencer.mode),UVM_LOW)
	endtask
endclass

class my_test extends uvm_test;
	`uvm_component_utils(my_test)
	my_sequencer seqr;

	function new(string name,uvm_component parent);
		super.new(name,parent);
	endfunction

	function void build_phase(uvm_phase phase);
		seqr = my_sequencer::type_id::create("seqr", this);
	endfunction

	task run_phase(uvm_phase phase);
		my_seq seq;

		phase.raise_objection(this);
		seq = my_seq::type_id::create("seq");
		seq.start(seqr);
		phase.drop_objection(this);
	endtask
endclass
```

```
module top;
initial
        run_test("my_test");
endmodule
```

Simulation Outputs:

```
UVM_INFO @ 0: reporter [RNTST] Running test my_test...
UVM_INFO p_sequencer.sv(32) @ 0: uvm_test_top.seqr@@seq [SEQ] mode = 100
UVM_INFO /usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_objection.svh(1274) @ 0: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase


--- UVM Report Summary ---


** Report counts by severity
UVM_INFO :    3
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
** Report counts by id
[RNTST]     1
[SEQ]     1
[TEST_DONE]     1
$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_root.svh", line 437.
$finish at simulation time                   0
          V C S   S i m u l a t i o n   R e p o r t
Time: 0
CPU Time:      0.140 seconds;       Data structure size:   0.5Mb
Sat Jun 20 11:10:25 2026
```

TLM FIFO in UVM

```
//
`include "uvm_macros.svh"
import uvm_pkg::*;

class txn extends uvm_sequence_item;
        rand bit [7:0] data;
        `uvm_object_utils(txn)

        function new(string name="txn");
                super.new(name);
        endfunction
endclass

class my_monitor extends uvm_component;
        `uvm_component_utils(my_monitor)

        uvm_tlm_fifo #(txn) fifo;

        function new(string name, uvm_component parent);
                super.new(name, parent);
        endfunction

        function void build_phase(uvm_phase phase);
                fifo = new("fifo", this);
        endfunction
```

```
	task run_phase(uvm_phase phase);
		txn t;
		repeat(5) begin
			t = txn::type_id::create("t");
			assert(t.randomize());
			`uvm_info("MON", $sformatf("Sending data = %0d", t.data), 
UVM_LOW)
			fifo.put(t);
			#10;
		end
	endtask
endclass

class my_scoreboard extends uvm_component;
	`uvm_component_utils(my_scoreboard)

	uvm_tlm_fifo #(txn) fifo;

	function new(string name, uvm_component parent);
		super.new(name, parent);
	endfunction

	task run_phase(uvm_phase phase);
		txn t;
		forever begin
			fifo.get(t);
			`uvm_info("SB", $sformatf("Received data = %0d", t.data), 
UVM_LOW)
```

```
            end
        endtask
endclass

class my_env extends uvm_env;
        `uvm_component_utils(my_env)
        my_monitor mon;
        my_scoreboard sb;

        function new(string name, uvm_component parent);
            super.new(name, parent);
        endfunction

        function void build_phase(uvm_phase phase);
            mon = my_monitor::type_id::create("mon", this);
            sb  = my_scoreboard::type_id::create("sb", this);
        endfunction
        function void connect_phase(uvm_phase phase);
            sb.fifo = mon.fifo;
        endfunction
endclass

class my_test extends uvm_test;
        `uvm_component_utils(my_test)
        my_env env;

        function new(string name, uvm_component parent);
```

```
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        env = my_env::type_id::create("env", this);
    endfunction

    task run_phase(uvm_phase phase);
        phase.raise_objection(this);
        #100;
        phase.drop_objection(this);
    endtask
endclass

module top;
initial begin
    run_test("my_test");
end
endmodule
```

Simulation Outputs:

```
UVM_INFO @ 0: reporter [RNTST] Running test my_test...
UVM_INFO tlm_fifo.sv(32) @ 0: uvm_test_top.env.mon [MON] Sending data = 246
UVM_INFO tlm_fifo.sv(52) @ 0: uvm_test_top.env.sb [SB] Received data = 246
UVM_INFO tlm_fifo.sv(32) @ 10: uvm_test_top.env.mon [MON] Sending data = 114
UVM_INFO tlm_fifo.sv(52) @ 10: uvm_test_top.env.sb [SB] Received data = 114
UVM_INFO tlm_fifo.sv(32) @ 20: uvm_test_top.env.mon [MON] Sending data = 66
UVM_INFO tlm_fifo.sv(52) @ 20: uvm_test_top.env.sb [SB] Received data = 66
```

UVM_INFO tlm_fifo.sv(32) @ 30: uvm_test_top.env.mon [MON] Sending data = 46

UVM_INFO tlm_fifo.sv(52) @ 30: uvm_test_top.env.sb [SB] Received data = 46

UVM_INFO tlm_fifo.sv(32) @ 40: uvm_test_top.env.mon [MON] Sending data = 212

UVM_INFO tlm_fifo.sv(52) @ 40: uvm_test_top.env.sb [SB] Received data = 212

UVM_INFO /usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_objection.svh(1274) @ 100: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase

--- UVM Report Summary ---

** Report counts by severity

UVM_INFO : 12

UVM_WARNING : 0

UVM_ERROR : 0

UVM_FATAL : 0

** Report counts by id

[MON] 5

[RNTST] 1

[SB] 5

[TEST_DONE] 1

$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_root.svh", line 437.

$finish at simulation time 100

V C S S i m u l a t i o n R e p o r t

Time: 100

CPU Time: 0.140 seconds; Data structure size: 0.5Mb

Sat Jun 20 11:55:17 2026

UVM Callback in UVM

```
//
`include "uvm_macros.svh"
import uvm_pkg::*;

class txn extends uvm_sequence_item;
        rand bit [7:0] data;
        `uvm_object_utils(txn)

        function new(string name="txn");
                super.new(name);
        endfunction
endclass

class my_callback extends uvm_callback;
        `uvm_object_utils(my_callback)

        function new(string name="my_callback");
                super.new(name);
        endfunction

        virtual function void modify_data(ref txn t);
        t.data = t.data + 10;
        `uvm_info("CALLBACK",$sformatf("Modified Data = %0d",
t.data),UVM_LOW)
endfunction

endclass
```

```
class my_driver extends uvm_component;
	`uvm_component_utils(my_driver)
	`uvm_register_cb(my_driver,my_callback)
	txn req;

	function new(string name,uvm_component parent);
		super.new(name,parent);
	endfunction

	task run_phase(uvm_phase phase);
		req = txn::type_id::create("req");
		req.data = 20;
		`uvm_info("DRIVER",$sformatf("Original Data = %0d",
req.data),UVM_LOW)
		`uvm_do_callbacks(my_driver,my_callback,modify_data(req))
		`uvm_info("DRIVER",$sformatf("Driving Data = %0d",
req.data),UVM_LOW)
	endtask
endclass

class my_test extends uvm_test;
	`uvm_component_utils(my_test)
	my_driver drv;
	my_callback cb;

	function new(string name,uvm_component parent);
		super.new(name,parent);
	endfunction
```

```
    function void build_phase(uvm_phase phase);
        drv = my_driver::type_id::create("drv",this);
        cb = my_callback::type_id::create("cb");
        uvm_callbacks#(my_driver,my_callback)::add(drv,cb);
    endfunction
endclass

module top;
initial
    run_test("my_test");
endmodule
```

Simulation Outputs:

```
UVM_INFO @ 0: reporter [RNTST] Running test my_test...
UVM_INFO callback.sv(40) @ 0: uvm_test_top.drv [DRIVER] Original Data = 20
UVM_INFO callback.sv(23) @ 0: reporter [CALLBACK] Modified Data = 30
UVM_INFO callback.sv(42) @ 0: uvm_test_top.drv [DRIVER] Driving Data = 30

--- UVM Report Summary ---

** Report counts by severity
UVM_INFO :    4
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
** Report counts by id
[CALLBACK]     1
```

[DRIVER] 2

[RNTST] 1

$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_root.svh", line 437.

$finish at simulation time 0

V C S S i m u l a t i o n R e p o r t

Time: 0

CPU Time: 0.130 seconds; Data structure size: 0.5Mb

Sat Jun 20 14:28:25 2026

Arbitration in UVM

```
//
`include "uvm_macros.svh"
import uvm_pkg::*;

class txn extends uvm_sequence_item;
    rand int data;
    `uvm_object_utils(txn)

    function new(string name="txn");
        super.new(name);
    endfunction
endclass

class seq1 extends uvm_sequence #(txn);
    `uvm_object_utils(seq1)
```

```
	function new(string name="seq1");
		super.new(name);
	endfunction

	task body();
		txn t;
		repeat(3) begin
			t = txn::type_id::create("t");
			start_item(t);
			t.data = 100;
			finish_item(t);
			`uvm_info("SEQ1","Sent 100",UVM_LOW)
		end
	endtask
endclass

class seq2 extends uvm_sequence #(txn);
	`uvm_object_utils(seq2)

	function new(string name="seq2");
		super.new(name);
	endfunction

	task body();
		txn t;
		repeat(3) begin
			t = txn::type_id::create("t");
```

```
                start_item(t);
                t.data = 200;
                finish_item(t);
                `uvm_info("SEQ2","Sent 200",UVM_LOW)
        end
    endtask
endclass

class drv extends uvm_driver #(txn);
    `uvm_component_utils(drv)

    function new(string name, uvm_component parent);
        super.new(name,parent);
    endfunction

    task run_phase(uvm_phase phase);
        txn t;
        forever begin
            seq_item_port.get_next_item(t);
            `uvm_info("DRV",$sformatf("Received = %0d",t.data),UVM_LOW)
            seq_item_port.item_done();
        end
    endtask
endclass

class env extends uvm_env;
    `uvm_component_utils(env)
```

```
    uvm_sequencer #(txn) seqr;
    drv d;

    function new(string name, uvm_component parent);
        super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
        seqr = uvm_sequencer#(txn)::type_id::create("seqr",this);
        d    = drv::type_id::create("d",this);
    endfunction

    function void connect_phase(uvm_phase phase);
        d.seq_item_port.connect(seqr.seq_item_export);
    endfunction
endclass

class test extends uvm_test;
    `uvm_component_utils(test)
    env e;

    function new(string name, uvm_component parent);
        super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
        e = env::type_id::create("e",this);
```

```
    endfunction

    task run_phase(uvm_phase phase);
        seq1 s1;
        seq2 s2;

        phase.raise_objection(this);
        s1 = seq1::type_id::create("s1");
        s2 = seq2::type_id::create("s2");
        e.seqr.set_arbitration(SEQ_ARB_FIFO);
        fork
            s1.start(e.seqr);
            s2.start(e.seqr);
        join
        phase.drop_objection(this);
    endtask
endclass

module top;
initial
    run_test("test");
endmodule
```

Simulation Outputs:

```
UVM_INFO @ 0: reporter [RNTST] Running test test...
UVM_INFO arbitration.sv(63) @ 0: uvm_test_top.e.d [DRV] Received = 100
UVM_INFO arbitration.sv(28) @ 0: uvm_test_top.e.seqr@@s1 [SEQ1] Sent 100
UVM_INFO arbitration.sv(63) @ 0: uvm_test_top.e.d [DRV] Received = 200
```

UVM_INFO arbitration.sv(47) @ 0: uvm_test_top.e.seqr@@s2 [SEQ2] Sent 200

UVM_INFO arbitration.sv(63) @ 0: uvm_test_top.e.d [DRV] Received = 100

UVM_INFO arbitration.sv(28) @ 0: uvm_test_top.e.seqr@@s1 [SEQ1] Sent 100

UVM_INFO arbitration.sv(63) @ 0: uvm_test_top.e.d [DRV] Received = 200

UVM_INFO arbitration.sv(47) @ 0: uvm_test_top.e.seqr@@s2 [SEQ2] Sent 200

UVM_INFO arbitration.sv(63) @ 0: uvm_test_top.e.d [DRV] Received = 100

UVM_INFO arbitration.sv(28) @ 0: uvm_test_top.e.seqr@@s1 [SEQ1] Sent 100

UVM_INFO arbitration.sv(63) @ 0: uvm_test_top.e.d [DRV] Received = 200

UVM_INFO arbitration.sv(47) @ 0: uvm_test_top.e.seqr@@s2 [SEQ2] Sent 200

UVM_INFO /usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_objection.svh(1274) @ 0: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase

--- UVM Report Summary ---

** Report counts by severity

UVM_INFO : 14

UVM_WARNING : 0

UVM_ERROR : 0

UVM_FATAL : 0

** Report counts by id

[DRV] 6

[RNTST] 1

[SEQ1] 3

[SEQ2] 3

[TEST_DONE] 1

$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_root.svh", line 437.

$finish at simulation time 0

V C S S i m u l a t i o n R e p o r t

Time: 0

CPU Time: 0.130 seconds; Data structure size: 0.5Mb

Sat Jun 20 16:11:40 2026

Case-2

In the above code only change this line in test class-

e.seqr.set_arbitration(SEQ_ARB_RANDOM);

Simulation Outputs:

UVM_INFO @ 0: reporter [RNTST] Running test test...

UVM_INFO arbitration.sv(63) @ 0: uvm_test_top.e.d [DRV] Received = 200

UVM_INFO arbitration.sv(47) @ 0: uvm_test_top.e.seqr@@s2 [SEQ2] Sent 200

UVM_INFO arbitration.sv(63) @ 0: uvm_test_top.e.d [DRV] Received = 200

UVM_INFO arbitration.sv(47) @ 0: uvm_test_top.e.seqr@@s2 [SEQ2] Sent 200

UVM_INFO arbitration.sv(63) @ 0: uvm_test_top.e.d [DRV] Received = 200

UVM_INFO arbitration.sv(47) @ 0: uvm_test_top.e.seqr@@s2 [SEQ2] Sent 200

UVM_INFO arbitration.sv(63) @ 0: uvm_test_top.e.d [DRV] Received = 100

UVM_INFO arbitration.sv(28) @ 0: uvm_test_top.e.seqr@@s1 [SEQ1] Sent 100

UVM_INFO arbitration.sv(63) @ 0: uvm_test_top.e.d [DRV] Received = 100

UVM_INFO arbitration.sv(28) @ 0: uvm_test_top.e.seqr@@s1 [SEQ1] Sent 100

UVM_INFO arbitration.sv(63) @ 0: uvm_test_top.e.d [DRV] Received = 100

UVM_INFO arbitration.sv(28) @ 0: uvm_test_top.e.seqr@@s1 [SEQ1] Sent 100

UVM_INFO /usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_objection.svh(1274) @ 0: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase

--- UVM Report Summary ---

** Report counts by severity

UVM_INFO : 14

UVM_WARNING : 0

UVM_ERROR : 0

UVM_FATAL : 0

** Report counts by id

[DRV] 6

[RNTST] 1

[SEQ1] 3

[SEQ2] 3

[TEST_DONE] 1

$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_root.svh", line 437.

$finish at simulation time 0

V C S S i m u l a t i o n R e p o r t

Time: 0

CPU Time: 0.130 seconds; Data structure size: 0.5Mb

Sat Jun 20 16:19:33 2026

Case-3

In the above code only change this line in test class-

e.seqr.set_arbitration(SEQ_ARB_STRICT_FIFO);

Simulation Outputs:

UVM_INFO @ 0: reporter [RNTST] Running test test...

UVM_INFO arbitration.sv(63) @ 0: uvm_test_top.e.d [DRV] Received = 100

UVM_INFO arbitration.sv(28) @ 0: uvm_test_top.e.seqr@@s1 [SEQ1] Sent 100

UVM_INFO arbitration.sv(63) @ 0: uvm_test_top.e.d [DRV] Received = 200

UVM_INFO arbitration.sv(47) @ 0: uvm_test_top.e.seqr@@s2 [SEQ2] Sent 200

UVM_INFO arbitration.sv(63) @ 0: uvm_test_top.e.d [DRV] Received = 100

UVM_INFO arbitration.sv(28) @ 0: uvm_test_top.e.seqr@@s1 [SEQ1] Sent 100

UVM_INFO arbitration.sv(63) @ 0: uvm_test_top.e.d [DRV] Received = 200

UVM_INFO arbitration.sv(47) @ 0: uvm_test_top.e.seqr@@s2 [SEQ2] Sent 200

UVM_INFO arbitration.sv(63) @ 0: uvm_test_top.e.d [DRV] Received = 100

UVM_INFO arbitration.sv(28) @ 0: uvm_test_top.e.seqr@@s1 [SEQ1] Sent 100

UVM_INFO arbitration.sv(63) @ 0: uvm_test_top.e.d [DRV] Received = 200

UVM_INFO arbitration.sv(47) @ 0: uvm_test_top.e.seqr@@s2 [SEQ2] Sent 200

UVM_INFO arbitration.sv(63) @ 0: uvm_test_top.e.d [DRV] Received = 200

UVM_INFO arbitration.sv(47) @ 0: uvm_test_top.e.seqr@@s2 [SEQ2] Sent 200

UVM_INFO /usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_objection.svh(1274) @ 0: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase

--- UVM Report Summary ---

** Report counts by severity

UVM_INFO :   16

UVM_WARNING :   0

UVM_ERROR :   0

UVM_FATAL :   0

** Report counts by id

[DRV]   7

[RNTST]   1

[SEQ1]   3

[SEQ2]   4

[TEST_DONE]   1

$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_root.svh", line 437.

$finish at simulation time 0

V C S S i m u l a t i o n R e p o r t

Time: 0

Virtual Sequence & Virtual sequencer-

```
//

`include "uvm_macros.svh"

import uvm_pkg::*;


class txn extends uvm_sequence_item;

        rand bit[4:0] data;

        `uvm_object_utils(txn)


        function new(string name="txn");

                super.new(name);

        endfunction

endclass


class seq1 extends uvm_sequence #(txn);

        `uvm_object_utils(seq1)
```

```
	function new(string name="seq1");
		super.new(name);
	endfunction

	task body();
		txn t;
		repeat(3) begin
			`uvm_do(t)
			`uvm_info("SEQ1",$sformatf("Generated data=%0d",
t.data),UVM_LOW)
		end
	endtask
endclass

class seq2 extends uvm_sequence #(txn);
	`uvm_object_utils(seq2)

	function new(string name="seq2");
		super.new(name);
	endfunction

	task body();
		txn t;
		repeat(3) begin
```

```
                `uvm_do(t)
                `uvm_info("SEQ2",$sformatf("Generated data=%0d",
t.data),UVM_LOW)
            end
        endtask
endclass

class drv extends uvm_driver #(txn);
        `uvm_component_utils(drv)

        function new(string name, uvm_component parent);
            super.new(name,parent);
        endfunction

        task run_phase(uvm_phase phase);
            txn t;
            forever begin
                seq_item_port.get_next_item(t);
                `uvm_info("DRV",$sformatf("Received %0d",t.data),UVM_LOW)
                #10;
                seq_item_port.item_done();
            end
        endtask
endclass
```

```
class virtual_sequencer extends uvm_sequencer;
        `uvm_component_utils(virtual_sequencer)

        uvm_sequencer #(txn) seqr1;
        uvm_sequencer #(txn) seqr2;

        function new(string name, uvm_component parent);
                super.new(name,parent);
        endfunction
endclass

class vseq extends uvm_sequence;
        `uvm_object_utils(vseq)
        `uvm_declare_p_sequencer(virtual_sequencer)

        function new(string name="vseq");
                super.new(name);
        endfunction

        task body();
                seq1 s1;
                seq2 s2;
```

```
        s1 = seq1::type_id::create("s1");
        s2 = seq2::type_id::create("s2");

        fork
            s1.start(p_sequencer.seqr1);
            s2.start(p_sequencer.seqr2);
        join
    endtask
endclass

class env extends uvm_env;
    `uvm_component_utils(env)

    uvm_sequencer #(txn) seqr1;
    uvm_sequencer #(txn) seqr2;
    virtual_sequencer vseqr;
    drv d1;
    drv d2;

    function new(string name, uvm_component parent);
        super.new(name,parent);
    endfunction
```

```
    function void build_phase(uvm_phase phase);
        seqr1 = uvm_sequencer#(txn)::type_id::create("seqr1",this);
        seqr2 = uvm_sequencer#(txn)::type_id::create("seqr2",this);
        vseqr = virtual_sequencer::type_id::create("vseqr",this);
        d1 = drv::type_id::create("d1",this);
        d2 = drv::type_id::create("d2",this);
    endfunction

    function void connect_phase(uvm_phase phase);
        d1.seq_item_port.connect(seqr1.seq_item_export);
        d2.seq_item_port.connect(seqr2.seq_item_export);
        vseqr.seqr1 = seqr1;
        vseqr.seqr2 = seqr2;
    endfunction
endclass

class test extends uvm_test;
    `uvm_component_utils(test)
    env e;

    function new(string name, uvm_component parent);
        super.new(name,parent);
```

```
    endfunction

    function void build_phase(uvm_phase phase);
        e = env::type_id::create("e",this);
    endfunction

    task run_phase(uvm_phase phase);
        vseq vs;

        phase.raise_objection(this);
        vs = vseq::type_id::create("vs");
        vs.start(e.vseqr);
        phase.drop_objection(this);
    endtask
endclass

module top;
initial
    run_test("test");
endmodule
```

Simulation Outputs:

```
UVM_INFO @ 0: reporter [RNTST] Running test test...
UVM_INFO virtual_seq_seqr.sv(57) @ 0: uvm_test_top.e.d2 [DRV] Received 4
```

UVM_INFO virtual_seq_seqr.sv(57) @ 0: uvm_test_top.e.d1 [DRV] Received 23

UVM_INFO virtual_seq_seqr.sv(41) @ 10: uvm_test_top.e.seqr2@@s2 [SEQ2] Generated data=4

UVM_INFO virtual_seq_seqr.sv(25) @ 10: uvm_test_top.e.seqr1@@s1 [SEQ1] Generated data=23

UVM_INFO virtual_seq_seqr.sv(57) @ 10: uvm_test_top.e.d1 [DRV] Received 29

UVM_INFO virtual_seq_seqr.sv(57) @ 10: uvm_test_top.e.d2 [DRV] Received 27

UVM_INFO virtual_seq_seqr.sv(25) @ 20: uvm_test_top.e.seqr1@@s1 [SEQ1] Generated data=29

UVM_INFO virtual_seq_seqr.sv(41) @ 20: uvm_test_top.e.seqr2@@s2 [SEQ2] Generated data=27

UVM_INFO virtual_seq_seqr.sv(57) @ 20: uvm_test_top.e.d2 [DRV] Received 21

UVM_INFO virtual_seq_seqr.sv(57) @ 20: uvm_test_top.e.d1 [DRV] Received 12

UVM_INFO virtual_seq_seqr.sv(41) @ 30: uvm_test_top.e.seqr2@@s2 [SEQ2] Generated data=21

UVM_INFO virtual_seq_seqr.sv(25) @ 30: uvm_test_top.e.seqr1@@s1 [SEQ1] Generated data=12

UVM_INFO /usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_objection.svh(1274) @ 30: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase

--- UVM Report Summary ---

** Report counts by severity

UVM_INFO :   14

UVM_WARNING :   0

UVM_ERROR :   0

UVM_FATAL :   0

** Report counts by id

[DRV]   6

[RNTST]   1

[SEQ1] 3

[SEQ2] 3

[TEST_DONE] 1

$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_root.svh", line 437.

$finish at simulation time 30

V C S S i m u l a t i o n R e p o r t

Time: 30

CPU Time: 0.150 seconds; Data structure size: 0.6Mb

Sat Jun 20 22:57:26 2026

UVM testbench for a Half Adder

[b23n_sarif_727@prac3 half_adder]$ ls

agent.sv driver.sv ha_if.sv ha_pkg.sv scoreboard.sv sequencer.sv simv test.sv ucli.key

csrc env.sv half_adder.sv monitor.sv sequence_item.sv sequence.sv simv.daidir top.sv vc_hdrs.h

[b23n_sarif_727@prac3 half_adder]$

[b23n_sarif_727@prac3 half_adder]$ vcs -sverilog half_adder.sv ha_if.sv ha_pkg.sv top.sv -ntb_opts uvm -kdb -lca -debug_access+all -full64

[b23n_sarif_727@prac3 half_adder]$ ./simv

//

module half_adder (

  input logic a,

 input logic b,

 output logic sum,

```
        output logic carry
);

assign sum   = a ^ b;
assign carry = a & b;

endmodule

//
interface ha_if;
          logic a;
        logic b;
        logic sum;
        logic carry;

endinterface

//
package ha_pkg;

import uvm_pkg::*;
`include "uvm_macros.svh"

`include "sequence_item.sv"
`include "sequence.sv"
`include "sequencer.sv"
`include "driver.sv"
```

```
`include "monitor.sv"
`include "scoreboard.sv"
`include "agent.sv"
`include "env.sv"
`include "test.sv"

endpackage

//
class ha_seq_item extends uvm_sequence_item;
        rand bit a;
        rand bit b;
        bit sum;
       bit carry;

        `uvm_object_utils(ha_seq_item)

        function new(string name="ha_seq_item");
                super.new(name);
        endfunction
endclass

//
class ha_sequence extends uvm_sequence #(ha_seq_item);
        `uvm_object_utils(ha_sequence)

        function new(string name="ha_sequence");
```

```
        super.new(name);
    endfunction

    task body();
        repeat(10) begin
            req = ha_seq_item::type_id::create("req");
            start_item(req);
            assert(req.randomize());
            finish_item(req);
        end
    endtask

endclass

//
class ha_sequencer extends uvm_sequencer #(ha_seq_item);
    `uvm_component_utils(ha_sequencer)

    function new(string name="ha_sequencer",uvm_component parent);
        super.new(name,parent);
    endfunction

endclass

//
class ha_driver extends uvm_driver #(ha_seq_item);
    `uvm_component_utils(ha_driver)
```

```
	virtual ha_if vif;

	function new(string name="ha_driver",uvm_component parent);
		super.new(name,parent);
	endfunction

	function void build_phase(uvm_phase phase);
		super.build_phase(phase);

		if(!uvm_config_db#(virtual ha_if)::get(this,"","vif",vif))
			`uvm_fatal("DRV","Interface not found")
	endfunction

	task run_phase(uvm_phase phase);
		forever begin
			seq_item_port.get_next_item(req);
			vif.a = req.a;
			vif.b = req.b;
			#10;
			seq_item_port.item_done();
			`uvm_info("DRV",$sformatf("a=%0b
b=%0b",req.a,req.b),UVM_NONE)
		end
	endtask

endclass

//
```

```
class ha_monitor extends uvm_monitor;
	`uvm_component_utils(ha_monitor)
	virtual ha_if vif;
	uvm_analysis_port #(ha_seq_item) send;
	ha_seq_item tx;

	function new(string name="ha_monitor", uvm_component parent);
		super.new(name,parent);
		send = new("send",this);
	endfunction

	function void build_phase(uvm_phase phase);
		super.build_phase(phase);
		if(!uvm_config_db#(virtual ha_if)::get(this,"","vif",vif))
			`uvm_fatal("MON","Interface not found")
	endfunction

	task run_phase(uvm_phase phase);
		forever begin
			#10;
			tx = ha_seq_item::type_id::create("tx");
			tx.a     = vif.a;
			tx.b     = vif.b;
			tx.sum   = vif.sum;
			tx.carry = vif.carry;
			send.write(tx);
			`uvm_info("MON",$sformatf("a=%0b b=%0b sum=%0b
carry=%0b",tx.a,tx.b,tx.sum,tx.carry),UVM_NONE)
```

```
        end
    endtask

endclass

//
class ha_scoreboard extends uvm_scoreboard;
    `uvm_component_utils(ha_scoreboard)
    uvm_analysis_imp #(ha_seq_item,ha_scoreboard) recv;

    function new(string name="ha_scoreboard",uvm_component parent);
        super.new(name,parent);
        recv = new("recv",this);
    endfunction

    function void write(ha_seq_item tx);
        if((tx.sum == (tx.a ^ tx.b)) && (tx.carry == (tx.a & tx.b))) begin
            `uvm_info("SCO",$sformatf("PASS a=%0b b=%0b sum=%0b carry=%0b", tx.a,tx.b,tx.sum,tx.carry), UVM_NONE)
        end
        else begin
            `uvm_error("SCO",$sformatf("FAIL a=%0b b=%0b sum=%0b carry=%0b",tx.a,tx.b,tx.sum,tx.carry))
        end
    endfunction

endclass
```

```
//
class ha_agent extends uvm_agent;
        `uvm_component_utils(ha_agent)

        ha_driver    drv;
        ha_monitor   mon;
        ha_sequencer seqr;

        function new(string name="ha_agent",uvm_component parent);
                super.new(name,parent);
        endfunction

        function void build_phase(uvm_phase phase);
                super.build_phase(phase);
                drv  = ha_driver::type_id::create("drv",this);
                mon  = ha_monitor::type_id::create("mon",this);
                seqr = ha_sequencer::type_id::create("seqr",this);
        endfunction

        function void connect_phase(uvm_phase phase);
                drv.seq_item_port.connect(seqr.seq_item_export);
        endfunction

endclass

//
class ha_env extends uvm_env;
```

```
	`uvm_component_utils(ha_env)

	ha_agent agent;
	ha_scoreboard sco;

	function new(string name="ha_env",uvm_component parent);
		super.new(name,parent);
	endfunction

	function void build_phase(uvm_phase phase);
		super.build_phase(phase);
		agent = ha_agent::type_id::create("agent",this);
		sco   = ha_scoreboard::type_id::create("sco",this);
	endfunction

	function void connect_phase(uvm_phase phase);
		agent.mon.send.connect(sco.recv);
	endfunction

endclass

//
class ha_test extends uvm_test;
	`uvm_component_utils(ha_test)

	ha_env env;
	ha_sequence seq;
```

```
    function new(string name="ha_test",uvm_component parent);
        super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        env = ha_env::type_id::create("env",this);
    endfunction

    task run_phase(uvm_phase phase);
        phase.raise_objection(this);
        seq = ha_sequence::type_id::create("seq");
        seq.start(env.agent.seqr);
        #20;
        phase.drop_objection(this);
    endtask

endclass

//
module top;

import uvm_pkg::*;
import ha_pkg::*;

ha_if vif();
```

```
half_adder dut(.a(vif.a),.b(vif.b),.sum(vif.sum),.carry(vif.carry));


initial begin
        uvm_config_db#(virtual ha_if)::set(null,"*","vif",vif);
        run_test("ha_test");
end


endmodule
```

Simulation Outputs:

```
UVM_INFO @ 0: reporter [RNTST] Running test ha_test...
UVM_INFO scoreboard.sv(13) @ 10: uvm_test_top.env.sco [SCO] PASS a=0 b=1 sum=1 carry=0
UVM_INFO monitor.sv(28) @ 10: uvm_test_top.env.agent.mon [MON] a=0 b=1 sum=1 carry=0
UVM_INFO driver.sv(24) @ 10: uvm_test_top.env.agent.drv [DRV] a=0 b=1
UVM_INFO scoreboard.sv(13) @ 20: uvm_test_top.env.sco [SCO] PASS a=1 b=1 sum=0 carry=1
UVM_INFO monitor.sv(28) @ 20: uvm_test_top.env.agent.mon [MON] a=1 b=1 sum=0 carry=1
UVM_INFO driver.sv(24) @ 20: uvm_test_top.env.agent.drv [DRV] a=1 b=1
UVM_INFO scoreboard.sv(13) @ 30: uvm_test_top.env.sco [SCO] PASS a=1 b=1 sum=0 carry=1
UVM_INFO monitor.sv(28) @ 30: uvm_test_top.env.agent.mon [MON] a=1 b=1 sum=0 carry=1
UVM_INFO driver.sv(24) @ 30: uvm_test_top.env.agent.drv [DRV] a=1 b=1
UVM_INFO scoreboard.sv(13) @ 40: uvm_test_top.env.sco [SCO] PASS a=0 b=1 sum=1 carry=0
```

UVM_INFO monitor.sv(28) @ 40: uvm_test_top.env.agent.mon [MON] a=0 b=1 sum=1 carry=0

UVM_INFO driver.sv(24) @ 40: uvm_test_top.env.agent.drv [DRV] a=0 b=1

UVM_INFO scoreboard.sv(13) @ 50: uvm_test_top.env.sco [SCO] PASS a=0 b=1 sum=1 carry=0

UVM_INFO monitor.sv(28) @ 50: uvm_test_top.env.agent.mon [MON] a=0 b=1 sum=1 carry=0

UVM_INFO driver.sv(24) @ 50: uvm_test_top.env.agent.drv [DRV] a=0 b=1

UVM_INFO scoreboard.sv(13) @ 60: uvm_test_top.env.sco [SCO] PASS a=1 b=1 sum=0 carry=1

UVM_INFO monitor.sv(28) @ 60: uvm_test_top.env.agent.mon [MON] a=1 b=1 sum=0 carry=1

UVM_INFO driver.sv(24) @ 60: uvm_test_top.env.agent.drv [DRV] a=1 b=1

UVM_INFO scoreboard.sv(13) @ 70: uvm_test_top.env.sco [SCO] PASS a=0 b=0 sum=0 carry=0

UVM_INFO monitor.sv(28) @ 70: uvm_test_top.env.agent.mon [MON] a=0 b=0 sum=0 carry=0

UVM_INFO driver.sv(24) @ 70: uvm_test_top.env.agent.drv [DRV] a=0 b=0

UVM_INFO scoreboard.sv(13) @ 80: uvm_test_top.env.sco [SCO] PASS a=0 b=1 sum=1 carry=0

UVM_INFO monitor.sv(28) @ 80: uvm_test_top.env.agent.mon [MON] a=0 b=1 sum=1 carry=0

UVM_INFO driver.sv(24) @ 80: uvm_test_top.env.agent.drv [DRV] a=0 b=1

UVM_INFO scoreboard.sv(13) @ 90: uvm_test_top.env.sco [SCO] PASS a=0 b=1 sum=1 carry=0

UVM_INFO monitor.sv(28) @ 90: uvm_test_top.env.agent.mon [MON] a=0 b=1 sum=1 carry=0

UVM_INFO driver.sv(24) @ 90: uvm_test_top.env.agent.drv [DRV] a=0 b=1

UVM_INFO scoreboard.sv(13) @ 100: uvm_test_top.env.sco [SCO] PASS a=1 b=1 sum=0 carry=1

UVM_INFO monitor.sv(28) @ 100: uvm_test_top.env.agent.mon [MON] a=1 b=1 sum=0 carry=1

UVM_INFO driver.sv(24) @ 100: uvm_test_top.env.agent.drv [DRV] a=1 b=1

UVM_INFO scoreboard.sv(13) @ 110: uvm_test_top.env.sco [SCO] PASS a=1 b=1 sum=0 carry=1

UVM_INFO monitor.sv(28) @ 110: uvm_test_top.env.agent.mon [MON] a=1 b=1 sum=0 carry=1

UVM_INFO scoreboard.sv(13) @ 120: uvm_test_top.env.sco [SCO] PASS a=1 b=1 sum=0 carry=1

UVM_INFO monitor.sv(28) @ 120: uvm_test_top.env.agent.mon [MON] a=1 b=1 sum=0 carry=1

UVM_INFO /usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_objection.svh(1274) @ 120: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase

--- UVM Report Summary ---

** Report counts by severity

UVM_INFO : 36

UVM_WARNING : 0

UVM_ERROR : 0

UVM_FATAL : 0

** Report counts by id

[DRV] 10

[MON] 12

[RNTST] 1

[SCO] 12

[TEST_DONE] 1

$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_root.svh", line 437.

$finish at simulation time 120

V C S S i m u l a t i o n R e p o r t

Time: 120

TLM Analysis Port

[b23n_sarif_727@prac3 tlm_analysis_port]$ ls

consumer.sv csrc env.sv producer.sv simv simv.daidir test.sv tlm_pkg.sv top.sv ucli.key vc_hdrs.h

[b23n_sarif_727@prac3 tlm_analysis_port]$

```
//
class producer extends uvm_component;
	`uvm_component_utils(producer)
	uvm_analysis_port #(int) ap;

	function new(string name="producer",uvm_component parent);
		super.new(name,parent);
	endfunction

	function void build_phase(uvm_phase phase);
		super.build_phase(phase);
		ap = new("ap",this);
	endfunction

	task run_phase(uvm_phase phase);
		int data;
		repeat(5) begin
			data = $urandom_range(1,100);
```

```
                `uvm_info("PRODUCER",$sformatf("Sending Data =
%0d",data),UVM_NONE)
                ap.write(data);
                #10;
        end
    endtask
endclass
//
class consumer extends uvm_component;
    `uvm_component_utils(consumer)
    uvm_analysis_imp #(int,consumer) imp;

    function new(string name="consumer",uvm_component parent);
        super.new(name,parent);
        imp = new("imp",this);
    endfunction

    function void write(int data);
        `uvm_info("CONSUMER",$sformatf("Received Data =
%0d",data),UVM_NONE)
    endfunction
endclass

//
class my_env extends uvm_env;
    `uvm_component_utils(my_env)

    producer p;
```

```
        consumer c;

        function new(string name="my_env",uvm_component parent);
                super.new(name,parent);
        endfunction

        function void build_phase(uvm_phase phase);
                p = producer::type_id::create("p",this);
                c = consumer::type_id::create("c",this);
        endfunction

        function void connect_phase(uvm_phase phase);
                p.ap.connect(c.imp);
        endfunction

endclass

//
class my_test extends uvm_test;
        `uvm_component_utils(my_test)
        my_env env;

        function new(string name="my_test",uvm_component parent);
                super.new(name,parent);
        endfunction

        function void build_phase(uvm_phase phase);
```

```
            env = my_env::type_id::create("env",this);
        endfunction

        task run_phase(uvm_phase phase);
            phase.raise_objection(this);
            #100;
            phase.drop_objection(this);
        endtask

endclass

//
package tlm_pkg;
        import uvm_pkg::*;
        `include "uvm_macros.svh"

       `include "producer.sv"
       `include "consumer.sv"
       `include "env.sv"
       `include "test.sv"
endpackage

//
module top;
import uvm_pkg::*;
import tlm_pkg::*;
```

```
initial begin
        run_test("my_test");
end
endmodule
```

```
[b23n_sarif_727@prac3 tlm_analysis_port]$ vcs -sverilog tlm_pkg.sv top.sv -ntb_opts uvm -kdb -lca -debug_access+all -full64
[b23n_sarif_727@prac3 tlm_analysis_port]$ ./simv
```

Simulation Outputs:

```
UVM_INFO @ 0: reporter [RNTST] Running test my_test...
UVM_INFO producer.sv(19) @ 0: uvm_test_top.env.p [PRODUCER] Sending Data = 96
UVM_INFO consumer.sv(12) @ 0: uvm_test_top.env.c [CONSUMER] Received Data = 96
UVM_INFO producer.sv(19) @ 10: uvm_test_top.env.p [PRODUCER] Sending Data = 53
UVM_INFO consumer.sv(12) @ 10: uvm_test_top.env.c [CONSUMER] Received Data = 53
UVM_INFO producer.sv(19) @ 20: uvm_test_top.env.p [PRODUCER] Sending Data = 89
UVM_INFO consumer.sv(12) @ 20: uvm_test_top.env.c [CONSUMER] Received Data = 89
UVM_INFO producer.sv(19) @ 30: uvm_test_top.env.p [PRODUCER] Sending Data = 8
UVM_INFO consumer.sv(12) @ 30: uvm_test_top.env.c [CONSUMER] Received Data = 8
UVM_INFO producer.sv(19) @ 40: uvm_test_top.env.p [PRODUCER] Sending Data = 64
UVM_INFO consumer.sv(12) @ 40: uvm_test_top.env.c [CONSUMER] Received Data = 64
```

UVM_INFO /usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_objection.svh(1274) @ 100: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase

--- UVM Report Summary ---

** Report counts by severity

UVM_INFO : 12

UVM_WARNING : 0

UVM_ERROR : 0

UVM_FATAL : 0

** Report counts by id

[CONSUMER] 5

[PRODUCER] 5

[RNTST] 1

[TEST_DONE] 1

$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_root.svh", line 437.

$finish at simulation time 100

V C S S i m u l a t i o n R e p o r t

Time: 100

CPU Time: 0.140 seconds; Data structure size: 0.5Mb

Sun Jun 21 00:43:53 2026

Method-2

write the complete code of TLM analysis port (1 producer to 2 consumer) of UVM

[b23n_sarif_727@prac3 tlm_analysis_port]$ ls

consumer1.sv consumer2.sv csrc env.sv packet.sv producer.sv simv simv.daidir test.sv tlm_pkg.sv top.sv ucli.key vc_hdrs.h

```
[b23n_sarif_727@prac3 tlm_analysis_port]$

//
class packet extends uvm_sequence_item;
	rand bit [7:0] addr;
	rand bit [7:0] data;

	`uvm_object_utils(packet)

	function new(string name="packet");
		super.new(name);
	endfunction

endclass

//
class producer extends uvm_component;
	`uvm_component_utils(producer)
	uvm_analysis_port #(packet) ap;

	function new(string name="producer",uvm_component parent);
		super.new(name,parent);
	endfunction

	function void build_phase(uvm_phase phase);
		super.build_phase(phase);
		ap = new("ap",this);
```

```
	endfunction

	task run_phase(uvm_phase phase);
		packet pkt;
		repeat(3) begin
			pkt = packet::type_id::create("pkt");
			assert(pkt.randomize());
			`uvm_info("PRODUCER",$sformatf("Sending addr=%0h data=%0h",pkt.addr,pkt.data),UVM_NONE)
			ap.write(pkt);
			#10;
		end
	endtask

endclass

//
class consumer1 extends uvm_component;
	`uvm_component_utils(consumer1)
	uvm_analysis_imp #(packet,consumer1) imp;

	function new(string name="consumer1",uvm_component parent);
		super.new(name,parent);
		imp = new("imp",this);
	endfunction

	function void write(packet pkt);
```

```
        `uvm_info("CONSUMER1",$sformatf("Received addr=%0h
data=%0h",pkt.addr,pkt.data),UVM_NONE)
    endfunction

endclass

//
class consumer2 extends uvm_component;
    `uvm_component_utils(consumer2)
    uvm_analysis_imp #(packet,consumer2) imp;

    function new(string name="consumer2",uvm_component parent);
        super.new(name,parent);
        imp = new("imp",this);
    endfunction

    function void write(packet pkt);
        `uvm_info("CONSUMER2",$sformatf("Received addr=%0h
data=%0h",pkt.addr,pkt.data),UVM_NONE)
    endfunction

endclass

//
class my_env extends uvm_env;
    `uvm_component_utils(my_env)

    producer  p;
```

```
    consumer1 c1;
    consumer2 c2;

    function new(string name="my_env",uvm_component parent);
        super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
        p  = producer ::type_id::create("p",this);
        c1 = consumer1::type_id::create("c1",this);
        c2 = consumer2::type_id::create("c2",this);
    endfunction

    function void connect_phase(uvm_phase phase);
        p.ap.connect(c1.imp);
        p.ap.connect(c2.imp);
    endfunction

endclass

//
class my_test extends uvm_test;
    `uvm_component_utils(my_test)
    my_env env;

    function new(string name="my_test",uvm_component parent);
        super.new(name,parent);
```

```
        endfunction

        function void build_phase(uvm_phase phase);
                env = my_env::type_id::create("env",this);
        endfunction

        task run_phase(uvm_phase phase);
                phase.raise_objection(this);
                #50;
                phase.drop_objection(this);
        endtask

endclass

//
package tlm_pkg;

  import uvm_pkg::*;
  `include "uvm_macros.svh"

  `include "packet.sv"
  `include "producer.sv"
  `include "consumer1.sv"
  `include "consumer2.sv"
  `include "env.sv"
  `include "test.sv"
```

```
endpackage


//
module top;
import uvm_pkg::*;
import tlm_pkg::*;

initial begin
        run_test("my_test");
end

endmodule
```

[b23n_sarif_727@prac3 tlm_analysis_port]$ vcs -sverilog tlm_pkg.sv top.sv -ntb_opts uvm -kdb -lca -debug_access+all -full64

[b23n_sarif_727@prac3 tlm_analysis_port]$ ./simv

Simulation Outputs:

UVM_INFO @ 0: reporter [RNTST] Running test my_test...

UVM_INFO producer.sv(20) @ 0: uvm_test_top.env.p [PRODUCER] Sending addr=aa data=96

UVM_INFO consumer1.sv(12) @ 0: uvm_test_top.env.c1 [CONSUMER1] Received addr=aa data=96

UVM_INFO consumer2.sv(12) @ 0: uvm_test_top.env.c2 [CONSUMER2] Received addr=aa data=96

UVM_INFO producer.sv(20) @ 10: uvm_test_top.env.p [PRODUCER] Sending addr=95 data=23

UVM_INFO consumer1.sv(12) @ 10: uvm_test_top.env.c1 [CONSUMER1] Received addr=95 data=23

UVM_INFO consumer2.sv(12) @ 10: uvm_test_top.env.c2 [CONSUMER2] Received addr=95 data=23

UVM_INFO producer.sv(20) @ 20: uvm_test_top.env.p [PRODUCER] Sending addr=cb data=5f

UVM_INFO consumer1.sv(12) @ 20: uvm_test_top.env.c1 [CONSUMER1] Received addr=cb data=5f

UVM_INFO consumer2.sv(12) @ 20: uvm_test_top.env.c2 [CONSUMER2] Received addr=cb data=5f

UVM_INFO /usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_objection.svh(1274) @ 50: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase

--- UVM Report Summary ---

** Report counts by severity

UVM_INFO : 11

UVM_WARNING : 0

UVM_ERROR : 0

UVM_FATAL : 0

** Report counts by id

[CONSUMER1] 3

[CONSUMER2] 3

[PRODUCER] 3

[RNTST] 1

[TEST_DONE] 1

$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_root.svh", line 437.

$finish at simulation time 50

V C S S i m u l a t i o n R e p o r t

Time: 50

CPU Time: 0.170 seconds; Data structure size: 0.5Mb

Sun Jun 21 12:44:14 2026

TLM Blocking Put Port

Component_A to Component_B

[b23n_sarif_727@prac3 tlm_blocking_port]$ ls

consumer.sv csrc env.sv packet.sv producer.sv simv simv.daidir test.sv tlm_pkg.sv top.sv ucli.key vc_hdrs.h

```
//
class packet extends uvm_object;
        rand bit [7:0] addr;
        rand bit [7:0] data;

        `uvm_object_utils(packet)

        function new(string name="packet");
                super.new(name);
        endfunction
endclass

//
class consumer extends uvm_component;
        `uvm_component_utils(consumer)
        uvm_blocking_put_imp #(packet,consumer) put_imp;

        function new(string name="consumer",uvm_component parent);
                super.new(name,parent);
```

```
        put_imp = new("put_imp",this);
    endfunction

    task put(packet pkt);
        `uvm_info("CONSUMER",$sformatf("Received addr=%0h data=%0h",pkt.addr,pkt.data),UVM_NONE)
    endtask

endclass

//
class producer extends uvm_component;
    `uvm_component_utils(producer)
    uvm_blocking_put_port #(packet) put_port;

    function new(string name="producer",uvm_component parent);
        super.new(name,parent);
        put_port = new("put_port",this);
    endfunction

    task run_phase(uvm_phase phase);
        packet pkt;
        repeat(5) begin
            pkt = packet::type_id::create("pkt");
            assert(pkt.randomize());
            `uvm_info("PRODUCER",$sformatf("Sending addr=%0h data=%0h",pkt.addr,pkt.data),UVM_NONE)
            put_port.put(pkt);
```

```
                #10;
            end
        endtask
endclass

//
class my_env extends uvm_env;
        `uvm_component_utils(my_env)
        producer p;
        consumer c;

        function new(string name="my_env",uvm_component parent);
            super.new(name,parent);
        endfunction

        function void build_phase(uvm_phase phase);
            p = producer::type_id::create("p",this);
            c = consumer::type_id::create("c",this);
        endfunction

        function void connect_phase(uvm_phase phase);
            p.put_port.connect(c.put_imp);
        endfunction
endclass

//
class my_test extends uvm_test;
```

```
	`uvm_component_utils(my_test)
	my_env env;

	function new(string name="my_test",uvm_component parent);
		super.new(name,parent);
	endfunction

	function void build_phase(uvm_phase phase);
		env = my_env::type_id::create("env",this);
	endfunction

	task run_phase(uvm_phase phase);
		phase.raise_objection(this);
		#100;
		phase.drop_objection(this);
	endtask

endclass

//
package tlm_pkg;
	import uvm_pkg::*;
	`include "uvm_macros.svh"

	`include "packet.sv"
	`include "consumer.sv"
	`include "producer.sv"
```

```
    `include "env.sv"
    `include "test.sv"

endpackage

//
module top;

import uvm_pkg::*;
import tlm_pkg::*;

initial begin
    run_test("my_test");
end
endmodule
```

```
[b23n_sarif_727@prac3 tlm_blocking_port]$ vcs -sverilog tlm_pkg.sv top.sv -ntb_opts uvm -kdb -lca -debug_access+all -full64
[b23n_sarif_727@prac3 tlm_blocking_port]$ ./simv
```

Simulation Outputs:

```
UVM_INFO @ 0: reporter [RNTST] Running test my_test...
UVM_INFO producer.sv(16) @ 0: uvm_test_top.env.p [PRODUCER] Sending addr=aa data=96
UVM_INFO consumer.sv(12) @ 0: uvm_test_top.env.c [CONSUMER] Received addr=aa data=96
UVM_INFO producer.sv(16) @ 10: uvm_test_top.env.p [PRODUCER] Sending addr=95 data=23
UVM_INFO consumer.sv(12) @ 10: uvm_test_top.env.c [CONSUMER] Received addr=95 data=23
```

UVM_INFO producer.sv(16) @ 20: uvm_test_top.env.p [PRODUCER] Sending addr=cb data=5f

UVM_INFO consumer.sv(12) @ 20: uvm_test_top.env.c [CONSUMER] Received addr=cb data=5f

UVM_INFO producer.sv(16) @ 30: uvm_test_top.env.p [PRODUCER] Sending addr=fe data=f2

UVM_INFO consumer.sv(12) @ 30: uvm_test_top.env.c [CONSUMER] Received addr=fe data=f2

UVM_INFO producer.sv(16) @ 40: uvm_test_top.env.p [PRODUCER] Sending addr=33 data=77

UVM_INFO consumer.sv(12) @ 40: uvm_test_top.env.c [CONSUMER] Received addr=33 data=77

UVM_INFO /usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_objection.svh(1274) @ 100: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase

--- UVM Report Summary ---

** Report counts by severity

UVM_INFO :   12

UVM_WARNING :   0

UVM_ERROR :   0

UVM_FATAL :   0

** Report counts by id

[CONSUMER]   5

[PRODUCER]   5

[RNTST]   1

[TEST_DONE]   1

$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_root.svh", line 437.

$finish at simulation time   100

V C S S i m u l a t i o n R e p o r t

Time: 100

Type Override: - (agent)

```
//
`include "uvm_macros.svh"
import uvm_pkg::*;

class base_agent extends uvm_agent;
        `uvm_component_utils(base_agent)

        function new(string name="base_agent", uvm_component parent=null);
                super.new(name,parent);
        endfunction
endclass

class child_agent extends base_agent;
       `uvm_component_utils(child_agent)

       function new(string name="child_agent", uvm_component parent=null);
              super.new(name,parent);
       endfunction
endclass

class env extends uvm_env;
        `uvm_component_utils(env)
        base_agent bagt;
```

```
    function new(string name="env",uvm_component parent=null);
        super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        bagt=base_agent::type_id::create("bagt",this);
    endfunction
endclass

class test extends uvm_test;
    `uvm_component_utils(test)
    env envh;

  function new(string name="test",uvm_component parent=null);
      super.new(name,parent);
  endfunction

  function void build_phase(uvm_phase phase);
      super.build_phase(phase);
      envh=env::type_id::create("envh",this);

        set_type_override_by_type(base_agent::get_type(),child_agent::get_type());
  endfunction

    function void end_of_elaboration_phase(uvm_phase phase);
```

```
        uvm_factory::get().print();
    endfunction

endclass

module top;
initial begin
    run_test("test");
end
endmodule
```

[b23n_sarif_727@prac3 youtube]$ vcs -sverilog factory_override.sv -ntb_opts uvm -kdb -lca -debug_access+all -full64

Simulation Outputs:

```
UVM_INFO @ 0: reporter [RNTST] Running test test...

#### Factory Configuration (*)

No instance overrides are registered with this factory

Type Overrides:

  Requested Type  Override Type
  --------------  -------------
  base_agent      child_agent

All types registered with the factory: 44 total
```

(types without type names will not be printed)

Type Name

---

base_agent

child_agent

env

snps_uvm_reg_bank_group

snps_uvm_reg_map

test

(*) Types with no associated type name will be printed as <unknown>

####

--- UVM Report Summary ---

** Report counts by severity

UVM_INFO : 1

UVM_WARNING : 0

UVM_ERROR : 0

UVM_FATAL : 0

** Report counts by id

[RNTST] 1

$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_root.svh", line 437.

$finish at simulation time 0

V C S S i m u l a t i o n R e p o r t

Time: 0

CPU Time: 0.170 seconds; Data structure size: 0.5Mb

Sun Jun 21 14:50:08 2026

Type Override: -

```
//
`include "uvm_macros.svh"
import uvm_pkg::*;

class base_driver extends uvm_driver;
        `uvm_component_utils(base_driver)

        function new(string name="base_driver",uvm_component parent=null);
                super.new(name,parent);
        endfunction
endclass

class driver1 extends base_driver;
        `uvm_component_utils(driver1)

       function new(string name="driver1",uvm_component parent=null);
              super.new(name,parent);
       endfunction
endclass

class base_agent extends uvm_agent;
        `uvm_component_utils(base_agent)
        base_driver bdh;
```

```
        function new(string name="base_agent", uvm_component parent=null);
                super.new(name,parent);
        endfunction

        function void build_phase(uvm_phase phase);
                super.build_phase(phase);
                bdh=base_driver::type_id::create("bdh",this);
        endfunction
endclass

class child_agent extends base_agent;
      `uvm_component_utils(child_agent)

      function new(string name="child_agent", uvm_component parent=null);
            super.new(name,parent);
      endfunction
endclass

class env extends uvm_env;
        `uvm_component_utils(env)
        base_agent bagt;

        function new(string name="env",uvm_component parent=null);
                super.new(name,parent);
        endfunction
```

```
    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        bagt=base_agent::type_id::create("bagt",this);
    endfunction
endclass

class test extends uvm_test;
    `uvm_component_utils(test)
    env envh;

  function new(string name="test",uvm_component parent=null);
      super.new(name,parent);
  endfunction

  function void build_phase(uvm_phase phase);
      super.build_phase(phase);
      envh=env::type_id::create("envh",this);

    //      set_type_override_by_type(base_agent::get_type(),child_agent::get_type());
        set_type_override_by_type(base_driver::get_type(),driver1::get_type());
    endfunction


    function void end_of_elaboration_phase(uvm_phase phase);
        uvm_factory::get().print();
    endfunction
```

```
endclass

module top;
initial begin
        run_test("test");
end
endmodule
```

Simulation Outputs:

```
UVM_INFO @ 0: reporter [RNTST] Running test test...

#### Factory Configuration (*)

No instance overrides are registered with this factory

Type Overrides:

  Requested Type  Override Type
  --------------  -------------
  base_driver     driver1

All types registered with the factory: 46 total
(types without type names will not be printed)

  Type Name
  ---------
  base_agent
  base_driver
```

child_agent

driver1

env

snps_uvm_reg_bank_group

snps_uvm_reg_map

test

(*) Types with no associated type name will be printed as <unknown>

####

--- UVM Report Summary ---

** Report counts by severity

UVM_INFO : 1

UVM_WARNING : 0

UVM_ERROR : 0

UVM_FATAL : 0

** Report counts by id

[RNTST] 1

$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_root.svh", line 437.

$finish at simulation time 0

V C S S i m u l a t i o n R e p o r t

Time: 0

instant Override: -

```
//
`include "uvm_macros.svh"
import uvm_pkg::*;

class base_driver extends uvm_driver;
        `uvm_component_utils(base_driver)

        function new(string name="base_driver",uvm_component parent=null);
                super.new(name,parent);
        endfunction
endclass

class driver1 extends base_driver;
        `uvm_component_utils(driver1)

       function new(string name="driver1",uvm_component parent=null);
             super.new(name,parent);
       endfunction
endclass

class base_agent extends uvm_agent;
        `uvm_component_utils(base_agent)
        base_driver bdh;

        function new(string name="base_agent", uvm_component parent=null);
                super.new(name,parent);
        endfunction
```

```
        function void build_phase(uvm_phase phase);
                super.build_phase(phase);
                bdh=base_driver::type_id::create("bdh",this);
        endfunction
endclass

class child_agent extends base_agent;
       `uvm_component_utils(child_agent)

       function new(string name="child_agent", uvm_component parent=null);
            super.new(name,parent);
       endfunction
endclass

class env extends uvm_env;
        `uvm_component_utils(env)
        base_agent bagt;

        function new(string name="env",uvm_component parent=null);
                super.new(name,parent);
        endfunction

        function void build_phase(uvm_phase phase);
                super.build_phase(phase);
                bagt=base_agent::type_id::create("bagt",this);
        endfunction
```

```
endclass

class test extends uvm_test;
        `uvm_component_utils(test)
        env envh;

      function new(string name="test",uvm_component parent=null);
            super.new(name,parent);
      endfunction

      function void build_phase(uvm_phase phase);
            super.build_phase(phase);
            envh=env::type_id::create("envh",this);

        //        set_type_override_by_type(base_agent::get_type(),child_agent::get_type());
      //      set_type_override_by_type(base_driver::get_type(),driver1::get_type());

        set_inst_override_by_type("envh.bagt",base_driver::get_type(),driver1::get_type())
;
        endfunction


        function void end_of_elaboration_phase(uvm_phase phase);
              uvm_factory::get().print();
        endfunction

endclass
```

```
module top;
initial begin
        run_test("test");
end
endmodule
```

Simulation Outputs:

```
UVM_INFO @ 0: reporter [RNTST] Running test test...

#### Factory Configuration (*)

Instance Overrides:

  Requested Type  Override Path           Override Type
  --------------  ----------------------  -------------
  base_driver     uvm_test_top.envh.bagt  driver1

No type overrides are registered with this factory

All types registered with the factory: 46 total
(types without type names will not be printed)

  Type Name
  ---------
  base_agent
  base_driver
  child_agent
```

driver1

env

snps_uvm_reg_bank_group

snps_uvm_reg_map

test

(*) Types with no associated type name will be printed as <unknown>

####

--- UVM Report Summary ---

** Report counts by severity

UVM_INFO : 1

UVM_WARNING : 0

UVM_ERROR : 0

UVM_FATAL : 0

** Report counts by id

[RNTST] 1

$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_root.svh", line 437.

$finish at simulation time 0

V C S S i m u l a t i o n R e p o r t

Time: 0

Copy Method

//

`include "uvm_macros.svh"

```
import uvm_pkg::*;

class transaction extends uvm_object;
        `uvm_object_utils(transaction)

        bit [3:0] addr;
        bit [7:0] data;

        function new(string name="transaction");
                super.new(name);
        endfunction

        function void display(string message);
                $display("[%s]: the value of addr : %0d and the value of data is
%0d",message,addr,data);
        endfunction

endclass

module test;
transaction h1,h2;

initial begin

        h1=transaction::type_id::create("h1");
       h2=transaction::type_id::create("h2");

        h1.data=50;
```

```
        h1.addr=10;

        h2.copy(h1);
        h1.display("h1");
        h2.display("h2");
end
endmodule
```

Simulation Outputs:

```
[h1]: the value of addr : 10 and the value of data is 50
[h2]: the value of addr : 0 and the value of data is 0
        V C S  S i m u l a t i o n  R e p o r t
Time: 0
CPU Time:      0.160 seconds;       Data structure size:   0.5Mb
Sun Jun 21 17:21:04 2026
```

Copy Method-2

```
//
`include "uvm_macros.svh"
import uvm_pkg::*;

class transaction extends uvm_object;
        `uvm_object_utils(transaction)

        bit [3:0] addr;
        bit [7:0] data;

        function new(string name="transaction");
```

```
        super.new(name);
    endfunction

    function void display(string message);
        $display("[%s]: the value of addr : %0d and the value of data is %0d",message,addr,data);
    endfunction

    virtual function void do_copy(uvm_object rhs);
      transaction t_h;
        $cast(t_h,rhs);

        this.addr=t_h.addr;
        this.data=t_h.data;
        super.do_copy(t_h);
    endfunction
endclass

module test;
transaction h1,h2;

initial begin

    h1=transaction::type_id::create("h1");
   h2=transaction::type_id::create("h2");

    h1.data=50;
    h1.addr=10;
```

```
        h2.copy(h1);
        h1.display("h1");
        h2.display("h2");
end
endmodule
```

Simulation Outputs:

```
[h1]: the value of addr : 10 and the value of data is 50
[h2]: the value of addr : 10 and the value of data is 50
        V C S  S i m u l a t i o n  R e p o r t
Time: 0
CPU Time:      0.160 seconds;       Data structure size:   0.5Mb
Sun Jun 21 17:31:47 2026
```

Clone() Method:-

```
//
`include "uvm_macros.svh"
import uvm_pkg::*;

class transaction extends uvm_object;
        `uvm_object_utils(transaction)

        bit [3:0] addr;
        bit [7:0] data;

        function new(string name="transaction");
```

```
            super.new(name);
        endfunction

        function void display(string message);
            $display("[%s]: the value of addr : %0d and the value of data is %0d",message,addr,data);
        endfunction

        virtual function void do_copy(uvm_object rhs);
            transaction t_h;
            $cast(t_h,rhs);

            this.addr=t_h.addr;
            this.data=t_h.data;
            super.do_copy(t_h);
        endfunction
endclass

module test;
transaction h1,h2;

initial begin

        h1=transaction::type_id::create("h1");
      // h2=transaction::type_id::create("h2");

        h1.data=50;
        h1.addr=10;
```

```
        //h2.copy(h1);
        $cast(h2,h1.clone());
        h1.display("h1");
        h2.display("h2");
end
endmodule
```

Simulation Outputs:

```
[h1]: the value of addr : 10 and the value of data is 50
[h2]: the value of addr : 10 and the value of data is 50
        V C S  S i m u l a t i o n  R e p o r t
Time: 0
CPU Time:      0.170 seconds;       Data structure size:   0.5Mb
Sun Jun 21 17:36:56 2026
```

Compare method without field registration

```
//
`include "uvm_macros.svh"
import uvm_pkg::*;

class base_packet extends uvm_object;
        `uvm_object_utils(base_packet)

        int a,b;

        function new(string name="base_packet");
```

```
        super.new(name);
    endfunction

    function bit do_compare(uvm_object rhs,uvm_comparer comparer);
        base_packet bph;

        if (!($cast(bph,rhs)))
            return 0;
        return (this.a ==bph.a && this.b ==bph.b);
    endfunction

endclass

module top;
base_packet bph1,bph2;

initial begin
    bph1=base_packet::type_id::create("bph1");
    bph2=base_packet::type_id::create("bph2");

    bph1.a = 10;
    bph1.b = 20;

    bph2.a = 10;
    bph2.b = 30;

    if (bph1.compare(bph2))
```

```
        $display("Comparision succesful");
    else
        $display("Comparision failed");

end
endmodule
```

Simulation Outputs:

```
Comparision failed
        V C S   S i m u l a t i o n   R e p o r t
Time: 0
CPU Time:      0.140 seconds;       Data structure size:   0.5Mb
Mon Jun 22 10:09:29 2026
```

In the above code change only this line of code-

```
        return (this.a ==bph.a);
```

Simulation Outputs:

```
Comparision succesful
        V C S   S i m u l a t i o n   R e p o r t
Time: 0
CPU Time:      0.130 seconds;       Data structure size:   0.5Mb
Mon Jun 22 10:45:06 2026
```

sformatf

```
//
`include "uvm_macros.svh"
import uvm_pkg::*;
```

```
module tb;

int a = 10;
int b = 20;
string msg;

initial begin
    $display("the value of a : %0d", a);
    msg = $sformatf("using $sformatf statement : b = %0d", b);
    $display(msg);
end

endmodule
```

Simulation Outputs:

```
the value of a : 10
using $sformatf statement : b = 20
        V C S   S i m u l a t i o n   R e p o r t
Time: 0
CPU Time:      0.130 seconds;       Data structure size:   0.5Mb
Mon Jun 22 10:57:47 2026
```

do_print() (and uvm_default_table_printer)

```
//
`include "uvm_macros.svh"
import uvm_pkg::*;
```

```
class base_packet extends uvm_object;
	`uvm_object_utils(base_packet)

	int a,b;

	function new(string name="base_packet");
		super.new(name);
	endfunction

	function bit do_compare(uvm_object rhs,uvm_comparer comparer);
		base_packet bph;

		if (!($cast(bph,rhs)))
			return 0;
		return (this.a ==bph.a && this.b ==bph.b);
	endfunction

	function void do_print(uvm_printer printer);
		printer.print_field("the value of a:",a, $bits(a),UVM_DEC);
	    printer.print_field("the value of b:",b, $bits(b),UVM_HEX);
	endfunction

endclass

module top;
base_packet bph1,bph2;
```

```
initial begin
        bph1=base_packet::type_id::create("bph1");
        bph2=base_packet::type_id::create("bph2");

        bph1.a = 10;
        bph1.b = 20;

        bph2.a = 10;
        bph2.b = 30;

        bph1.print();
        bph2.print();

        if (bph1.compare(bph2))
                $display("Comparision succesful");
        else
                $display("Comparision failed");

end
endmodule
```

Simulation Outputs:

```
----------------------------------------
Name             Type        Size  Value
----------------------------------------
bph1             base_packet  -     @456
  the value of a:  integral     32    'd10
```

```
  the value of b:  integral     32    'h14
----------------------------------------
----------------------------------------
Name             Type        Size  Value
----------------------------------------
bph2             base_packet  -    @457
  the value of a:  integral     32    'd10
  the value of b:  integral     32    'h1e
----------------------------------------
Comparision failed
        V C S   S i m u l a t i o n   R e p o r t
Time: 0
CPU Time:      0.130 seconds;       Data structure size:   0.5Mb
Mon Jun 22 11:13:42 2026
```

uvm_default_tree/line_printer

```
//
`include "uvm_macros.svh"
import uvm_pkg::*;

class base_packet extends uvm_object;
        `uvm_object_utils(base_packet)

        int a,b;

        function new(string name="base_packet");
                super.new(name);
```

```
    endfunction

    function bit do_compare(uvm_object rhs,uvm_comparer comparer);
        base_packet bph;

        if (!($cast(bph,rhs)))
            return 0;
        return (this.a ==bph.a && this.b ==bph.b);
    endfunction

    function void do_print(uvm_printer printer);
        printer.print_field("the value of a:",a, $bits(a),UVM_DEC);
      printer.print_field("the value of b:",b, $bits(b),UVM_HEX);
    endfunction

endclass

module top;
base_packet bph1,bph2;

initial begin
    bph1=base_packet::type_id::create("bph1");
    bph2=base_packet::type_id::create("bph2");

    bph1.a = 10;
    bph1.b = 20;
```

```
        bph2.a = 10;
        bph2.b = 30;

        bph1.print(uvm_default_tree_printer);
        bph2.print(uvm_default_line_printer);

        if (bph1.compare(bph2))
                $display("Comparision succesful");
        else
                $display("Comparision failed");

end
endmodule
```

Simulation Outputs:

```
bph1: (base_packet@456) {
  the value of a:: 'd10
  the value of b:: 'h14
}
bph2: (base_packet@457) { the value of a:: 'd10  the value of b:: 'h1e  }
Comparision failed
        V C S  S i m u l a t i o n  R e p o r t
Time: 0
CPU Time:      0.130 seconds;       Data structure size:   0.5Mb
```

Field_macros

```
//
```

```
`include "uvm_macros.svh"
import uvm_pkg::*;

class packet extends uvm_object;

        rand int data;
        rand reg[3:0]addr;
        rand logic enable;

        `uvm_object_utils_begin(packet)
        `uvm_field_int(data,UVM_ALL_ON)
        `uvm_field_int(addr,UVM_NOCOPY)
        `uvm_field_int(enable,UVM_NOCOMPARE)
        `uvm_object_utils_end

        function new(string name="packet");
                super.new(name);
        endfunction

        function void display(string handle);
                $display("%s the value of data is %0d and value of addr :%0d and value of
enale is %d",handle,data,addr,enable);
        endfunction

endclass

module tb;
packet p1,p2;
```

```
initial begin
        p1 = packet::type_id::create("p1");
        p2 = packet::type_id::create("p2");

        p1.randomize();

        p2.copy(p1);
        p1.display("p1");
        p2.display("p2");
end

endmodule
```

Simulation Outputs:

```
p1 the value of data is -384116807 and value of addr :12 and value of enale is 1
p2 the value of data is -384116807 and value of addr :x and value of enale is 1
        V C S  S i m u l a t i o n  R e p o r t
Time: 0
CPU Time:      0.130 seconds;       Data structure size:   0.5Mb
Mon Jun 22 11:53:30 2026
```

UVM Phase (Build_phase)

```
//
`include "uvm_macros.svh"
import uvm_pkg::*;

class driver extends uvm_driver;
```

```
    `uvm_component_utils(driver)

    function new(string name="driver",uvm_component parent=null);
        super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        `uvm_info("build_phase", "Build phase called from driver component",UVM_LOW)
    endfunction
endclass

class monitor extends uvm_monitor;
    `uvm_component_utils(monitor)

    function new(string name="monitor",uvm_component parent=null);
        super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        `uvm_info("build_phase", "Build phase called from monitor component",UVM_LOW)
    endfunction

endclass
```

```
class agent extends uvm_agent;
    `uvm_component_utils(agent)

     driver drvh;
     monitor monh;

    function new(string name="agent",uvm_component parent=null);
        super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        `uvm_info("build_phase", "Build phase called from agent
component",UVM_LOW)

            drvh=driver::type_id::create("drvh",this);
            monh=monitor::type_id::create("monh",this);
    endfunction

endclass

class env extends uvm_env;
    `uvm_component_utils(env)

     agent agth;

    function new(string name="env",uvm_component parent=null);
        super.new(name,parent);
```

```
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        `uvm_info("build_phase", "Build phase called from env component",UVM_LOW)
          agth=agent::type_id::create("agth",this);
    endfunction

endclass


class test extends uvm_test;
    `uvm_component_utils(test)
     env envh;

    function new(string name="test",uvm_component parent=null);
        super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        `uvm_info("build_phase", "Build phase called from test component",UVM_LOW)
        envh=env::type_id::create("envh",this);
    endfunction

endclass
```

```
module top;
initial begin
        run_test("test");
end
endmodule
```

Simulation Outputs:

```
UVM_INFO @ 0: reporter [RNTST] Running test test...
UVM_INFO uvm_phase.sv(80) @ 0: uvm_test_top [build_phase] Build phase called from test component
UVM_INFO uvm_phase.sv(63) @ 0: uvm_test_top.envh [build_phase] Build phase called from env component
UVM_INFO uvm_phase.sv(44) @ 0: uvm_test_top.envh.agth [build_phase] Build phase called from agent component
UVM_INFO uvm_phase.sv(14) @ 0: uvm_test_top.envh.agth.drvh [build_phase] Build phase called from driver component
UVM_INFO uvm_phase.sv(27) @ 0: uvm_test_top.envh.agth.monh [build_phase] Build phase called from monitor component


--- UVM Report Summary ---


** Report counts by severity
UVM_INFO :    6
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
** Report counts by id
[RNTST]     1
[build_phase]     5
```

$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_root.svh", line 437.

$finish at simulation time 0

V C S S i m u l a t i o n R e p o r t

Time: 0

CPU Time: 0.130 seconds; Data structure size: 0.5Mb

Tue Jun 23 14:43:01 2026

UVM Phase (Build_phase + Connect_phase)

```
//
`include "uvm_macros.svh"
import uvm_pkg::*;


class driver extends uvm_driver;
	`uvm_component_utils(driver)


	function new(string name="driver",uvm_component parent=null);
		super.new(name,parent);
	endfunction


	function void build_phase(uvm_phase phase);
		super.build_phase(phase);
		`uvm_info("build_phase", "Build phase called from driver component",UVM_LOW)
	endfunction


	function void connect_phase(uvm_phase phase);
		super.connect_phase(phase);
```

```
        `uvm_info("connect_phase", "Connect phase called from driver component",UVM_LOW)
      endfunction

endclass

class monitor extends uvm_monitor;
    `uvm_component_utils(monitor)

    function new(string name="monitor",uvm_component parent=null);
        super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        `uvm_info("build_phase", "Build phase called from monitor component",UVM_LOW)
    endfunction

    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        `uvm_info("connect_phase", "Connect phase called from monitor component",UVM_LOW)
    endfunction


endclass
```

```
class agent extends uvm_agent;
	`uvm_component_utils(agent)

	  driver drvh;
	  monitor monh;

	function new(string name="agent",uvm_component parent=null);
		super.new(name,parent);
	endfunction

	function void build_phase(uvm_phase phase);
		super.build_phase(phase);
		`uvm_info("build_phase", "Build phase called from agent component",UVM_LOW)

		    drvh=driver::type_id::create("drvh",this);
		    monh=monitor::type_id::create("monh",this);
	endfunction

	function void connect_phase(uvm_phase phase);
		super.connect_phase(phase);
		`uvm_info("connect_phase", "Connect phase called from agent component",UVM_LOW)
	endfunction


endclass
```

```
class env extends uvm_env;
	`uvm_component_utils(env)

	  agent agth;

	function new(string name="env",uvm_component parent=null);
		super.new(name,parent);
	endfunction

	function void build_phase(uvm_phase phase);
		super.build_phase(phase);
		`uvm_info("build_phase", "Build phase called from env component",UVM_LOW)
		    agth=agent::type_id::create("agth",this);
	endfunction

	function void connect_phase(uvm_phase phase);
		super.connect_phase(phase);
		`uvm_info("connect_phase", "Connect phase called from env component",UVM_LOW)
	endfunction


endclass


class test extends uvm_test;
	`uvm_component_utils(test)
```

```
  env envh;

 function new(string name="test",uvm_component parent=null);
      super.new(name,parent);
 endfunction

 function void build_phase(uvm_phase phase);
      super.build_phase(phase);
      `uvm_info("build_phase", "Build phase called from test component",UVM_LOW)
      envh=env::type_id::create("envh",this);
 endfunction

 function void connect_phase(uvm_phase phase);
      super.connect_phase(phase);
      `uvm_info("connect_phase", "Connect phase called from test component",UVM_LOW)
 endfunction



endclass

module top;
initial begin
   run_test("test");
end
endmodule
```

Simulation Outputs:

UVM_INFO @ 0: reporter [RNTST] Running test test...

UVM_INFO uvm_phase.sv(104) @ 0: uvm_test_top [build_phase] Build phase called from test component

UVM_INFO uvm_phase.sv(81) @ 0: uvm_test_top.envh [build_phase] Build phase called from env component

UVM_INFO uvm_phase.sv(56) @ 0: uvm_test_top.envh.agth [build_phase] Build phase called from agent component

UVM_INFO uvm_phase.sv(14) @ 0: uvm_test_top.envh.agth.drvh [build_phase] Build phase called from driver component

UVM_INFO uvm_phase.sv(33) @ 0: uvm_test_top.envh.agth.monh [build_phase] Build phase called from monitor component

UVM_INFO uvm_phase.sv(19) @ 0: uvm_test_top.envh.agth.drvh [connect_phase] Connect phase called from driver component

UVM_INFO uvm_phase.sv(38) @ 0: uvm_test_top.envh.agth.monh [connect_phase] Connect phase called from monitor component

UVM_INFO uvm_phase.sv(64) @ 0: uvm_test_top.envh.agth [connect_phase] Connect phase called from agent component

UVM_INFO uvm_phase.sv(87) @ 0: uvm_test_top.envh [connect_phase] Connect phase called from env component

UVM_INFO uvm_phase.sv(110) @ 0: uvm_test_top [connect_phase] Connect phase called from test component

--- UVM Report Summary ---

** Report counts by severity

UVM_INFO :   11

UVM_WARNING :   0

UVM_ERROR :   0

UVM_FATAL :   0

** Report counts by id

[RNTST]   1

[build_phase] 5

[connect_phase] 5

$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_root.svh", line 437.

$finish at simulation time 0

V C S S i m u l a t i o n R e p o r t

Time: 0

CPU Time: 0.130 seconds; Data structure size: 0.5Mb

Tue Jun 23 14:54:52 2026

UVM Phase (Build_phase + Connect_phase + end_of_elaboration_phase)

```
//
`include "uvm_macros.svh"
import uvm_pkg::*;


class driver extends uvm_driver;
	`uvm_component_utils(driver)


	function new(string name="driver",uvm_component parent=null);
		super.new(name,parent);
	endfunction


	function void build_phase(uvm_phase phase);
		super.build_phase(phase);
		`uvm_info("build_phase", "Build phase called from driver component",UVM_LOW)
	endfunction
```

```
    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        `uvm_info("connect_phase", "Connect phase called from driver component",UVM_LOW)
    endfunction

  function void end_of_elaboration_phase(uvm_phase phase);
      super.end_of_elaboration_phase(phase);

      `uvm_info("end of elaboration phase", "End of elaboration phase called from driver component",UVM_LOW)
  endfunction

endclass

class monitor extends uvm_monitor;
  `uvm_component_utils(monitor)

  function new(string name="monitor",uvm_component parent=null);
      super.new(name,parent);
  endfunction

  function void build_phase(uvm_phase phase);
      super.build_phase(phase);
      `uvm_info("build_phase", "Build phase called from monitor component",UVM_LOW)
  endfunction
```

```
    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        `uvm_info("connect_phase", "Connect phase called from monitor component",UVM_LOW)
    endfunction

    function void end_of_elaboration_phase(uvm_phase phase);
        super.end_of_elaboration_phase(phase);

        `uvm_info("end of elaboration phase", "End of elaboration phase called from monitor component",UVM_LOW)
    endfunction
endclass

class agent extends uvm_agent;
    `uvm_component_utils(agent)

      driver drvh;
      monitor monh;

    function new(string name="agent",uvm_component parent=null);
        super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
```

```
        `uvm_info("build_phase", "Build phase called from agent component",UVM_LOW)

            drvh=driver::type_id::create("drvh",this);
            monh=monitor::type_id::create("monh",this);
    endfunction

    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        `uvm_info("connect_phase", "Connect phase called from agent component",UVM_LOW)
    endfunction

    function void end_of_elaboration_phase(uvm_phase phase);
        super.end_of_elaboration_phase(phase);

        `uvm_info("end of elaboration phase", "End of elaboration phase called from agent component",UVM_LOW)
    endfunction
endclass

class env extends uvm_env;
    `uvm_component_utils(env)

      agent agth;

    function new(string name="env",uvm_component parent=null);
        super.new(name,parent);
```

```
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        `uvm_info("build_phase", "Build phase called from env component",UVM_LOW)
            agth=agent::type_id::create("agth",this);
    endfunction

    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        `uvm_info("connect_phase", "Connect phase called from env component",UVM_LOW)
    endfunction

    function void end_of_elaboration_phase(uvm_phase phase);
        super.end_of_elaboration_phase(phase);

            `uvm_info("end of elaboration phase", "End of elaboration phase called from env component",UVM_LOW)
    endfunction
endclass

class test extends uvm_test;
    `uvm_component_utils(test)
      env envh;

    function new(string name="test",uvm_component parent=null);
```

```
        super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        `uvm_info("build_phase", "Build phase called from test component",UVM_LOW)
        envh=env::type_id::create("envh",this);
    endfunction

    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        `uvm_info("connect_phase", "Connect phase called from test component",UVM_LOW)
    endfunction

     function void end_of_elaboration_phase(uvm_phase phase);
            super.end_of_elaboration_phase(phase);

            uvm_top.print_topology();
            `uvm_info("end of elaboration phase", "End of elaboration phase called from test component",UVM_LOW)
     endfunction

endclass

module top;
initial begin
```

```
    run_test("test");
end
endmodule
```

Simulation Outputs:

UVM_INFO @ 0: reporter [RNTST] Running test test...

UVM_INFO uvm_phase.sv(121) @ 0: uvm_test_top [build_phase] Build phase called from test component

UVM_INFO uvm_phase.sv(95) @ 0: uvm_test_top.envh [build_phase] Build phase called from env component

UVM_INFO uvm_phase.sv(66) @ 0: uvm_test_top.envh.agth [build_phase] Build phase called from agent component

UVM_INFO uvm_phase.sv(14) @ 0: uvm_test_top.envh.agth.drvh [build_phase] Build phase called from driver component

UVM_INFO uvm_phase.sv(39) @ 0: uvm_test_top.envh.agth.monh [build_phase] Build phase called from monitor component

UVM_INFO uvm_phase.sv(19) @ 0: uvm_test_top.envh.agth.drvh [connect_phase] Connect phase called from driver component

UVM_INFO uvm_phase.sv(44) @ 0: uvm_test_top.envh.agth.monh [connect_phase] Connect phase called from monitor component

UVM_INFO uvm_phase.sv(74) @ 0: uvm_test_top.envh.agth [connect_phase] Connect phase called from agent component

UVM_INFO uvm_phase.sv(101) @ 0: uvm_test_top.envh [connect_phase] Connect phase called from env component

UVM_INFO uvm_phase.sv(127) @ 0: uvm_test_top [connect_phase] Connect phase called from test component

UVM_INFO uvm_phase.sv(25) @ 0: uvm_test_top.envh.agth.drvh [end of elaboration phase] End of elaboration phase called from driver component

UVM_INFO uvm_phase.sv(50) @ 0: uvm_test_top.envh.agth.monh [end of elaboration phase] End of elaboration phase called from monitor component

UVM_INFO uvm_phase.sv(80) @ 0: uvm_test_top.envh.agth [end of elaboration phase] End of elaboration phase called from agent component

UVM_INFO uvm_phase.sv(107) @ 0: uvm_test_top.envh [end of elaboration phase] End of elaboration phase called from env component

UVM_INFO @ 0: reporter [UVMTOP] UVM testbench topology:

```
----------------------------------------------------------
Name                Type                Size  Value
----------------------------------------------------------
uvm_test_top        test                -     @456
  envh              env                 -     @464
    agth            agent               -     @472
      drvh          driver              -     @484
        rsp_port      uvm_analysis_port     -     @501
        seq_item_port uvm_seq_item_pull_port -     @492
      monh          monitor             -     @510
----------------------------------------------------------
```

UVM_INFO uvm_phase.sv(134) @ 0: uvm_test_top [end of elaboration phase] End of elaboration phase called from test component

--- UVM Report Summary ---

** Report counts by severity

UVM_INFO :   17

UVM_WARNING :   0

UVM_ERROR :   0

UVM_FATAL :   0

** Report counts by id

[RNTST]   1

[UVMTOP]   1

[build_phase] 5

[connect_phase] 5

[end of elaboration phase] 5

$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_root.svh", line 437.

$finish at simulation time 0

V C S S i m u l a t i o n R e p o r t

Time: 0

CPU Time: 0.140 seconds; Data structure size: 0.5Mb

Tue Jun 23 15:13:30 2026

UVM Phase (Build_phase + Connect_phase + end_of_elaboration_phase + run_phase)

```
//
`include "uvm_macros.svh"
import uvm_pkg::*;

class driver extends uvm_driver;
    `uvm_component_utils(driver)

    function new(string name="driver",uvm_component parent=null);
        super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        `uvm_info("build_phase", "Build phase called from driver component",UVM_LOW)
```

```
    endfunction

    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        `uvm_info("connect_phase", "Connect phase called from driver component",UVM_LOW)
    endfunction

  function void end_of_elaboration_phase(uvm_phase phase);
        super.end_of_elaboration_phase(phase);

        `uvm_info("end of elaboration phase", "End of elaboration phase called from driver component",UVM_LOW)
  endfunction

    task run_phase(uvm_phase phase);
        super.run_phase(phase);
        `uvm_info("Run phase", "Run phase called from driver component",UVM_LOW)
    endtask

endclass

class monitor extends uvm_monitor;
  `uvm_component_utils(monitor)

  function new(string name="monitor",uvm_component parent=null);
        super.new(name,parent);
```

```
    endfunction


    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        `uvm_info("build_phase", "Build phase called from monitor component",UVM_LOW)
    endfunction


    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        `uvm_info("connect_phase", "Connect phase called from monitor component",UVM_LOW)
    endfunction


    function void end_of_elaboration_phase(uvm_phase phase);
        super.end_of_elaboration_phase(phase);


        `uvm_info("end of elaboration phase", "End of elaboration phase called from monitor component",UVM_LOW)
     endfunction


     task run_phase(uvm_phase phase);
          super.run_phase(phase);
        `uvm_info("Run phase", "Run phase called from monitor component",UVM_LOW)
    endtask


endclass
```

```
class agent extends uvm_agent;
    `uvm_component_utils(agent)

     driver drvh;
     monitor monh;

    function new(string name="agent",uvm_component parent=null);
        super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        `uvm_info("build_phase", "Build phase called from agent component",UVM_LOW)

            drvh=driver::type_id::create("drvh",this);
            monh=monitor::type_id::create("monh",this);
    endfunction

    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        `uvm_info("connect_phase", "Connect phase called from agent component",UVM_LOW)
    endfunction

    function void end_of_elaboration_phase(uvm_phase phase);
        super.end_of_elaboration_phase(phase);
```

```
        `uvm_info("end of elaboration phase", "End of elaboration phase called from agent component",UVM_LOW)
    endfunction

    task run_phase(uvm_phase phase);
        super.run_phase(phase);
        `uvm_info("Run phase", "Run phase called from agent component",UVM_LOW)
    endtask

endclass

class env extends uvm_env;
    `uvm_component_utils(env)

     agent agth;

    function new(string name="env",uvm_component parent=null);
        super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        `uvm_info("build_phase", "Build phase called from env component",UVM_LOW)
            agth=agent::type_id::create("agth",this);
    endfunction
```

```
    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        `uvm_info("connect_phase", "Connect phase called from env component",UVM_LOW)
    endfunction

    function void end_of_elaboration_phase(uvm_phase phase);
        super.end_of_elaboration_phase(phase);

          `uvm_info("end of elaboration phase", "End of elaboration phase called from env component",UVM_LOW)
    endfunction

    task run_phase(uvm_phase phase);
        super.run_phase(phase);
        `uvm_info("Run phase", "Run phase called from env component",UVM_LOW)
    endtask

endclass

class test extends uvm_test;
    `uvm_component_utils(test)
     env envh;

    function new(string name="test",uvm_component parent=null);
        super.new(name,parent);
    endfunction
```

```
    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        `uvm_info("build_phase", "Build phase called from test component",UVM_LOW)
        envh=env::type_id::create("envh",this);
    endfunction

    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        `uvm_info("connect_phase", "Connect phase called from test component",UVM_LOW)
    endfunction

     function void end_of_elaboration_phase(uvm_phase phase);
            super.end_of_elaboration_phase(phase);

            uvm_top.print_topology();
            `uvm_info("end of elaboration phase", "End of elaboration phase called from test component",UVM_LOW)
     endfunction

    task run_phase(uvm_phase phase);
        super.run_phase(phase);
        `uvm_info("Run phase", "Run phase called from test component",UVM_LOW)
    endtask

endclass
```

```
module top;
initial begin
        run_test("test");
end
endmodule
```

Simulation Outputs:

```
UVM_INFO @ 0: reporter [RNTST] Running test test...
UVM_INFO uvm_phase.sv(144) @ 0: uvm_test_top [build_phase] Build phase called from test component
UVM_INFO uvm_phase.sv(112) @ 0: uvm_test_top.envh [build_phase] Build phase called from env component
UVM_INFO uvm_phase.sv(77) @ 0: uvm_test_top.envh.agth [build_phase] Build phase called from agent component
UVM_INFO uvm_phase.sv(14) @ 0: uvm_test_top.envh.agth.drvh [build_phase] Build phase called from driver component
UVM_INFO uvm_phase.sv(44) @ 0: uvm_test_top.envh.agth.monh [build_phase] Build phase called from monitor component
UVM_INFO uvm_phase.sv(19) @ 0: uvm_test_top.envh.agth.drvh [connect_phase] Connect phase called from driver component
UVM_INFO uvm_phase.sv(49) @ 0: uvm_test_top.envh.agth.monh [connect_phase] Connect phase called from monitor component
UVM_INFO uvm_phase.sv(85) @ 0: uvm_test_top.envh.agth [connect_phase] Connect phase called from agent component
UVM_INFO uvm_phase.sv(118) @ 0: uvm_test_top.envh [connect_phase] Connect phase called from env component
UVM_INFO uvm_phase.sv(150) @ 0: uvm_test_top [connect_phase] Connect phase called from test component
UVM_INFO uvm_phase.sv(25) @ 0: uvm_test_top.envh.agth.drvh [end of elaboration phase] End of elaboration phase called from driver component
UVM_INFO uvm_phase.sv(55) @ 0: uvm_test_top.envh.agth.monh [end of elaboration phase] End of elaboration phase called from monitor component
```

UVM_INFO uvm_phase.sv(91) @ 0: uvm_test_top.envh.agth [end of elaboration phase] End of elaboration phase called from agent component

UVM_INFO uvm_phase.sv(124) @ 0: uvm_test_top.envh [end of elaboration phase] End of elaboration phase called from env component

UVM_INFO @ 0: reporter [UVMTOP] UVM testbench topology:

```
----------------------------------------------------------
Name                Type                Size  Value
----------------------------------------------------------
uvm_test_top        test                -     @456
  envh              env                 -     @464
    agth            agent               -     @472
      drvh          driver              -     @484
        rsp_port      uvm_analysis_port   -     @501
        seq_item_port uvm_seq_item_pull_port -  @492
      monh          monitor             -     @510
----------------------------------------------------------
```

UVM_INFO uvm_phase.sv(157) @ 0: uvm_test_top [end of elaboration phase] End of elaboration phase called from test component

UVM_INFO uvm_phase.sv(30) @ 0: uvm_test_top.envh.agth.drvh [Run phase] Run phase called from driver component

UVM_INFO uvm_phase.sv(60) @ 0: uvm_test_top.envh.agth.monh [Run phase] Run phase called from monitor component

UVM_INFO uvm_phase.sv(96) @ 0: uvm_test_top.envh.agth [Run phase] Run phase called from agent component

UVM_INFO uvm_phase.sv(129) @ 0: uvm_test_top.envh [Run phase] Run phase called from env component

UVM_INFO uvm_phase.sv(162) @ 0: uvm_test_top [Run phase] Run phase called from test component

--- UVM Report Summary ---

** Report counts by severity

UVM_INFO : 22

UVM_WARNING : 0

UVM_ERROR : 0

UVM_FATAL : 0

** Report counts by id

[RNTST] 1

[Run phase] 5

[UVMTOP] 1

[build_phase] 5

[connect_phase] 5

[end of elaboration phase] 5

$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_root.svh", line 437.

$finish at simulation time 0

V C S S i m u l a t i o n R e p o r t

Time: 0

CPU Time: 0.130 seconds; Data structure size: 0.5Mb

Tue Jun 23 15:31:56 2026

UVM Phase (run_phase + UVM_(raise/drop) objection)

//

`include "uvm_macros.svh"

import uvm_pkg::*;

```
class driver extends uvm_driver;
        `uvm_component_utils(driver)

        function new(string name="driver",uvm_component parent=null);
                super.new(name,parent);
        endfunction

        function void build_phase(uvm_phase phase);
                super.build_phase(phase);
                `uvm_info("build_phase", "Build phase called from driver component",UVM_LOW)
        endfunction

        function void connect_phase(uvm_phase phase);
                super.connect_phase(phase);
            `uvm_info("connect_phase", "Connect phase called from driver component",UVM_LOW)
        endfunction

      function void end_of_elaboration_phase(uvm_phase phase);
            super.end_of_elaboration_phase(phase);

            `uvm_info("end of elaboration phase", "End of elaboration phase called from driver component",UVM_LOW)
      endfunction

        task run_phase(uvm_phase phase);
                super.run_phase(phase);
```

```
        phase.raise_objection(this);
    `uvm_info("Run phase", "Run phase raise_objection called from driver component",UVM_LOW)
        #10;
        phase.drop_objection(this);
    `uvm_info("Run phase", "Run phase drop_objection called from driver component",UVM_LOW)

    `uvm_info("Run phase", "Run phase called from driver component",UVM_LOW)
  endtask

endclass

class monitor extends uvm_monitor;
  `uvm_component_utils(monitor)

  function new(string name="monitor",uvm_component parent=null);
    super.new(name,parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    `uvm_info("build_phase", "Build phase called from monitor component",UVM_LOW)
  endfunction

  function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
```

```
        `uvm_info("connect_phase", "Connect phase called from monitor component",UVM_LOW)
    endfunction

    function void end_of_elaboration_phase(uvm_phase phase);
        super.end_of_elaboration_phase(phase);

        `uvm_info("end of elaboration phase", "End of elaboration phase called from monitor component",UVM_LOW)
     endfunction

     task run_phase(uvm_phase phase);
         super.run_phase(phase);
        `uvm_info("Run phase", "Run phase raise_objection called from monitor component",UVM_LOW)
        #50;
        phase.drop_objection(this);
        `uvm_info("Run phase", "Run phase drop_objection called from monitorr component",UVM_LOW)

        `uvm_info("Run phase", "Run phase called from monitor component",UVM_LOW)
    endtask

endclass

class agent extends uvm_agent;
    `uvm_component_utils(agent)
```

```
  driver drvh;
  monitor monh;

function new(string name="agent",uvm_component parent=null);
    super.new(name,parent);
endfunction

function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    `uvm_info("build_phase", "Build phase called from agent component",UVM_LOW)

      drvh=driver::type_id::create("drvh",this);
      monh=monitor::type_id::create("monh",this);
endfunction

function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    `uvm_info("connect_phase", "Connect phase called from agent component",UVM_LOW)
endfunction

function void end_of_elaboration_phase(uvm_phase phase);
    super.end_of_elaboration_phase(phase);

    `uvm_info("end of elaboration phase", "End of elaboration phase called from agent component",UVM_LOW)
endfunction
```

```
    task run_phase(uvm_phase phase);
        super.run_phase(phase);
        `uvm_info("Run phase", "Run phase raise_objection called from agent component",UVM_LOW)
        #10;
        phase.drop_objection(this);
        `uvm_info("Run phase", "Run phase drop_objection called from agent component",UVM_LOW)

        `uvm_info("Run phase", "Run phase called from agent component",UVM_LOW)
    endtask

endclass

class env extends uvm_env;
    `uvm_component_utils(env)

      agent agth;

    function new(string name="env",uvm_component parent=null);
        super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        `uvm_info("build_phase", "Build phase called from env component",UVM_LOW)
```

```
        agth=agent::type_id::create("agth",this);
    endfunction

    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        `uvm_info("connect_phase", "Connect phase called from env component",UVM_LOW)
    endfunction

    function void end_of_elaboration_phase(uvm_phase phase);
        super.end_of_elaboration_phase(phase);

        `uvm_info("end of elaboration phase", "End of elaboration phase called from env component",UVM_LOW)
    endfunction

    task run_phase(uvm_phase phase);
        super.run_phase(phase);
        `uvm_info("Run phase", "Run phase raise_objection called from env component",UVM_LOW)
        #10;
        phase.drop_objection(this);
        `uvm_info("Run phase", "Run phase drop_objection called from env component",UVM_LOW)

        `uvm_info("Run phase", "Run phase called from env component",UVM_LOW)
    endtask

endclass
```

```
class test extends uvm_test;
	`uvm_component_utils(test)
	  env envh;

	function new(string name="test",uvm_component parent=null);
		super.new(name,parent);
	endfunction

	function void build_phase(uvm_phase phase);
		super.build_phase(phase);
		`uvm_info("build_phase", "Build phase called from test component",UVM_LOW)
		envh=env::type_id::create("envh",this);
	endfunction

	function void connect_phase(uvm_phase phase);
		super.connect_phase(phase);
		`uvm_info("connect_phase", "Connect phase called from test component",UVM_LOW)
	endfunction

	  function void end_of_elaboration_phase(uvm_phase phase);
			super.end_of_elaboration_phase(phase);

			uvm_top.print_topology();
			`uvm_info("end of elaboration phase", "End of elaboration phase called from test component",UVM_LOW)
```

```
    endfunction


  task run_phase(uvm_phase phase);
      super.run_phase(phase);
      `uvm_info("Run phase", "Run phase raise_objection called from test component",UVM_LOW)
      #10;
      phase.drop_objection(this);
      `uvm_info("Run phase", "Run phase drop_objection called from test component",UVM_LOW)


      `uvm_info("Run phase", "Run phase called from test component",UVM_LOW)
  endtask


endclass


module top;
initial begin
    run_test("test");
end
endmodule
```

Simulation Outputs:

```
UVM_INFO @ 0: reporter [RNTST] Running test test...
UVM_INFO uvm_phase.sv(165) @ 0: uvm_test_top [build_phase] Build phase called from test component
UVM_INFO uvm_phase.sv(128) @ 0: uvm_test_top.envh [build_phase] Build phase called from env component
UVM_INFO uvm_phase.sv(88) @ 0: uvm_test_top.envh.agth [build_phase] Build phase called from agent component
```

UVM_INFO uvm_phase.sv(14) @ 0: uvm_test_top.envh.agth.drvh [build_phase] Build phase called from driver component

UVM_INFO uvm_phase.sv(50) @ 0: uvm_test_top.envh.agth.monh [build_phase] Build phase called from monitor component

UVM_INFO uvm_phase.sv(19) @ 0: uvm_test_top.envh.agth.drvh [connect_phase] Connect phase called from driver component

UVM_INFO uvm_phase.sv(55) @ 0: uvm_test_top.envh.agth.monh [connect_phase] Connect phase called from monitor component

UVM_INFO uvm_phase.sv(96) @ 0: uvm_test_top.envh.agth [connect_phase] Connect phase called from agent component

UVM_INFO uvm_phase.sv(134) @ 0: uvm_test_top.envh [connect_phase] Connect phase called from env component

UVM_INFO uvm_phase.sv(171) @ 0: uvm_test_top [connect_phase] Connect phase called from test component

UVM_INFO uvm_phase.sv(25) @ 0: uvm_test_top.envh.agth.drvh [end of elaboration phase] End of elaboration phase called from driver component

UVM_INFO uvm_phase.sv(61) @ 0: uvm_test_top.envh.agth.monh [end of elaboration phase] End of elaboration phase called from monitor component

UVM_INFO uvm_phase.sv(102) @ 0: uvm_test_top.envh.agth [end of elaboration phase] End of elaboration phase called from agent component

UVM_INFO uvm_phase.sv(140) @ 0: uvm_test_top.envh [end of elaboration phase] End of elaboration phase called from env component

UVM_INFO @ 0: reporter [UVMTOP] UVM testbench topology:

Name Type Size Value

---

uvm_test_top test - @456

envh env - @464

agth agent - @472

drvh driver - @484

rsp_port uvm_analysis_port - @501

seq_item_port uvm_seq_item_pull_port - @492

monh monitor - @510

UVM_INFO uvm_phase.sv(178) @ 0: uvm_test_top [end of elaboration phase] End of elaboration phase called from test component

UVM_INFO uvm_phase.sv(31) @ 0: uvm_test_top.envh.agth.drvh [Run phase] Run phase raise_objection called from driver component

UVM_INFO uvm_phase.sv(66) @ 0: uvm_test_top.envh.agth.monh [Run phase] Run phase raise_objection called from monitor component

UVM_INFO uvm_phase.sv(107) @ 0: uvm_test_top.envh.agth [Run phase] Run phase raise_objection called from agent component

UVM_INFO uvm_phase.sv(145) @ 0: uvm_test_top.envh [Run phase] Run phase raise_objection called from env component

UVM_INFO uvm_phase.sv(183) @ 0: uvm_test_top [Run phase] Run phase raise_objection called from test component

UVM_INFO uvm_phase.sv(34) @ 10: uvm_test_top.envh.agth.drvh [Run phase] Run phase drop_objection called from driver component

UVM_INFO uvm_phase.sv(36) @ 10: uvm_test_top.envh.agth.drvh [Run phase] Run phase called from driver component

UVM_FATAL @ 10: run [OBJTN_ZERO] Object "uvm_test_top.envh.agth" attempted to drop objection 'run' count below zero

--- UVM Report Summary ---

** Report counts by severity

UVM_INFO : 24

UVM_WARNING : 0

UVM_ERROR : 0

UVM_FATAL : 1

```
** Report counts by id
[OBJTN_ZERO]     1
[RNTST]     1
[Run phase]     7
[UVMTOP]     1
[build_phase]     5
[connect_phase]     5
[end of elaboration phase]     5
$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_report_object.svh", line 292.
$finish at simulation time                  10
           V C S   S i m u l a t i o n   R e p o r t
Time: 10
CPU Time:      0.140 seconds;       Data structure size:   0.5Mb
Tue Jun 23 15:51:10 2026
```

(UVM_Report Functions & Macros || uvm_info, uvm_error, uvm_warning, uvm_fatal)

```
//
`include "uvm_macros.svh"
import uvm_pkg::*;

class test extends uvm_test;
    `uvm_component_utils(test)

    function new(string name="test",uvm_component parent=null);
        super.new(name,parent);
```

```
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);

        uvm_report_info(get_type_name(),"printing from build phase",UVM_HIGH,"uvm_report.sv",15);
    endfunction
endclass

module top;
initial begin
    run_test("test");
end
endmodule
```

Simulation Outputs:

```
UVM_INFO @ 0: reporter [RNTST] Running test test...

--- UVM Report Summary ---

** Report counts by severity
UVM_INFO :    1
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
** Report counts by id
[RNTST]     1
```

$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_root.svh", line 437.

$finish at simulation time 0

V C S S i m u l a t i o n R e p o r t

Time: 0

CPU Time: 0.130 seconds; Data structure size: 0.5Mb

Tue Jun 23 18:18:50 2026

In the above code only change one line:- (verbosity level)

uvm_report_info(get_type_name(),"printing from build phase",UVM_MEDIUM,"uvm_report.sv",15);

Simulation Outputs:

UVM_INFO @ 0: reporter [RNTST] Running test test...

UVM_INFO uvm_report.sv(15) @ 0: uvm_test_top [test] printing from build phase

--- UVM Report Summary ---

** Report counts by severity

UVM_INFO : 2

UVM_WARNING : 0

UVM_ERROR : 0

UVM_FATAL : 0

** Report counts by id

[RNTST] 1

[test] 1

$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_root.svh", line 437.

$finish at simulation time 0

V C S S i m u l a t i o n R e p o r t

Time: 0

CPU Time: 0.130 seconds;

Change the verbosity level-

```
//
`include "uvm_macros.svh"
import uvm_pkg::*;

class test extends uvm_test;
    `uvm_component_utils(test)

    function new(string name="test",uvm_component parent=null);
        super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);

        uvm_report_info(get_type_name(),"printing from build phase",UVM_MEDIUM,"uvm_report.sv",15);
    endfunction
endclass

module top;
initial begin
    uvm_top.set_report_verbosity_level(UVM_LOW);
```

```
    run_test("test");
end
endmodule
```

Simulation Outputs:

```
UVM_INFO @ 0: reporter [RNTST] Running test test...


--- UVM Report Summary ---


** Report counts by severity
UVM_INFO :    1
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
** Report counts by id
[RNTST]     1
$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_root.svh", line 437.
$finish at simulation time                   0
           V C S   S i m u l a t i o n   R e p o r t
Time: 0
CPU Time:      0.120 seconds;       Data structure size:   0.5Mb
Tue Jun 23 18:30:55 2026
```

Change the verbosity level- and remove (file name & line number)

```
//
`include "uvm_macros.svh"
```

```
import uvm_pkg::*;

class test extends uvm_test;
	`uvm_component_utils(test)

	function new(string name="test",uvm_component parent=null);
		super.new(name,parent);
	endfunction

	function void build_phase(uvm_phase phase);
		super.build_phase(phase);

		uvm_report_info(get_type_name(),"printing from build phase", UVM_NONE);
//		`uvm_info();
	endfunction
endclass

module top;
initial begin
	uvm_top.set_report_verbosity_level(UVM_LOW);
	run_test("test");
end
endmodule
```

Simulation Outputs:

```
UVM_INFO @ 0: reporter [RNTST] Running test test...
UVM_INFO @ 0: uvm_test_top [test] printing from build phase
```

--- UVM Report Summary ---

** Report counts by severity

UVM_INFO : 2

UVM_WARNING : 0

UVM_ERROR : 0

UVM_FATAL : 0

** Report counts by id

[RNTST] 1

[test] 1

$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_root.svh", line 437.

$finish at simulation time 0

V C S S i m u l a t i o n R e p o r t

Time: 0

CPU Time: 0.130 seconds; Data structure size: 0.5Mb

Tue Jun 23 18:38:03 2026

Change the verbosity level- and use `uvm_info

```
//
`include "uvm_macros.svh"
import uvm_pkg::*;

class test extends uvm_test;
    `uvm_component_utils(test)

    function new(string name="test",uvm_component parent=null);
```

```
        super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);

//          uvm_report_info(get_type_name(),"printing from build phase",UVM_NONE);
            `uvm_info(get_type_name(),"printing from build phase",UVM_NONE);
    endfunction
endclass

module top;
initial begin
    uvm_top.set_report_verbosity_level(UVM_LOW);
    run_test("test");
end
endmodule
```

Simulation Outputs:

```
UVM_INFO @ 0: reporter [RNTST] Running test test...
UVM_INFO uvm_report.sv(16) @ 0: uvm_test_top [test] printing from build phase

--- UVM Report Summary ---

** Report counts by severity
UVM_INFO :    2
UVM_WARNING :    0
UVM_ERROR :    0
```

UVM_FATAL : 0

** Report counts by id

[RNTST] 1

[test] 1

$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_root.svh", line 437.

$finish at simulation time 0

V C S S i m u l a t i o n R e p o r t

Time: 0

CPU Time: 0.130 seconds; Data structure size: 0.5Mb

Tue Jun 23 18:42:42 2026

Printing the data by using $sformatf

```
//
`include "uvm_macros.svh"
import uvm_pkg::*;

class test extends uvm_test;
    `uvm_component_utils(test)
    int a = 50;

    function new(string name="test",uvm_component parent=null);
        super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
```

```
//              uvm_report_info(get_type_name(),"printing from build phase",UVM_NONE);
                `uvm_info(get_type_name(),"printing from build phase",UVM_NONE);
                `uvm_info(get_type_name(),$sformatf("the value of a:%0d",a),UVM_NONE);


        endfunction
endclass

module top;
initial begin
        uvm_top.set_report_verbosity_level(UVM_LOW);
        run_test("test");
end
endmodule
```

Simulation Outputs:

```
UVM_INFO @ 0: reporter [RNTST] Running test test...
UVM_INFO uvm_report.sv(17) @ 0: uvm_test_top [test] printing from build phase
UVM_INFO uvm_report.sv(18) @ 0: uvm_test_top [test] the value of a:50

--- UVM Report Summary ---

** Report counts by severity
UVM_INFO :    3
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
** Report counts by id
```

```
[RNTST]     1
[test]     2
$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_root.svh", line 437.
$finish at simulation time                   0
           V C S   S i m u l a t i o n   R e p o r t
Time: 0
CPU Time:      0.130 seconds;       Data structure size:   0.5Mb
Tue Jun 23 18:50:07 2026
```

Seniority level of verbosity level

```
//
`include "uvm_macros.svh"
import uvm_pkg::*;

class test extends uvm_test;
        `uvm_component_utils(test)
        int a = 50;

        function new(string name="test",uvm_component parent=null);
                super.new(name,parent);
        endfunction

        function void build_phase(uvm_phase phase);
                super.build_phase(phase);

//              uvm_report_info(get_type_name(),"printing from build phase",UVM_NONE);
```

```
        `uvm_info(get_type_name(),"printing from build phase",UVM_NONE);
        `uvm_info(get_type_name(),$sformatf("the value of a:%0d",a),UVM_NONE);

    endfunction
endclass

module top;
initial begin
    uvm_top.set_report_verbosity_level(UVM_LOW);
    uvm_top.set_report_verbosity_level(UVM_NO_ACTION);
    run_test("test");
end
endmodule
```

Simulation Outputs:

```
UVM_INFO uvm_report.sv(17) @ 0: uvm_test_top [test] printing from build phase
UVM_INFO uvm_report.sv(18) @ 0: uvm_test_top [test] the value of a:50

--- UVM Report Summary ---

** Report counts by severity
UVM_INFO :    2
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
** Report counts by id
[test]     2
```

$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_root.svh", line 437.

$finish at simulation time 0

V C S S i m u l a t i o n R e p o r t

Time: 0

CPU Time: 0.130 seconds; Data structure size: 0.5Mb

Tue Jun 23 18:58:36 2026

UVM TLM Ports || put & put_imp (push-based)

```
//
`include "uvm_macros.svh"
import uvm_pkg::*;

class transaction extends uvm_object;
    `uvm_object_utils(transaction)

    rand bit data;

    function new(string name="transaction");
        super.new(name);
    endfunction

    function void display(string message);
        $display("%s : the value of data is :%0d", message,data);
    endfunction
endclass
```

```
class generator extends uvm_component;
        `uvm_component_utils(generator)
        uvm_blocking_put_port #(transaction) put_port;

        function new(string name="generator",uvm_component parent=null);
                super.new(name,parent);
                put_port = new("put_port",this);
        endfunction

        task run_phase(uvm_phase phase);
                transaction txn;

                txn = transaction::type_id::create("txn");
                assert(txn.randomize());
                put_port.put(txn);
        endtask
endclass

class driver extends uvm_driver;
        `uvm_component_utils(driver)
        uvm_blocking_put_imp #(transaction,driver) put_imp;

       function new(string name="driver",uvm_component parent=null);
            super.new(name,parent);
            put_imp = new("put_imp",this);
       endfunction
```

```
        task put(transaction txn);

                txn.display("driver");

        endtask
endclass

class agent extends uvm_agent;
        `uvm_component_utils(agent)
        generator genh;
        driver drvh;

      function new(string name="agent",uvm_component parent=null);
            super.new(name,parent);
      endfunction

        function void build_phase(uvm_phase phase);
                super.build_phase(phase);

                genh = generator::type_id::create("genh",this);
            drvh = driver::type_id::create("drvh",this);
        endfunction

        function void connect_phase(uvm_phase phase);
                super.connect_phase(phase);

                genh.put_port.connect(drvh.put_imp);
```

```
        endfunction
endclass

class test extends uvm_test;
        `uvm_component_utils(test)
        agent agth;

      function new(string name="test",uvm_component parent=null);
            super.new(name,parent);
      endfunction

      function void build_phase(uvm_phase phase);
            super.build_phase(phase);

            agth = agent::type_id::create("agth",this);
      endfunction
endclass

module top;
initial begin
        run_test("test");
end
endmodule
```

Simulation Outputs:

```
UVM_INFO @ 0: reporter [RNTST] Running test test...
driver : the value of data is :1
```

--- UVM Report Summary ---

** Report counts by severity

UVM_INFO : 1

UVM_WARNING : 0

UVM_ERROR : 0

UVM_FATAL : 0

** Report counts by id

[RNTST] 1

$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_root.svh", line 437.

$finish at simulation time 0

V C S S i m u l a t i o n R e p o r t

Time: 0

CPU Time: 0.130 seconds; Data structure size: 0.5Mb

Tue Jun 23 20:06:50 2026

UVM TLM Ports || put & put_imp (Pull-based)

uvm_get_port & uvm_get_imp belongs to the pull-based TLM interface. (get)

1.0 data will flow from target to inititor (get)

2.0 data will flow from initiator to target (put)

uvm_tlm_fifo

//

`include "uvm_macros.svh"

import uvm_pkg::*;

```
class packet extends uvm_object;
	`uvm_object_utils(packet)

	rand int data;
	function new(string name="packet");
		super.new(name);
	endfunction

	function void display(string s);
		$display("%s : the value of data is :%0d",s,data);
	endfunction
endclass

class generator extends uvm_component;
	`uvm_component_utils(generator)
	uvm_blocking_put_port #(packet) put_port;
	packet pkt;

	function new(string name="generator",uvm_component parent=null);
		super.new(name,parent);
		put_port = new("put_port",this);
	endfunction

	function void build_phase(uvm_phase phase);
		super.build_phase(phase);
		pkt = packet::type_id::create("pkt",this);
	endfunction
```

```
        task run_phase(uvm_phase phase);
                pkt.randomize();
                put_port.put(pkt);
        endtask
endclass

class driver extends uvm_driver;
        `uvm_component_utils(driver)
        uvm_blocking_get_port #(packet) get_port;
        packet pkt;

      function new(string name="driver",uvm_component parent=null);
            super.new(name,parent);
            get_port = new("get_port",this);
      endfunction

      task run_phase(uvm_phase phase);
            get_port.get(pkt);
                pkt.display("driver");
      endtask
endclass

class agent extends uvm_agent;
        `uvm_component_utils(agent)

        generator genh;
```

```
  driver drvh;
  uvm_tlm_fifo #(packet) fifoh;

  function new(string name="agent",uvm_component parent=null);
      super.new(name,parent);
      fifoh = new("fifoh",this);
  endfunction

  function void build_phase(uvm_phase phase);
      super.build_phase(phase);
      genh = generator::type_id::create("genh",this);
      drvh = driver::type_id::create("drvh",this);
  endfunction

  function void connect_phase (uvm_phase phase);
        super.connect_phase(phase);
        genh.put_port.connect(fifoh.put_export);
      drvh.get_port.connect(fifoh.get_export);
  endfunction

endclass

class test extends uvm_test;
    `uvm_component_utils(test)

  agent agth;
  function new(string name="agent",uvm_component parent=null);
```

```
        super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        agth = agent::type_id::create("agth",this);
    endfunction
endclass

module top;
initial begin
      run_test("test");
end
endmodule
```

Simulation Outputs:

```
UVM_INFO @ 0: reporter [RNTST] Running test test...
driver : the value of data is :848306379

--- UVM Report Summary ---

** Report counts by severity
UVM_INFO :    1
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
** Report counts by id
[RNTST]     1
```

$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_root.svh", line 437.

$finish at simulation time 0

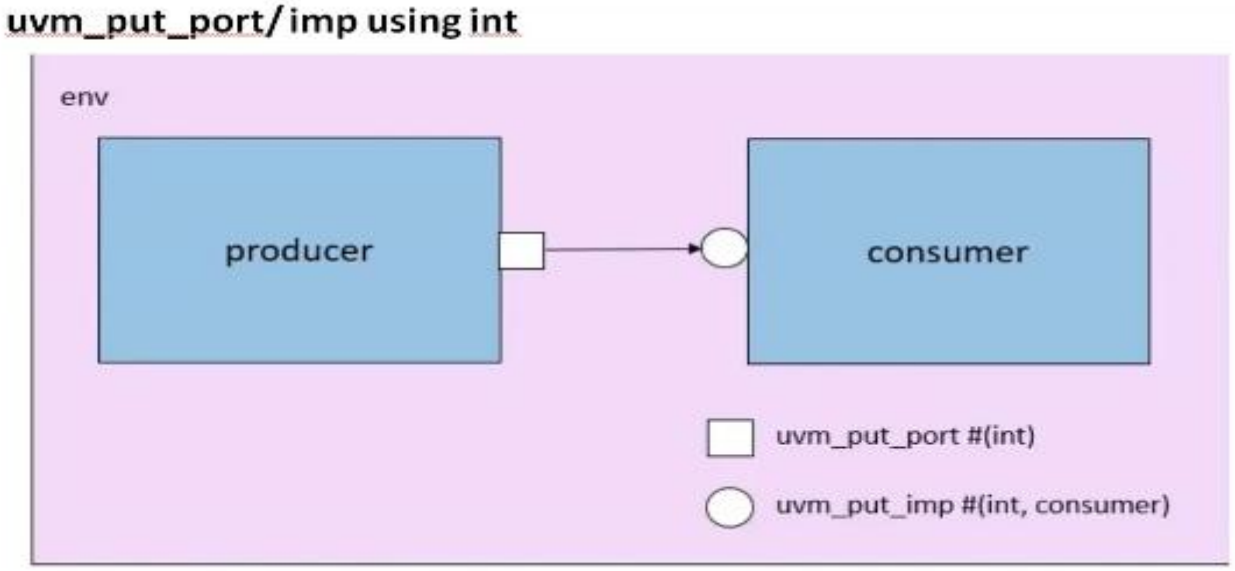
V C S S i m u l a t i o n R e p o r t

Time: 0

CPU Time: 0.140 seconds; Data structure size: 0.5Mb

Wed Jun 24 00:20:40 2026

uvm_put_port / imp using int



//

`include "uvm_macros.svh"

import uvm_pkg::*;

class producer extends uvm_component;

uvm_put_port #(int) tlm_put;

`uvm_component_utils(producer)

int data;

```
        function new(string name="producer",uvm_component parent=null);
                super.new(name,parent);
                tlm_put = new("tlm_put",this);
        endfunction

        task run_phase(uvm_phase phase);
                super.run_phase(phase);


                data = 20;
                `uvm_info(get_type_name(),$sformatf("the value of data is
%0d",data),UVM_LOW);
                tlm_put.put(data);
                tlm_put.try_put(data);
                tlm_put.can_put();
        endtask
endclass

class consumer extends uvm_component;
        `uvm_component_utils(consumer)
        uvm_put_imp #(int,consumer) tlm_imp;

       function new(string name="consumer",uvm_component parent=null);
            super.new(name,parent);
                tlm_imp = new("tlm_imp",this);
       endfunction

        task put(int val);
```

```
            #10;
            `uvm_info(get_type_name(),$sformatf("received the data: %0d",
val),UVM_LOW);
        endtask

        function bit try_put(int val);

            `uvm_info(get_type_name(),$sformatf("received the try_put data: %0d",
val),UVM_LOW);
            return 1;
        endfunction

        function bit can_put();

            `uvm_info(get_type_name(),"inside can_put ",UVM_LOW);
            return 1;
        endfunction
endclass

class env extends uvm_env;
        `uvm_component_utils(env)

        producer prh;
        consumer crh;

      function new(string name="env",uvm_component parent=null);
          super.new(name,parent);
      endfunction
```

```
    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        prh = producer::type_id::create("prh",this);
      crh = consumer::type_id::create("crh",this);
    endfunction

    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        prh.tlm_put.connect(crh.tlm_imp);
    endfunction
endclass

class test extends uvm_test;
    `uvm_component_utils(test)
    env envh;

  function new(string name="test",uvm_component parent=null);
      super.new(name,parent);
  endfunction

  function void build_phase(uvm_phase phase);
      super.build_phase(phase);
      envh = env::type_id::create("envh",this);
  endfunction

    task run_phase(uvm_phase phase);
```

```
        super.run_phase(phase);


        phase.raise_objection(this);
        #50;
        phase.drop_objection(this);
    endtask
endclass


module top;
initial begin
    run_test("test");
end
endmodule
```

Simulation Outputs:

```
UVM_INFO @ 0: reporter [RNTST] Running test test...
UVM_INFO tlm_fifo_2.sv(19) @ 0: uvm_test_top.envh.prh [producer] the value of data is 20
UVM_INFO tlm_fifo_2.sv(38) @ 10: uvm_test_top.envh.crh [consumer] received the data: 20
UVM_INFO tlm_fifo_2.sv(43) @ 10: uvm_test_top.envh.crh [consumer] received the try_put data: 20
UVM_INFO tlm_fifo_2.sv(49) @ 10: uvm_test_top.envh.crh [consumer] inside can_put
UVM_INFO /usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_objection.svh(1274) @ 50: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase


--- UVM Report Summary ---


** Report counts by severity
```

```
UVM_INFO :    6
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
** Report counts by id
[RNTST]     1
[TEST_DONE]     1
[consumer]     3
[producer]     1
$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_root.svh", line 437.
$finish at simulation time                   50
           V C S   S i m u l a t i o n   R e p o r t
Time: 50
CPU Time:      0.170 seconds;       Data structure size:   0.5Mb
Wed Jun 24 20:02:24 2026
```

Virtual sequence & Virtual Sequencer

```
//
`include "uvm_macros.svh"
import uvm_pkg::*;


class seq_item extends uvm_sequence_item;
        `uvm_object_utils(seq_item)


        rand int data1;
        rand int data2;
```

```
    function new(string name="seq_item");
        super.new(name);
    endfunction
endclass

class seq1 extends uvm_sequence #(seq_item);
    `uvm_object_utils(seq1)
    seq_item req;

    function new(string name="seq1");
        super.new(name);
    endfunction

    task body();
        `uvm_info(get_type_name(),"inside body task of : seq1",UVM_LOW);

        req = seq_item::type_id::create("req");
        wait_for_grant();
        req.randomize();
        send_request(req);
        wait_for_item_done();
    endtask
endclass

class seq2 extends uvm_sequence #(seq_item);
    `uvm_object_utils(seq2)
```

```
    seq_item req;

    function new(string name="seq2");
        super.new(name);
    endfunction

    task body();
        `uvm_info(get_type_name(),"inside body task of : seq2",UVM_LOW);

        req = seq_item::type_id::create("req");
        wait_for_grant();
        req.randomize();
        send_request(req);
        wait_for_item_done();
    endtask
endclass

class seqr1 extends uvm_sequencer #(seq_item);
    `uvm_component_utils(seqr1)

    function new(string name="seqr1", uvm_component parent=null);
        super.new(name,parent);
    endfunction
endclass

class seqr2 extends uvm_sequencer #(seq_item);
    `uvm_component_utils(seqr2)
```

```
        function new(string name="seqr2", uvm_component parent=null);
                super.new(name,parent);
        endfunction
endclass

class virtual_seqr extends uvm_sequencer;
        `uvm_component_utils(virtual_seqr)
        seqr1 seqrh1;
        seqr2 seqrh2;

        function new(string name="virtual_seqr", uvm_component parent=null);
                super.new(name,parent);
        endfunction
endclass

class virtual_seq extends uvm_sequence;
          `uvm_object_utils(virtual_seq)
          `uvm_declare_p_sequencer(virtual_seqr)

          seq1 seqh1;
          seq2 seqh2;
        seqr1 seqrh1;
        seqr2 seqrh2;

        function new(string name="virtual_seq");
                super.new(name);
```

```
    endfunction

      task body();
          `uvm_info(get_type_name(),"inside body of : Virtual sequence",UVM_LOW);

              seqh1 = seq1::type_id::create("seqh1");
          seqh2 = seq2::type_id::create("seqh2");

              seqh1.start(p_sequencer.seqrh1);
          seqh2.start(p_sequencer.seqrh2);
      endtask

endclass

class base_driver extends uvm_driver #(seq_item);
      `uvm_component_utils(base_driver)
      seq_item req;

    function new(string name="base_driver", uvm_component parent=null);
          super.new(name,parent);
    endfunction

      function void build_phase(uvm_phase phase);
              super.build_phase(phase);
      endfunction

      task run_phase(uvm_phase phase);
```

```
            forever begin
                seq_item_port.get_next_item(req);

                drive(req);

                seq_item_port.item_done();
            end
        endtask

        virtual task drive(seq_item req);
            `uvm_info(get_type_name(),"Driving from the base driver",UVM_LOW);
            #50;
        endtask
endclass

class driver1 extends base_driver;
        `uvm_component_utils(driver1)

    function new(string name="driver1", uvm_component parent=null);
        super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
    endfunction

    task drive(seq_item req);
```

```
        `uvm_info(get_type_name(),"Driving from the driver1",UVM_LOW);
        #50;
    endtask
endclass

class driver2 extends base_driver;
    `uvm_component_utils(driver2)

    function new(string name="driver2", uvm_component parent=null);
        super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
    endfunction

    task drive(seq_item req);
        `uvm_info(get_type_name(),"Driving from the driver2",UVM_LOW);
        #50;
    endtask
endclass

class agent1 extends uvm_agent;
      `uvm_component_utils(agent1)

      driver1 drvh1;
      seqr1 seqrh1;
```

```
    function new(string name="agent1", uvm_component parent=null);
        super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
            drvh1 = driver1::type_id::create("drvh1",this);
        seqrh1 = seqr1::type_id::create("seqrh1",this);
    endfunction

      function void connect_phase(uvm_phase phase);
            super.connect_phase(phase);

            drvh1.seq_item_port.connect(seqrh1.seq_item_export);
      endfunction
endclass

class agent2 extends uvm_agent;
    `uvm_component_utils(agent2)

    driver2 drvh2;
    seqr2 seqrh2;

    function new(string name="agent2", uvm_component parent=null);
        super.new(name,parent);
    endfunction
```

```
    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        drvh2 = driver2::type_id::create("drvh2",this);
        seqrh2 = seqr2::type_id::create("seqrh2",this);
    endfunction

    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);

        drvh2.seq_item_port.connect(seqrh2.seq_item_export);
    endfunction
endclass

class env extends uvm_env;
      `uvm_component_utils(env)
      agent1 agth1;
      agent2 agth2;
      virtual_seqr vseqrh;

    function new(string name="env", uvm_component parent=null);
        super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        agth1 = agent1::type_id::create("agth1",this);
```

```
        agth2 =agent2::type_id::create("agth2",this);
            vseqrh=virtual_seqr::type_id::create("vseqrh",this);

    endfunction

    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);

            vseqrh.seqrh1 = agth1.seqrh1;
            vseqrh.seqrh2 = agth2.seqrh2;
    endfunction
endclass

class test extends uvm_test;
      `uvm_component_utils(test)
      env envh;
      virtual_seq vseqh;

    function new(string name="test", uvm_component parent=null);
        super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        envh = env::type_id::create("envh",this);
    endfunction
```

```
    task run_phase(uvm_phase phase);

        phase.raise_objection(this);
        vseqh = virtual_seq ::type_id::create("vseqh",this);

        vseqh.start(envh.vseqrh);
        phase.drop_objection(this);
    endtask
endclass

module top;
initial begin
    run_test("test");
end
endmodule
```

Run Command: -

```
[b23n_sarif_727@prac3 youtube]$ vcs -sverilog virt_seqc_seqr.sv -ntb_opts uvm -kdb -lca -debug_access+all -full64
[b23n_sarif_727@prac3 youtube]$ ./simv
```

Simulation Outputs:

```
UVM_INFO @ 0: reporter [RNTST] Running test test...
UVM_INFO virt_seqc_seqr.sv(94) @ 0: uvm_test_top.envh.vseqrh@@vseqh [virtual_seq] inside body of : Virtual sequence
UVM_INFO virt_seqc_seqr.sv(25) @ 0: uvm_test_top.envh.agth1.seqrh1@@seqh1 [seq1] inside body task of : seq1
UVM_INFO virt_seqc_seqr.sv(145) @ 0: uvm_test_top.envh.agth1.drvh1 [driver1] Driving from the driver1
```

UVM_INFO virt_seqc_seqr.sv(44) @ 50: uvm_test_top.envh.agth2.seqrh2@@seqh2 [seq2] inside body task of : seq2

UVM_INFO virt_seqc_seqr.sv(162) @ 50: uvm_test_top.envh.agth2.drvh2 [driver2] Driving from the driver2

UVM_INFO /usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_objection.svh(1274) @ 100: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase

--- UVM Report Summary ---

** Report counts by severity

UVM_INFO : 7

UVM_WARNING : 0

UVM_ERROR : 0

UVM_FATAL : 0

** Report counts by id

[RNTST] 1

[TEST_DONE] 1

[driver1] 1

[driver2] 1

[seq1] 1

[seq2] 1

[virtual_seq] 1

$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_root.svh", line 437.

$finish at simulation time 100

V C S S i m u l a t i o n R e p o r t

Time: 100

CPU Time: 0.220 seconds; Data structure size: 0.6Mb

Thu Jun 25 17:23:17 2026

VYOM SIR- ADDER UVM CODE KITTU PATEL, DUAL PORT RAM, APB PROTOCOL. --- IMPORTANT

1.0 Adder Verification of UVM Code

Command line for Run code

[b23n_sarif_727@prac3 adder]$ ls

adder_design.sv agent.sv dump.fsdb monitor.sv novas.rc sequencer.sv simv.daidir ucli.key

adder_pkg.sv csrc environment.sv novas.conf scoreboard.sv sequence.sv test.sv vc_hdrs.h

add_if.sv driver.sv inter.fsdb novas_dump.log sequence_item.sv simv top.sv verdiLog

[b23n_sarif_727@prac3 adder]$

```
//
module adder(input clk,reset,input [7:0] in1,in2, output reg [8:0] out);

always@(posedge clk or posedge reset) begin
        if(reset)
                out <= 0;
        else
                out <= in1 + in2;
end
endmodule

//
interface add_if(input logic clk, reset);
        logic [7:0] in1;
```

```
        logic [7:0] in2;
        logic [8:0] out;

endinterface

//
class seq_item extends uvm_sequence_item;
        rand bit [7:0] in1,in2;
        bit [8:0] out;

        function new(string name="seq_item");
                super.new(name);
        endfunction

        `uvm_object_utils_begin(seq_item)
        `uvm_field_int(in1,UVM_ALL_ON);
        `uvm_field_int(in2,UVM_ALL_ON);
        `uvm_object_utils_end

        constraint c1 {in1 <100; in2<100;}
endclass

//
class base_seq extends uvm_sequence #(seq_item);
        seq_item req;
        `uvm_object_utils(base_seq)
```

```
        function new(string name="base_seq");
                super.new(name);
        endfunction

        task body();
                `uvm_info(get_type_name(), "Base seq: Inside Body", UVM_LOW);
                `uvm_do(req);
        endtask
endclass

//
class seqcr extends uvm_sequencer #(seq_item);
        `uvm_component_utils(seqcr)

        function new(string name = "seqcr", uvm_component parent = null);
                super.new(name, parent);
        endfunction

        function void build_phase(uvm_phase phase);
                super.build_phase(phase);
        endfunction

endclass

//
class driver extends uvm_driver#(seq_item);
        virtual add_if vif;
```

```
	`uvm_component_utils(driver)

	function new(string name = "driver", uvm_component parent = null);
		super.new(name, parent);
	endfunction

	function void build_phase(uvm_phase phase);
		super.build_phase(phase);
		if(!uvm_config_db#(virtual add_if) :: get(this, "", "vif", vif))
			`uvm_fatal(get_type_name(), "Not set at top level");
	endfunction

	task run_phase (uvm_phase phase);
		forever begin
			seq_item_port.get_next_item(req);
			`uvm_info(get_type_name, $sformatf("in1 = %0d, in2 = %0d", req.in1, req.in2), UVM_LOW);
			vif.in1 <= req.in1;
			vif.in2 <= req.in2;
			seq_item_port.item_done();
		end
	endtask
endclass

//
class monitor extends uvm_monitor;
	virtual add_if vif;
	uvm_analysis_port #(seq_item) mon_ap;
```

```
    seq_item mon_item;
    `uvm_component_utils(monitor)

    function new(string name = "monitor", uvm_component parent = null);
        super.new(name, parent);
        mon_ap = new("mon_ap", this);
        mon_item = new();
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if(!uvm_config_db#(virtual add_if) :: get(this, "", "vif", vif))
            `uvm_fatal(get_type_name(), "Not set at top level");
    endfunction

    task run_phase (uvm_phase phase);
        forever begin
            wait(!vif.reset);
            @(posedge vif.clk);
            mon_item.in1 = vif.in1;
            mon_item.in2 = vif.in2;
            `uvm_info(get_type_name, $sformatf("in1 = %0d, in2 = %0d",
mon_item.in1, mon_item.in2), UVM_HIGH);
            @(posedge vif.clk);
            mon_item.out = vif.out;
            mon_ap.write(mon_item);
        end
    endtask
```

```
endclass

//
class scoreboard extends uvm_scoreboard;
	uvm_analysis_imp #(seq_item, scoreboard) sb_imp;
	seq_item item_q[$];
	`uvm_component_utils(scoreboard)

	function new(string name = "scoreboard", uvm_component parent = null);
		super.new(name, parent);
		sb_imp = new("sb_imp", this);
	endfunction

	function void build_phase(uvm_phase phase);
		super.build_phase(phase);
	endfunction

	function void write(seq_item req);
		if(req.in1 + req.in2 == req.out) begin
			`uvm_info(get_type_name, $sformatf("Matched: in1 = %0d, in2 = %0d, out = %0d", req.in1, req.in2, req.out),UVM_LOW);
		end
		else begin
			`uvm_error(get_name, $sformatf("NOT matched: in1 = %0d, in2 = %0d, out = %0d", req.in1, req.in2, req.out));
		end
	endfunction
endclass
```

```
//
class agent extends uvm_agent;
	`uvm_component_utils(agent)
	driver drv;
	seqcr seqr;
	monitor mon;

	function new(string name = "agent", uvm_component parent = null);
		super.new(name, parent);
	endfunction

	function void build_phase(uvm_phase phase);
		super.build_phase(phase);
		if(get_is_active == UVM_ACTIVE) begin
			drv = driver::type_id::create("drv", this);
			seqr = seqcr::type_id::create("seqr", this);
		end
		mon = monitor::type_id::create("mon", this);
	endfunction

	function void connect_phase(uvm_phase phase);
		if(get_is_active == UVM_ACTIVE) begin
			drv.seq_item_port.connect(seqr.seq_item_export);
		end
	endfunction
endclass
```

```
//
class env extends uvm_env;
	`uvm_component_utils(env)
	agent agt;
	scoreboard sb;

	function new(string name = "env", uvm_component parent = null);
		super.new(name, parent);
	endfunction

	function void build_phase(uvm_phase phase);
		super.build_phase(phase);
		agt = agent::type_id::create("agt", this);
		sb = scoreboard::type_id::create("sb", this);
	endfunction

	function void connect_phase(uvm_phase phase);
		agt.mon.mon_ap.connect(sb.sb_imp);
	endfunction
endclass

//
class base_test extends uvm_test;
	env env_o;
	base_seq bseq;
	`uvm_component_utils(base_test)
```

```
    function new(string name = "base_test", uvm_component parent = null);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        env_o = env::type_id::create("env_o", this);
    endfunction

    task run_phase(uvm_phase phase);
        phase.raise_objection(this);
        bseq = base_seq::type_id::create("bseq");
        repeat(10) begin
            #5; bseq.start(env_o.agt.seqr);
        end

        phase.drop_objection(this);
        `uvm_info(get_type_name, "End of testcase", UVM_LOW);
    endtask
endclass

//
`include "uvm_macros.svh"
import uvm_pkg::*;
import adder_pkg::*;

interface add_if(input logic clk, reset);
```

```
        logic [7:0] in1;
        logic [7:0] in2;
        logic [8:0] out;

endinterface

module tb_top;
bit clk;
bit reset;

always #2 clk = ~clk;

initial begin
        reset = 1;
        #5;
        reset = 0;
end

add_if vif(clk, reset);

adder DUT(.clk(vif.clk),.reset(vif.reset),.in1(vif.in1),.in2(vif.in2),.out(vif.out));

initial begin
        uvm_config_db#(virtual add_if)::set(null, "*", "vif", vif);
end

initial begin
```

```
        run_test("base_test");
end

initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,tb_top);
end
endmodule

//
package adder_pkg;

`include "uvm_macros.svh"
import uvm_pkg::*;
`include "sequence_item.sv";
`include "sequence.sv";
`include "sequencer.sv";
`include "driver.sv";
`include "monitor.sv";
`include "agent.sv";
`include "scoreboard.sv";
`include "environment.sv";
`include "test.sv";
endpackage
```

Run-Command

[b23n_sarif_727@prac3 adder]$ vcs -sverilog adder_design.sv adder_pkg.sv top.sv -ntb_opts uvm -kdb -lca -debug_access+all -full64

[b23n_sarif_727@prac3 adder]$ ./simv

[b23n_sarif_727@prac3 adder]$ ./simv -verdi

Simulation Outputs

UVM_INFO sequence.sv(11) @ 5: uvm_test_top.env_o.agt.seqr@@bseq [base_seq] Base seq: Inside Body

UVM_INFO driver.sv(19) @ 5: uvm_test_top.env_o.agt.drv [driver] in1 = 5, in2 = 45

UVM_INFO scoreboard.sv(18) @ 10: uvm_test_top.env_o.sb [scoreboard] Matched: in1 = 5, in2 = 45, out = 50

UVM_INFO sequence.sv(11) @ 10: uvm_test_top.env_o.agt.seqr@@bseq [base_seq] Base seq: Inside Body

UVM_INFO driver.sv(19) @ 10: uvm_test_top.env_o.agt.drv [driver] in1 = 99, in2 = 71

UVM_INFO sequence.sv(11) @ 15: uvm_test_top.env_o.agt.seqr@@bseq [base_seq] Base seq: Inside Body

UVM_INFO driver.sv(19) @ 15: uvm_test_top.env_o.agt.drv [driver] in1 = 20, in2 = 5

UVM_INFO scoreboard.sv(18) @ 18: uvm_test_top.env_o.sb [scoreboard] Matched: in1 = 99, in2 = 71, out = 170

UVM_INFO sequence.sv(11) @ 20: uvm_test_top.env_o.agt.seqr@@bseq [base_seq] Base seq: Inside Body

UVM_INFO driver.sv(19) @ 20: uvm_test_top.env_o.agt.drv [driver] in1 = 58, in2 = 99

UVM_INFO sequence.sv(11) @ 25: uvm_test_top.env_o.agt.seqr@@bseq [base_seq] Base seq: Inside Body

UVM_INFO driver.sv(19) @ 25: uvm_test_top.env_o.agt.drv [driver] in1 = 37, in2 = 74

UVM_INFO scoreboard.sv(18) @ 26: uvm_test_top.env_o.sb [scoreboard] Matched: in1 = 58, in2 = 99, out = 157

UVM_INFO sequence.sv(11) @ 30: uvm_test_top.env_o.agt.seqr@@bseq [base_seq] Base seq: Inside Body

UVM_INFO driver.sv(19) @ 30: uvm_test_top.env_o.agt.drv [driver] in1 = 69, in2 = 78

UVM_INFO scoreboard.sv(18) @ 34: uvm_test_top.env_o.sb [scoreboard] Matched: in1 = 37, in2 = 74, out = 111

UVM_INFO sequence.sv(11) @ 35: uvm_test_top.env_o.agt.seqr@@bseq [base_seq] Base seq: Inside Body

UVM_INFO driver.sv(19) @ 35: uvm_test_top.env_o.agt.drv [driver] in1 = 72, in2 = 11

UVM_INFO sequence.sv(11) @ 40: uvm_test_top.env_o.agt.seqr@@bseq [base_seq] Base seq: Inside Body

UVM_INFO driver.sv(19) @ 40: uvm_test_top.env_o.agt.drv [driver] in1 = 79, in2 = 27

UVM_INFO scoreboard.sv(18) @ 42: uvm_test_top.env_o.sb [scoreboard] Matched: in1 = 72, in2 = 11, out = 83

UVM_INFO sequence.sv(11) @ 45: uvm_test_top.env_o.agt.seqr@@bseq [base_seq] Base seq: Inside Body

UVM_INFO driver.sv(19) @ 45: uvm_test_top.env_o.agt.drv [driver] in1 = 47, in2 = 82

UVM_INFO scoreboard.sv(18) @ 50: uvm_test_top.env_o.sb [scoreboard] Matched: in1 = 47, in2 = 82, out = 129

UVM_INFO sequence.sv(11) @ 50: uvm_test_top.env_o.agt.seqr@@bseq [base_seq] Base seq: Inside Body

UVM_INFO driver.sv(19) @ 50: uvm_test_top.env_o.agt.drv [driver] in1 = 65, in2 = 95

UVM_INFO test.sv(24) @ 50: uvm_test_top [base_test] End of testcase

UVM_INFO /usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_objection.svh(1274) @ 50: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase

--- UVM Report Summary ---

** Report counts by severity

UVM_INFO :   29

UVM_WARNING :   0

UVM_ERROR :   0

UVM_FATAL :   0

** Report counts by id

[RNTST]   1

[TEST_DONE]   1

[base_seq] 10

[base_test] 1

[driver] 10

[scoreboard] 6

$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_root.svh", line 437.

$finish at simulation time 50

V C S S i m u l a t i o n R e p o r t

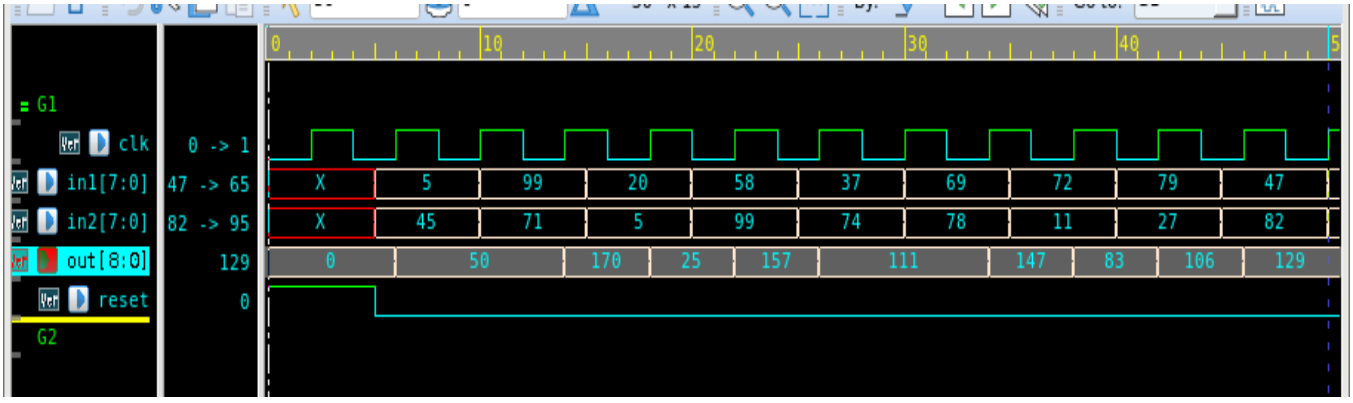
Time: 50

CPU Time: 0.150 seconds; Data structure size: 0.5Mb

Mon Aug 17 17:32:06 2026

[b23n_sarif_727@prac3 adder]$ vi top.sv

Simulation waveform



APB Protocol Verification using UVM

Command line for Run code

[b23n_sarif_727@prac3 apb_protocol]$ ls

agent.sv apb_pkg.sv driver.sv monitor.sv novas.rc sequencer.sv simv.cst test.sv verdiLog

apb_dump.fsdb apb_slave.sv environment.sv novas.conf scoreboard.sv sequence.sv simv.daidir ucli.key

apb_if.sv csrc inter.fsdb novas_dump.log sequence_item.sv simv tb_top.sv vc_hdrs.h

Design Code

```
//
module apb_slave #(parameter integer WAIT_CYCLES = 2)(
        input logic       pclk,
      input logic       rst_n,
      input logic [31:0] paddr,
      input logic       psel,
      input logic       penable,
      input logic       pwrite,
      input logic [31:0] pwdata,
        output logic [31:0]prdata,
      output logic      pready,
      output logic      pslverr
);

logic [31:0] mem [0:255];
typedef enum logic [1:0] { SETUP = 2'b00, W_ENABLE = 2'b01, R_ENABLE = 2'b10 } state_t;
state_t apb_st;
integer wait_count;

always_ff @(posedge pclk or negedge rst_n) begin
        if (!rst_n) begin
              apb_st    <= SETUP;
              prdata    <= 32'd0;
           pready    <= 1'b1;
```

```
    pslverr    <= 1'b0;
    wait_count <= 0;

        for (int i = 0; i < 256; i++)
                mem[i] <= i;
end

else begin
        case (apb_st)
                SETUP:
                begin
                        pready  <= 1'b0;
                        pslverr <= 1'b0;
                        prdata  <= 32'd0;

                        if (psel && !penable) begin
                                if (paddr > 32'd255) begin
                                        pslverr <= 1'b1;
                                        pready  <= 1'b1;
                                        apb_st <= SETUP;
                                end
                                else begin
                                        wait_count <= WAIT_CYCLES;
                                        if (pwrite)
                                                apb_st <= W_ENABLE;
                                        else
                                                apb_st <= R_ENABLE;
```

```
            end
        end
    end
    W_ENABLE:
    begin
        if (psel && penable && pwrite) begin
            if (wait_count > 0) begin
                pready     <= 1'b0;
                wait_count <= wait_count - 1;
            end
            else begin
                pready <= 1'b1;
                if (paddr <= 32'd255)
                    mem[paddr] <= pwdata;
                apb_st <= SETUP;
            end
        end
    end
    R_ENABLE:
    begin
        if (psel && penable && !pwrite) begin
            if (wait_count > 0) begin
                pready     <= 1'b0;
                wait_count <= wait_count - 1;
            end
            else begin
                pready <= 1'b1;
```

```
                                        if (paddr <= 32'd255)
                                                prdata <= mem[paddr];
                                        apb_st <= SETUP;
                                end
                        end
                end
                default:
                begin
                        apb_st     <= SETUP;
                        pready     <= 1'b1;
                        pslverr    <= 1'b0;
                        prdata     <= 32'd0;
                        wait_count <= 0;
                end
        endcase
    end
end
endmodule

//
class apb_seq_item extends uvm_sequence_item;
    rand bit [31:0] paddr;
    rand bit        pwrite;
    rand bit [31:0] pwdata;
    bit [31:0] prdata;
   bit        pready;
   bit        pslverr;
```

```
  `uvm_object_utils_begin(apb_seq_item)
  `uvm_field_int(paddr,   UVM_ALL_ON)
 `uvm_field_int(pwrite,  UVM_ALL_ON)
 `uvm_field_int(pwdata,  UVM_ALL_ON)
 `uvm_field_int(prdata,  UVM_ALL_ON)
 `uvm_field_int(pready,  UVM_ALL_ON)
 `uvm_field_int(pslverr, UVM_ALL_ON)
  `uvm_object_utils_end

  function new(string name = "apb_seq_item");
        super.new(name);
  endfunction

endclass

//
class apb_base_seq extends uvm_sequence #(apb_seq_item);
  apb_seq_item req;
  `uvm_object_utils(apb_base_seq)

  function new(string name = "apb_base_seq");
        super.new(name);
  endfunction

  task body();
        repeat(10) begin
```

```
                `uvm_do_with(req, { pwrite == 1; paddr inside {[0:255]}; })
        end

        repeat(10) begin
                `uvm_do_with(req, { pwrite == 0; paddr inside {[0:255]}; })
        end

        `uvm_do_with(req, { pwrite == 0; paddr > 255; })
    endtask
endclass

//
class apb_seqcr extends uvm_sequencer #(apb_seq_item);
    `uvm_component_utils(apb_seqcr)

    function new(string name = "apb_seqcr", uvm_component parent = null);
        super.new(name, parent);
    endfunction

endclass

//
class apb_driver extends uvm_driver #(apb_seq_item);
    virtual apb_if vif;
    `uvm_component_utils(apb_driver)

    function new(string name = "apb_driver", uvm_component parent = null);
```

```
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if (!uvm_config_db#(virtual apb_if)::get(this, "", "vif", vif))
            `uvm_fatal(get_type_name(), "Virtual interface not found")
    endfunction

    task run_phase(uvm_phase phase);
        forever begin
            seq_item_port.get_next_item(req);
            drive_to_dut(req);
            seq_item_port.item_done();
        end
    endtask

    task drive_to_dut(apb_seq_item req);
        @(posedge vif.pclk);
        vif.psel    <= 1'b1;
        vif.penable <= 1'b0;
        vif.paddr   <= req.paddr;
        vif.pwrite  <= req.pwrite;
        vif.pwdata  <= req.pwdata;
        `uvm_info(get_type_name(), $sformatf("SETUP: addr=%0d write=%0d
data=%0h",req.paddr,req.pwrite,req.pwdata),UVM_LOW)

        @(posedge vif.pclk);
```

```
            vif.penable <= 1'b1;

        while (!vif.pready) begin
            @(posedge vif.pclk);
            `uvm_info(get_type_name(), "Waiting for PREADY...", UVM_HIGH)
        end

      req.prdata  = vif.prdata;
      req.pready  = vif.pready;
      req.pslverr = vif.pslverr;
        `uvm_info(get_type_name(),$sformatf("ACCESS DONE: addr=%0d pwrite=%0d
prdata=%0h PSLVERR=%0d",req.paddr,req.pwrite,req.prdata,req.pslverr),UVM_LOW)

        @(posedge vif.pclk);
      vif.psel    <= 1'b0;
      vif.penable <= 1'b0;
      vif.paddr   <= 32'd0;
      vif.pwdata  <= 32'd0;
endtask
endclass

//
class apb_monitor extends uvm_monitor;
        virtual apb_if vif;
        uvm_analysis_port #(apb_seq_item) mon_ap;
        `uvm_component_utils(apb_monitor)

        function new(string name = "apb_monitor", uvm_component parent = null);
```

```
        super.new(name, parent);
        mon_ap = new("mon_ap", this);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if (!uvm_config_db#(virtual apb_if)::get(this, "", "vif", vif))
            `uvm_fatal(get_type_name(),"Virtual interface not found")
    endfunction

    task run_phase(uvm_phase phase);
        apb_seq_item mon_item;
        forever begin
            @(posedge vif.pclk);
            if (vif.psel && vif.penable && vif.pready) begin
                mon_item = apb_seq_item::type_id::create("mon_item");

                mon_item.paddr   = vif.paddr;
                mon_item.pwrite  = vif.pwrite;
                mon_item.pwdata  = vif.pwdata;
                mon_item.prdata  = vif.prdata;
                mon_item.pready  = vif.pready;
                mon_item.pslverr = vif.pslverr;
                `uvm_info(get_type_name(),$sformatf("MONITOR: addr=%0d write=%0d pwdata=%0h prdata=%0h PSLVERR=%0d",mon_item.paddr,mon_item.pwrite,mon_item.pwdata,mon_item.prdata, mon_item.pslverr),UVM_HIGH)
                mon_ap.write(mon_item);
```

```
                end
            end
        endtask
endclass

//
class apb_scoreboard extends uvm_scoreboard;
        uvm_analysis_imp #(apb_seq_item, apb_scoreboard) sb_imp;
        bit [31:0] ref_mem [0:255];
        `uvm_component_utils(apb_scoreboard)

        function new(string name = "apb_scoreboard", uvm_component parent = null);
                super.new(name, parent);
                sb_imp = new("sb_imp", this);
        endfunction

        function void build_phase(uvm_phase phase);
                super.build_phase(phase);

                for (int i = 0; i < 256; i++)
                        ref_mem[i] = i;
        endfunction

        function void write(apb_seq_item req);
                if (req.paddr > 255) begin
                        if (req.pslverr == 1'b1) begin
                                `uvm_info(get_type_name(),$sformatf("INVALID ADDRESS
PASS: addr=%0d PSLVERR=%0d",req.paddr,req.pslverr),UVM_LOW)
```

```
                end
                else begin
                        `uvm_error(get_type_name(),$sformatf("INVALID ADDRESS FAIL: addr=%0d PSLVERR=%0d",req.paddr,req.pslverr))
                end
                return;
            end

            if (req.pwrite) begin
                ref_mem[req.paddr] = req.pwdata;
                `uvm_info(get_type_name(),$sformatf("WRITE PASS: addr=%0d data=%0h",req.paddr,req.pwdata),UVM_LOW)
            end
            else begin
                if (req.prdata == ref_mem[req.paddr]) begin
                    `uvm_info(get_type_name(),$sformatf("READ PASS: addr=%0d expected=%0h actual=%0h",req.paddr,ref_mem[req.paddr],req.prdata),UVM_LOW)
                end
                else begin
                    `uvm_error(get_type_name(),$sformatf("READ FAIL: addr=%0d expected=%0h actual=%0h",req.paddr,ref_mem[req.paddr],req.prdata))
                end
            end
        endfunction

endclass

//
```

```
class apb_agent extends uvm_agent;
	`uvm_component_utils(apb_agent)
	apb_driver drv;
	apb_seqcr  seqr;
	apb_monitor mon;

	function new(string name = "apb_agent", uvm_component parent = null);
		super.new(name, parent);
	endfunction

	function void build_phase(uvm_phase phase);
		super.build_phase(phase);
		mon = apb_monitor::type_id::create("mon", this);
		if (get_is_active() == UVM_ACTIVE) begin
			drv = apb_driver::type_id::create("drv", this);
			seqr = apb_seqcr::type_id::create("seqr", this);
		end
	endfunction

	function void connect_phase(uvm_phase phase);
		super.connect_phase(phase);
		if (get_is_active() == UVM_ACTIVE) begin
			drv.seq_item_port.connect(seqr.seq_item_export);
		end
	endfunction

endclass
```

```
//
class apb_env extends uvm_env;
	`uvm_component_utils(apb_env)
	apb_agent	agt;
	apb_scoreboard sb;

	function new(string name = "apb_env", uvm_component parent = null);
		super.new(name, parent);
	endfunction

	function void build_phase(uvm_phase phase);
		super.build_phase(phase);
		agt = apb_agent::type_id::create("agt", this);
		sb = apb_scoreboard::type_id::create("sb", this);
	endfunction

	function void connect_phase(uvm_phase phase);
		super.connect_phase(phase);
		agt.mon.mon_ap.connect(sb.sb_imp);
	endfunction

endclass

//
class apb_base_test extends uvm_test;
	apb_env	env_o;
```

```
    apb_base_seq bseq;
    `uvm_component_utils(apb_base_test)

    function new(string name = "apb_base_test", uvm_component parent = null);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        env_o = apb_env::type_id::create("env_o", this);
    endfunction

    task run_phase(uvm_phase phase);
        phase.raise_objection(this);
        bseq = apb_base_seq::type_id::create("bseq");
        bseq.start(env_o.agt.seqr);
        #50;
        phase.drop_objection(this);

        `uvm_info(get_type_name(),"END OF TESTCASE",UVM_LOW)
    endtask

endclass

//
`include "uvm_macros.svh"
import uvm_pkg::*;
```

```
import apb_pkg::*;

module tb_top;

bit pclk;
always #5 pclk = ~pclk;

bit rst_n;

initial begin
	rst_n = 1'b0;
	#20;
	rst_n = 1'b1;
end

apb_if vif(.pclk (pclk),.rst_n (rst_n));

apb_slave #(.WAIT_CYCLES(2)) DUT (.pclk (vif.pclk),.rst_n (vif.rst_n),.paddr (vif.paddr),.psel (vif.psel),.penable (vif.penable),.pwrite (vif.pwrite),.pwdata (vif.pwdata),.prdata (vif.prdata),.pready (vif.pready),.pslverr (vif.pslverr));

initial begin
	uvm_config_db#(virtual apb_if)::set( null,"*","vif",vif);
end
initial begin
	run_test("apb_base_test");
end
initial begin
```

```
        $fsdbDumpfile("apb_dump.fsdb");
      $fsdbDumpvars(0, tb_top);
end

endmodule

//
interface apb_if(input logic pclk, rst_n);
        logic [31:0] paddr;
        logic        psel;
      logic        penable;
      logic        pwrite;
      logic [31:0] pwdata;

      logic [31:0] prdata;
      logic        pready;
      logic        pslverr;

endinterface
//
package apb_pkg;
         `include "uvm_macros.svh"
         import uvm_pkg::*;
         `include "sequence_item.sv"
      `include "sequence.sv"
      `include "sequencer.sv"
      `include "driver.sv"
```

```
    `include "monitor.sv"
    `include "scoreboard.sv"
    `include "agent.sv"
    `include "environment.sv"
    `include "test.sv"
endpackage
//
class apb_coverage extends uvm_subscriber #(apb_seq_item);
      `uvm_component_utils(apb_coverage)

      bit       cov_pwrite;
    bit [31:0] cov_paddr;
    bit [31:0] cov_pwdata;
    bit [31:0] cov_prdata;
    bit       cov_pready;
    bit       cov_pslverr;

      covergroup apb_cg;

    cp_operation:
       coverpoint cov_pwrite {
          bins READ  = {0};
          bins WRITE = {1};
       }

    cp_address:
       coverpoint cov_paddr {
```

```
        bins LOW_ADDR  = {[0:63]};
        bins MID_ADDR  = {[64:191]};
        bins HIGH_ADDR = {[192:255]};
        bins INVALID   = {[256:$]};
    }

cp_boundary_address:
    coverpoint cov_paddr {
        bins ADDR_0   = {0};
        bins ADDR_1   = {1};
        bins ADDR_254 = {254};
        bins ADDR_255 = {255};
        bins INVALID  = {[256:$]};
    }

cp_pwdata:
    coverpoint cov_pwdata {
        bins ZERO      = {32'h00000000};
        bins MAX       = {32'hFFFFFFFF};
        bins ALL_A     = {32'hAAAAAAAA};
        bins ALL_5     = {32'h55555555};
        bins LOW_VALUE = {[1:255]};
        bins OTHER     = default;
    }

cp_prdata:
    coverpoint cov_prdata {
```

```
        bins ZERO      = {32'h00000000};
        bins MAX       = {32'hFFFFFFFF};
        bins LOW_VALUE = {[1:255]};
        bins OTHER     = default;
    }

cp_pready:
    coverpoint cov_pready {
        bins READY    = {1};
        bins NOT_READY = {0};
    }

cp_pslverr:
    coverpoint cov_pslverr {
        bins NO_ERROR = {0};
        bins ERROR    = {1};
    }

cross_operation_address:
    cross cp_operation, cp_address;

cross_operation_boundary:
    cross cp_operation, cp_boundary_address;

cross_operation_data:
    cross cp_operation, cp_pwdata;
```

```
    cross_operation_pready:
        cross cp_operation, cp_pready;

    cross_operation_error:
        cross cp_operation, cp_pslverr;

    cross_address_error:
        cross cp_address, cp_pslverr;

    cross_address_pready:
        cross cp_address, cp_pready;

    cross_operation_address_error:
        cross cp_operation, cp_address, cp_pslverr;

    cross_operation_address_pready:
        cross cp_operation, cp_address, cp_pready;

    cross_write_addr_data:
        cross cp_operation, cp_address, cp_pwdata;

endgroup

function new(string name = "apb_coverage", uvm_component parent = null);
          super.new(name, parent);
          apb_cg = new();
endfunction
```

```
    function void write(apb_seq_item t);
            cov_pwrite  = t.pwrite;
          cov_paddr   = t.paddr;
          cov_pwdata  = t.pwdata;
          cov_prdata  = t.prdata;
          cov_pready  = t.pready;
          cov_pslverr = t.pslverr;

            apb_cg.sample();
    endfunction
endclass
```

Run-Command

[b23n_sarif_727@prac3 apb_protocol]$ vcs -sverilog apb_slave.sv apb_if.sv apb_pkg.sv tb_top.sv -ntb_opts uvm -kdb -lca -debug_access+all -full64

[b23n_sarif_727@prac3 apb_protocol]$ ./simv

[b23n_sarif_727@prac3 apb_protocol]$ ./simv -verdi

Simulation Outputs

UVM_INFO driver.sv(31) @ 5: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=86 write=1 data=3e08d426

UVM_INFO driver.sv(44) @ 15: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=86 pwrite=1 prdata=0 PSLVERR=0

UVM_INFO scoreboard.sv(32) @ 25: uvm_test_top.env_o.sb [apb_scoreboard] WRITE PASS: addr=86 data=3e08d426

UVM_INFO driver.sv(31) @ 35: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=240 write=1 data=c6d1d15e

UVM_INFO scoreboard.sv(32) @ 85: uvm_test_top.env_o.sb [apb_scoreboard] WRITE PASS: addr=240 data=c6d1d15e

UVM_INFO driver.sv(44) @ 85: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=240 pwrite=1 prdata=0 PSLVERR=0

UVM_INFO driver.sv(31) @ 105: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=73 write=1 data=f1d20d11

UVM_INFO scoreboard.sv(32) @ 155: uvm_test_top.env_o.sb [apb_scoreboard] WRITE PASS: addr=73 data=f1d20d11

UVM_INFO driver.sv(44) @ 155: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=73 pwrite=1 prdata=0 PSLVERR=0

UVM_INFO driver.sv(31) @ 175: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=191 write=1 data=8dc5a05a

UVM_INFO scoreboard.sv(32) @ 225: uvm_test_top.env_o.sb [apb_scoreboard] WRITE PASS: addr=191 data=8dc5a05a

UVM_INFO driver.sv(44) @ 225: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=191 pwrite=1 prdata=0 PSLVERR=0

UVM_INFO driver.sv(31) @ 245: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=116 write=1 data=5e8b780d

UVM_INFO scoreboard.sv(32) @ 295: uvm_test_top.env_o.sb [apb_scoreboard] WRITE PASS: addr=116 data=5e8b780d

UVM_INFO driver.sv(44) @ 295: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=116 pwrite=1 prdata=0 PSLVERR=0

UVM_INFO driver.sv(31) @ 315: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=13 write=1 data=5c213a1d

UVM_INFO scoreboard.sv(32) @ 365: uvm_test_top.env_o.sb [apb_scoreboard] WRITE PASS: addr=13 data=5c213a1d

UVM_INFO driver.sv(44) @ 365: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=13 pwrite=1 prdata=0 PSLVERR=0

UVM_INFO driver.sv(31) @ 385: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=211 write=1 data=1225fc42

UVM_INFO scoreboard.sv(32) @ 435: uvm_test_top.env_o.sb [apb_scoreboard] WRITE PASS: addr=211 data=1225fc42

UVM_INFO driver.sv(44) @ 435: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=211 pwrite=1 prdata=0 PSLVERR=0

UVM_INFO driver.sv(31) @ 455: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=83 write=1 data=e959b00e

UVM_INFO scoreboard.sv(32) @ 505: uvm_test_top.env_o.sb [apb_scoreboard] WRITE PASS: addr=83 data=e959b00e

UVM_INFO driver.sv(44) @ 505: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=83 pwrite=1 prdata=0 PSLVERR=0

UVM_INFO driver.sv(31) @ 525: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=58 write=1 data=93e4ece1

UVM_INFO scoreboard.sv(32) @ 575: uvm_test_top.env_o.sb [apb_scoreboard] WRITE PASS: addr=58 data=93e4ece1

UVM_INFO driver.sv(44) @ 575: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=58 pwrite=1 prdata=0 PSLVERR=0

UVM_INFO driver.sv(31) @ 595: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=71 write=1 data=abb0b331

UVM_INFO scoreboard.sv(32) @ 645: uvm_test_top.env_o.sb [apb_scoreboard] WRITE PASS: addr=71 data=abb0b331

UVM_INFO driver.sv(44) @ 645: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=71 pwrite=1 prdata=0 PSLVERR=0

UVM_INFO driver.sv(31) @ 665: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=148 write=0 data=4afd08d5

UVM_INFO scoreboard.sv(36) @ 715: uvm_test_top.env_o.sb [apb_scoreboard] READ PASS: addr=148 expected=94 actual=94

UVM_INFO driver.sv(44) @ 715: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=148 pwrite=0 prdata=94 PSLVERR=0

UVM_INFO driver.sv(31) @ 735: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=54 write=0 data=4a272a28

UVM_INFO scoreboard.sv(36) @ 785: uvm_test_top.env_o.sb [apb_scoreboard] READ PASS: addr=54 expected=36 actual=36

UVM_INFO driver.sv(44) @ 785: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=54 pwrite=0 prdata=36 PSLVERR=0

UVM_INFO driver.sv(31) @ 805: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=61 write=0 data=3d481d16

UVM_INFO scoreboard.sv(36) @ 855: uvm_test_top.env_o.sb [apb_scoreboard] READ PASS: addr=61 expected=3d actual=3d

UVM_INFO driver.sv(44) @ 855: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=61 pwrite=0 prdata=3d PSLVERR=0

UVM_INFO driver.sv(31) @ 875: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=10 write=0 data=29a690a2

UVM_INFO scoreboard.sv(36) @ 925: uvm_test_top.env_o.sb [apb_scoreboard] READ PASS: addr=10 expected=a actual=a

UVM_INFO driver.sv(44) @ 925: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=10 pwrite=0 prdata=a PSLVERR=0

UVM_INFO driver.sv(31) @ 945: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=26 write=0 data=350e24b6

UVM_INFO scoreboard.sv(36) @ 995: uvm_test_top.env_o.sb [apb_scoreboard] READ PASS: addr=26 expected=1a actual=1a

UVM_INFO driver.sv(44) @ 995: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=26 pwrite=0 prdata=1a PSLVERR=0

UVM_INFO driver.sv(31) @ 1015: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=90 write=0 data=9d117889

UVM_INFO scoreboard.sv(36) @ 1065: uvm_test_top.env_o.sb [apb_scoreboard] READ PASS: addr=90 expected=5a actual=5a

UVM_INFO driver.sv(44) @ 1065: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=90 pwrite=0 prdata=5a PSLVERR=0

UVM_INFO driver.sv(31) @ 1085: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=35 write=0 data=4e2ce6a9

UVM_INFO scoreboard.sv(36) @ 1135: uvm_test_top.env_o.sb [apb_scoreboard] READ PASS: addr=35 expected=23 actual=23

UVM_INFO driver.sv(44) @ 1135: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=35 pwrite=0 prdata=23 PSLVERR=0

UVM_INFO driver.sv(31) @ 1155: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=166 write=0 data=da704690

UVM_INFO scoreboard.sv(36) @ 1205: uvm_test_top.env_o.sb [apb_scoreboard] READ PASS: addr=166 expected=a6 actual=a6

UVM_INFO driver.sv(44) @ 1205: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=166 pwrite=0 prdata=a6 PSLVERR=0

UVM_INFO driver.sv(31) @ 1225: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=37 write=0 data=fb4bb2e7

UVM_INFO scoreboard.sv(36) @ 1275: uvm_test_top.env_o.sb [apb_scoreboard] READ PASS: addr=37 expected=25 actual=25

UVM_INFO driver.sv(44) @ 1275: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=37 pwrite=0 prdata=25 PSLVERR=0

UVM_INFO driver.sv(31) @ 1295: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=115 write=0 data=6fd565b6

UVM_INFO scoreboard.sv(36) @ 1345: uvm_test_top.env_o.sb [apb_scoreboard] READ PASS: addr=115 expected=73 actual=73

UVM_INFO driver.sv(44) @ 1345: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=115 pwrite=0 prdata=73 PSLVERR=0

UVM_INFO driver.sv(31) @ 1365: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=1060706736 write=0 data=bea2638

UVM_INFO scoreboard.sv(22) @ 1385: uvm_test_top.env_o.sb [apb_scoreboard] INVALID ADDRESS PASS: addr=1060706736 PSLVERR=1

UVM_INFO driver.sv(44) @ 1385: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=1060706736 pwrite=0 prdata=0 PSLVERR=1

UVM_INFO test.sv(23) @ 1445: uvm_test_top [apb_base_test] END OF TESTCASE

UVM_INFO /usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_objection.svh(1274) @ 1445: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase

--- UVM Report Summary ---

** Report counts by severity

UVM_INFO :   66

UVM_WARNING :   0

UVM_ERROR :   0

UVM_FATAL :   0

** Report counts by id

[RNTST]   1

[TEST_DONE] 1

[apb_base_test] 1

[apb_driver] 42

[apb_scoreboard] 21

$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_root.svh", line 437.

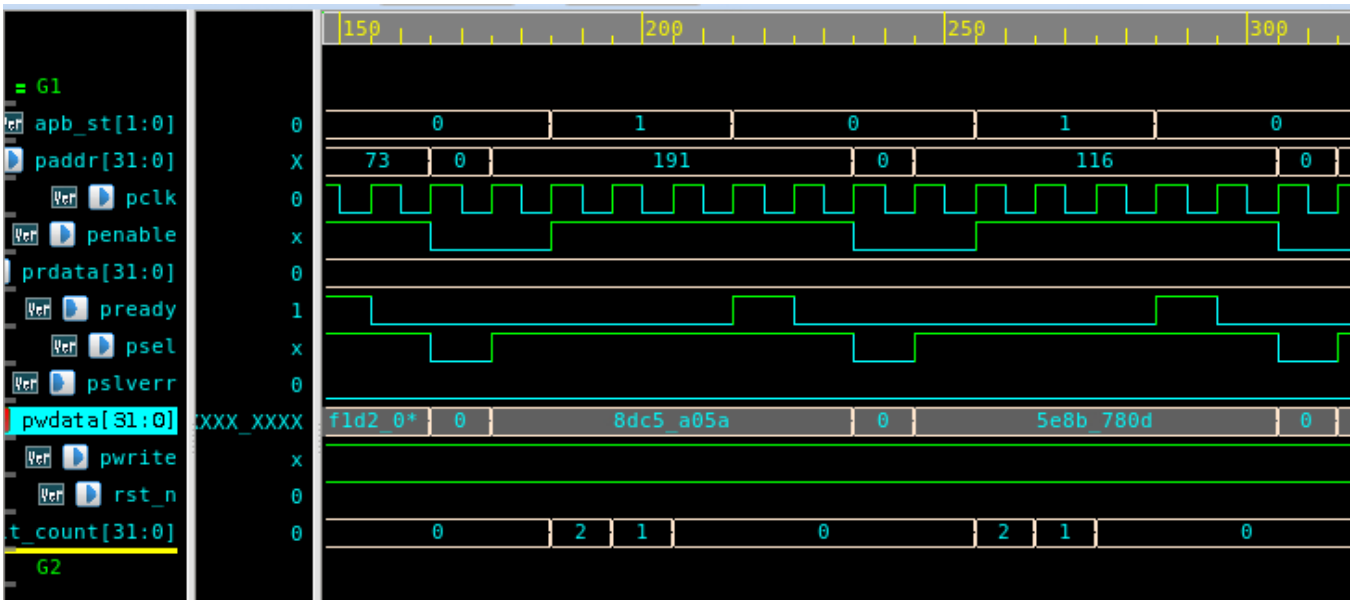
$finish at simulation time 1445

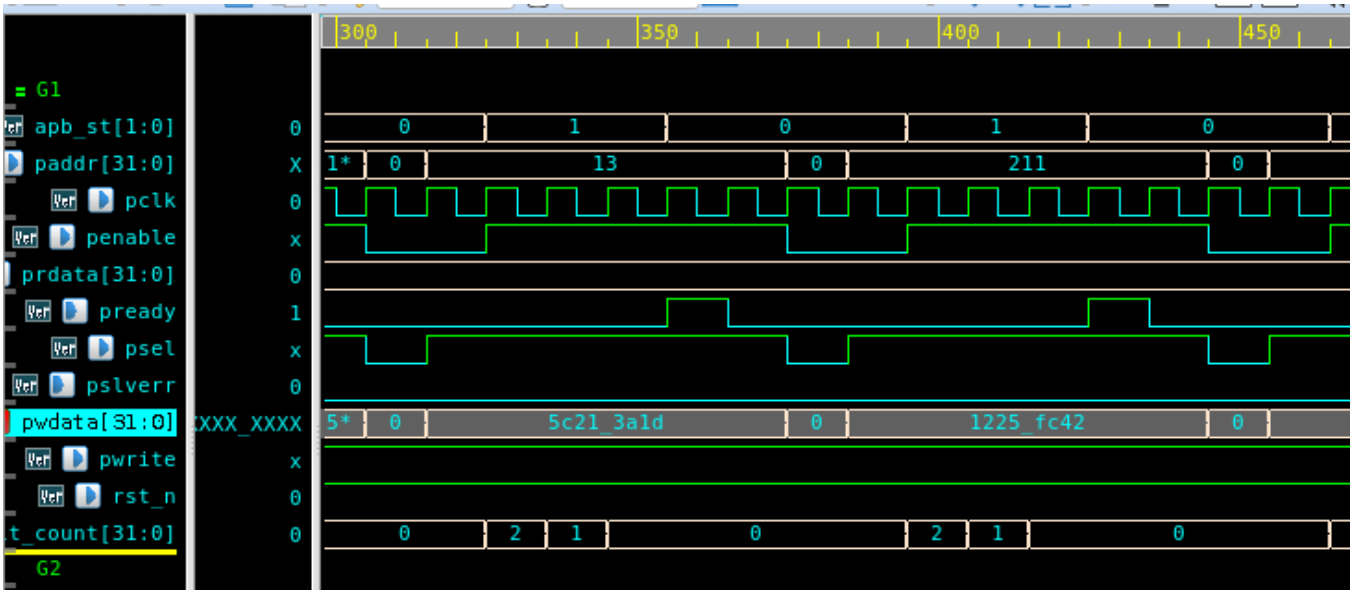
V C S S i m u l a t i o n R e p o r t

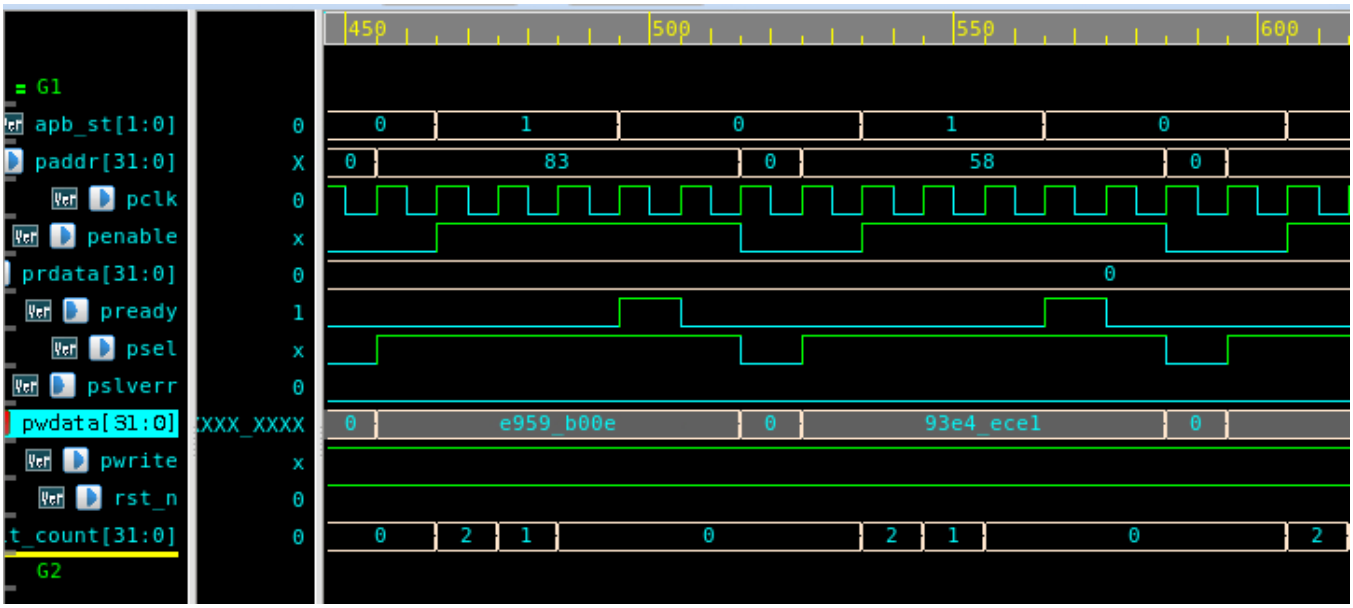
Time: 1445

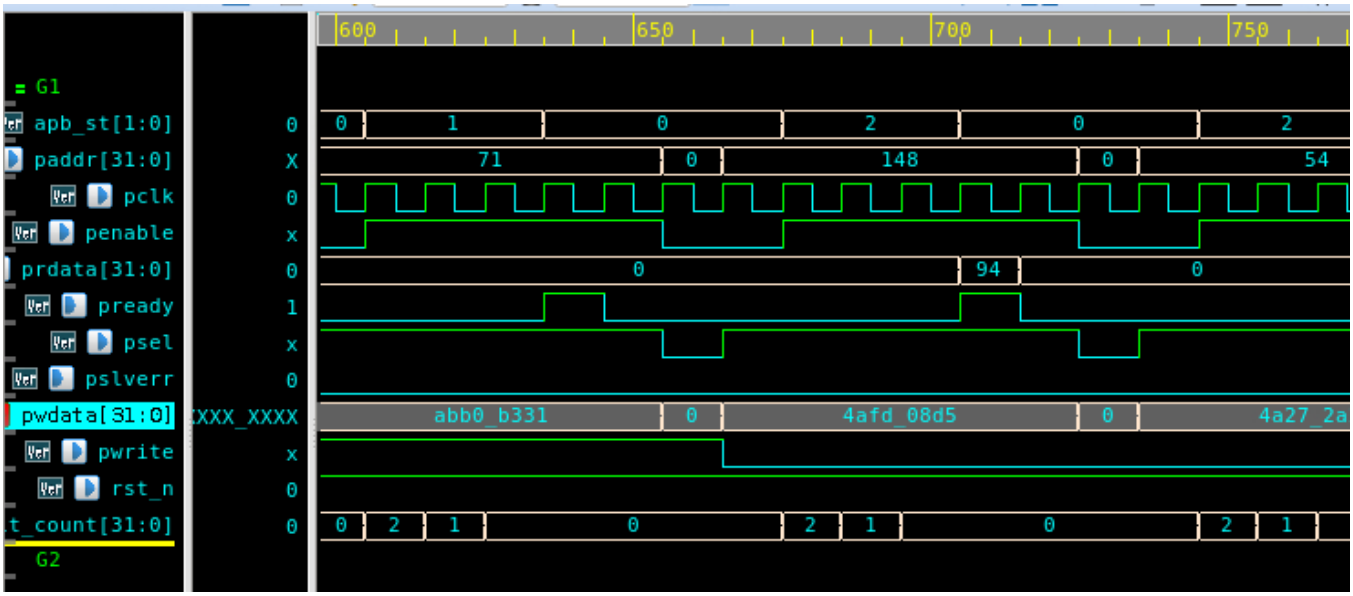
Simulation waveform

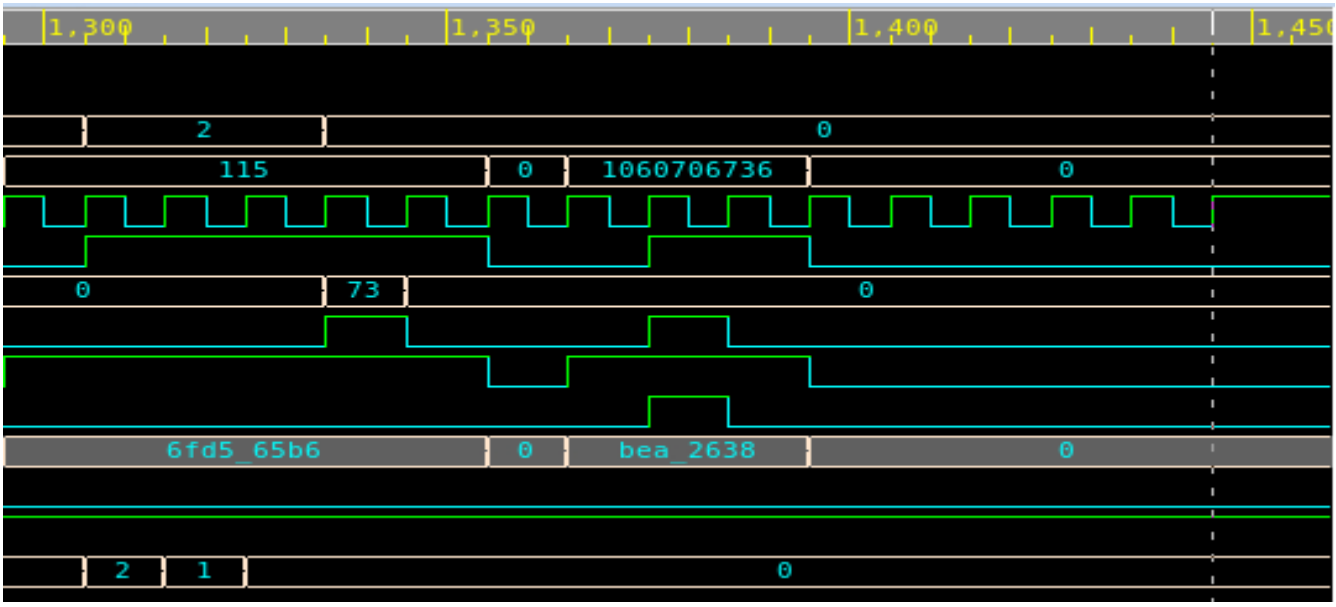



150
200
250
300
G1
apb_st[1:0]
paddr[31:0]
pclk
penable
prdata[31:0]
pready
psel
pslverr
pwdata[31:0]
pwrite
rst_n
t_count[31:0]
G2
0
X
0
x
0
1
x
0
XXX_XXXX
x
0
0
0 1 0 1 0
73 0 191 0 116 0
f1d2_0* 0 8dc5_a05a 0 5e8b_780d 0
0 2 1 0 2 1 0


300
350
400
450
G1
apb_st[1:0]
paddr[31:0]
pclk
penable
prdata[31:0]
pready
psel
pslverr
pwdata[31:0]
pwrite
rst_n
t_count[31:0]
G2
0
X
0
x
0
1
x
0
XXX_XXXX
x
0
0
0 1 0 1 0
1* 0 13 0 211 0
5* 0 5c21_3a1d 0 1225_fc42 0
0 2 1 0 2 1 0


450
500
550
600
G1
apb_st[1:0]
paddr[31:0]
pclk
penable
prdata[31:0]
pready
psel
pslverr
pwdata[31:0]
pwrite
rst_n
t_count[31:0]
G2
0
X
0
x
0
1
x
0
XXX_XXXX
x
0
0
0 1 0 1 0
0 83 0 58 0
0
0 e959_b00e 0 93e4_ece1 0
0 2 1 0 2 1 0 2





Avg. Group Score:48.33% U+C:147 U:103 C:44 X:0
Avg. Group Inst. Score:48.33% U+C:147 U:103 C:44 X:0

Group	Score	Instances	U+C	U	C	X	Goal	Weight	AtLeast	PerIns
cp_operation	100.00%		2	0	2	0	100%	1	1	
cp_prdata	66.67%		3	1	2	0	100%	1	1	
cp_pready	50.00%		2	1	1	0	100%	1	1	
cp_pslverr	100.00%		2	0	2	0	100%	1	1	
cp_pwdata	0.00%		5	5	0	0	100%	1	1	
cross_address_error	50.00%		8	4	4	0	100%	1	1	
cross_address_pready	50.00%		8	4	4	0	100%	1	1	
cross_operation_addr...	75.00%		8	2	6	0	100%	1	1	
cross_operation_addr...	37.50%		16	10	6	0	100%	1	1	
cross_operation_addr...	37.50%		16	10	6	0	100%	1	1	
cross_operation_bou...	10.00%		10	9	1	0	100%	1	1	
cross_operation_data	0.00%		10	10	0	0	100%	1	1	
cross_operation_error	75.00%		4	1	3	0	100%	1	1	
cross_operation_prea...	50.00%		4	2	2	0	100%	1	1	
cross_write_addr_data	0.00%		40	40	0	0	100%	1	1	

APB PROTOCOL WAIT-STATE(N=0)

Simulation Output

UVM_INFO driver.sv(31) @ 5: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=86 write=1 data=3e08d426

UVM_INFO driver.sv(44) @ 15: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=86 pwrite=1 prdata=0 PSLVERR=0

UVM_INFO scoreboard.sv(32) @ 25: uvm_test_top.env_o.sb [apb_scoreboard] WRITE PASS: addr=86 data=3e08d426

UVM_INFO driver.sv(31) @ 35: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=240 write=1 data=c6d1d15e

UVM_INFO scoreboard.sv(32) @ 65: uvm_test_top.env_o.sb [apb_scoreboard] WRITE PASS: addr=240 data=c6d1d15e

UVM_INFO driver.sv(44) @ 65: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=240 pwrite=1 prdata=0 PSLVERR=0

UVM_INFO driver.sv(31) @ 85: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=73 write=1 data=f1d20d11

UVM_INFO scoreboard.sv(32) @ 115: uvm_test_top.env_o.sb [apb_scoreboard] WRITE PASS: addr=73 data=f1d20d11

UVM_INFO driver.sv(44) @ 115: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=73 pwrite=1 prdata=0 PSLVERR=0

UVM_INFO driver.sv(31) @ 135: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=191 write=1 data=8dc5a05a

UVM_INFO scoreboard.sv(32) @ 165: uvm_test_top.env_o.sb [apb_scoreboard] WRITE PASS: addr=191 data=8dc5a05a

UVM_INFO driver.sv(44) @ 165: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=191 pwrite=1 prdata=0 PSLVERR=0

UVM_INFO driver.sv(31) @ 185: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=116 write=1 data=5e8b780d

UVM_INFO scoreboard.sv(32) @ 215: uvm_test_top.env_o.sb [apb_scoreboard] WRITE PASS: addr=116 data=5e8b780d

UVM_INFO driver.sv(44) @ 215: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=116 pwrite=1 prdata=0 PSLVERR=0

UVM_INFO driver.sv(31) @ 235: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=13 write=0 data=5c213a1d

UVM_INFO scoreboard.sv(36) @ 265: uvm_test_top.env_o.sb [apb_scoreboard] READ PASS: addr=13 expected=d actual=d

UVM_INFO driver.sv(44) @ 265: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=13 pwrite=0 prdata=d PSLVERR=0

UVM_INFO driver.sv(31) @ 285: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=211 write=0 data=1225fc42

UVM_INFO scoreboard.sv(36) @ 315: uvm_test_top.env_o.sb [apb_scoreboard] READ PASS: addr=211 expected=d3 actual=d3

UVM_INFO driver.sv(44) @ 315: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=211 pwrite=0 prdata=d3 PSLVERR=0

UVM_INFO driver.sv(31) @ 335: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=83 write=0 data=e959b00e

UVM_INFO scoreboard.sv(36) @ 365: uvm_test_top.env_o.sb [apb_scoreboard] READ PASS: addr=83 expected=53 actual=53

UVM_INFO driver.sv(44) @ 365: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=83 pwrite=0 prdata=53 PSLVERR=0

UVM_INFO driver.sv(31) @ 385: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=58 write=0 data=93e4ece1

UVM_INFO scoreboard.sv(36) @ 415: uvm_test_top.env_o.sb [apb_scoreboard] READ PASS: addr=58 expected=3a actual=3a

UVM_INFO driver.sv(44) @ 415: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=58 pwrite=0 prdata=3a PSLVERR=0

UVM_INFO driver.sv(31) @ 435: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=71 write=0 data=abb0b331

UVM_INFO scoreboard.sv(36) @ 465: uvm_test_top.env_o.sb [apb_scoreboard] READ PASS: addr=71 expected=47 actual=47

UVM_INFO driver.sv(44) @ 465: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=71 pwrite=0 prdata=47 PSLVERR=0

UVM_INFO driver.sv(31) @ 485: uvm_test_top.env_o.agt.drv [apb_driver] SETUP: addr=348408063 write=0 data=4afd08d5

UVM_INFO scoreboard.sv(22) @ 505: uvm_test_top.env_o.sb [apb_scoreboard] INVALID ADDRESS PASS: addr=348408063 PSLVERR=1

UVM_INFO driver.sv(44) @ 505: uvm_test_top.env_o.agt.drv [apb_driver] ACCESS DONE: addr=348408063 pwrite=0 prdata=0 PSLVERR=1

UVM_INFO test.sv(23) @ 565: uvm_test_top [apb_base_test] END OF TESTCASE

UVM_INFO /usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_objection.svh(1274) @ 565: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase

--- UVM Report Summary ---

** Report counts by severity

UVM_INFO : 36

UVM_WARNING : 0

UVM_ERROR : 0

UVM_FATAL : 0

** Report counts by id

[RNTST] 1

[TEST_DONE] 1

[apb_base_test] 1

[apb_driver] 22

[apb_scoreboard] 11

$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_root.svh", line 437.

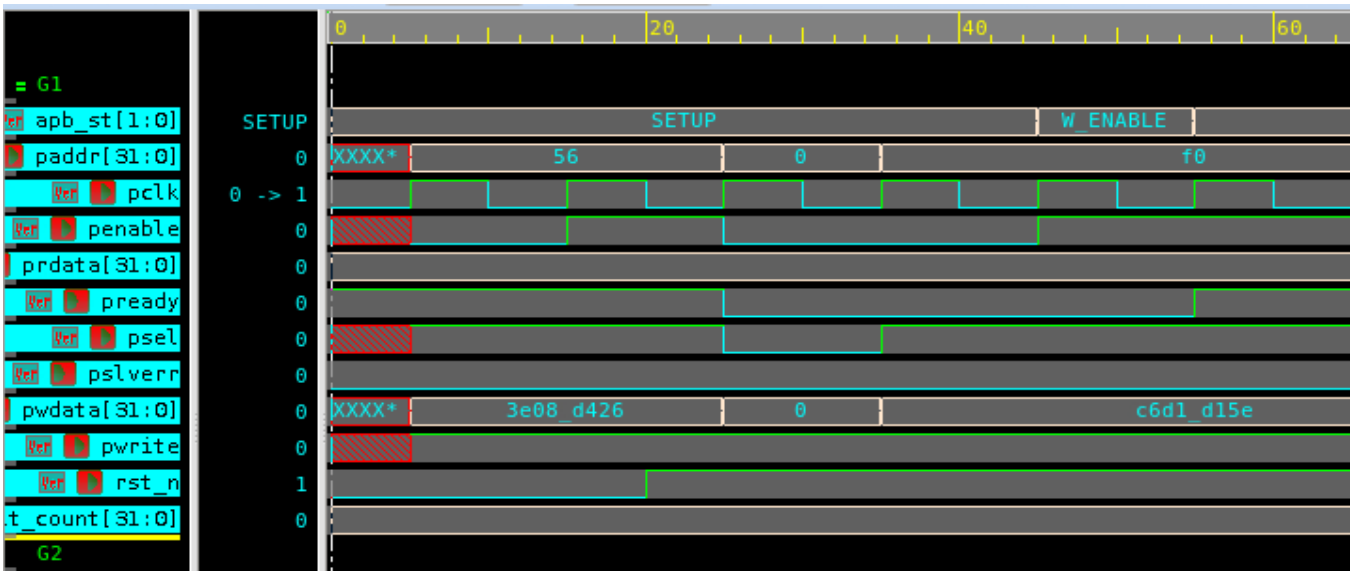
$finish at simulation time 565

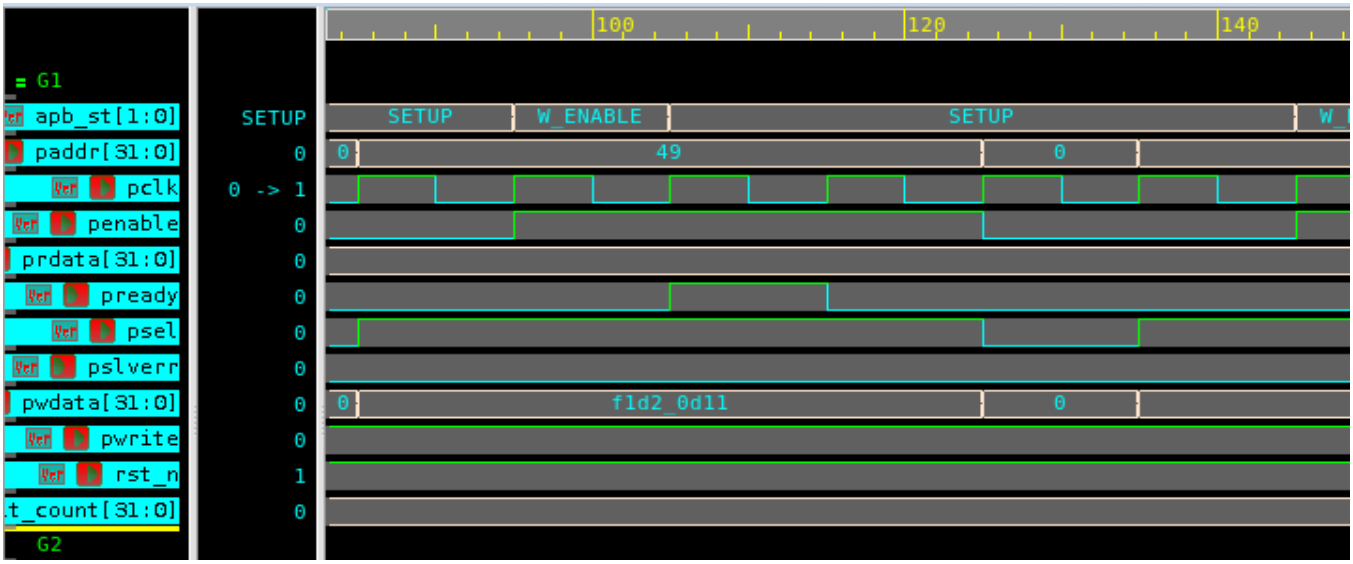
V C S S i m u l a t i o n R e p o r t

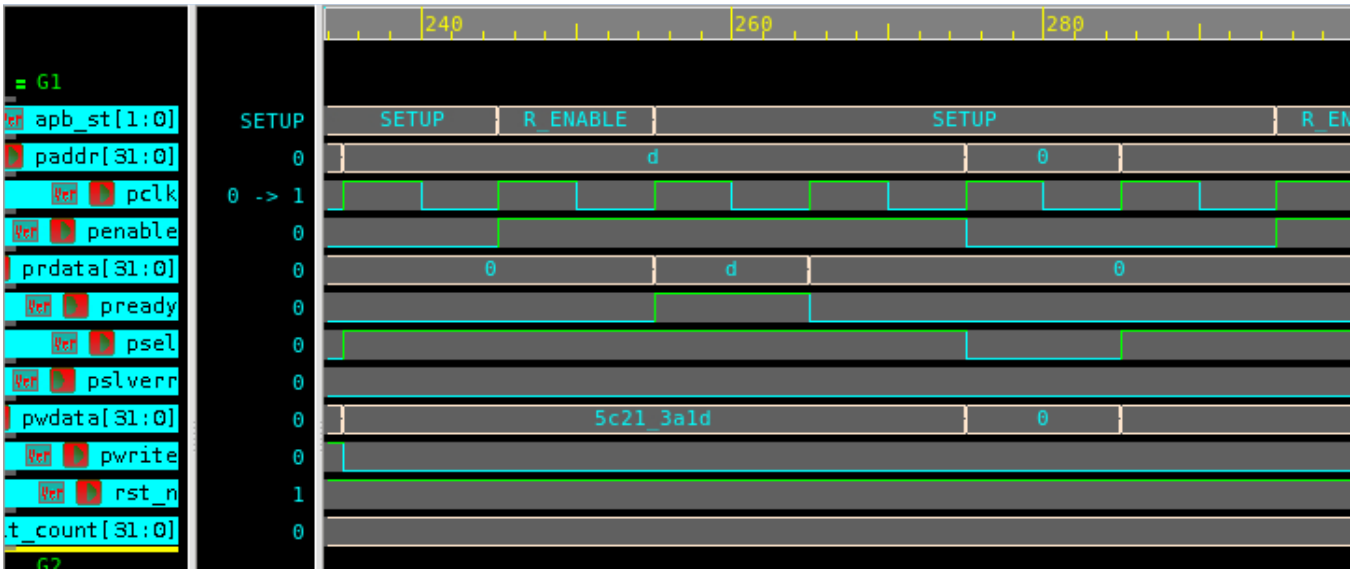
Time: 565

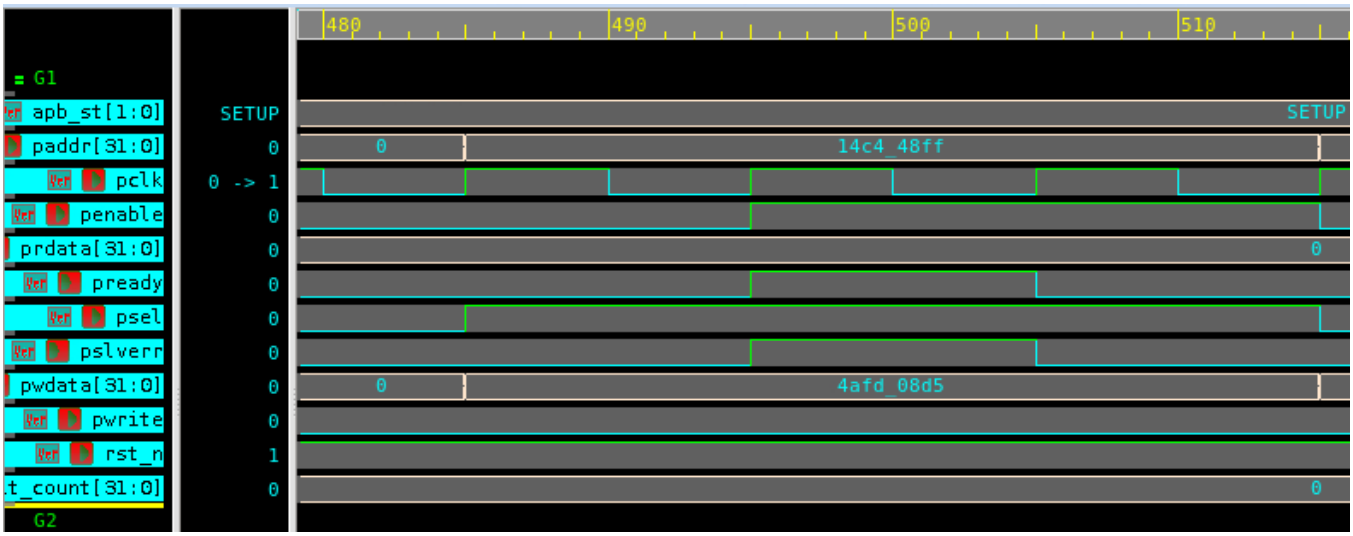
CPU Time: 0.150 seconds; Data structure size: 0.6Mb

Waveform Analysis (wait-state= 0)(min 2 clock cycle)


G1
apb_st[1:0]
paddr[31:0]
pclk
penable
prdata[31:0]
pready
psel
pslverr
pwdata[31:0]
pwrite
rst_n
t_count[31:0]
G2
SETUP
0
0 -> 1
0
0
0
0
0
0
0
1
0
0
20
40
60
SETUP
W_ENABLE
XXXX*
56
0
f0
XXXX*
3e08_d426
0
c6d1_d15e


G1
apb_st[1:0]
paddr[31:0]
pclk
penable
prdata[31:0]
pready
psel
pslverr
pwdata[31:0]
pwrite
rst_n
t_count[31:0]
G2
SETUP
0
0 -> 1
0
0
0
0
0
0
0
1
0
100
120
140
SETUP
W_ENABLE
SETUP
0
49
0
0
f1d2_0d11
0


G1
apb_st[1:0]
paddr[31:0]
pclk
penable
prdata[31:0]
pready
psel
pslverr
pwdata[31:0]
pwrite
rst_n
t_count[31:0]
G2
SETUP
0
0 -> 1
0
0
0
0
0
0
0
1
0
240
260
280
SETUP
R_ENABLE
SETUP
d
0
0
d
0
5c21_3a1d
0



Dual-Port RAM Verification using UVM

// READ-WRITE on memory

```
module dual_port_ram #(parameter DATA_WIDTH = 8,parameter ADDR_WIDTH = 8,parameter RAM_DEPTH  = 256)(
        input logic clk,rst_n,wr_en_a,rd_en_a,wr_en_b,rd_en_b,
        input logic [DATA_WIDTH-1:0] data_in_a,data_in_b,
        input logic [ADDR_WIDTH-1:0] addr_a,addr_b,
        output logic [DATA_WIDTH-1:0] data_out_a,data_out_b );


logic [DATA_WIDTH-1:0] ram [0:RAM_DEPTH-1];
integer i;


always_ff @(posedge clk) begin
        if (!rst_n) begin
                data_out_a <= '0;
                data_out_b <= '0;


                for (i = 0; i < RAM_DEPTH; i = i + 1) begin
```

```
                ram[i] <= '0;
            end
        end
    else begin

            if (wr_en_a)
                ram[addr_a] <= data_in_a;

            if (wr_en_b && !(wr_en_a && (addr_a == addr_b)))
                ram[addr_b] <= data_in_b;

            if (rd_en_a)
                data_out_a <= ram[addr_a];
            if (rd_en_b)
                data_out_b <= ram[addr_b];
        end
end

endmodule
```

Testbench

```
//
interface ram_if(input logic clk, input logic rst_n);
        logic wr_en_a;
        logic rd_en_a;
        logic wr_en_b;
        logic rd_en_b;
        logic [7:0] data_in_a;
```

```
        logic [7:0] data_in_b;
         logic [7:0] addr_a;
        logic [7:0] addr_b;
          logic [7:0] data_out_a;
        logic [7:0] data_out_b;

endinterface

//
class seq_item extends uvm_sequence_item;
          rand bit       wr_en_a;
        rand bit       rd_en_a;
        rand bit       wr_en_b;
        rand bit       rd_en_b;
          rand bit [7:0] data_in_a;
        rand bit [7:0] data_in_b;
        rand bit [7:0] addr_a;
        rand bit [7:0] addr_b;
        bit [7:0] data_out_a;
        bit [7:0] data_out_b;

          `uvm_object_utils_begin(seq_item)
          `uvm_field_int(wr_en_a,UVM_ALL_ON)
        `uvm_field_int(rd_en_a,UVM_ALL_ON)
        `uvm_field_int(wr_en_b,UVM_ALL_ON)
        `uvm_field_int(rd_en_b,UVM_ALL_ON)
        `uvm_field_int(data_in_a,UVM_ALL_ON)
```

```
        `uvm_field_int(data_in_b,UVM_ALL_ON)
        `uvm_field_int(addr_a, UVM_ALL_ON)
        `uvm_field_int(addr_b, UVM_ALL_ON)
        `uvm_field_int(data_out_a,UVM_ALL_ON)
        `uvm_field_int(data_out_b,UVM_ALL_ON)
          `uvm_object_utils_end

          function new(string name = "seq_item");
                super.new(name);
          endfunction

endclass

//
class base_seq extends uvm_sequence #(seq_item);
          seq_item req;
          `uvm_object_utils(base_seq)

          function new(string name = "base_seq");
                super.new(name);
          endfunction

          task body();

                `uvm_info(get_type_name(), "Base sequence started :", UVM_LOW)

                repeat(100) begin
```

```
                `uvm_do_with(req, { wr_en_a == 1; rd_en_a == 0; wr_en_b == 0;
rd_en_b == 0;})
                end

                repeat(5) begin
                `uvm_do_with(req, { wr_en_a == 0; rd_en_a == 0; wr_en_b == 1;
rd_en_b == 0;})
                end

                repeat(50) begin
                `uvm_do_with(req, { wr_en_a == 0; rd_en_a == 1; wr_en_b == 0;
rd_en_b == 0;})
                end

                repeat(5) begin
                `uvm_do_with(req, { wr_en_a == 0; rd_en_a == 0; wr_en_b == 0;
rd_en_b == 1;})
                end

                repeat(5) begin
                `uvm_do_with(req, { wr_en_a == 1; rd_en_a == 0; wr_en_b == 1;
rd_en_b == 0; addr_a != addr_b;})
                end

                repeat(5) begin
                `uvm_do_with(req, { wr_en_a == 0; rd_en_a == 1; wr_en_b == 0;
rd_en_b == 1;})
                end
```

```
		repeat(5) begin
			`uvm_do_with(req, { wr_en_a == 1; rd_en_a == 0; wr_en_b == 1; rd_en_b == 0; addr_a == addr_b;})
		end

		repeat(5) begin
			`uvm_do_with(req, { wr_en_a == 1; rd_en_a == 1; wr_en_b == 0; rd_en_b == 0;})
		end

		repeat(5) begin
			`uvm_do(req)
		end

		`uvm_info(get_type_name(), "Base sequence completed",UVM_LOW)
	endtask

endclass

//
class seqcr extends uvm_sequencer #(seq_item);
	`uvm_component_utils(seqcr)

	function new(string name = "seqcr", uvm_component parent = null);
		super.new(name, parent);
	endfunction

	function void build_phase(uvm_phase phase);
```

```
        super.build_phase(phase);
    endfunction

endclass

//
class driver extends uvm_driver #(seq_item);
    virtual ram_if vif;
    `uvm_component_utils(driver)

    function new(string name = "driver", uvm_component parent = null);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if (!uvm_config_db#(virtual ram_if)::get(this, "", "vif", vif))
            `uvm_fatal(get_type_name(), "Virtual interface not found")
    endfunction

    task run_phase(uvm_phase phase);
        forever begin
            seq_item_port.get_next_item(req);
            @(negedge vif.clk);
            vif.wr_en_a   <= req.wr_en_a;
          vif.rd_en_a   <= req.rd_en_a;
          vif.wr_en_b   <= req.wr_en_b;
```

```
                vif.rd_en_b   <= req.rd_en_b;
                    vif.data_in_a <= req.data_in_a;
                    vif.data_in_b <= req.data_in_b;
                    vif.addr_a    <= req.addr_a;
                    vif.addr_b    <= req.addr_b;

                    `uvm_info(get_type_name(), $sformatf("DRIVER: A[WR=%0d RD=%0d ADDR=%0h DATA=%0h] B[WR=%0d RD=%0d ADDR=%0h DATA=%0h]",req.wr_en_a,req.rd_en_a,req.addr_a,req.data_in_a,req.wr_en_b,req.rd_en_b,req.addr_b,req.data_in_b), UVM_LOW)

                    @(posedge vif.clk);
                    seq_item_port.item_done();
            end
        endtask

endclass

//
class monitor extends uvm_monitor;
        virtual ram_if vif;
        uvm_analysis_port #(seq_item) mon_ap;
        `uvm_component_utils(monitor)

        function new(string name = "monitor", uvm_component parent = null);
                super.new(name, parent);
                mon_ap = new("mon_ap", this);
        endfunction
```

```
function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if (!uvm_config_db#(virtual ram_if)::get(this, "", "vif", vif))
                `uvm_fatal(get_type_name(), "Virtual interface not found")
endfunction

task run_phase(uvm_phase phase);
        seq_item mon_item;
        forever begin
                @(posedge vif.clk);
                if (!vif.rst_n)
                        continue;
                #1;
                if (vif.wr_en_a || vif.rd_en_a || vif.wr_en_b || vif.rd_en_b) begin
                        mon_item = seq_item::type_id::create("mon_item");

                        mon_item.wr_en_a   = vif.wr_en_a;
                        mon_item.rd_en_a   = vif.rd_en_a;
                        mon_item.wr_en_b   = vif.wr_en_b;
                        mon_item.rd_en_b   = vif.rd_en_b;
                        mon_item.data_in_a = vif.data_in_a;
                        mon_item.data_in_b = vif.data_in_b;
                        mon_item.addr_a    = vif.addr_a;
                        mon_item.addr_b    = vif.addr_b;
                        mon_item.data_out_a = vif.data_out_a;
                        mon_item.data_out_b = vif.data_out_b;
```

```
                        mon_ap.write(mon_item);
                    end
                end
            endtask

endclass

//
class scoreboard extends uvm_scoreboard;
        `uvm_component_utils(scoreboard)
        uvm_analysis_imp #(seq_item, scoreboard) imp;

        bit [7:0] ref_ram [0:255];

        function new(string name = "ram_scoreboard", uvm_component parent);
                super.new(name, parent);
                imp = new("imp", this);
        endfunction

        function void build_phase(uvm_phase phase);
                super.build_phase(phase);

                foreach (ref_ram[i]) begin
                        ref_ram[i] = 8'h00;
                end
        endfunction
```

```
function void write(seq_item tx);

    bit [7:0] exp_a;
    bit [7:0] exp_b;

    if (tx.rd_en_a) begin
        exp_a = ref_ram[tx.addr_a];
    end

    if (tx.rd_en_b) begin
        exp_b = ref_ram[tx.addr_b];
    end

    if (tx.rd_en_a) begin
        if (tx.data_out_a === exp_a) begin
            `uvm_info("SCOREBOARD",$sformatf("PORT A PASS:
ADDR=%0h EXPECTED=%0h ACTUAL=%0h",
tx.addr_a,exp_a,tx.data_out_a),UVM_LOW)
        end
        else begin
            `uvm_error("SCOREBOARD",$sformatf("PORT A FAIL:
ADDR=%0h EXPECTED=%0h ACTUAL=%0h", tx.addr_a,exp_a,tx.data_out_a))
        end
    end

    if (tx.rd_en_b) begin
        if (tx.data_out_b === exp_b) begin
```

```
                `uvm_info("SCOREBOARD", $sformatf("PORT B PASS:
ADDR=%0h EXPECTED=%0h ACTUAL=%0h",
tx.addr_b,exp_b,tx.data_out_b),UVM_LOW)
            end
            else begin
                `uvm_error("SCOREBOARD",$sformatf("PORT B FAIL:
ADDR=%0h EXPECTED=%0h ACTUAL=%0h", tx.addr_b,exp_b,tx.data_out_b))
            end
        end

        if (tx.wr_en_a) begin
            ref_ram[tx.addr_a] = tx.data_in_a;
        end

        if (tx.wr_en_b && !(tx.wr_en_a && (tx.addr_a == tx.addr_b))) begin
            ref_ram[tx.addr_b] = tx.data_in_b;
        end

    endfunction

endclass

//
class agent extends uvm_agent;
    `uvm_component_utils(agent)
    driver  drv;
    seqcr   seqr;
    monitor mon;
```

```
    function new(string name = "agent", uvm_component parent = null);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        drv = driver::type_id::create("drv", this);
        seqr = seqcr::type_id::create("seqr", this);
        mon = monitor::type_id::create("mon", this);
    endfunction

    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        drv.seq_item_port.connect(seqr.seq_item_export);
    endfunction

endclass

//
class env extends uvm_env;
    `uvm_component_utils(env)
    agent      agt;
    scoreboard sb;

    function new(string name = "env", uvm_component parent = null);
        super.new(name, parent);
```

```
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        agt = agent::type_id::create("agt", this);
        sb = scoreboard::type_id::create("sb", this);
    endfunction

    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        agt.mon.mon_ap.connect(sb.imp);
    endfunction

endclass

//
class base_test extends uvm_test;
    `uvm_component_utils(base_test)
    env env_o;

    function new(string name = "base_test", uvm_component parent = null);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        env_o = env::type_id::create("env_o", this);
```

```
        endfunction

        task run_phase(uvm_phase phase);
            base_seq bseq;

            phase.raise_objection(this);
            bseq = base_seq::type_id::create("bseq");
          bseq.start(env_o.agt.seqr);
            phase.drop_objection(this);
            `uvm_info(get_type_name(), "End of testcase", UVM_LOW)
        endtask

endclass

//
package ram_pkg;

    `include "uvm_macros.svh"

    import uvm_pkg::*;

    `include "sequence_item.sv"
    `include "sequence.sv"
    `include "sequencer.sv"
    `include "driver.sv"
    `include "monitor.sv"
    `include "scoreboard.sv"
```

```
    `include "agent.sv"
    `include "environment.sv"
    `include "test.sv"

endpackage

//
`include "uvm_macros.svh"

import uvm_pkg::*;
import ram_pkg::*;

module tb_top;

bit clk;
bit rst_n;

always #5 clk = ~clk;

ram_if vif(clk, rst_n);

dual_port_ram #(.DATA_WIDTH(8),.ADDR_WIDTH(8),.RAM_DEPTH(256)) DUT
(.clk (vif.clk),.rst_n (vif.rst_n),.wr_en_a (vif.wr_en_a),.rd_en_a (vif.rd_en_a),.wr_en_b
(vif.wr_en_b),.rd_en_b (vif.rd_en_b),.data_in_a (vif.data_in_a),.data_in_b
(vif.data_in_b), .addr_a (vif.addr_a),.addr_b (vif.addr_b),.data_out_a
(vif.data_out_a),.data_out_b (vif.data_out_b));

initial begin
        rst_n = 1'b0;
```

```
        vif.wr_en_a = 0;
        vif.rd_en_a = 0;
        vif.wr_en_b = 0;
        vif.rd_en_b = 0;
        vif.data_in_a = 0;
        vif.data_in_b = 0;
        vif.addr_a = 0;
        vif.addr_b = 0;

          repeat(3)
          @(posedge clk);
          rst_n = 1'b1;
end

initial begin
        uvm_config_db#(virtual ram_if)::set(null,"*","vif",vif);
end

initial begin
        run_test("base_test");
end

initial begin
        $fsdbDumpfile("dual_port_ram.fsdb");
        $fsdbDumpvars(0, tb_top);
end
```

endmodule

Run-Command

[b23n_sarif_727@prac3 dual_port_ram]$ ls

agent.sv dual_port_ram_design.sv monitor.sv ram_pkg.sv sequencer.sv simv.cst test.sv

csrc dual_port_ram.fsdb novas_dump.log scoreboard.sv sequence.sv simv.daidir ucli.key

driver.sv environment.sv ram_if.sv sequence_item.sv simv tb_top.sv vc_hdrs.h

[b23n_sarif_727@prac3 dual_port_ram]$ vcs -sverilog dual_port_ram_design.sv ram_if.sv ram_pkg.sv tb_top.sv -ntb_opts uvm -kdb -lca -debug_access+all -full64

[b23n_sarif_727@prac3 dual_port_ram]$ ./simv

[b23n_sarif_727@prac3 dual_port_ram]$ ./simv -verdi

Simulation Outputs

UVM_INFO sequence.sv(12) @ 0: uvm_test_top.env_o.agt.seqr@@bseq [base_seq] Base sequence started :

UVM_INFO driver.sv(29) @ 10: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=8 DATA=53] B[WR=0 RD=0 ADDR=34 DATA=a6]

UVM_INFO driver.sv(29) @ 20: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=2f DATA=a7] B[WR=0 RD=0 ADDR=37 DATA=7d]

UVM_INFO driver.sv(29) @ 30: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=31 DATA=e8] B[WR=0 RD=0 ADDR=47 DATA=d]

UVM_INFO driver.sv(29) @ 40: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=23 DATA=26] B[WR=0 RD=0 ADDR=9 DATA=2f]

UVM_INFO driver.sv(29) @ 50: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=ec DATA=bd] B[WR=0 RD=0 ADDR=4e DATA=d5]

UVM_INFO driver.sv(29) @ 60: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=10 DATA=bc] B[WR=0 RD=0 ADDR=7b DATA=9d]

UVM_INFO driver.sv(29) @ 70: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=5b DATA=9a] B[WR=0 RD=0 ADDR=bd DATA=94]

UVM_INFO driver.sv(29) @ 80: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=8a DATA=c4] B[WR=0 RD=0 ADDR=82 DATA=33]

UVM_INFO driver.sv(29) @ 90: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=1f DATA=95] B[WR=0 RD=0 ADDR=4c DATA=e]

UVM_INFO driver.sv(29) @ 100: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=41 DATA=46] B[WR=0 RD=0 ADDR=ef DATA=f6]

UVM_INFO driver.sv(29) @ 110: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=2f DATA=98] B[WR=0 RD=0 ADDR=e DATA=4e]

UVM_INFO driver.sv(29) @ 120: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=79 DATA=bc] B[WR=0 RD=0 ADDR=7e DATA=fd]

UVM_INFO driver.sv(29) @ 130: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=36 DATA=90] B[WR=0 RD=0 ADDR=be DATA=86]

UVM_INFO driver.sv(29) @ 140: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=34 DATA=7e] B[WR=0 RD=0 ADDR=28 DATA=7d]

UVM_INFO driver.sv(29) @ 150: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=f2 DATA=66] B[WR=0 RD=0 ADDR=62 DATA=96]

UVM_INFO driver.sv(29) @ 160: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=44 DATA=44] B[WR=0 RD=0 ADDR=a8 DATA=ec]

UVM_INFO driver.sv(29) @ 170: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=2e DATA=b4] B[WR=0 RD=0 ADDR=0 DATA=22]

UVM_INFO driver.sv(29) @ 180: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=b7 DATA=e0] B[WR=0 RD=0 ADDR=db DATA=72]

UVM_INFO driver.sv(29) @ 190: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=95 DATA=c2] B[WR=0 RD=0 ADDR=18 DATA=2c]

UVM_INFO driver.sv(29) @ 200: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=7a DATA=69] B[WR=0 RD=0 ADDR=4d DATA=a7]

UVM_INFO driver.sv(29) @ 210: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=ac DATA=1b] B[WR=0 RD=0 ADDR=44 DATA=7c]

UVM_INFO driver.sv(29) @ 220: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=63 DATA=27] B[WR=0 RD=0 ADDR=7f DATA=8c]

UVM_INFO driver.sv(29) @ 230: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=33 DATA=f0] B[WR=0 RD=0 ADDR=4c DATA=4d]

UVM_INFO driver.sv(29) @ 240: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=d4 DATA=97] B[WR=0 RD=0 ADDR=72 DATA=8a]

UVM_INFO driver.sv(29) @ 250: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=f6 DATA=55] B[WR=0 RD=0 ADDR=be DATA=33]

UVM_INFO driver.sv(29) @ 260: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=8c DATA=ef] B[WR=0 RD=0 ADDR=b5 DATA=75]

UVM_INFO driver.sv(29) @ 270: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=4f DATA=c1] B[WR=0 RD=0 ADDR=49 DATA=82]

UVM_INFO driver.sv(29) @ 280: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=e4 DATA=96] B[WR=0 RD=0 ADDR=cd DATA=91]

UVM_INFO driver.sv(29) @ 290: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=82 DATA=fd] B[WR=0 RD=0 ADDR=1b DATA=1a]

UVM_INFO driver.sv(29) @ 300: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=48 DATA=7f] B[WR=0 RD=0 ADDR=41 DATA=23]

UVM_INFO driver.sv(29) @ 310: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=e4 DATA=e7] B[WR=0 RD=0 ADDR=15 DATA=85]

UVM_INFO driver.sv(29) @ 320: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=64 DATA=53] B[WR=0 RD=0 ADDR=72 DATA=35]

UVM_INFO driver.sv(29) @ 330: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=b4 DATA=31] B[WR=0 RD=0 ADDR=23 DATA=f4]

UVM_INFO driver.sv(29) @ 340: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=20 DATA=77] B[WR=0 RD=0 ADDR=9b DATA=40]

UVM_INFO driver.sv(29) @ 350: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=ba DATA=d3] B[WR=0 RD=0 ADDR=6e DATA=7c]

UVM_INFO driver.sv(29) @ 360: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=94 DATA=9b] B[WR=0 RD=0 ADDR=17 DATA=d3]

UVM_INFO driver.sv(29) @ 370: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=6d DATA=4c] B[WR=0 RD=0 ADDR=b2 DATA=73]

UVM_INFO driver.sv(29) @ 380: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=5d DATA=9a] B[WR=0 RD=0 ADDR=57 DATA=72]

UVM_INFO driver.sv(29) @ 390: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=e0 DATA=64] B[WR=0 RD=0 ADDR=a5 DATA=0]

UVM_INFO driver.sv(29) @ 400: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=b0 DATA=47] B[WR=0 RD=0 ADDR=f DATA=ce]

UVM_INFO driver.sv(29) @ 410: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=27 DATA=2f] B[WR=0 RD=0 ADDR=e DATA=b0]

UVM_INFO driver.sv(29) @ 420: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=8e DATA=68] B[WR=0 RD=0 ADDR=b2 DATA=27]

UVM_INFO driver.sv(29) @ 430: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=5a DATA=e1] B[WR=0 RD=0 ADDR=8a DATA=71]

UVM_INFO driver.sv(29) @ 440: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=6d DATA=2c] B[WR=0 RD=0 ADDR=4e DATA=7c]

UVM_INFO driver.sv(29) @ 450: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=b1 DATA=ef] B[WR=0 RD=0 ADDR=98 DATA=f8]

UVM_INFO driver.sv(29) @ 460: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=61 DATA=a8] B[WR=0 RD=0 ADDR=a6 DATA=41]

UVM_INFO driver.sv(29) @ 470: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=93 DATA=18] B[WR=0 RD=0 ADDR=54 DATA=ce]

UVM_INFO driver.sv(29) @ 480: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=7f DATA=33] B[WR=0 RD=0 ADDR=84 DATA=af]

UVM_INFO driver.sv(29) @ 490: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=94 DATA=5e] B[WR=0 RD=0 ADDR=13 DATA=25]

UVM_INFO driver.sv(29) @ 500: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=e4 DATA=2b] B[WR=0 RD=0 ADDR=48 DATA=5b]

UVM_INFO driver.sv(29) @ 510: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=ba DATA=23] B[WR=0 RD=0 ADDR=95 DATA=18]

UVM_INFO driver.sv(29) @ 520: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=3e DATA=ce] B[WR=0 RD=0 ADDR=e4 DATA=5a]

UVM_INFO driver.sv(29) @ 530: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=cc DATA=d0] B[WR=0 RD=0 ADDR=a2 DATA=4d]

UVM_INFO driver.sv(29) @ 540: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=9c DATA=cf] B[WR=0 RD=0 ADDR=f4 DATA=21]

UVM_INFO driver.sv(29) @ 550: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=80 DATA=d9] B[WR=0 RD=0 ADDR=a7 DATA=89]

UVM_INFO driver.sv(29) @ 560: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=e6 DATA=65] B[WR=0 RD=0 ADDR=6c DATA=a8]

UVM_INFO driver.sv(29) @ 570: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=de DATA=26] B[WR=0 RD=0 ADDR=9 DATA=b0]

UVM_INFO driver.sv(29) @ 580: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=f9 DATA=5e] B[WR=0 RD=0 ADDR=7f DATA=21]

UVM_INFO driver.sv(29) @ 590: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=95 DATA=7c] B[WR=0 RD=0 ADDR=e5 DATA=58]

UVM_INFO driver.sv(29) @ 600: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=db DATA=9c] B[WR=0 RD=0 ADDR=4e DATA=f7]

UVM_INFO driver.sv(29) @ 610: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=d7 DATA=c0] B[WR=0 RD=0 ADDR=a8 DATA=da]

UVM_INFO driver.sv(29) @ 620: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=8a DATA=5a] B[WR=0 RD=0 ADDR=fe DATA=cc]

UVM_INFO driver.sv(29) @ 630: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=af DATA=d9] B[WR=0 RD=0 ADDR=9e DATA=1d]

UVM_INFO driver.sv(29) @ 640: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=2d DATA=2b] B[WR=0 RD=0 ADDR=11 DATA=3e]

UVM_INFO driver.sv(29) @ 650: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=b6 DATA=f6] B[WR=0 RD=0 ADDR=55 DATA=5d]

UVM_INFO driver.sv(29) @ 660: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=2f DATA=80] B[WR=0 RD=0 ADDR=da DATA=31]

UVM_INFO driver.sv(29) @ 670: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=66 DATA=b1] B[WR=0 RD=0 ADDR=8f DATA=7c]

UVM_INFO driver.sv(29) @ 680: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=fc DATA=7a] B[WR=0 RD=0 ADDR=e0 DATA=e6]

UVM_INFO driver.sv(29) @ 690: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=37 DATA=84] B[WR=0 RD=0 ADDR=94 DATA=3b]

UVM_INFO driver.sv(29) @ 700: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=ab DATA=e7] B[WR=0 RD=0 ADDR=83 DATA=1f]

UVM_INFO driver.sv(29) @ 710: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=aa DATA=cd] B[WR=0 RD=0 ADDR=68 DATA=2b]

UVM_INFO driver.sv(29) @ 720: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=11 DATA=ae] B[WR=0 RD=0 ADDR=b6 DATA=65]

UVM_INFO driver.sv(29) @ 730: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=d1 DATA=1a] B[WR=0 RD=0 ADDR=cf DATA=ea]

UVM_INFO driver.sv(29) @ 740: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=6a DATA=63] B[WR=0 RD=0 ADDR=c3 DATA=5e]

UVM_INFO driver.sv(29) @ 750: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=d8 DATA=f8] B[WR=0 RD=0 ADDR=c DATA=d0]

UVM_INFO driver.sv(29) @ 760: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=54 DATA=b4] B[WR=0 RD=0 ADDR=8f DATA=68]

UVM_INFO driver.sv(29) @ 770: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=16 DATA=c4] B[WR=0 RD=0 ADDR=2e DATA=5a]

UVM_INFO driver.sv(29) @ 780: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=a7 DATA=51] B[WR=0 RD=0 ADDR=f8 DATA=49]

UVM_INFO driver.sv(29) @ 790: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=8e DATA=49] B[WR=0 RD=0 ADDR=57 DATA=a0]

UVM_INFO driver.sv(29) @ 800: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=2d DATA=13] B[WR=0 RD=0 ADDR=c1 DATA=a8]

UVM_INFO driver.sv(29) @ 810: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=85 DATA=d9] B[WR=0 RD=0 ADDR=9c DATA=f2]

UVM_INFO driver.sv(29) @ 820: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=28 DATA=42] B[WR=0 RD=0 ADDR=c4 DATA=a]

UVM_INFO driver.sv(29) @ 830: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=e5 DATA=3b] B[WR=0 RD=0 ADDR=2b DATA=78]

UVM_INFO driver.sv(29) @ 840: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=45 DATA=d] B[WR=0 RD=0 ADDR=dc DATA=1]

UVM_INFO driver.sv(29) @ 850: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=a4 DATA=9f] B[WR=0 RD=0 ADDR=12 DATA=ea]

UVM_INFO driver.sv(29) @ 860: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=b9 DATA=c] B[WR=0 RD=0 ADDR=6e DATA=63]

UVM_INFO driver.sv(29) @ 870: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=ac DATA=42] B[WR=0 RD=0 ADDR=7 DATA=a7]

UVM_INFO driver.sv(29) @ 880: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=9a DATA=de] B[WR=0 RD=0 ADDR=8d DATA=61]

UVM_INFO driver.sv(29) @ 890: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=c6 DATA=d3] B[WR=0 RD=0 ADDR=4e DATA=7d]

UVM_INFO driver.sv(29) @ 900: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=76 DATA=33] B[WR=0 RD=0 ADDR=ae DATA=f6]

UVM_INFO driver.sv(29) @ 910: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=a8 DATA=16] B[WR=0 RD=0 ADDR=a9 DATA=53]

UVM_INFO driver.sv(29) @ 920: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=3c DATA=84] B[WR=0 RD=0 ADDR=c0 DATA=eb]

UVM_INFO driver.sv(29) @ 930: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=59 DATA=be] B[WR=0 RD=0 ADDR=1c DATA=f4]

UVM_INFO driver.sv(29) @ 940: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=f4 DATA=b2] B[WR=0 RD=0 ADDR=dc DATA=89]

UVM_INFO driver.sv(29) @ 950: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=4e DATA=4d] B[WR=0 RD=0 ADDR=e2 DATA=e0]

UVM_INFO driver.sv(29) @ 960: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=5d DATA=fa] B[WR=0 RD=0 ADDR=3c DATA=e1]

UVM_INFO driver.sv(29) @ 970: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=cf DATA=6a] B[WR=0 RD=0 ADDR=4c DATA=2d]

UVM_INFO driver.sv(29) @ 980: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=d7 DATA=aa] B[WR=0 RD=0 ADDR=f9 DATA=13]

UVM_INFO driver.sv(29) @ 990: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=4b DATA=8a] B[WR=0 RD=0 ADDR=11 DATA=7]

UVM_INFO driver.sv(29) @ 1000: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=ca DATA=b1] B[WR=0 RD=0 ADDR=a1 DATA=31]

UVM_INFO driver.sv(29) @ 1010: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=0 ADDR=23 DATA=3e] B[WR=1 RD=0 ADDR=42 DATA=b0]

UVM_INFO driver.sv(29) @ 1020: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=0 ADDR=20 DATA=8e] B[WR=1 RD=0 ADDR=2d DATA=f6]

UVM_INFO driver.sv(29) @ 1030: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=0 ADDR=c DATA=3d] B[WR=1 RD=0 ADDR=84 DATA=3a]

UVM_INFO driver.sv(29) @ 1040: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=0 ADDR=13 DATA=fe] B[WR=1 RD=0 ADDR=f8 DATA=ab]

UVM_INFO driver.sv(29) @ 1050: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=0 ADDR=53 DATA=54] B[WR=1 RD=0 ADDR=bf DATA=1f]

UVM_INFO driver.sv(29) @ 1060: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=f2 DATA=e7] B[WR=0 RD=0 ADDR=57 DATA=4d]

UVM_INFO scoreboard.sv(36) @ 1066: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=f2 EXPECTED=66 ACTUAL=66

UVM_INFO driver.sv(29) @ 1070: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=2b DATA=b3] B[WR=0 RD=0 ADDR=b1 DATA=29]

UVM_INFO scoreboard.sv(36) @ 1076: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=2b EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(29) @ 1080: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=53 DATA=ec] B[WR=0 RD=0 ADDR=f9 DATA=e3]

UVM_INFO scoreboard.sv(36) @ 1086: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=53 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(29) @ 1090: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=ad DATA=fb] B[WR=0 RD=0 ADDR=4e DATA=b3]

UVM_INFO scoreboard.sv(36) @ 1096: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=ad EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(29) @ 1100: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=c8 DATA=f0] B[WR=0 RD=0 ADDR=b0 DATA=24]

UVM_INFO scoreboard.sv(36) @ 1106: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=c8 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(29) @ 1110: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=92 DATA=9e] B[WR=0 RD=0 ADDR=33 DATA=3f]

UVM_INFO scoreboard.sv(36) @ 1116: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=92 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(29) @ 1120: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=bd DATA=14] B[WR=0 RD=0 ADDR=81 DATA=2a]

UVM_INFO scoreboard.sv(36) @ 1126: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=bd EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(29) @ 1130: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=d6 DATA=91] B[WR=0 RD=0 ADDR=be DATA=14]

UVM_INFO scoreboard.sv(36) @ 1136: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=d6 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(29) @ 1140: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=21 DATA=da] B[WR=0 RD=0 ADDR=83 DATA=d0]

UVM_INFO scoreboard.sv(36) @ 1146: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=21 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(29) @ 1150: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=3 DATA=9f] B[WR=0 RD=0 ADDR=21 DATA=b9]

UVM_INFO scoreboard.sv(36) @ 1156: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=3 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(29) @ 1160: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=15 DATA=98] B[WR=0 RD=0 ADDR=9b DATA=9]

UVM_INFO scoreboard.sv(36) @ 1166: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=15 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(29) @ 1170: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=62 DATA=be] B[WR=0 RD=0 ADDR=f0 DATA=fe]

UVM_INFO scoreboard.sv(36) @ 1176: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=62 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(29) @ 1180: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=5e DATA=c6] B[WR=0 RD=0 ADDR=f1 DATA=ed]

UVM_INFO scoreboard.sv(36) @ 1186: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=5e EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(29) @ 1190: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=a4 DATA=98] B[WR=0 RD=0 ADDR=3b DATA=c2]

UVM_INFO scoreboard.sv(36) @ 1196: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=a4 EXPECTED=9f ACTUAL=9f

UVM_INFO driver.sv(29) @ 1200: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=3d DATA=6b] B[WR=0 RD=0 ADDR=e8 DATA=f3]

UVM_INFO scoreboard.sv(36) @ 1206: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=3d EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(29) @ 1210: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=a6 DATA=cb] B[WR=0 RD=0 ADDR=7c DATA=45]

UVM_INFO scoreboard.sv(36) @ 1216: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=a6 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(29) @ 1220: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=86 DATA=e9] B[WR=0 RD=0 ADDR=2b DATA=ee]

UVM_INFO scoreboard.sv(36) @ 1226: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=86 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(29) @ 1230: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=8d DATA=ab] B[WR=0 RD=0 ADDR=e5 DATA=c]

UVM_INFO scoreboard.sv(36) @ 1236: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=8d EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(29) @ 1240: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=1c DATA=86] B[WR=0 RD=0 ADDR=98 DATA=d4]

UVM_INFO scoreboard.sv(36) @ 1246: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=1c EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(29) @ 1250: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=1d DATA=25] B[WR=0 RD=0 ADDR=2d DATA=2b]

UVM_INFO scoreboard.sv(36) @ 1256: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=1d EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(29) @ 1260: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=3f DATA=2d] B[WR=0 RD=0 ADDR=a3 DATA=7c]

UVM_INFO scoreboard.sv(36) @ 1266: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=3f EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(29) @ 1270: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=ba DATA=d4] B[WR=0 RD=0 ADDR=2f DATA=b1]

UVM_INFO scoreboard.sv(36) @ 1276: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=ba EXPECTED=23 ACTUAL=23

UVM_INFO driver.sv(29) @ 1280: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=8c DATA=97] B[WR=0 RD=0 ADDR=43 DATA=90]

UVM_INFO scoreboard.sv(36) @ 1286: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=8c EXPECTED=ef ACTUAL=ef

UVM_INFO driver.sv(29) @ 1290: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=a8 DATA=1] B[WR=0 RD=0 ADDR=d3 DATA=aa]

UVM_INFO scoreboard.sv(36) @ 1296: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=a8 EXPECTED=16 ACTUAL=16

UVM_INFO driver.sv(29) @ 1300: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=e8 DATA=bc] B[WR=0 RD=0 ADDR=12 DATA=40]

UVM_INFO scoreboard.sv(36) @ 1306: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=e8 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(29) @ 1310: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=c8 DATA=35] B[WR=0 RD=0 ADDR=2a DATA=aa]

UVM_INFO scoreboard.sv(36) @ 1316: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=c8 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(29) @ 1320: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=41 DATA=70] B[WR=0 RD=0 ADDR=e3 DATA=7d]

UVM_INFO scoreboard.sv(36) @ 1326: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=41 EXPECTED=46 ACTUAL=46

UVM_INFO driver.sv(29) @ 1330: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=5e DATA=87] B[WR=0 RD=0 ADDR=46 DATA=4c]

UVM_INFO scoreboard.sv(36) @ 1336: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=5e EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(29) @ 1340: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=2c DATA=89] B[WR=0 RD=0 ADDR=5d DATA=ad]

UVM_INFO scoreboard.sv(36) @ 1346: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=2c EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(29) @ 1350: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=6 DATA=7d] B[WR=0 RD=0 ADDR=ae DATA=7a]

UVM_INFO scoreboard.sv(36) @ 1356: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=6 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(29) @ 1360: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=9d DATA=5f] B[WR=0 RD=0 ADDR=38 DATA=e6]

UVM_INFO scoreboard.sv(36) @ 1366: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=9d EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(29) @ 1370: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=46 DATA=4e] B[WR=0 RD=0 ADDR=d7 DATA=c4]

UVM_INFO scoreboard.sv(36) @ 1376: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=46 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(29) @ 1380: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=68 DATA=cb] B[WR=0 RD=0 ADDR=4d DATA=92]

UVM_INFO scoreboard.sv(36) @ 1386: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=68 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(29) @ 1390: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=ee DATA=84] B[WR=0 RD=0 ADDR=a DATA=f0]

UVM_INFO scoreboard.sv(36) @ 1396: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=ee EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(29) @ 1400: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=1c DATA=db] B[WR=0 RD=0 ADDR=fd DATA=96]

UVM_INFO scoreboard.sv(36) @ 1406: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=1c EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(29) @ 1410: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=15 DATA=14] B[WR=0 RD=0 ADDR=59 DATA=8e]

UVM_INFO scoreboard.sv(36) @ 1416: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=15 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(29) @ 1420: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=16 DATA=c5] B[WR=0 RD=0 ADDR=fe DATA=4c]

UVM_INFO scoreboard.sv(36) @ 1426: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=16 EXPECTED=c4 ACTUAL=c4

UVM_INFO driver.sv(29) @ 1430: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=13 DATA=a3] B[WR=0 RD=0 ADDR=5a DATA=63]

UVM_INFO scoreboard.sv(36) @ 1436: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=13 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(29) @ 1440: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=8a DATA=72] B[WR=0 RD=0 ADDR=76 DATA=bc]

UVM_INFO scoreboard.sv(36) @ 1446: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=8a EXPECTED=5a ACTUAL=5a

UVM_INFO driver.sv(29) @ 1450: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=2c DATA=4f] B[WR=0 RD=0 ADDR=1f DATA=f9]

UVM_INFO scoreboard.sv(36) @ 1456: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=2c EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(29) @ 1460: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=c9 DATA=a4] B[WR=0 RD=0 ADDR=a6 DATA=62]

UVM_INFO scoreboard.sv(36) @ 1466: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=c9 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(29) @ 1470: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=2 DATA=b1] B[WR=0 RD=0 ADDR=e7 DATA=fb]

UVM_INFO scoreboard.sv(36) @ 1476: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=2 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(29) @ 1480: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=93 DATA=5] B[WR=0 RD=0 ADDR=bc DATA=6b]

UVM_INFO scoreboard.sv(36) @ 1486: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=93 EXPECTED=18 ACTUAL=18

UVM_INFO driver.sv(29) @ 1490: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=1c DATA=e5] B[WR=0 RD=0 ADDR=89 DATA=c3]

UVM_INFO scoreboard.sv(36) @ 1496: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=1c EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(29) @ 1500: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=21 DATA=4e] B[WR=0 RD=0 ADDR=a7 DATA=d9]

UVM_INFO scoreboard.sv(36) @ 1506: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=21 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(29) @ 1510: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=7a DATA=0] B[WR=0 RD=0 ADDR=2d DATA=d1]

UVM_INFO scoreboard.sv(36) @ 1516: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=7a EXPECTED=69 ACTUAL=69

UVM_INFO driver.sv(29) @ 1520: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=e3 DATA=4] B[WR=0 RD=0 ADDR=16 DATA=4]

UVM_INFO scoreboard.sv(36) @ 1526: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=e3 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(29) @ 1530: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=a9 DATA=fe] B[WR=0 RD=0 ADDR=48 DATA=5a]

UVM_INFO scoreboard.sv(36) @ 1536: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=a9 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(29) @ 1540: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=b1 DATA=db] B[WR=0 RD=0 ADDR=83 DATA=34]

UVM_INFO scoreboard.sv(36) @ 1546: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=b1 EXPECTED=ef ACTUAL=ef

UVM_INFO driver.sv(29) @ 1550: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=e6 DATA=9d] B[WR=0 RD=0 ADDR=cd DATA=c2]

UVM_INFO scoreboard.sv(36) @ 1556: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=e6 EXPECTED=65 ACTUAL=65

UVM_INFO driver.sv(29) @ 1560: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=0 ADDR=ad DATA=19] B[WR=0 RD=1 ADDR=21 DATA=16]

UVM_INFO scoreboard.sv(45) @ 1566: uvm_test_top.env_o.sb [SCOREBOARD] PORT B PASS: ADDR=21 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(29) @ 1570: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=0 ADDR=9 DATA=13] B[WR=0 RD=1 ADDR=9f DATA=1c]

UVM_INFO scoreboard.sv(45) @ 1576: uvm_test_top.env_o.sb [SCOREBOARD] PORT B PASS: ADDR=9f EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(29) @ 1580: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=0 ADDR=f5 DATA=db] B[WR=0 RD=1 ADDR=b2 DATA=9b]

UVM_INFO scoreboard.sv(45) @ 1586: uvm_test_top.env_o.sb [SCOREBOARD] PORT B PASS: ADDR=b2 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(29) @ 1590: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=0 ADDR=68 DATA=fd] B[WR=0 RD=1 ADDR=b3 DATA=97]

UVM_INFO scoreboard.sv(45) @ 1596: uvm_test_top.env_o.sb [SCOREBOARD] PORT B PASS: ADDR=b3 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(29) @ 1600: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=0 ADDR=8b DATA=df] B[WR=0 RD=1 ADDR=4a DATA=99]

UVM_INFO scoreboard.sv(45) @ 1606: uvm_test_top.env_o.sb [SCOREBOARD] PORT B PASS: ADDR=4a EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(29) @ 1610: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=81 DATA=82] B[WR=1 RD=0 ADDR=2a DATA=f]

UVM_INFO driver.sv(29) @ 1620: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=e5 DATA=ea] B[WR=1 RD=0 ADDR=55 DATA=20]

UVM_INFO driver.sv(29) @ 1630: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=d9 DATA=b9] B[WR=1 RD=0 ADDR=9f DATA=37]

UVM_INFO driver.sv(29) @ 1640: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=a8 DATA=a6] B[WR=1 RD=0 ADDR=3e DATA=bc]

UVM_INFO driver.sv(29) @ 1650: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=df DATA=14] B[WR=1 RD=0 ADDR=27 DATA=6e]

UVM_INFO driver.sv(29) @ 1660: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=ba DATA=56] B[WR=0 RD=1 ADDR=8c DATA=81]

UVM_INFO scoreboard.sv(36) @ 1666: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=ba EXPECTED=23 ACTUAL=23

UVM_INFO scoreboard.sv(45) @ 1666: uvm_test_top.env_o.sb [SCOREBOARD] PORT B PASS: ADDR=8c EXPECTED=ef ACTUAL=ef

UVM_INFO driver.sv(29) @ 1670: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=e7 DATA=79] B[WR=0 RD=1 ADDR=b7 DATA=a6]

UVM_INFO scoreboard.sv(36) @ 1676: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=e7 EXPECTED=0 ACTUAL=0

UVM_INFO scoreboard.sv(45) @ 1676: uvm_test_top.env_o.sb [SCOREBOARD] PORT B PASS: ADDR=b7 EXPECTED=e0 ACTUAL=e0

UVM_INFO driver.sv(29) @ 1680: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=73 DATA=8a] B[WR=0 RD=1 ADDR=51 DATA=a3]

UVM_INFO scoreboard.sv(36) @ 1686: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=73 EXPECTED=0 ACTUAL=0

UVM_INFO scoreboard.sv(45) @ 1686: uvm_test_top.env_o.sb [SCOREBOARD] PORT B PASS: ADDR=51 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(29) @ 1690: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=c5 DATA=85] B[WR=0 RD=1 ADDR=e4 DATA=6d]

UVM_INFO scoreboard.sv(36) @ 1696: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=c5 EXPECTED=0 ACTUAL=0

UVM_INFO scoreboard.sv(45) @ 1696: uvm_test_top.env_o.sb [SCOREBOARD] PORT B PASS: ADDR=e4 EXPECTED=2b ACTUAL=2b

UVM_INFO driver.sv(29) @ 1700: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=db DATA=9a] B[WR=0 RD=1 ADDR=39 DATA=60]

UVM_INFO scoreboard.sv(36) @ 1706: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=db EXPECTED=9c ACTUAL=9c

UVM_INFO scoreboard.sv(45) @ 1706: uvm_test_top.env_o.sb [SCOREBOARD] PORT B PASS: ADDR=39 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(29) @ 1710: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=a7 DATA=95] B[WR=1 RD=0 ADDR=a7 DATA=2c]

UVM_INFO driver.sv(29) @ 1720: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=92 DATA=d4] B[WR=1 RD=0 ADDR=92 DATA=2d]

UVM_INFO driver.sv(29) @ 1730: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=a2 DATA=eb] B[WR=1 RD=0 ADDR=a2 DATA=2a]

UVM_INFO driver.sv(29) @ 1740: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=fb DATA=53] B[WR=1 RD=0 ADDR=fb DATA=4f]

UVM_INFO driver.sv(29) @ 1750: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=b3 DATA=fe] B[WR=1 RD=0 ADDR=b3 DATA=4c]

UVM_INFO driver.sv(29) @ 1760: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=1 ADDR=2f DATA=1d] B[WR=0 RD=0 ADDR=9f DATA=2f]

UVM_INFO scoreboard.sv(36) @ 1766: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=2f EXPECTED=80 ACTUAL=80

UVM_INFO driver.sv(29) @ 1770: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=1 ADDR=e9 DATA=6e] B[WR=0 RD=0 ADDR=e DATA=b6]

UVM_INFO scoreboard.sv(36) @ 1776: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=e9 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(29) @ 1780: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=1 ADDR=56 DATA=b2] B[WR=0 RD=0 ADDR=5f DATA=fc]

UVM_INFO scoreboard.sv(36) @ 1786: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=56 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(29) @ 1790: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=1 ADDR=ec DATA=c4] B[WR=0 RD=0 ADDR=56 DATA=ad]

UVM_INFO scoreboard.sv(36) @ 1796: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=ec EXPECTED=bd ACTUAL=bd

UVM_INFO driver.sv(29) @ 1800: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=1 ADDR=6c DATA=90] B[WR=0 RD=0 ADDR=b4 DATA=7e]

UVM_INFO scoreboard.sv(36) @ 1806: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=6c EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(29) @ 1810: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=1c DATA=cc] B[WR=1 RD=1 ADDR=48 DATA=da]

UVM_INFO scoreboard.sv(36) @ 1816: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=1c EXPECTED=0 ACTUAL=0

UVM_INFO scoreboard.sv(45) @ 1816: uvm_test_top.env_o.sb [SCOREBOARD] PORT B PASS: ADDR=48 EXPECTED=7f ACTUAL=7f

UVM_INFO driver.sv(29) @ 1820: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=0 ADDR=44 DATA=bb] B[WR=1 RD=0 ADDR=16 DATA=eb]

UVM_INFO driver.sv(29) @ 1830: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=1 ADDR=e9 DATA=78] B[WR=0 RD=0 ADDR=b3 DATA=38]

UVM_INFO scoreboard.sv(36) @ 1836: uvm_test_top.env_o.sb [SCOREBOARD] PORT A PASS: ADDR=e9 EXPECTED=6e ACTUAL=6e

UVM_INFO driver.sv(29) @ 1840: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=0 ADDR=70 DATA=f6] B[WR=0 RD=0 ADDR=d3 DATA=d9]

UVM_INFO driver.sv(29) @ 1850: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=3b DATA=39] B[WR=1 RD=0 ADDR=56 DATA=af]

UVM_INFO sequence.sv(50) @ 1855: uvm_test_top.env_o.agt.seqr@@bseq [base_seq] Base sequence completed

UVM_INFO test.sv(22) @ 1855: uvm_test_top [base_test] End of testcase

UVM_INFO /usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_objection.svh(1274) @ 1855: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase

--- UVM Report Summary ---

** Report counts by severity

UVM_INFO : 263

UVM_WARNING : 0

UVM_ERROR : 0

UVM_FATAL : 0

** Report counts by id

[RNTST] 1

[SCOREBOARD] 73

[TEST_DONE] 1

[base_seq] 2

[base_test] 1

[driver] 185

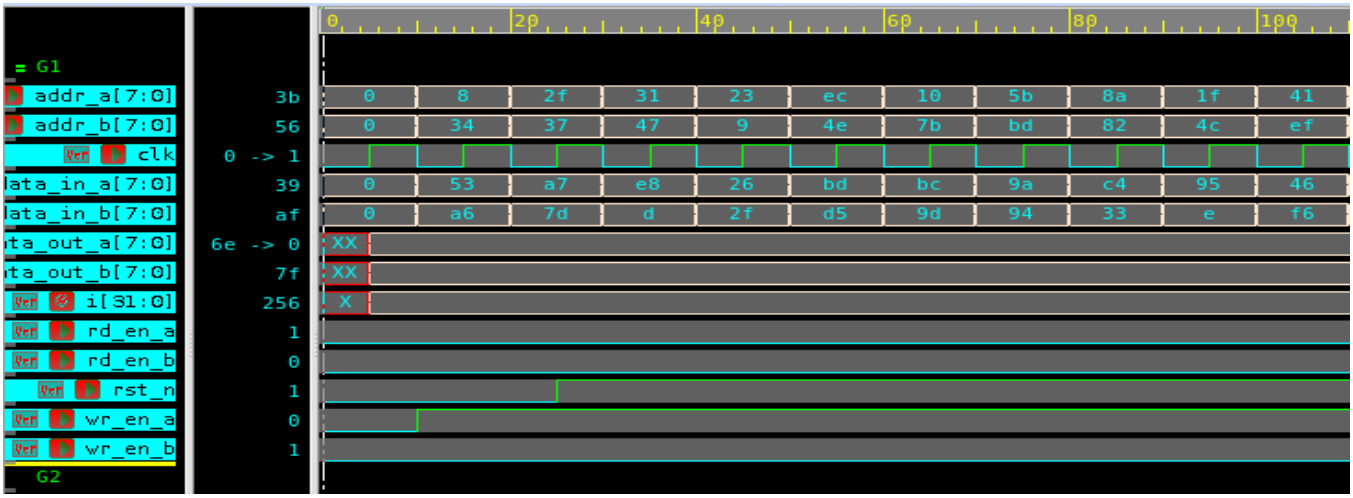
$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_root.svh", line 437.

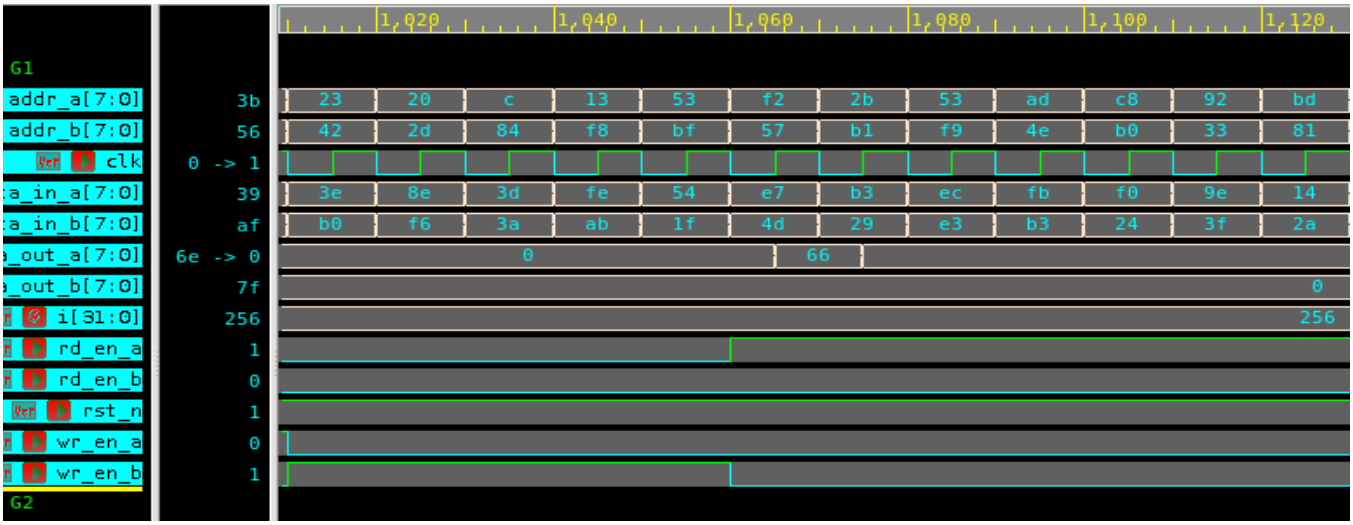
$finish at simulation time 1855

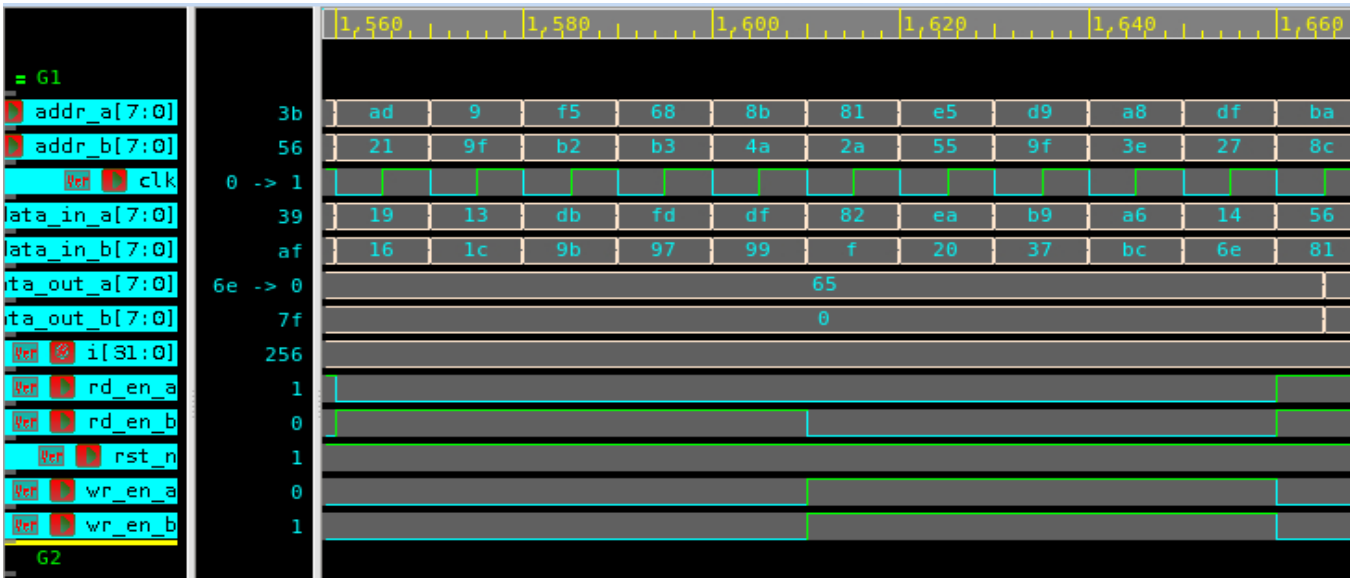
V C S S i m u l a t i o n R e p o r t

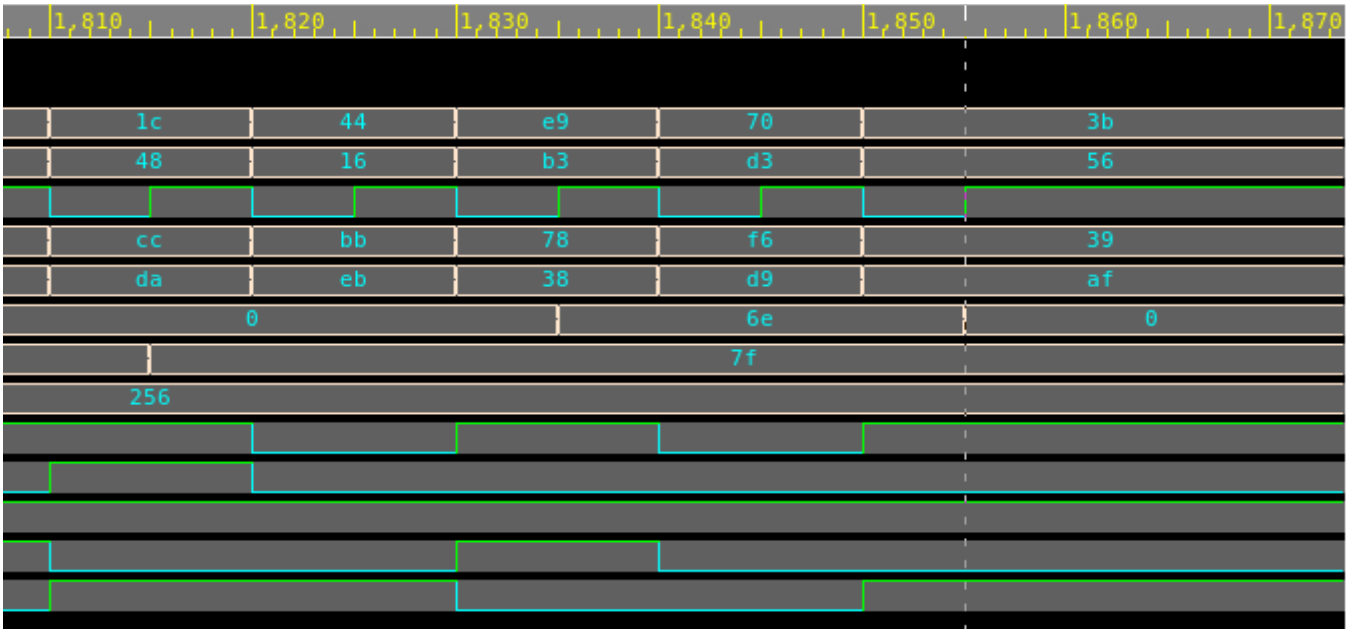
Time: 1855

Simulation waveform




1,020
1,040
1,060
1,080
1,100
1,120
G1
addr_a[7:0]
addr_b[7:0]
clk
a_in_a[7:0]
a_in_b[7:0]
_out_a[7:0]
_out_b[7:0]
i[31:0]
rd_en_a
rd_en_b
rst_n
wr_en_a
wr_en_b
G2
3b
56
0 -> 1
39
af
6e -> 0
7f
256
1
0
1
0
1
23 20 c 13 53 f2 2b 53 ad c8 92 bd
42 2d 84 f8 bf 57 b1 f9 4e b0 33 81
3e 8e 3d fe 54 e7 b3 ec fb f0 9e 14
b0 f6 3a ab 1f 4d 29 e3 b3 24 3f 2a
0
66
0
256


1,560
1,580
1,600
1,620
1,640
1,660
= G1
addr_a[7:0]
addr_b[7:0]
clk
lata_in_a[7:0]
lata_in_b[7:0]
ta_out_a[7:0]
ta_out_b[7:0]
i[31:0]
rd_en_a
rd_en_b
rst_n
wr_en_a
wr_en_b
G2
3b
56
0 -> 1
39
af
6e -> 0
7f
256
1
0
1
0
1
ad 9 f5 68 8b 81 e5 d9 a8 df ba
21 9f b2 b3 4a 2a 55 9f 3e 27 8c
19 13 db fd df 82 ea b9 a6 14 56
16 1c 9b 97 99 f 20 37 bc 6e 81
65
0


1,810
1,820
1,830
1,840
1,850
1,860
1,870
1c 44 e9 70 3b
48 16 b3 d3 56
cc bb 78 f6 39
da eb 38 d9 af
0 6e 0
7f
256

WRITE-READ CODE

```
//
module dual_port_ram #(parameter DATA_WIDTH = 8,parameter ADDR_WIDTH = 8,parameter RAM_DEPTH  = 256)(
        input  logic clk,rst_n,wr_en_a,rd_en_a,wr_en_b,rd_en_b,
      input  logic [DATA_WIDTH-1:0] data_in_a,data_in_b,
      input  logic [ADDR_WIDTH-1:0] addr_a,addr_b,
      output logic [DATA_WIDTH-1:0] data_out_a,
      output logic [DATA_WIDTH-1:0] data_out_b
);

logic [DATA_WIDTH-1:0] ram [0:RAM_DEPTH-1];
integer i;

always_ff @(posedge clk) begin
        if (!rst_n) begin
                data_out_a <= '0;
                data_out_b <= '0;

                for (i = 0; i < RAM_DEPTH; i = i + 1)
                        ram[i] <= '0;
        end
        else begin

                if (wr_en_a)
                        ram[addr_a] <= data_in_a;
```

```
        if (wr_en_b && !(wr_en_a && (addr_a == addr_b)))
            ram[addr_b] <= data_in_b;

        if (rd_en_a) begin
            if (wr_en_a)
                data_out_a <= data_in_a;
            else if (wr_en_b && (addr_a == addr_b))
                data_out_a <= data_in_b;
            else
                data_out_a <= ram[addr_a];
        end

        if (rd_en_b) begin
            if (wr_en_b && !(wr_en_a && (addr_a == addr_b)))
                data_out_b <= data_in_b;
            else if (wr_en_a && (addr_a == addr_b))
                data_out_b <= data_in_a;
            else
                data_out_b <= ram[addr_b];
        end
    end
end

endmodule
```

Run-Command

```
[b23n_sarif_727@prac3 dual_port_ram2]$ ls
```

agent.sv dual_port_ram.fsdb monitor.sv ram_pkg.sv sequencer.sv simv.cst test.sv

csrc dual_port_ram.sv novas_dump.log scoreboard.sv sequence.sv simv.daidir ucli.key

driver.sv environment.sv ram_if.sv sequence_item.sv simv tb_top.sv vc_hdrs.h

[b23n_sarif_727@prac3 dual_port_ram2]$ vcs -sverilog dual_port_ram.sv ram_if.sv ram_pkg.sv tb_top.sv -ntb_opts uvm -kdb -lca -debug_access+all -full64

[b23n_sarif_727@prac3 dual_port_ram2]$ ./simv

[b23n_sarif_727@prac3 dual_port_ram2]$ ./simv -verdi

Testbench

//

```
interface ram_if(input logic clk, input logic rst_n);
        logic wr_en_a;
      logic rd_en_a;


      logic wr_en_b;
      logic rd_en_b;


      logic [7:0] data_in_a;
      logic [7:0] data_in_b;
      logic [7:0] addr_a;
      logic [7:0] addr_b;
      logic [7:0] data_out_a;
      logic [7:0] data_out_b;


endinterface
```

```
//
class seq_item extends uvm_sequence_item;
        rand bit       wr_en_a;
       rand bit       rd_en_a;
        rand bit       wr_en_b;
       rand bit       rd_en_b;
        rand bit [7:0] data_in_a;
        rand bit [7:0] data_in_b;
        rand bit [7:0] addr_a;
        rand bit [7:0] addr_b;
        bit [7:0] data_out_a;
        bit [7:0] data_out_b;

         `uvm_object_utils_begin(seq_item)
         `uvm_field_int(wr_en_a, UVM_ALL_ON)
       `uvm_field_int(rd_en_a, UVM_ALL_ON)
       `uvm_field_int(wr_en_b, UVM_ALL_ON)
       `uvm_field_int(rd_en_b, UVM_ALL_ON)
       `uvm_field_int(data_in_a,UVM_ALL_ON)
       `uvm_field_int(data_in_b,UVM_ALL_ON)
       `uvm_field_int(addr_a, UVM_ALL_ON)
       `uvm_field_int(addr_b, UVM_ALL_ON)
       `uvm_field_int(data_out_a,UVM_ALL_ON)
       `uvm_field_int(data_out_b,UVM_ALL_ON)
         `uvm_object_utils_end

        function new(string name = "seq_item");
```

```
        super.new(name);
    endfunction

//      constraint c1 { !(wr_en_a && rd_en_a); !(wr_en_b && rd_en_b);}

endclass

//
class base_seq extends uvm_sequence #(seq_item);
    seq_item req;
    `uvm_object_utils(base_seq)

    function new(string name = "base_seq");
        super.new(name);
    endfunction

    task body();
        `uvm_info(get_type_name(), "Base sequence started", UVM_LOW)

        repeat(10) begin
            `uvm_do_with(req, { wr_en_a == 1; rd_en_a == 0; wr_en_b == 0;
rd_en_b == 0;})
        end

        repeat(10) begin
            `uvm_do_with(req, { wr_en_a == 0; rd_en_a == 0; wr_en_b == 1;
rd_en_b == 0;})
        end
```

```
		repeat(10) begin
			`uvm_do_with(req, { wr_en_a == 0; rd_en_a == 1; wr_en_b == 0; rd_en_b == 0;})
		end

		repeat(5) begin
			`uvm_do_with(req, { wr_en_a == 0; rd_en_a == 0; wr_en_b == 0; rd_en_b == 1;})
		end

		repeat(5) begin
			`uvm_do_with(req, { wr_en_a == 1; rd_en_a == 0; wr_en_b == 1; rd_en_b == 0; addr_a != addr_b;})
		end

		repeat(5) begin
			`uvm_do_with(req, { wr_en_a == 0; rd_en_a == 1; wr_en_b == 0; rd_en_b == 1;})
		end

		repeat(5) begin
			`uvm_do_with(req, { wr_en_a == 1; rd_en_a == 0; wr_en_b == 1; rd_en_b == 0; addr_a == addr_b;})
		end

		repeat(5) begin
			`uvm_do_with(req, { wr_en_a == 1; rd_en_a == 1;  wr_en_b == 0; rd_en_b == 0;})
```

```
            end

            repeat(20) begin
                `uvm_do(req)
            end
        endtask

endclass

//
class seqcr extends uvm_sequencer #(seq_item);
        `uvm_component_utils(seqcr)

        function new(string name = "seqcr", uvm_component parent = null);
            super.new(name, parent);
        endfunction

endclass

//
class driver extends uvm_driver #(seq_item);
        virtual ram_if vif;
        `uvm_component_utils(driver)

        function new(string name = "driver", uvm_component parent = null);
            super.new(name, parent);
        endfunction
```

```
function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if (!uvm_config_db#(virtual ram_if)::get(this, "", "vif", vif))
                `uvm_fatal(get_type_name(),"Virtual interface not found")
endfunction

task run_phase(uvm_phase phase);
        forever begin
                seq_item_port.get_next_item(req);
                @(negedge vif.clk);
                vif.wr_en_a   <= req.wr_en_a;
           vif.rd_en_a   <= req.rd_en_a;

           vif.wr_en_b   <= req.wr_en_b;
           vif.rd_en_b   <= req.rd_en_b;

           vif.data_in_a <= req.data_in_a;
           vif.data_in_b <= req.data_in_b;

                vif.addr_a    <= req.addr_a;
                vif.addr_b    <= req.addr_b;

                `uvm_info(get_type_name(),$sformatf("DRIVER: A[WR=%0d
RD=%0d ADDR=%0d DATA=%0h] B[WR=%0d RD=%0d ADDR=%0d
DATA=%0h]",
req.wr_en_a,req.rd_en_a,req.addr_a,req.data_in_a,req.wr_en_b,req.rd_en_b,req.addr_b,re
q.data_in_b), UVM_LOW)
```

```
                @(posedge vif.clk);
                seq_item_port.item_done();
            end
        endtask

endclass

//
class monitor extends uvm_monitor;
        virtual ram_if vif;
        uvm_analysis_port #(seq_item) mon_ap;
        `uvm_component_utils(monitor)

        function new(string name = "monitor", uvm_component parent = null);
            super.new(name, parent);
            mon_ap = new("mon_ap", this);
        endfunction

        function void build_phase(uvm_phase phase);
            super.build_phase(phase);
            if (!uvm_config_db#(virtual ram_if)::get(this, "", "vif", vif))
                `uvm_fatal(get_type_name(),"Virtual interface not found")
        endfunction

        task run_phase(uvm_phase phase);
            seq_item mon_item;
            forever begin
```

```
@(posedge vif.clk);

if (!vif.rst_n)
        continue;
#1;

if (vif.wr_en_a || vif.rd_en_a || vif.wr_en_b || vif.rd_en_b) begin
        mon_item = seq_item::type_id::create("mon_item");

        mon_item.wr_en_a = vif.wr_en_a;
 mon_item.rd_en_a = vif.rd_en_a;
 mon_item.wr_en_b = vif.wr_en_b;
 mon_item.rd_en_b = vif.rd_en_b;

 mon_item.data_in_a = vif.data_in_a;
 mon_item.data_in_b = vif.data_in_b;

 mon_item.addr_a = vif.addr_a;
 mon_item.addr_b = vif.addr_b;

 mon_item.data_out_a = vif.data_out_a;
 mon_item.data_out_b = vif.data_out_b;

        `uvm_info(get_type_name(),$sformatf("MONITOR:
A[WR=%0d RD=%0d ADDR=%0d OUT=%0h] B[WR=%0d RD=%0d ADDR=%0d
OUT=%0h]", mon_item.wr_en_a, mon_item.rd_en_a, mon_item.addr_a,
mon_item.data_out_a, mon_item.wr_en_b, mon_item.rd_en_b, mon_item.addr_b,
mon_item.data_out_b),UVM_HIGH)
```

```
                        mon_ap.write(mon_item);
                    end
                end
            endtask

endclass

//
class scoreboard extends uvm_scoreboard;
            uvm_analysis_imp #(seq_item, scoreboard) sb_imp;
            bit [7:0] ref_mem [0:255];
            `uvm_component_utils(scoreboard)

            function new(string name = "scoreboard", uvm_component parent = null);
                        super.new(name, parent);
                        sb_imp = new("sb_imp", this);
            endfunction

            function void build_phase(uvm_phase phase);
                        super.build_phase(phase);

                        for (int i = 0; i < 256; i++)
                                    ref_mem[i] = 8'h00;
            endfunction

            function void write(seq_item req);
                        bit [7:0] expected_a;
```

```
        bit [7:0] expected_b;

        expected_a = ref_mem[req.addr_a];
        expected_b = ref_mem[req.addr_b];

        if (req.wr_en_a && req.rd_en_a)
            expected_a = req.data_in_a;

        if (req.wr_en_b && req.rd_en_b)
            expected_b = req.data_in_b;

        if (req.wr_en_a && req.wr_en_b && (req.addr_a == req.addr_b)) begin
            if (req.rd_en_a)
                expected_a = req.data_in_a;
            if (req.rd_en_b)
                expected_b = req.data_in_a;
        end

        if (req.rd_en_a) begin
            if (req.data_out_a == expected_a) begin
                `uvm_info(get_type_name(),$sformatf("PORT A READ PASS:
ADDR=%0d EXPECTED=%0h ACTUAL=%0h",
req.addr_a,expected_a,req.data_out_a), UVM_LOW)
            end
            else begin
                `uvm_error(get_type_name(),$sformatf("PORT A READ
FAIL: ADDR=%0d EXPECTED=%0h ACTUAL=%0h",
req.addr_a,expected_a,req.data_out_a))
            end
```

```
        end


        if (req.rd_en_b) begin
            if (req.data_out_b == expected_b) begin
                `uvm_info(get_type_name(),$sformatf("PORT B READ PASS:
ADDR=%0d EXPECTED=%0h ACTUAL=%0h",
req.addr_b,expected_b,req.data_out_b),UVM_LOW)
            end
            else begin
                `uvm_error(get_type_name(),$sformatf("PORT B READ FAIL:
ADDR=%0d EXPECTED=%0h
ACTUAL=%0h",req.addr_b,expected_b,req.data_out_b))
            end
        end


        if (req.wr_en_a)
            ref_mem[req.addr_a] = req.data_in_a;


        if (req.wr_en_b) begin
            if (!(req.wr_en_a && (req.addr_a == req.addr_b))) begin
                ref_mem[req.addr_b] = req.data_in_b;
            end
        end


        if (req.wr_en_a && req.wr_en_b && (req.addr_a == req.addr_b)) begin
            `uvm_info(get_type_name(), $sformatf("WRITE COLLISION:
ADDR=%0d PORT A PRIORITY DATA=%0h",
req.addr_a,req.data_in_a),UVM_LOW)
        end
```

```
        endfunction

endclass

//
class agent extends uvm_agent;
        `uvm_component_utils(agent)
        driver  drv;
      seqcr   seqr;
      monitor mon;

        function new(string name = "agent", uvm_component parent = null);
                super.new(name, parent);
        endfunction

        function void build_phase(uvm_phase phase);
                super.build_phase(phase);

                if (get_is_active() == UVM_ACTIVE) begin
                        drv = driver::type_id::create("drv", this);
                        seqr = seqcr::type_id::create("seqr", this);
                end
                mon = monitor::type_id::create("mon", this);
        endfunction

        function void connect_phase(uvm_phase phase);
                super.connect_phase(phase);
```

```
        if (get_is_active() == UVM_ACTIVE)
            drv.seq_item_port.connect(seqr.seq_item_export);
    endfunction

endclass

//
class env extends uvm_env;
    `uvm_component_utils(env)
    agent      agt;
    scoreboard sb;

    function new(string name = "env", uvm_component parent = null);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        agt = agent::type_id::create("agt", this);
        sb = scoreboard::type_id::create("sb", this);
    endfunction

    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        agt.mon.mon_ap.connect(sb.sb_imp);
    endfunction
```

```
endclass

//
class base_test extends uvm_test;
	`uvm_component_utils(base_test)
	env env_o;

	function new(string name = "base_test",uvm_component parent = null);
		super.new(name, parent);
	endfunction

	function void build_phase(uvm_phase phase);
		super.build_phase(phase);
		env_o = env::type_id::create("env_o", this);
	endfunction

	task run_phase(uvm_phase phase);
		base_seq bseq;

		phase.raise_objection(this);
		bseq = base_seq::type_id::create("bseq");
		bseq.start(env_o.agt.seqr);
		#20;
		phase.drop_objection(this);
		`uvm_info(get_type_name(),"End of testcases", UVM_LOW)
	endtask
```

```
endclass

//
package ram_pkg;

    `include "uvm_macros.svh"
     import uvm_pkg::*;
    `include "sequence_item.sv"
    `include "sequence.sv"
    `include "sequencer.sv"
    `include "driver.sv"
    `include "monitor.sv"
    `include "scoreboard.sv"
    `include "agent.sv"
    `include "environment.sv"
    `include "test.sv"

endpackage

//
`include "uvm_macros.svh"

import uvm_pkg::*;
import ram_pkg::*;

module tb_top;
```

```
bit clk;
bit rst_n;
always #5 clk = ~clk;

ram_if vif(clk, rst_n);

dual_port_ram #(.DATA_WIDTH(8),.ADDR_WIDTH(8),.RAM_DEPTH(256)) DUT
(.clk (vif.clk),.rst_n (vif.rst_n),.wr_en_a (vif.wr_en_a), .rd_en_a (vif.rd_en_a),.wr_en_b
(vif.wr_en_b),.rd_en_b (vif.rd_en_b),.data_in_a (vif.data_in_a),.data_in_b
(vif.data_in_b),.addr_a (vif.addr_a),.addr_b (vif.addr_b),.data_out_a
(vif.data_out_a),.data_out_b (vif.data_out_b));

initial begin
      rst_n = 1'b0;
      vif.wr_en_a = 0;
    vif.rd_en_a = 0;

    vif.wr_en_b = 0;
    vif.rd_en_b = 0;

    vif.data_in_a = 0;
    vif.data_in_b = 0;

    vif.addr_a = 0;
    vif.addr_b = 0;

    repeat(3)
      @(posedge clk);
      rst_n = 1'b1;
```

```
end

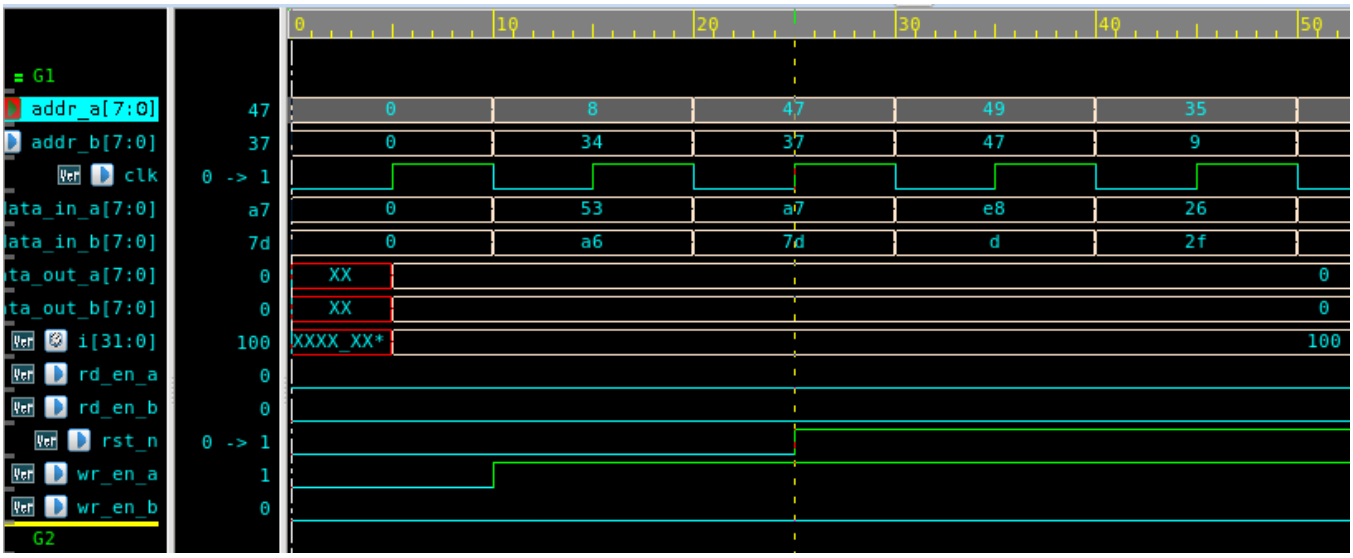
initial begin
        uvm_config_db#(virtual ram_if)::set(null, "*","vif",vif);
end

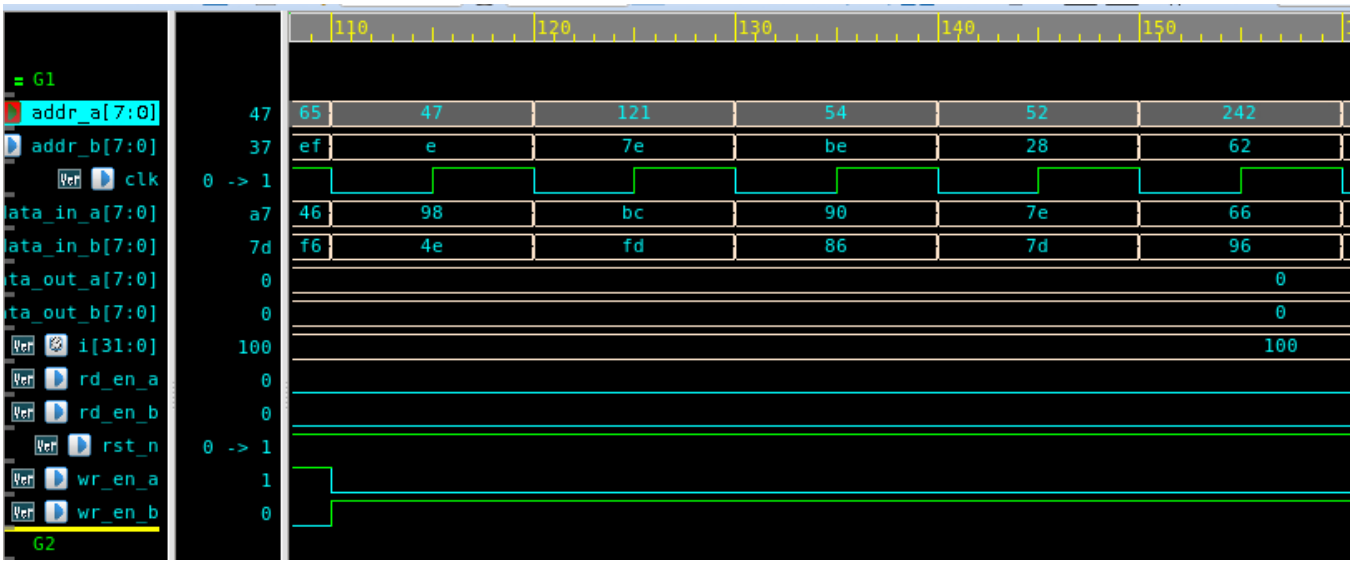
initial begin
        run_test("base_test");
end

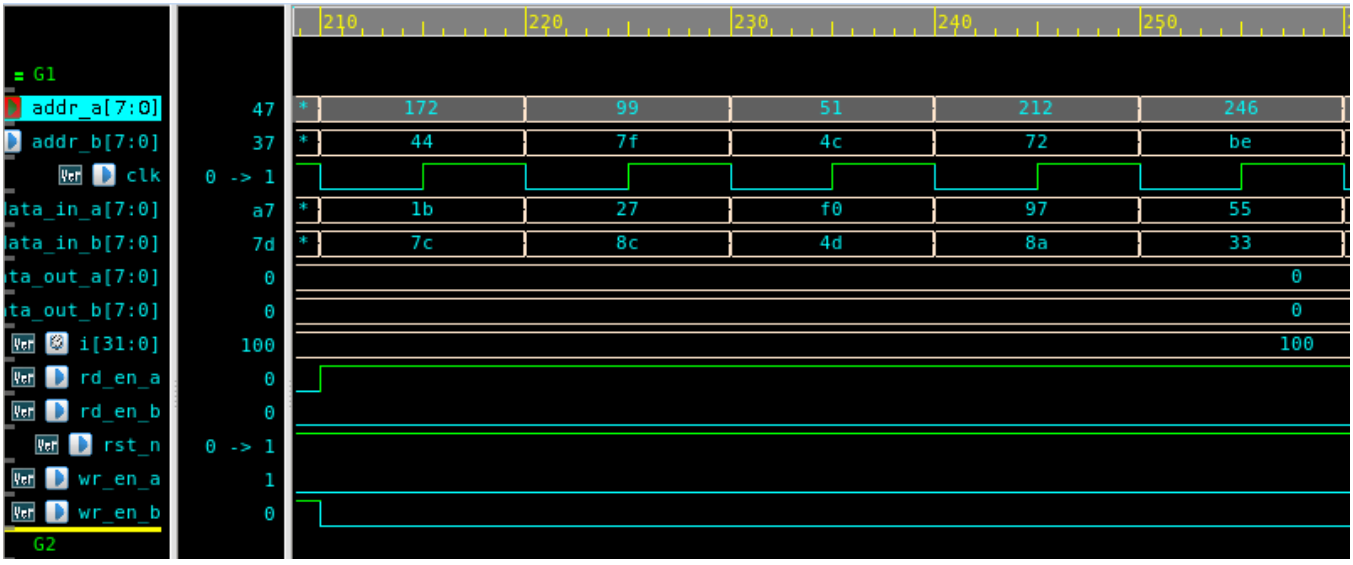
initial begin
        $fsdbDumpfile("dual_port_ram.fsdb");
      $fsdbDumpvars(0, tb_top);
end

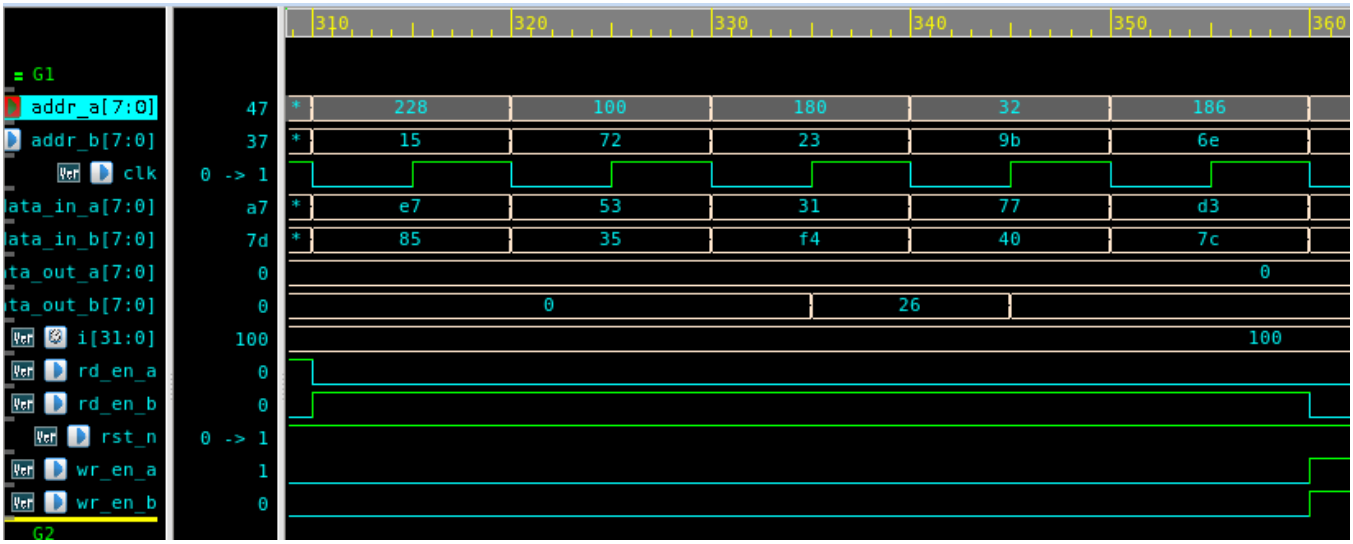
endmodule
```

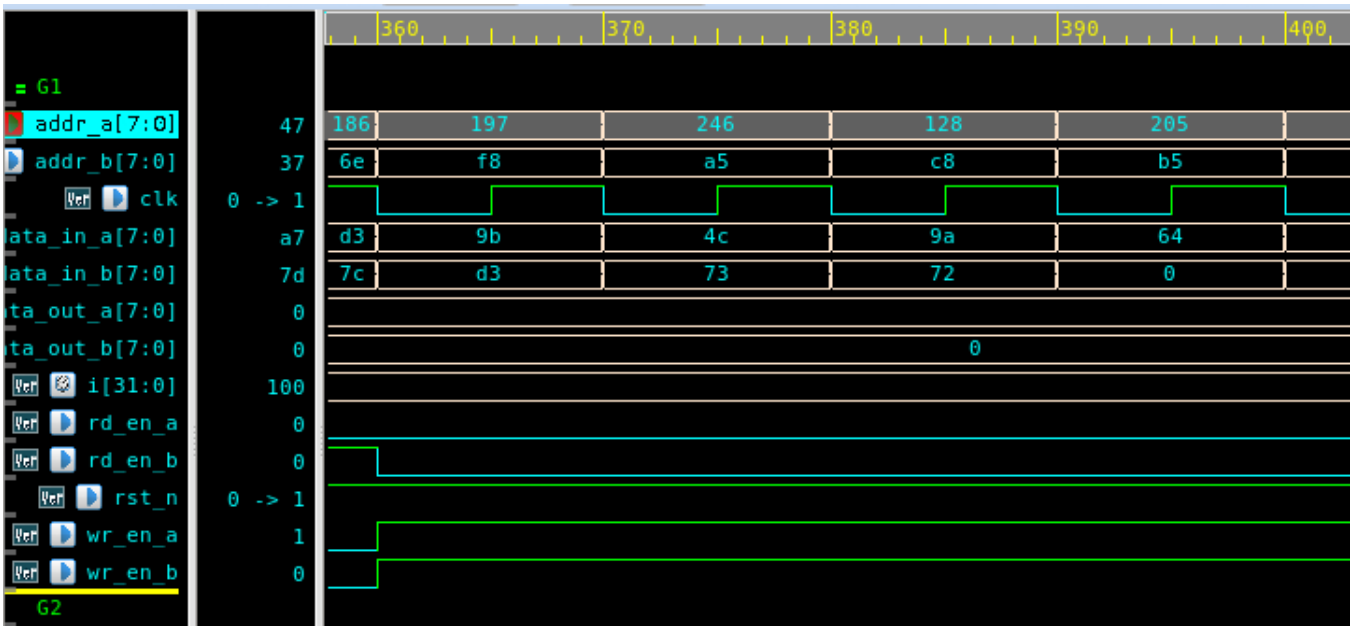
Simulation waveform

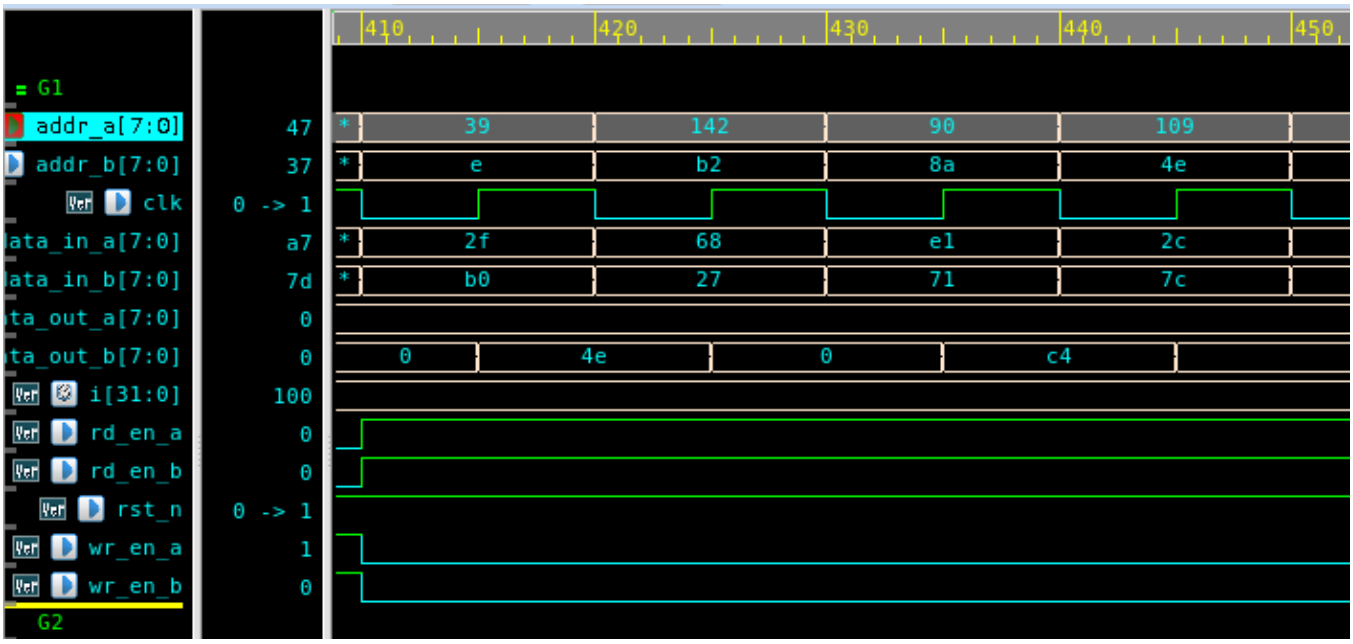


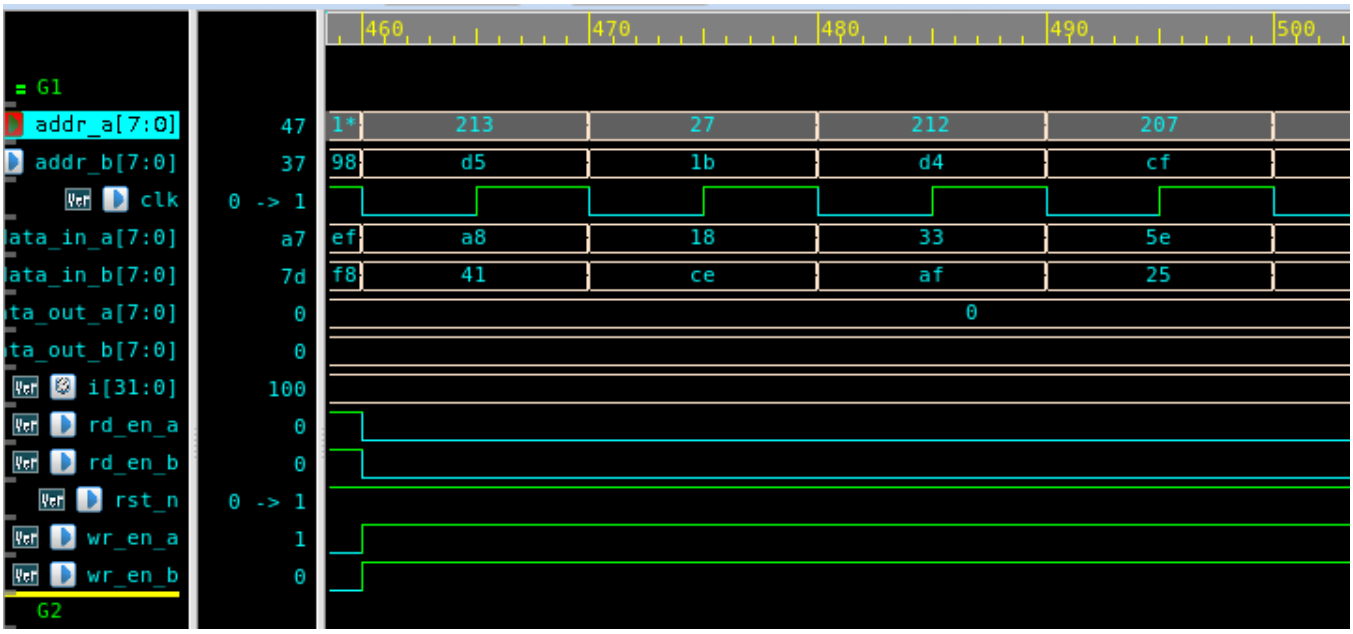

110
120
130
140
150
G1
addr_a[7:0] 47 65 47 121 54 52 242
addr_b[7:0] 37 ef e 7e be 28 62
clk 0 -> 1
ata_in_a[7:0] a7 46 98 bc 90 7e 66
ata_in_b[7:0] 7d f6 4e fd 86 7d 96
ta_out_a[7:0] 0 0
ta_out_b[7:0] 0 0
i[31:0] 100 100
rd_en_a 0
rd_en_b 0
rst_n 0 -> 1
wr_en_a 1
wr_en_b 0
G2

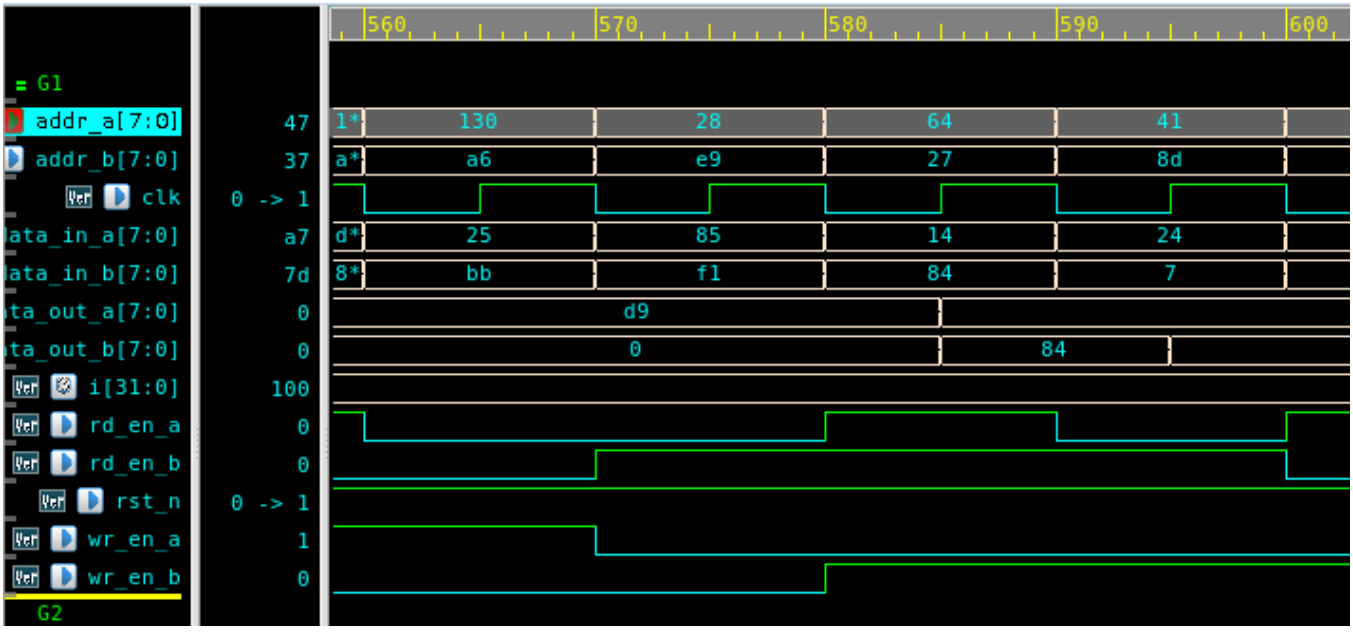

210
220
230
240
250
G1
addr_a[7:0] 47 * 172 99 51 212 246
addr_b[7:0] 37 * 44 7f 4c 72 be
clk 0 -> 1
ata_in_a[7:0] a7 * 1b 27 f0 97 55
ata_in_b[7:0] 7d * 7c 8c 4d 8a 33
ta_out_a[7:0] 0 0
ta_out_b[7:0] 0 0
i[31:0] 100 100
rd_en_a 0
rd_en_b 0
rst_n 0 -> 1
wr_en_a 1
wr_en_b 0
G2

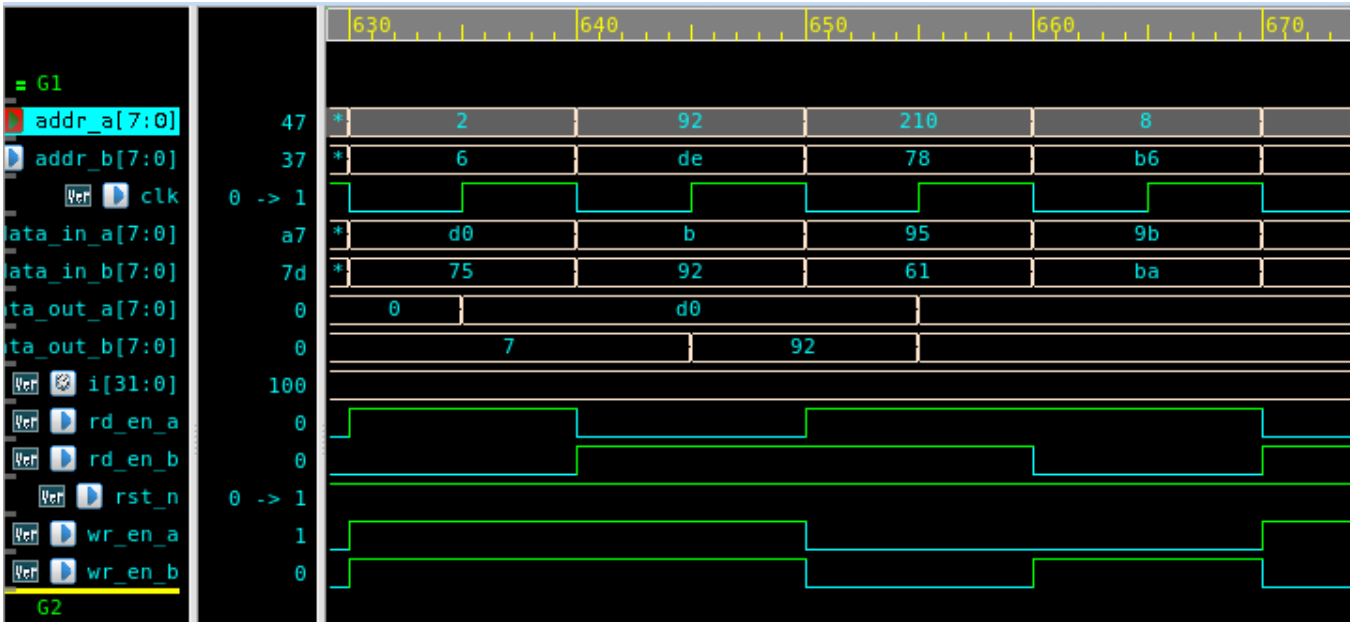

310
320
330
340
350
360
G1
addr_a[7:0] 47 * 228 100 180 32 186
addr_b[7:0] 37 * 15 72 23 9b 6e
clk 0 -> 1
ata_in_a[7:0] a7 * e7 53 31 77 d3
ata_in_b[7:0] 7d * 85 35 f4 40 7c
ta_out_a[7:0] 0 0
ta_out_b[7:0] 0 0 26
i[31:0] 100 100
rd_en_a 0
rd_en_b 0
rst_n 0 -> 1
wr_en_a 1
wr_en_b 0
G2


360
370
380
390
400
G1
addr_a[7:0]
47
186
197
246
128
205
addr_b[7:0]
37
6e
f8
a5
c8
b5
clk
0 -> 1
ata_in_a[7:0]
a7
d3
9b
4c
9a
64
ata_in_b[7:0]
7d
7c
d3
73
72
0
ta_out_a[7:0]
0
ta_out_b[7:0]
0
0
i[31:0]
100
rd_en_a
0
rd_en_b
0
rst_n
0 -> 1
wr_en_a
1
wr_en_b
0
G2


410
420
430
440
450
G1
addr_a[7:0]
47
*
39
142
90
109
addr_b[7:0]
37
*
e
b2
8a
4e
clk
0 -> 1
ata_in_a[7:0]
a7
*
2f
68
e1
2c
ata_in_b[7:0]
7d
*
b0
27
71
7c
ta_out_a[7:0]
0
ta_out_b[7:0]
0
0
4e
0
c4
i[31:0]
100
rd_en_a
0
rd_en_b
0
rst_n
0 -> 1
wr_en_a
1
wr_en_b
0
G2


G1
addr_a[7:0]
addr_b[7:0]
clk
data_in_a[7:0]
data_in_b[7:0]
data_out_a[7:0]
data_out_b[7:0]
i[31:0]
rd_en_a
rd_en_b
rst_n
wr_en_a
wr_en_b
G2
47
37
0 -> 1
a7
7d
0
0
100
0
0
0 -> 1
1
0
460
470
480
490
500
1*
213
27
212
207
98
d5
1b
d4
cf
ef
a8
18
33
5e
f8
41
ce
af
25
0


G1
addr_a[7:0]
addr_b[7:0]
clk
data_in_a[7:0]
data_in_b[7:0]
data_out_a[7:0]
data_out_b[7:0]
i[31:0]
rd_en_a
rd_en_b
rst_n
wr_en_a
wr_en_b
G2
47
37
0 -> 1
a7
7d
0
0
100
0
0
0 -> 1
1
0
560
570
580
590
600
1*
130
28
64
41
a*
a6
e9
27
8d
d*
25
85
14
24
8*
bb
f1
84
7
d9
0
84



Simulation Outputs

UVM_INFO sequence.sv(11) @ 0: uvm_test_top.env_o.agt.seqr@@bseq [base_seq] Base sequence started

UVM_INFO driver.sv(32) @ 10: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=8 DATA=53] B[WR=0 RD=0 ADDR=52 DATA=a6]

UVM_INFO driver.sv(32) @ 20: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=47 DATA=a7] B[WR=0 RD=0 ADDR=55 DATA=7d]

UVM_INFO driver.sv(32) @ 30: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=49 DATA=e8] B[WR=0 RD=0 ADDR=71 DATA=d]

UVM_INFO driver.sv(32) @ 40: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=35 DATA=26] B[WR=0 RD=0 ADDR=9 DATA=2f]

UVM_INFO driver.sv(32) @ 50: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=236 DATA=bd] B[WR=0 RD=0 ADDR=78 DATA=d5]

UVM_INFO driver.sv(32) @ 60: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=16 DATA=bc] B[WR=0 RD=0 ADDR=123 DATA=9d]

UVM_INFO driver.sv(32) @ 70: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=91 DATA=9a] B[WR=0 RD=0 ADDR=189 DATA=94]

UVM_INFO driver.sv(32) @ 80: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=138 DATA=c4] B[WR=0 RD=0 ADDR=130 DATA=33]

UVM_INFO driver.sv(32) @ 90: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=31 DATA=95] B[WR=0 RD=0 ADDR=76 DATA=e]

UVM_INFO driver.sv(32) @ 100: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=65 DATA=46] B[WR=0 RD=0 ADDR=239 DATA=f6]

UVM_INFO driver.sv(32) @ 110: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=0 ADDR=47 DATA=98] B[WR=1 RD=0 ADDR=14 DATA=4e]

UVM_INFO driver.sv(32) @ 120: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=0 ADDR=121 DATA=bc] B[WR=1 RD=0 ADDR=126 DATA=fd]

UVM_INFO driver.sv(32) @ 130: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=0 ADDR=54 DATA=90] B[WR=1 RD=0 ADDR=190 DATA=86]

UVM_INFO driver.sv(32) @ 140: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=0 ADDR=52 DATA=7e] B[WR=1 RD=0 ADDR=40 DATA=7d]

UVM_INFO driver.sv(32) @ 150: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=0 ADDR=242 DATA=66] B[WR=1 RD=0 ADDR=98 DATA=96]

UVM_INFO driver.sv(32) @ 160: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=0 ADDR=68 DATA=44] B[WR=1 RD=0 ADDR=168 DATA=ec]

UVM_INFO driver.sv(32) @ 170: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=0 ADDR=46 DATA=b4] B[WR=1 RD=0 ADDR=0 DATA=22]

UVM_INFO driver.sv(32) @ 180: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=0 ADDR=183 DATA=e0] B[WR=1 RD=0 ADDR=219 DATA=72]

UVM_INFO driver.sv(32) @ 190: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=0 ADDR=149 DATA=c2] B[WR=1 RD=0 ADDR=24 DATA=2c]

UVM_INFO driver.sv(32) @ 200: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=0 ADDR=122 DATA=69] B[WR=1 RD=0 ADDR=77 DATA=a7]

UVM_INFO driver.sv(32) @ 210: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=172 DATA=1b] B[WR=0 RD=0 ADDR=68 DATA=7c]

UVM_INFO scoreboard.sv(41) @ 216: uvm_test_top.env_o.sb [scoreboard] PORT A READ PASS: ADDR=172 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(32) @ 220: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=99 DATA=27] B[WR=0 RD=0 ADDR=127 DATA=8c]

UVM_INFO scoreboard.sv(41) @ 226: uvm_test_top.env_o.sb [scoreboard] PORT A READ PASS: ADDR=99 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(32) @ 230: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=51 DATA=f0] B[WR=0 RD=0 ADDR=76 DATA=4d]

UVM_INFO scoreboard.sv(41) @ 236: uvm_test_top.env_o.sb [scoreboard] PORT A READ PASS: ADDR=51 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(32) @ 240: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=212 DATA=97] B[WR=0 RD=0 ADDR=114 DATA=8a]

UVM_INFO scoreboard.sv(41) @ 246: uvm_test_top.env_o.sb [scoreboard] PORT A READ PASS: ADDR=212 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(32) @ 250: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=246 DATA=55] B[WR=0 RD=0 ADDR=190 DATA=33]

UVM_INFO scoreboard.sv(41) @ 256: uvm_test_top.env_o.sb [scoreboard] PORT A READ PASS: ADDR=246 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(32) @ 260: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=140 DATA=ef] B[WR=0 RD=0 ADDR=181 DATA=75]

UVM_INFO scoreboard.sv(41) @ 266: uvm_test_top.env_o.sb [scoreboard] PORT A READ PASS: ADDR=140 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(32) @ 270: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=79 DATA=c1] B[WR=0 RD=0 ADDR=73 DATA=82]

UVM_INFO scoreboard.sv(41) @ 276: uvm_test_top.env_o.sb [scoreboard] PORT A READ PASS: ADDR=79 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(32) @ 280: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=228 DATA=96] B[WR=0 RD=0 ADDR=205 DATA=91]

UVM_INFO scoreboard.sv(41) @ 286: uvm_test_top.env_o.sb [scoreboard] PORT A READ PASS: ADDR=228 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(32) @ 290: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=130 DATA=fd] B[WR=0 RD=0 ADDR=27 DATA=1a]

UVM_INFO scoreboard.sv(41) @ 296: uvm_test_top.env_o.sb [scoreboard] PORT A READ PASS: ADDR=130 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(32) @ 300: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=72 DATA=7f] B[WR=0 RD=0 ADDR=65 DATA=23]

UVM_INFO scoreboard.sv(41) @ 306: uvm_test_top.env_o.sb [scoreboard] PORT A READ PASS: ADDR=72 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(32) @ 310: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=0 ADDR=228 DATA=e7] B[WR=0 RD=1 ADDR=21 DATA=85]

UVM_INFO scoreboard.sv(50) @ 316: uvm_test_top.env_o.sb [scoreboard] PORT B READ PASS: ADDR=21 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(32) @ 320: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=0 ADDR=100 DATA=53] B[WR=0 RD=1 ADDR=114 DATA=35]

UVM_INFO scoreboard.sv(50) @ 326: uvm_test_top.env_o.sb [scoreboard] PORT B READ PASS: ADDR=114 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(32) @ 330: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=0 ADDR=180 DATA=31] B[WR=0 RD=1 ADDR=35 DATA=f4]

UVM_INFO scoreboard.sv(50) @ 336: uvm_test_top.env_o.sb [scoreboard] PORT B READ PASS: ADDR=35 EXPECTED=26 ACTUAL=26

UVM_INFO driver.sv(32) @ 340: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=0 ADDR=32 DATA=77] B[WR=0 RD=1 ADDR=155 DATA=40]

UVM_INFO scoreboard.sv(50) @ 346: uvm_test_top.env_o.sb [scoreboard] PORT B READ PASS: ADDR=155 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(32) @ 350: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=0 ADDR=186 DATA=d3] B[WR=0 RD=1 ADDR=110 DATA=7c]

UVM_INFO scoreboard.sv(50) @ 356: uvm_test_top.env_o.sb [scoreboard] PORT B READ PASS: ADDR=110 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(32) @ 360: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=197 DATA=9b] B[WR=1 RD=0 ADDR=248 DATA=d3]

UVM_INFO driver.sv(32) @ 370: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=246 DATA=4c] B[WR=1 RD=0 ADDR=165 DATA=73]

UVM_INFO driver.sv(32) @ 380: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=128 DATA=9a] B[WR=1 RD=0 ADDR=200 DATA=72]

UVM_INFO driver.sv(32) @ 390: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=205 DATA=64] B[WR=1 RD=0 ADDR=181 DATA=0]

UVM_INFO driver.sv(32) @ 400: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=195 DATA=47] B[WR=1 RD=0 ADDR=209 DATA=ce]

UVM_INFO driver.sv(32) @ 410: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=39 DATA=2f] B[WR=0 RD=1 ADDR=14 DATA=b0]

UVM_INFO scoreboard.sv(41) @ 416: uvm_test_top.env_o.sb [scoreboard] PORT A READ PASS: ADDR=39 EXPECTED=0 ACTUAL=0

UVM_INFO scoreboard.sv(50) @ 416: uvm_test_top.env_o.sb [scoreboard] PORT B READ PASS: ADDR=14 EXPECTED=4e ACTUAL=4e

UVM_INFO driver.sv(32) @ 420: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=142 DATA=68] B[WR=0 RD=1 ADDR=178 DATA=27]

UVM_INFO scoreboard.sv(41) @ 426: uvm_test_top.env_o.sb [scoreboard] PORT A READ PASS: ADDR=142 EXPECTED=0 ACTUAL=0

UVM_INFO scoreboard.sv(50) @ 426: uvm_test_top.env_o.sb [scoreboard] PORT B READ PASS: ADDR=178 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(32) @ 430: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=90 DATA=e1] B[WR=0 RD=1 ADDR=138 DATA=71]

UVM_INFO scoreboard.sv(41) @ 436: uvm_test_top.env_o.sb [scoreboard] PORT A READ PASS: ADDR=90 EXPECTED=0 ACTUAL=0

UVM_INFO scoreboard.sv(50) @ 436: uvm_test_top.env_o.sb [scoreboard] PORT B READ PASS: ADDR=138 EXPECTED=c4 ACTUAL=c4

UVM_INFO driver.sv(32) @ 440: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=109 DATA=2c] B[WR=0 RD=1 ADDR=78 DATA=7c]

UVM_INFO scoreboard.sv(41) @ 446: uvm_test_top.env_o.sb [scoreboard] PORT A READ PASS: ADDR=109 EXPECTED=0 ACTUAL=0

UVM_INFO scoreboard.sv(50) @ 446: uvm_test_top.env_o.sb [scoreboard] PORT B READ PASS: ADDR=78 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(32) @ 450: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=177 DATA=ef] B[WR=0 RD=1 ADDR=152 DATA=f8]

UVM_INFO scoreboard.sv(41) @ 456: uvm_test_top.env_o.sb [scoreboard] PORT A READ PASS: ADDR=177 EXPECTED=0 ACTUAL=0

UVM_INFO scoreboard.sv(50) @ 456: uvm_test_top.env_o.sb [scoreboard] PORT B READ PASS: ADDR=152 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(32) @ 460: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=213 DATA=a8] B[WR=1 RD=0 ADDR=213 DATA=41]

UVM_INFO scoreboard.sv(67) @ 466: uvm_test_top.env_o.sb [scoreboard] WRITE COLLISION: ADDR=213 PORT A PRIORITY DATA=a8

UVM_INFO driver.sv(32) @ 470: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=27 DATA=18] B[WR=1 RD=0 ADDR=27 DATA=ce]

UVM_INFO scoreboard.sv(67) @ 476: uvm_test_top.env_o.sb [scoreboard] WRITE COLLISION: ADDR=27 PORT A PRIORITY DATA=18

UVM_INFO driver.sv(32) @ 480: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=212 DATA=33] B[WR=1 RD=0 ADDR=212 DATA=af]

UVM_INFO scoreboard.sv(67) @ 486: uvm_test_top.env_o.sb [scoreboard] WRITE COLLISION: ADDR=212 PORT A PRIORITY DATA=33

UVM_INFO driver.sv(32) @ 490: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=207 DATA=5e] B[WR=1 RD=0 ADDR=207 DATA=25]

UVM_INFO scoreboard.sv(67) @ 496: uvm_test_top.env_o.sb [scoreboard] WRITE COLLISION: ADDR=207 PORT A PRIORITY DATA=5e

UVM_INFO driver.sv(32) @ 500: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=152 DATA=2b] B[WR=1 RD=0 ADDR=152 DATA=5b]

UVM_INFO scoreboard.sv(67) @ 506: uvm_test_top.env_o.sb [scoreboard] WRITE COLLISION: ADDR=152 PORT A PRIORITY DATA=2b

UVM_INFO driver.sv(32) @ 510: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=1 ADDR=186 DATA=23] B[WR=0 RD=0 ADDR=149 DATA=18]

UVM_INFO scoreboard.sv(41) @ 516: uvm_test_top.env_o.sb [scoreboard] PORT A READ PASS: ADDR=186 EXPECTED=23 ACTUAL=23

UVM_INFO driver.sv(32) @ 520: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=1 ADDR=62 DATA=ce] B[WR=0 RD=0 ADDR=228 DATA=5a]

UVM_INFO scoreboard.sv(41) @ 526: uvm_test_top.env_o.sb [scoreboard] PORT A READ PASS: ADDR=62 EXPECTED=ce ACTUAL=ce

UVM_INFO driver.sv(32) @ 530: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=1 ADDR=204 DATA=d0] B[WR=0 RD=0 ADDR=162 DATA=4d]

UVM_INFO scoreboard.sv(41) @ 536: uvm_test_top.env_o.sb [scoreboard] PORT A READ PASS: ADDR=204 EXPECTED=d0 ACTUAL=d0

UVM_INFO driver.sv(32) @ 540: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=1 ADDR=156 DATA=cf] B[WR=0 RD=0 ADDR=244 DATA=21]

UVM_INFO scoreboard.sv(41) @ 546: uvm_test_top.env_o.sb [scoreboard] PORT A READ PASS: ADDR=156 EXPECTED=cf ACTUAL=cf

UVM_INFO driver.sv(32) @ 550: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=1 ADDR=128 DATA=d9] B[WR=0 RD=0 ADDR=167 DATA=89]

UVM_INFO scoreboard.sv(41) @ 556: uvm_test_top.env_o.sb [scoreboard] PORT A READ PASS: ADDR=128 EXPECTED=d9 ACTUAL=d9

UVM_INFO driver.sv(32) @ 560: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=130 DATA=25] B[WR=0 RD=0 ADDR=166 DATA=bb]

UVM_INFO driver.sv(32) @ 570: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=0 ADDR=28 DATA=85] B[WR=0 RD=1 ADDR=233 DATA=f1]

UVM_INFO scoreboard.sv(50) @ 576: uvm_test_top.env_o.sb [scoreboard] PORT B READ PASS: ADDR=233 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(32) @ 580: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=64 DATA=14] B[WR=1 RD=1 ADDR=39 DATA=84]

UVM_INFO scoreboard.sv(41) @ 586: uvm_test_top.env_o.sb [scoreboard] PORT A READ PASS: ADDR=64 EXPECTED=0 ACTUAL=0

UVM_INFO scoreboard.sv(50) @ 586: uvm_test_top.env_o.sb [scoreboard] PORT B READ PASS: ADDR=39 EXPECTED=84 ACTUAL=84

UVM_INFO driver.sv(32) @ 590: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=0 ADDR=41 DATA=24] B[WR=1 RD=1 ADDR=141 DATA=7]

UVM_INFO scoreboard.sv(50) @ 596: uvm_test_top.env_o.sb [scoreboard] PORT B READ PASS: ADDR=141 EXPECTED=7 ACTUAL=7

UVM_INFO driver.sv(32) @ 600: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=54 DATA=5b] B[WR=1 RD=0 ADDR=35 DATA=80]

UVM_INFO scoreboard.sv(41) @ 606: uvm_test_top.env_o.sb [scoreboard] PORT A READ PASS: ADDR=54 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(32) @ 610: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=0 ADDR=237 DATA=36] B[WR=1 RD=0 ADDR=250 DATA=af]

UVM_INFO driver.sv(32) @ 620: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=0 ADDR=25 DATA=3a] B[WR=0 RD=0 ADDR=254 DATA=e2]

UVM_INFO driver.sv(32) @ 630: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=1 ADDR=2 DATA=d0] B[WR=1 RD=0 ADDR=6 DATA=75]

UVM_INFO scoreboard.sv(41) @ 636: uvm_test_top.env_o.sb [scoreboard] PORT A READ PASS: ADDR=2 EXPECTED=d0 ACTUAL=d0

UVM_INFO driver.sv(32) @ 640: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=92 DATA=b] B[WR=1 RD=1 ADDR=222 DATA=92]

UVM_INFO scoreboard.sv(50) @ 646: uvm_test_top.env_o.sb [scoreboard] PORT B READ PASS: ADDR=222 EXPECTED=92 ACTUAL=92

UVM_INFO driver.sv(32) @ 650: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=210 DATA=95] B[WR=0 RD=1 ADDR=120 DATA=61]

UVM_INFO scoreboard.sv(41) @ 656: uvm_test_top.env_o.sb [scoreboard] PORT A READ PASS: ADDR=210 EXPECTED=0 ACTUAL=0

UVM_INFO scoreboard.sv(50) @ 656: uvm_test_top.env_o.sb [scoreboard] PORT B READ PASS: ADDR=120 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(32) @ 660: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=8 DATA=9b] B[WR=1 RD=0 ADDR=182 DATA=ba]

UVM_INFO scoreboard.sv(41) @ 666: uvm_test_top.env_o.sb [scoreboard] PORT A READ PASS: ADDR=8 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(32) @ 670: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=250 DATA=13] B[WR=0 RD=1 ADDR=87 DATA=1f]

UVM_INFO scoreboard.sv(50) @ 676: uvm_test_top.env_o.sb [scoreboard] PORT B READ PASS: ADDR=87 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(32) @ 680: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=0 ADDR=61 DATA=60] B[WR=0 RD=0 ADDR=158 DATA=e8]

UVM_INFO driver.sv(32) @ 690: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=29 DATA=0] B[WR=1 RD=0 ADDR=183 DATA=28]

UVM_INFO scoreboard.sv(41) @ 696: uvm_test_top.env_o.sb [scoreboard] PORT A READ PASS: ADDR=29 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(32) @ 700: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=1 ADDR=191 DATA=7a] B[WR=1 RD=1 ADDR=23 DATA=12]

UVM_INFO scoreboard.sv(41) @ 706: uvm_test_top.env_o.sb [scoreboard] PORT A READ PASS: ADDR=191 EXPECTED=7a ACTUAL=7a

UVM_INFO scoreboard.sv(50) @ 706: uvm_test_top.env_o.sb [scoreboard] PORT B READ PASS: ADDR=23 EXPECTED=12 ACTUAL=12

UVM_INFO driver.sv(32) @ 710: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=1 ADDR=96 DATA=85] B[WR=0 RD=0 ADDR=89 DATA=86]

UVM_INFO scoreboard.sv(41) @ 716: uvm_test_top.env_o.sb [scoreboard] PORT A READ PASS: ADDR=96 EXPECTED=85 ACTUAL=85

UVM_INFO driver.sv(32) @ 720: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=1 ADDR=43 DATA=e4] B[WR=1 RD=0 ADDR=82 DATA=f1]

UVM_INFO scoreboard.sv(41) @ 726: uvm_test_top.env_o.sb [scoreboard] PORT A READ PASS: ADDR=43 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(32) @ 730: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=0 ADDR=161 DATA=3c] B[WR=1 RD=1 ADDR=23 DATA=2]

UVM_INFO scoreboard.sv(50) @ 736: uvm_test_top.env_o.sb [scoreboard] PORT B READ PASS: ADDR=23 EXPECTED=2 ACTUAL=2

UVM_INFO driver.sv(32) @ 740: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=1 RD=0 ADDR=18 DATA=b7] B[WR=0 RD=1 ADDR=142 DATA=a3]

UVM_INFO scoreboard.sv(50) @ 746: uvm_test_top.env_o.sb [scoreboard] PORT B READ PASS: ADDR=142 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(32) @ 750: uvm_test_top.env_o.agt.drv [driver] DRIVER: A[WR=0 RD=0 ADDR=239 DATA=ac] B[WR=0 RD=0 ADDR=195 DATA=aa]

UVM_INFO test.sv(23) @ 775: uvm_test_top [base_test] End of testcases

UVM_INFO /usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_objection.svh(1274) @ 775: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase

--- UVM Report Summary ---

** Report counts by severity

UVM_INFO : 132

UVM_WARNING : 0

UVM_ERROR : 0

UVM_FATAL : 0

** Report counts by id

[RNTST] 1

[TEST_DONE] 1

[base_seq] 1

[base_test] 1

[driver] 75

[scoreboard] 53

$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_root.svh", line 437.

$finish at simulation time 775

V C S S i m u l a t i o n R e p o r t

Time: 775

CPU Time: 0.150 seconds; Data structure size: 0.6Mb

```
************************************  Steps to run UVM Code in VCS+Verdi
*****************************

1) Whenever you open a new terminal, always source the environment file with the
command "source <environment file>".(In our case name of environment file is
env_file).

2) After successfully sourcing environment file, run the command to compile in vcs
 "vcs -sverilog <UVM_code_file_name> -lca -kdb -ntb_opts uvm -debug_access+all
-full64"

3) Once you get this message "./simv up to date", you will find your simv file
ready for further simulation(you can check that in directory by using ls command
there).

4)Run command "./simv" to see results on terminal or else run "./run -verdi" to do
simulation using verdi.

5) To add coverage in your simulation result run command
   " vcs -sverilog adder.sv tb_adder.sv -full64 -debug_access+all -kdb -lca  -cm
line+tgl+fsm+cond" and then
  "verdi -cov -covdir simv.vdb/".(Before using this command, Do check if simv.vdb
file has been generated after compilation using vcs command or not).




export NOVAS_HOME=/usr/synopsys/verdi/W-2024.09-1
export VERDI_HOME=/usr/synopsys/verdi/W-2024.09-1
export VCS_HOME=/usr/synopsys/vcs/W-2024.09-1
```

```
module tb;
  Parent p_h;
  Child  c_h1, c_h2;

  initial begin
    c_h1 = new();
    c_h1.id = 99;

    p_h = c_h1;

    // Downcast to access child properties
    if ($cast(c_h2, p_h)) begin
      $display("Cast Success! Accessing child id via c_h2: %0d", c_h2.id);
    end else begin
      $error("Cast Failed!");
    end
```

```
  class Parent;
    int data = 100;
  endclass

  class Child extends Parent;
    int id = 5;
  endclass

  module tb;
    Parent p_h;
    Child  c_h;

    initial begin
      c_h = new();


      p_h = c_h;


      $display("p_h.data = %0d", p_h.data);

       $display("p_h.id = %0d", p_h.id);


      $display("c_h.id = %0d", c_h.id);
    end
  endmodule
```

```
  end
endmodule
```

```
class ParentWithMsg;
  string msg = "Hello from Parent";

  function void display();
    $display("Method: %s", msg);
  endfunction
endclass

class ChildWithMsg extends ParentWithMsg;
  string msg = "Hello from Child";

  function void display();
    $display("Method: %s", msg);
  endfunction
endclass

  module tb;
    ParentWithMsg p_h;
    ChildWithMsg  c_h;

    initial begin
      c_h = new();
      p_h = c_h;


      $display("p_h.msg -> %s", p_h.msg);
      $display("c_h.msg -> %s", c_h.msg);


      p_h.display();
      c_h.display();
    end
  endmodule
```

```
class BaseDriver;
  virtual task send_pkt();
    $display("Sending a standard packet.");
  endtask
endclass

class BadPktDriver extends BaseDriver;
  task send_pkt();



    $display("Sending an error/corrupted packet.");
  endtask
endclass

module tb;
  BaseDriver b_h;
  BadPktDriver child_h;

  initial begin
    child_h = new();
    b_h = child_h;


    b_h.send_pkt();

  end
endmodule
```

Single Port RAM Verification by using UVM

// Dut of single port ram

module dut_ram #(parameter DATA_WIDTH = 8,parameter ADDR_WIDTH = 4)(

input logic clk,

input logic rst_n,

input logic write_en,

input logic [ADDR_WIDTH-1:0] addr,

input logic [DATA_WIDTH-1:0] data_in,

output logic [DATA_WIDTH-1:0] data_out

);

logic [DATA_WIDTH-1:0] mem [0:(1<<ADDR_WIDTH)-1];

always_ff @(posedge clk or negedge rst_n) begin

```
        if (!rst_n) begin
                data_out <= '0;

                for (int i = 0; i < (1<<ADDR_WIDTH); i++) begin
                        mem[i] <= '0;
                end
        end
        else begin
                if (write_en) begin
                        mem[addr] <= data_in;
                end
                else begin
                        data_out <= mem[addr];
                end
        end
end

endmodule
```

Run-Command

[b23n_sarif_727@prac3 single_port_ram]$ ls

agent.sv dut_ram.fsdb novas.rc sequence_item.sv simv.daidir test.sv vdCovLog

coverage.sv environment.sv ram_if.sv sequencer.sv simv.vdb ucli.key

csrc monitor.sv ram_pkg.sv sequence.sv single_port_ram_design.sv vc_hdrs.h

driver.sv novas_dump.log scoreboard.sv simv tb_top.sv vdCov.conf

[b23n_sarif_727@prac3 single_port_ram]$ vcs -sverilog single_port_ram_design.sv ram_if.sv ram_pkg.sv tb_top.sv -ntb_opts uvm -kdb -lca -debug_access+all -full64

[b23n_sarif_727@prac3 single_port_ram]$ ./simv

[b23n_sarif_727@prac3 single_port_ram]$ ./simv -verdi

[b23n_sarif_727@prac3 single_port_ram]$ verdi -cov -covdir simv.vdb/

Testbench

```
// interface of single port ram
interface ram_if #(parameter DATA_WIDTH = 8, parameter ADDR_WIDTH = 4)(input logic clk);
    logic               rst_n;
   logic               write_en;
   logic [ADDR_WIDTH-1:0] addr;
   logic [DATA_WIDTH-1:0] data_in;
   logic [DATA_WIDTH-1:0] data_out;

    clocking drv_cb @(posedge clk);
        default input #1step output #1ns;
        output write_en, addr, data_in;
    endclocking

    clocking mon_cb @(posedge clk);
        default input #1step output #1ns;
        input write_en, addr, data_in, data_out;
    endclocking

    modport DRV (clocking drv_cb,output rst_n);
    modport MON (clocking mon_cb,input rst_n);

endinterface
```

```
// sequence_item of single port ram
class seq_item extends uvm_sequence_item;
        rand bit       write_en;
        rand bit [3:0] addr;
       rand bit [7:0] data_in;
        bit [7:0]      data_out;

        `uvm_object_utils_begin(seq_item)
        `uvm_field_int(write_en, UVM_ALL_ON)
        `uvm_field_int(addr,     UVM_ALL_ON)
        `uvm_field_int(data_in,  UVM_ALL_ON)
        `uvm_field_int(data_out, UVM_ALL_ON)
        `uvm_object_utils_end

        function new(string name = "seq_item");
                super.new(name);
        endfunction

endclass

//
class base_seq extends uvm_sequence #(seq_item);
        seq_item req;
        `uvm_object_utils(base_seq)

        function new(string name = "base_seq");
```

```
        super.new(name);
    endfunction

    task body();
        `uvm_info(get_type_name(), "Base sequence has been started:", UVM_LOW)

        for (int i = 0; i < 16; i++) begin
            `uvm_do_with(req, { write_en == 1; addr == i; data_in == (8'hAA + 2*i);})
        end

        for (int i = 0; i < 16; i++) begin
            `uvm_do_with(req, { write_en == 0; addr == i;})
        end

        repeat(5) begin
            `uvm_do_with(req, { write_en== 1;})
        end

        repeat(5) begin
            `uvm_do_with(req, { write_en == 0;})
        end

        `uvm_info(get_type_name(), "All the Base sequence completely generated", UVM_LOW)
    endtask
endclass
```

```
//
class sequencer extends uvm_sequencer #(seq_item);
	`uvm_component_utils(sequencer)

	function new(string name = "sequencer", uvm_component parent = null);
		super.new(name, parent);
	endfunction

	function void build_phase(uvm_phase phase);
		super.build_phase(phase);
	endfunction

endclass

//
class driver extends uvm_driver #(seq_item);
	virtual ram_if.DRV vif;
	`uvm_component_utils(driver)

	function new(string name = "driver", uvm_component parent = null);
		super.new(name, parent);
	endfunction

	function void build_phase(uvm_phase phase);
		super.build_phase(phase);
		if (!uvm_config_db#(virtual ram_if.DRV)::get(this,"","vif",vif))
```

```
                `uvm_fatal(get_type_name(),"Virtual interface is not found from top level")
        endfunction

        task run_phase(uvm_phase phase);
                wait(vif.rst_n == 1'b1);
                @(vif.drv_cb);

                forever begin
                        seq_item_port.get_next_item(req);
                        @(vif.drv_cb);
                        vif.drv_cb.write_en <= req.write_en;
                    vif.drv_cb.addr     <= req.addr;
                    vif.drv_cb.data_in  <= req.data_in;
                        `uvm_info(get_type_name(),$sformatf("DRIVER: Write_en=%0d ADDR=%0d DATA_IN=%0h",req.write_en,req.addr,req.data_in), UVM_LOW)
                        if(!req.write_en) begin
                                @(vif.drv_cb);
                        end
                        seq_item_port.item_done();
                end
        endtask

endclass

//
class monitor extends uvm_monitor;
        virtual ram_if.MON vif;
```

```
	uvm_analysis_port #(seq_item) mon_ap;
	`uvm_component_utils(monitor)

	function new(string name = "monitor",uvm_component parent = null);
		super.new(name, parent);
		mon_ap = new("mon_ap", this);
	endfunction

	function void build_phase(uvm_phase phase);
		super.build_phase(phase);
		if (!uvm_config_db#(virtual ram_if.MON)::get(this,"","vif",vif))
			`uvm_fatal(get_type_name(),"Virtual interface is not found from top level")
	endfunction

	task run_phase(uvm_phase phase);
		seq_item mon_item;
		forever begin
			@(vif.mon_cb);
			if (!vif.rst_n)
				continue;
			mon_item = seq_item::type_id::create("mon_item");

			mon_item.write_en = vif.mon_cb.write_en;
			mon_item.addr = vif.mon_cb.addr;
			mon_item.data_in = vif.mon_cb.data_in;

			if(!vif.mon_cb.write_en) begin
```

```
                @(vif.mon_cb);
            end
            mon_item.data_out = vif.mon_cb.data_out;
            `uvm_info(get_type_name(),$sformatf("MONITOR: Write_en=%0d ADDR=%0d DATA_IN=%0h DATA_OUT=%0h",mon_item.write_en,mon_item.addr,mon_item.data_in,mon_item.data_out), UVM_LOW)

            mon_ap.write(mon_item);
        end
    endtask

endclass

//
class scoreboard extends uvm_scoreboard;
    `uvm_component_utils(scoreboard)
    uvm_analysis_imp #(seq_item, scoreboard) sb_imp;

    bit [7:0] ref_mem [0:15];

    function new(string name = "scoreboard",uvm_component parent = null);
        super.new(name, parent);
        sb_imp = new("sb_imp", this);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
```

```
            foreach (ref_mem[i]) begin
                ref_mem[i] = 8'h00;
            end
        endfunction

        function void write(seq_item req);

            if (req.write_en) begin
                ref_mem[req.addr] = req.data_in;
                `uvm_info("SCOREBOARD",$sformatf("WRITE PASS:
ADDR=%0d DATA=%0h",req.addr,req.data_in),UVM_LOW)
            end
            else begin
                if (req.data_out === ref_mem[req.addr]) begin
                    `uvm_info("SCOREBOARD",$sformatf("READ PASS:
ADDR=%0d EXPECTED=%0h
ACTUAL=%0h",req.addr,ref_mem[req.addr],req.data_out),UVM_LOW)
                end
                else begin
                    `uvm_error("SCOREBOARD",$sformatf("READ FAIL:
ADDR=%0d EXPECTED=%0h
ACTUAL=%0h",req.addr,ref_mem[req.addr],req.data_out))
                end
            end
        endfunction

endclass
```

```
//
class ram_coverage extends uvm_subscriber #(seq_item);
        `uvm_component_utils(ram_coverage)

        bit       cov_write_en;
      bit [3:0]  cov_addr;
      bit [7:0]  cov_data_in;

        covergroup ram_cg;

      cp_operation:
         coverpoint cov_write_en {
            bins read  = {0};
            bins write = {1};
         }

      cp_address:
         coverpoint cov_addr {
            bins low_addr  = {[0:3]};
            bins mid_addr  = {[4:11]};
            bins high_addr = {[12:15]};
         }

      cp_data_in:
         coverpoint cov_data_in {
            bins zero     = {8'h00};
            bins max      = {8'hFF};
```

```
        bins low_data = {[8'h01:8'h3F]};
        bins mid_data = {[8'h40:8'hBF]};
        bins high_data = {[8'hC0:8'hFE]};
      }

  cross_operation_address:
      cross cp_operation, cp_address;

  cross_operation_data:
      cross cp_operation, cp_data_in;

  cross_write_addr_data:
      cross cp_operation, cp_address, cp_data_in;

endgroup

function new(string name = "ram_coverage",uvm_component parent = null);
        super.new(name, parent);
        ram_cg = new();
endfunction

function void write(seq_item t);
        cov_write_en = t.write_en;
      cov_addr     = t.addr;
      cov_data_in  = t.data_in;
        ram_cg.sample();
endfunction
```

```
endclass
//
class agent extends uvm_agent;
	`uvm_component_utils(agent)

	driver    drv;
	sequencer seqr;
	monitor   mon;

	function new(string name = "agent", uvm_component parent = null);
		super.new(name,parent);
	endfunction

	function void build_phase(uvm_phase phase);
		super.build_phase(phase);
		drv = driver::type_id::create("drv", this);
		seqr =sequencer::type_id::create("seqr", this);
		mon =monitor::type_id::create("mon", this);
	endfunction

	function void connect_phase(uvm_phase phase);
		super.connect_phase(phase);
		drv.seq_item_port.connect(seqr.seq_item_export);
	endfunction

endclass
```

```
//
class env extends uvm_env;
	`uvm_component_utils(env)
	agent		agt;
	scoreboard	sb;
	ram_coverage	cov;

	function new(string name = "env", uvm_component parent = null);
		super.new(name, parent);
	endfunction

	function void build_phase(uvm_phase phase);
		super.build_phase(phase);
		agt =agent::type_id::create("agt", this);
		sb =scoreboard::type_id::create("sb", this);
		cov =ram_coverage::type_id::create("cov", this);
	endfunction

	function void connect_phase(uvm_phase phase);
		super.connect_phase(phase);
		agt.mon.mon_ap.connect(sb.sb_imp);
		agt.mon.mon_ap.connect(cov.analysis_export);
	endfunction

endclass
```

```
//
class base_test extends uvm_test;
	`uvm_component_utils(base_test)
	env env_o;

	function new(string name = "base_test",uvm_component parent = null);
		super.new(name, parent);
	endfunction

	function void build_phase(uvm_phase phase);
		super.build_phase(phase);
		env_o =env::type_id::create("env_o",this);
	endfunction

	task run_phase(uvm_phase phase);
		base_seq bseq;
		phase.raise_objection(this);
		bseq =base_seq::type_id::create("bseq");
		bseq.start(env_o.agt.seqr);
		phase.drop_objection(this);
		`uvm_info(get_type_name(),"End of testcases",UVM_LOW)
	endtask

endclass

//
`include "uvm_macros.svh"
```

```
import uvm_pkg::*;
import ram_pkg::*;

module tb_top;

bit clk;
always #5 clk = ~clk;

ram_if #(.DATA_WIDTH(8),.ADDR_WIDTH(4)) vif(clk);
dut_ram #(.DATA_WIDTH(8),.ADDR_WIDTH(4)) DUT
(.clk(vif.clk),.rst_n(vif.rst_n),.write_en(vif.write_en),.addr(vif.addr),.data_in(vif.data_in),
.data_out(vif.data_out));

initial begin
        vif.rst_n = 1'b0;
        vif.write_en = 1'b0;
      vif.addr     = '0;
      vif.data_in  = '0;

        repeat(3) @(negedge clk);
        vif.rst_n = 1'b1;
end

initial begin
        uvm_config_db#(virtual ram_if.DRV)::set(null,"*","vif",vif);
        uvm_config_db#(virtual ram_if.MON)::set(null,"*","vif",vif);
end
```

```
initial begin
        run_test("base_test");
end

initial begin
        $fsdbDumpfile("dut_ram.fsdb");
      $fsdbDumpvars(0,tb_top);
end

endmodule

//
package ram_pkg;

        `include "uvm_macros.svh"
         import uvm_pkg::*;
        `include "sequence_item.sv"
      `include "sequence.sv"
      `include "sequencer.sv"
      `include "driver.sv"
      `include "monitor.sv"
      `include "scoreboard.sv"
      `include "coverage.sv"
      `include "agent.sv"
      `include "environment.sv"
      `include "test.sv"
```

endpackage

Simulation Outputs

UVM_INFO sequence.sv(11) @ 0: uvm_test_top.env_o.agt.seqr@@bseq [base_seq] Base sequence has been started:

UVM_INFO monitor.sv(34) @ 45: uvm_test_top.env_o.agt.mon [monitor] MONITOR: Write_en=0 ADDR=0 DATA_IN=0 DATA_OUT=0

UVM_INFO scoreboard.sv(29) @ 45: uvm_test_top.env_o.sb [SCOREBOARD] READ PASS: ADDR=0 EXPECTED=0 ACTUAL=0

UVM_INFO driver.sv(26) @ 45: uvm_test_top.env_o.agt.drv [driver] DRIVER: Write_en=1 ADDR=0 DATA_IN=aa

UVM_INFO monitor.sv(34) @ 55: uvm_test_top.env_o.agt.mon [monitor] MONITOR: Write_en=1 ADDR=0 DATA_IN=aa DATA_OUT=0

UVM_INFO scoreboard.sv(25) @ 55: uvm_test_top.env_o.sb [SCOREBOARD] WRITE PASS: ADDR=0 DATA=aa

UVM_INFO driver.sv(26) @ 55: uvm_test_top.env_o.agt.drv [driver] DRIVER: Write_en=1 ADDR=1 DATA_IN=ac

UVM_INFO monitor.sv(34) @ 65: uvm_test_top.env_o.agt.mon [monitor] MONITOR: Write_en=1 ADDR=1 DATA_IN=ac DATA_OUT=0

UVM_INFO scoreboard.sv(25) @ 65: uvm_test_top.env_o.sb [SCOREBOARD] WRITE PASS: ADDR=1 DATA=ac

UVM_INFO driver.sv(26) @ 65: uvm_test_top.env_o.agt.drv [driver] DRIVER: Write_en=1 ADDR=2 DATA_IN=ae

UVM_INFO monitor.sv(34) @ 75: uvm_test_top.env_o.agt.mon [monitor] MONITOR: Write_en=1 ADDR=2 DATA_IN=ae DATA_OUT=0

UVM_INFO scoreboard.sv(25) @ 75: uvm_test_top.env_o.sb [SCOREBOARD] WRITE PASS: ADDR=2 DATA=ae

UVM_INFO driver.sv(26) @ 75: uvm_test_top.env_o.agt.drv [driver] DRIVER: Write_en=1 ADDR=3 DATA_IN=b0

UVM_INFO monitor.sv(34) @ 85: uvm_test_top.env_o.agt.mon [monitor] MONITOR: Write_en=1 ADDR=3 DATA_IN=b0 DATA_OUT=0

UVM_INFO scoreboard.sv(25) @ 85: uvm_test_top.env_o.sb [SCOREBOARD] WRITE PASS: ADDR=3 DATA=b0

UVM_INFO driver.sv(26) @ 85: uvm_test_top.env_o.agt.drv [driver] DRIVER: Write_en=1 ADDR=4 DATA_IN=b2

UVM_INFO monitor.sv(34) @ 95: uvm_test_top.env_o.agt.mon [monitor] MONITOR: Write_en=1 ADDR=4 DATA_IN=b2 DATA_OUT=0

UVM_INFO scoreboard.sv(25) @ 95: uvm_test_top.env_o.sb [SCOREBOARD] WRITE PASS: ADDR=4 DATA=b2

UVM_INFO driver.sv(26) @ 95: uvm_test_top.env_o.agt.drv [driver] DRIVER: Write_en=1 ADDR=5 DATA_IN=b4

UVM_INFO monitor.sv(34) @ 105: uvm_test_top.env_o.agt.mon [monitor] MONITOR: Write_en=1 ADDR=5 DATA_IN=b4 DATA_OUT=0

UVM_INFO scoreboard.sv(25) @ 105: uvm_test_top.env_o.sb [SCOREBOARD] WRITE PASS: ADDR=5 DATA=b4

UVM_INFO driver.sv(26) @ 105: uvm_test_top.env_o.agt.drv [driver] DRIVER: Write_en=1 ADDR=6 DATA_IN=b6

UVM_INFO monitor.sv(34) @ 115: uvm_test_top.env_o.agt.mon [monitor] MONITOR: Write_en=1 ADDR=6 DATA_IN=b6 DATA_OUT=0

UVM_INFO scoreboard.sv(25) @ 115: uvm_test_top.env_o.sb [SCOREBOARD] WRITE PASS: ADDR=6 DATA=b6

UVM_INFO driver.sv(26) @ 115: uvm_test_top.env_o.agt.drv [driver] DRIVER: Write_en=1 ADDR=7 DATA_IN=b8

UVM_INFO monitor.sv(34) @ 125: uvm_test_top.env_o.agt.mon [monitor] MONITOR: Write_en=1 ADDR=7 DATA_IN=b8 DATA_OUT=0

UVM_INFO scoreboard.sv(25) @ 125: uvm_test_top.env_o.sb [SCOREBOARD] WRITE PASS: ADDR=7 DATA=b8

UVM_INFO driver.sv(26) @ 125: uvm_test_top.env_o.agt.drv [driver] DRIVER: Write_en=1 ADDR=8 DATA_IN=ba

UVM_INFO monitor.sv(34) @ 135: uvm_test_top.env_o.agt.mon [monitor] MONITOR: Write_en=1 ADDR=8 DATA_IN=ba DATA_OUT=0

UVM_INFO scoreboard.sv(25) @ 135: uvm_test_top.env_o.sb [SCOREBOARD] WRITE PASS: ADDR=8 DATA=ba

UVM_INFO driver.sv(26) @ 135: uvm_test_top.env_o.agt.drv [driver] DRIVER: Write_en=1 ADDR=9 DATA_IN=bc

UVM_INFO monitor.sv(34) @ 145: uvm_test_top.env_o.agt.mon [monitor] MONITOR: Write_en=1 ADDR=9 DATA_IN=bc DATA_OUT=0

UVM_INFO scoreboard.sv(25) @ 145: uvm_test_top.env_o.sb [SCOREBOARD] WRITE PASS: ADDR=9 DATA=bc

UVM_INFO driver.sv(26) @ 145: uvm_test_top.env_o.agt.drv [driver] DRIVER: Write_en=1 ADDR=10 DATA_IN=be

UVM_INFO monitor.sv(34) @ 155: uvm_test_top.env_o.agt.mon [monitor] MONITOR: Write_en=1 ADDR=10 DATA_IN=be DATA_OUT=0

UVM_INFO scoreboard.sv(25) @ 155: uvm_test_top.env_o.sb [SCOREBOARD] WRITE PASS: ADDR=10 DATA=be

UVM_INFO driver.sv(26) @ 155: uvm_test_top.env_o.agt.drv [driver] DRIVER: Write_en=1 ADDR=11 DATA_IN=c0

UVM_INFO monitor.sv(34) @ 165: uvm_test_top.env_o.agt.mon [monitor] MONITOR: Write_en=1 ADDR=11 DATA_IN=c0 DATA_OUT=0

UVM_INFO scoreboard.sv(25) @ 165: uvm_test_top.env_o.sb [SCOREBOARD] WRITE PASS: ADDR=11 DATA=c0

UVM_INFO driver.sv(26) @ 165: uvm_test_top.env_o.agt.drv [driver] DRIVER: Write_en=1 ADDR=12 DATA_IN=c2

UVM_INFO monitor.sv(34) @ 175: uvm_test_top.env_o.agt.mon [monitor] MONITOR: Write_en=1 ADDR=12 DATA_IN=c2 DATA_OUT=0

UVM_INFO scoreboard.sv(25) @ 175: uvm_test_top.env_o.sb [SCOREBOARD] WRITE PASS: ADDR=12 DATA=c2

UVM_INFO driver.sv(26) @ 175: uvm_test_top.env_o.agt.drv [driver] DRIVER: Write_en=1 ADDR=13 DATA_IN=c4

UVM_INFO monitor.sv(34) @ 185: uvm_test_top.env_o.agt.mon [monitor] MONITOR: Write_en=1 ADDR=13 DATA_IN=c4 DATA_OUT=0

UVM_INFO scoreboard.sv(25) @ 185: uvm_test_top.env_o.sb [SCOREBOARD] WRITE PASS: ADDR=13 DATA=c4

UVM_INFO driver.sv(26) @ 185: uvm_test_top.env_o.agt.drv [driver] DRIVER: Write_en=1 ADDR=14 DATA_IN=c6

UVM_INFO monitor.sv(34) @ 195: uvm_test_top.env_o.agt.mon [monitor] MONITOR: Write_en=1 ADDR=14 DATA_IN=c6 DATA_OUT=0

UVM_INFO scoreboard.sv(25) @ 195: uvm_test_top.env_o.sb [SCOREBOARD] WRITE PASS: ADDR=14 DATA=c6

UVM_INFO driver.sv(26) @ 195: uvm_test_top.env_o.agt.drv [driver] DRIVER: Write_en=1 ADDR=15 DATA_IN=c8

UVM_INFO monitor.sv(34) @ 205: uvm_test_top.env_o.agt.mon [monitor] MONITOR: Write_en=1 ADDR=15 DATA_IN=c8 DATA_OUT=0

UVM_INFO scoreboard.sv(25) @ 205: uvm_test_top.env_o.sb [SCOREBOARD] WRITE PASS: ADDR=15 DATA=c8

UVM_INFO driver.sv(26) @ 205: uvm_test_top.env_o.agt.drv [driver] DRIVER: Write_en=0 ADDR=0 DATA_IN=b4

UVM_INFO monitor.sv(34) @ 225: uvm_test_top.env_o.agt.mon [monitor] MONITOR: Write_en=0 ADDR=0 DATA_IN=b4 DATA_OUT=aa

UVM_INFO scoreboard.sv(29) @ 225: uvm_test_top.env_o.sb [SCOREBOARD] READ PASS: ADDR=0 EXPECTED=aa ACTUAL=aa

UVM_INFO driver.sv(26) @ 225: uvm_test_top.env_o.agt.drv [driver] DRIVER: Write_en=0 ADDR=1 DATA_IN=e0

UVM_INFO monitor.sv(34) @ 245: uvm_test_top.env_o.agt.mon [monitor] MONITOR: Write_en=0 ADDR=1 DATA_IN=e0 DATA_OUT=ac

UVM_INFO scoreboard.sv(29) @ 245: uvm_test_top.env_o.sb [SCOREBOARD] READ PASS: ADDR=1 EXPECTED=ac ACTUAL=ac

UVM_INFO driver.sv(26) @ 245: uvm_test_top.env_o.agt.drv [driver] DRIVER: Write_en=0 ADDR=2 DATA_IN=c2

UVM_INFO monitor.sv(34) @ 265: uvm_test_top.env_o.agt.mon [monitor] MONITOR: Write_en=0 ADDR=2 DATA_IN=c2 DATA_OUT=ae

UVM_INFO scoreboard.sv(29) @ 265: uvm_test_top.env_o.sb [SCOREBOARD] READ PASS: ADDR=2 EXPECTED=ae ACTUAL=ae

UVM_INFO driver.sv(26) @ 265: uvm_test_top.env_o.agt.drv [driver] DRIVER: Write_en=0 ADDR=3 DATA_IN=69

UVM_INFO monitor.sv(34) @ 285: uvm_test_top.env_o.agt.mon [monitor] MONITOR: Write_en=0 ADDR=3 DATA_IN=69 DATA_OUT=b0

UVM_INFO scoreboard.sv(29) @ 285: uvm_test_top.env_o.sb [SCOREBOARD] READ PASS: ADDR=3 EXPECTED=b0 ACTUAL=b0

UVM_INFO driver.sv(26) @ 285: uvm_test_top.env_o.agt.drv [driver] DRIVER: Write_en=0 ADDR=4 DATA_IN=1b

UVM_INFO monitor.sv(34) @ 305: uvm_test_top.env_o.agt.mon [monitor] MONITOR: Write_en=0 ADDR=4 DATA_IN=1b DATA_OUT=b2

UVM_INFO scoreboard.sv(29) @ 305: uvm_test_top.env_o.sb [SCOREBOARD] READ PASS: ADDR=4 EXPECTED=b2 ACTUAL=b2

UVM_INFO driver.sv(26) @ 305: uvm_test_top.env_o.agt.drv [driver] DRIVER: Write_en=0 ADDR=5 DATA_IN=27

UVM_INFO monitor.sv(34) @ 325: uvm_test_top.env_o.agt.mon [monitor] MONITOR: Write_en=0 ADDR=5 DATA_IN=27 DATA_OUT=b4

UVM_INFO scoreboard.sv(29) @ 325: uvm_test_top.env_o.sb [SCOREBOARD] READ PASS: ADDR=5 EXPECTED=b4 ACTUAL=b4

UVM_INFO driver.sv(26) @ 325: uvm_test_top.env_o.agt.drv [driver] DRIVER: Write_en=0 ADDR=6 DATA_IN=f0

UVM_INFO monitor.sv(34) @ 345: uvm_test_top.env_o.agt.mon [monitor] MONITOR: Write_en=0 ADDR=6 DATA_IN=f0 DATA_OUT=b6

UVM_INFO scoreboard.sv(29) @ 345: uvm_test_top.env_o.sb [SCOREBOARD] READ PASS: ADDR=6 EXPECTED=b6 ACTUAL=b6

UVM_INFO driver.sv(26) @ 345: uvm_test_top.env_o.agt.drv [driver] DRIVER: Write_en=0 ADDR=7 DATA_IN=97

UVM_INFO monitor.sv(34) @ 365: uvm_test_top.env_o.agt.mon [monitor] MONITOR: Write_en=0 ADDR=7 DATA_IN=97 DATA_OUT=b8

UVM_INFO scoreboard.sv(29) @ 365: uvm_test_top.env_o.sb [SCOREBOARD] READ PASS: ADDR=7 EXPECTED=b8 ACTUAL=b8

UVM_INFO driver.sv(26) @ 365: uvm_test_top.env_o.agt.drv [driver] DRIVER: Write_en=0 ADDR=8 DATA_IN=55

UVM_INFO monitor.sv(34) @ 385: uvm_test_top.env_o.agt.mon [monitor] MONITOR: Write_en=0 ADDR=8 DATA_IN=55 DATA_OUT=ba

UVM_INFO scoreboard.sv(29) @ 385: uvm_test_top.env_o.sb [SCOREBOARD] READ PASS: ADDR=8 EXPECTED=ba ACTUAL=ba

UVM_INFO driver.sv(26) @ 385: uvm_test_top.env_o.agt.drv [driver] DRIVER: Write_en=0 ADDR=9 DATA_IN=ef

UVM_INFO monitor.sv(34) @ 405: uvm_test_top.env_o.agt.mon [monitor] MONITOR: Write_en=0 ADDR=9 DATA_IN=ef DATA_OUT=bc

UVM_INFO scoreboard.sv(29) @ 405: uvm_test_top.env_o.sb [SCOREBOARD] READ PASS: ADDR=9 EXPECTED=bc ACTUAL=bc

UVM_INFO driver.sv(26) @ 405: uvm_test_top.env_o.agt.drv [driver] DRIVER: Write_en=0 ADDR=10 DATA_IN=c1

UVM_INFO monitor.sv(34) @ 425: uvm_test_top.env_o.agt.mon [monitor] MONITOR: Write_en=0 ADDR=10 DATA_IN=c1 DATA_OUT=be

UVM_INFO scoreboard.sv(29) @ 425: uvm_test_top.env_o.sb [SCOREBOARD] READ PASS: ADDR=10 EXPECTED=be ACTUAL=be

UVM_INFO driver.sv(26) @ 425: uvm_test_top.env_o.agt.drv [driver] DRIVER: Write_en=0 ADDR=11 DATA_IN=96

UVM_INFO monitor.sv(34) @ 445: uvm_test_top.env_o.agt.mon [monitor] MONITOR: Write_en=0 ADDR=11 DATA_IN=96 DATA_OUT=c0

UVM_INFO scoreboard.sv(29) @ 445: uvm_test_top.env_o.sb [SCOREBOARD] READ PASS: ADDR=11 EXPECTED=c0 ACTUAL=c0

UVM_INFO driver.sv(26) @ 445: uvm_test_top.env_o.agt.drv [driver] DRIVER: Write_en=0 ADDR=12 DATA_IN=fd

UVM_INFO monitor.sv(34) @ 465: uvm_test_top.env_o.agt.mon [monitor] MONITOR: Write_en=0 ADDR=12 DATA_IN=fd DATA_OUT=c2

UVM_INFO scoreboard.sv(29) @ 465: uvm_test_top.env_o.sb [SCOREBOARD] READ PASS: ADDR=12 EXPECTED=c2 ACTUAL=c2

UVM_INFO driver.sv(26) @ 465: uvm_test_top.env_o.agt.drv [driver] DRIVER: Write_en=0 ADDR=13 DATA_IN=7f

UVM_INFO monitor.sv(34) @ 485: uvm_test_top.env_o.agt.mon [monitor] MONITOR: Write_en=0 ADDR=13 DATA_IN=7f DATA_OUT=c4

UVM_INFO scoreboard.sv(29) @ 485: uvm_test_top.env_o.sb [SCOREBOARD] READ PASS: ADDR=13 EXPECTED=c4 ACTUAL=c4

UVM_INFO driver.sv(26) @ 485: uvm_test_top.env_o.agt.drv [driver] DRIVER: Write_en=0 ADDR=14 DATA_IN=e7

UVM_INFO monitor.sv(34) @ 505: uvm_test_top.env_o.agt.mon [monitor] MONITOR: Write_en=0 ADDR=14 DATA_IN=e7 DATA_OUT=c6

UVM_INFO scoreboard.sv(29) @ 505: uvm_test_top.env_o.sb [SCOREBOARD] READ PASS: ADDR=14 EXPECTED=c6 ACTUAL=c6

UVM_INFO driver.sv(26) @ 505: uvm_test_top.env_o.agt.drv [driver] DRIVER: Write_en=0 ADDR=15 DATA_IN=53

UVM_INFO monitor.sv(34) @ 525: uvm_test_top.env_o.agt.mon [monitor] MONITOR: Write_en=0 ADDR=15 DATA_IN=53 DATA_OUT=c8

UVM_INFO scoreboard.sv(29) @ 525: uvm_test_top.env_o.sb [SCOREBOARD] READ PASS: ADDR=15 EXPECTED=c8 ACTUAL=c8

UVM_INFO driver.sv(26) @ 525: uvm_test_top.env_o.agt.drv [driver] DRIVER: Write_en=1 ADDR=1 DATA_IN=f4

UVM_INFO monitor.sv(34) @ 535: uvm_test_top.env_o.agt.mon [monitor] MONITOR: Write_en=1 ADDR=1 DATA_IN=f4 DATA_OUT=c8

UVM_INFO scoreboard.sv(25) @ 535: uvm_test_top.env_o.sb [SCOREBOARD] WRITE PASS: ADDR=1 DATA=f4

UVM_INFO driver.sv(26) @ 535: uvm_test_top.env_o.agt.drv [driver] DRIVER: Write_en=1 ADDR=7 DATA_IN=40

UVM_INFO monitor.sv(34) @ 545: uvm_test_top.env_o.agt.mon [monitor] MONITOR: Write_en=1 ADDR=7 DATA_IN=40 DATA_OUT=c8

UVM_INFO scoreboard.sv(25) @ 545: uvm_test_top.env_o.sb [SCOREBOARD] WRITE PASS: ADDR=7 DATA=40

UVM_INFO driver.sv(26) @ 545: uvm_test_top.env_o.agt.drv [driver] DRIVER: Write_en=1 ADDR=3 DATA_IN=7c

UVM_INFO monitor.sv(34) @ 555: uvm_test_top.env_o.agt.mon [monitor] MONITOR: Write_en=1 ADDR=3 DATA_IN=7c DATA_OUT=c8

UVM_INFO scoreboard.sv(25) @ 555: uvm_test_top.env_o.sb [SCOREBOARD] WRITE PASS: ADDR=3 DATA=7c

UVM_INFO driver.sv(26) @ 555: uvm_test_top.env_o.agt.drv [driver] DRIVER: Write_en=1 ADDR=11 DATA_IN=d3

UVM_INFO monitor.sv(34) @ 565: uvm_test_top.env_o.agt.mon [monitor] MONITOR: Write_en=1 ADDR=11 DATA_IN=d3 DATA_OUT=c8

UVM_INFO scoreboard.sv(25) @ 565: uvm_test_top.env_o.sb [SCOREBOARD] WRITE PASS: ADDR=11 DATA=d3

UVM_INFO driver.sv(26) @ 565: uvm_test_top.env_o.agt.drv [driver] DRIVER: Write_en=1 ADDR=12 DATA_IN=73

UVM_INFO monitor.sv(34) @ 575: uvm_test_top.env_o.agt.mon [monitor] MONITOR: Write_en=1 ADDR=12 DATA_IN=73 DATA_OUT=c8

UVM_INFO scoreboard.sv(25) @ 575: uvm_test_top.env_o.sb [SCOREBOARD] WRITE PASS: ADDR=12 DATA=73

UVM_INFO driver.sv(26) @ 575: uvm_test_top.env_o.agt.drv [driver] DRIVER: Write_en=0 ADDR=10 DATA_IN=72

UVM_INFO monitor.sv(34) @ 595: uvm_test_top.env_o.agt.mon [monitor] MONITOR: Write_en=0 ADDR=10 DATA_IN=72 DATA_OUT=be

UVM_INFO scoreboard.sv(29) @ 595: uvm_test_top.env_o.sb [SCOREBOARD] READ PASS: ADDR=10 EXPECTED=be ACTUAL=be

UVM_INFO driver.sv(26) @ 595: uvm_test_top.env_o.agt.drv [driver] DRIVER: Write_en=0 ADDR=4 DATA_IN=0

UVM_INFO monitor.sv(34) @ 615: uvm_test_top.env_o.agt.mon [monitor] MONITOR: Write_en=0 ADDR=4 DATA_IN=0 DATA_OUT=b2

UVM_INFO scoreboard.sv(29) @ 615: uvm_test_top.env_o.sb [SCOREBOARD] READ PASS: ADDR=4 EXPECTED=b2 ACTUAL=b2

UVM_INFO driver.sv(26) @ 615: uvm_test_top.env_o.agt.drv [driver] DRIVER: Write_en=0 ADDR=7 DATA_IN=ce

UVM_INFO monitor.sv(34) @ 635: uvm_test_top.env_o.agt.mon [monitor] MONITOR: Write_en=0 ADDR=7 DATA_IN=ce DATA_OUT=40

UVM_INFO scoreboard.sv(29) @ 635: uvm_test_top.env_o.sb [SCOREBOARD] READ PASS: ADDR=7 EXPECTED=40 ACTUAL=40

UVM_INFO driver.sv(26) @ 635: uvm_test_top.env_o.agt.drv [driver] DRIVER: Write_en=0 ADDR=15 DATA_IN=b0

UVM_INFO monitor.sv(34) @ 655: uvm_test_top.env_o.agt.mon [monitor] MONITOR: Write_en=0 ADDR=15 DATA_IN=b0 DATA_OUT=c8

UVM_INFO scoreboard.sv(29) @ 655: uvm_test_top.env_o.sb [SCOREBOARD] READ PASS: ADDR=15 EXPECTED=c8 ACTUAL=c8

UVM_INFO driver.sv(26) @ 655: uvm_test_top.env_o.agt.drv [driver] DRIVER: Write_en=0 ADDR=8 DATA_IN=27

UVM_INFO sequence.sv(29) @ 665: uvm_test_top.env_o.agt.seqr@@bseq [base_seq] All the Base sequence completely generated

UVM_INFO test.sv(21) @ 665: uvm_test_top [base_test] End of testcases

UVM_INFO /usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_objection.svh(1274) @ 665: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase

--- UVM Report Summary ---

** Report counts by severity

UVM_INFO : 131

UVM_WARNING : 0

UVM_ERROR : 0

UVM_FATAL : 0

** Report counts by id

[RNTST] 1

[SCOREBOARD] 42

[TEST_DONE] 1

[base_seq] 2

[base_test] 1

[driver] 42

[monitor] 42

$finish called from file "/usr/synopsys/vcs/W-2024.09-1/etc/uvm/base/uvm_root.svh", line 437.

$finish at simulation time 665

V C S S i m u l a t i o n R e p o r t

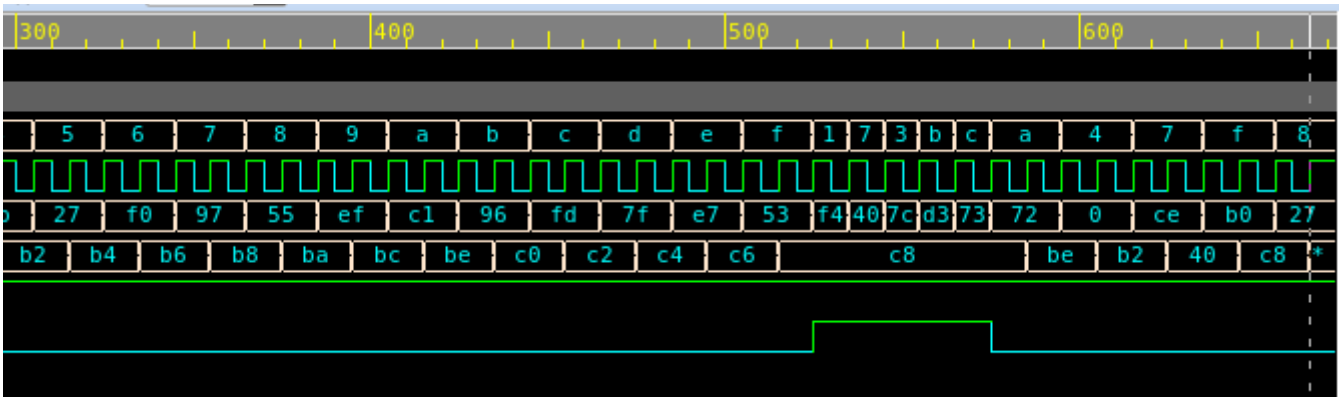
Time: 665

CPU Time: 0.170 seconds; Data structure size: 0.6Mb

Wed Aug 26 15:50:04 2026

Simulation waveform

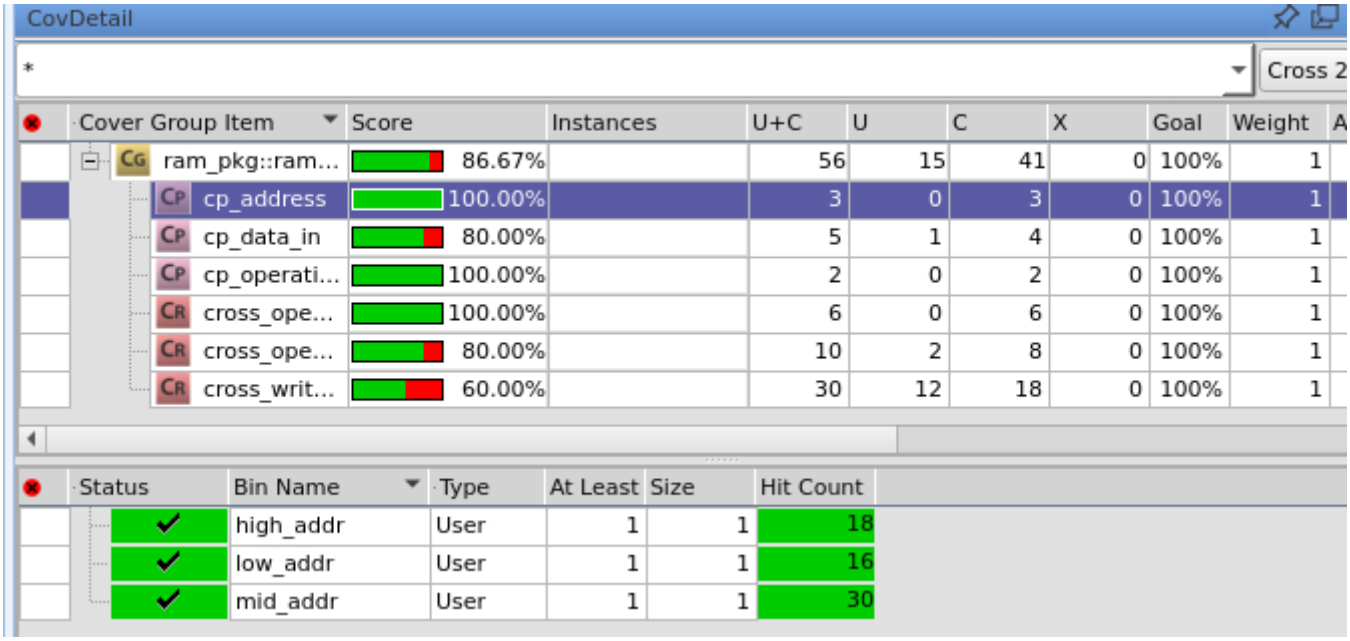




Coverage Report:-

Avg. Group Score:86.67% U+C:56 U:15 C:41 X:0
Avg. Group Inst. Score:86.67% U+C:56 U:15 C:41 X:0

Group	Score	Instances	U+C	U	C	X	Goal	Weight
ram_pkg::ram_coverage:...	86.67%		56	15	41	0	100%	1
cp_address	100.00%		3	0	3	0	100%	1
cp_data_in	80.00%		5	1	4	0	100%	1
cp_operation	100.00%		2	0	2	0	100%	1
cross_operation_addr...	100.00%		6	0	6	0	100%	1
cross_operation_data	80.00%		10	2	8	0	100%	1
cross_write_addr_data	60.00%		30	12	18	0	100%	1


CovDetail
*
Cross 2
Cover Group Item
Score
Instances
U+C
U
C
X
Goal
Weight
ram_pkg::ram...
86.67%
56
15
41
0
100%
1
cp_address
100.00%
3
0
3
0
100%
1
cp_data_in
80.00%
5
1
4
0
100%
1
cp_operati...
100.00%
2
0
2
0
100%
1
cross_ope...
100.00%
6
0
6
0
100%
1
cross_ope...
80.00%
10
2
8
0
100%
1
cross_writ...
60.00%
30
12
18
0
100%
1
Status
Bin Name
Type
At Least
Size
Hit Count
high_addr
User
1
1
18
low_addr
User
1
1
16
mid_addr
User
1
1
30